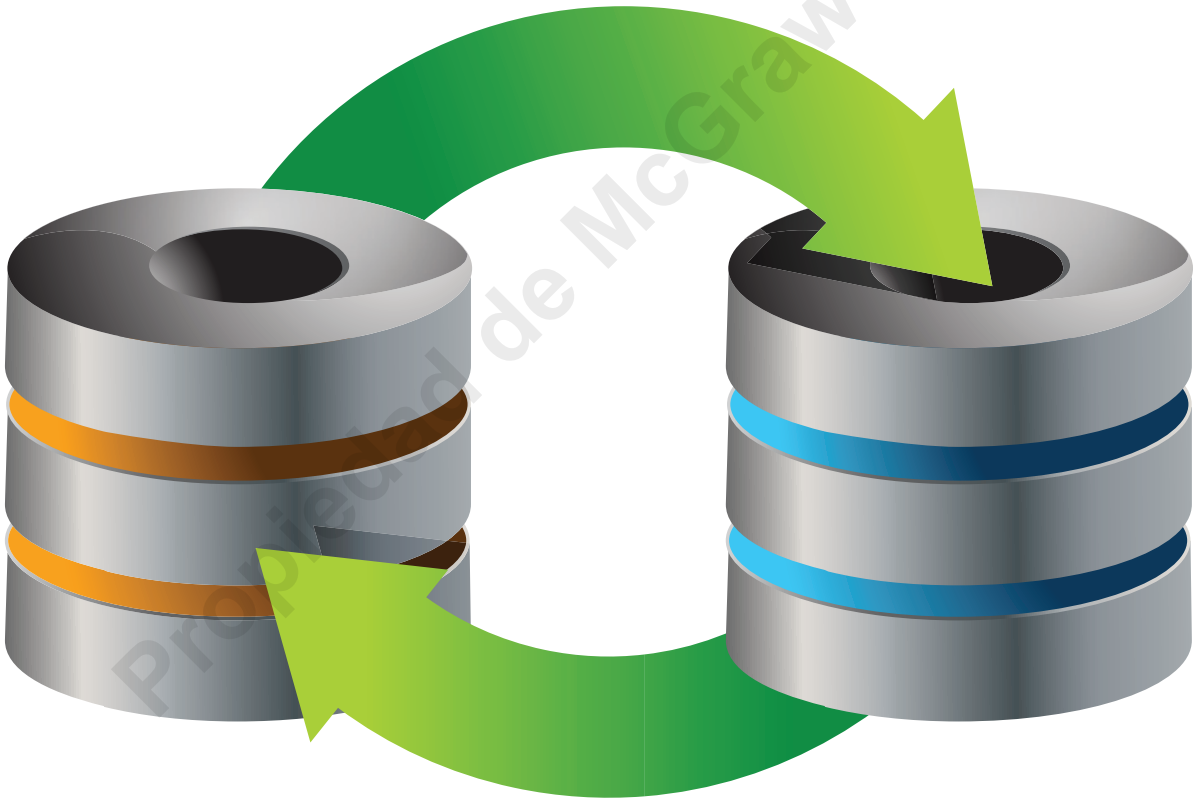
www.mheducation.es Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

CFGS | INFORMÁTICA

Gestión de Bases de Datos

Fidel Oltra Landete
Jose Chamorro Molina
Raül V. Lerma Blasco
José Luis Gil Gil



Propiedad de McGraw-Hill®

Mc Graw Hill

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Propiedad de McGraw-Hill®

Gestión de Bases de Datos

No está permitida la reproducción total o parcial de este libro, ni su tratamiento informático, ni la transmisión de ninguna forma o por cualquier medio, ya sea electrónico, mecánico, por fotocopia, por registro u otros métodos, sin el permiso previo y por escrito de los titulares del *copyright*.

Diríjase a CEDRO (Centro Español de Derechos Reprográficos, www.cedro.org) si necesita fotocopiar o escanear algún fragmento de esta obra.

Nota: Este libro se atiene al artículo 32 del derecho de cita de la Ley de Propiedad Intelectual de 1996 (R. D. Leg. 1/1996 de 12 de abril).

Derechos reservados © 2022, respecto a la primera edición en español, por:

McGraw Hill/Interamericana de España, S. L.

Basauri, 17 Edificio A, 1ª planta

28023 Aravaca (Madrid)

ISBN digital: 9788448626624

© Fidel Oltra Landete, Jose Chamorro Molina, Raül V. Lerma-Blasco y José Luis Gil Gil

Corrección técnica: Joaquim Matalí Ortiz

Equipo editorial: Miguel Puigmitjá Dávila y Óscar Álvarez Alonso

Corrección de estilo y ortotipográfica: Àlex López Frutuoso

Ilustración: Pablo Vázquez Rodríguez, Archivo McGraw-Hill, 123rf

Fotografías: Archivo McGraw-Hill, 123rf

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Unidad 1.

Introducción a las bases de datos

Propiedad de McGraw-Hill®

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

1. Persistencia y gestión de la información

En los últimos años, varias frases sobre datos e información aparecen de forma recurrente en publicaciones tanto económicas y empresariales como relacionadas con la informática. Entre ellas encontramos afirmaciones como "Los datos son el nuevo petróleo" o "La información es el bien más valioso de las empresas". De manera más general, se suele decir que "la información es poder". Estas afirmaciones, que podrían parecer exageradas, se refieren a la enorme importancia que tiene la gestión de los datos y la información por parte de todo tipo de empresas. Algo muy evidente en el caso de las tecnológicas, pero que en realidad se puede aplicar ya a casi cualquier tipo de negocio. Dejando aparte los motivos de este elevado interés de las empresas por los datos, que puedes ampliar en un artículo de la Universidad de Burgos que encontrarás entre los enlaces recomendados, lo que interesa en este apartado es dilucidar tres cuestiones: ¿qué son los datos? ¿Qué es la información? ¿Por qué es importante gestionarla adecuadamente?

1.1. Los datos

Un dato es una representación simbólica de un atributo que puede ser cuantificable o calificable. Por ejemplo, "verde" es un dato y "100" también es un dato, aunque uno sea una palabra y otro un número.



Sin embargo, los datos por sí mismos no ofrecen ninguna información valiosa si no se ponen en relación entre sí y con el contexto en el que adquieren significado. Solo entonces, cuando son contextualizados, estructurados, analizados e interpretados podemos decir que se convierten en información relevante.

En la página web del Instituto Nacional de Estadística puedes encontrar numerosas fuentes de datos, tablas, informes y estadísticas sobre una gran variedad de temas.

1.2. La información

Un error común es confundir los datos con información o conocimiento. ¿Qué podemos deducir de una serie como 410583, 393181, 372777, 360617 y 339206? No mucho, solo que son números ordenados de manera descendente. Sin embargo, si los contextualizamos diciendo que se han sacado de un informe del Instituto Nacional de Estadística sobre demografía y que representan el número de nacimientos en España entre 2016 y 2020, entonces empiezan a tomar sentido. Ahora sabemos algo que antes no sabíamos: en los últimos añ

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

la natalidad en España ha descendido. Podemos obtener también el porcentaje de descenso año a año. Estamos recibiendo información.

Puedes consultar un ejemplo como este, sobre el movimiento natural de la población en 2020, en la sección enlaces de interés del curso.

La obtención de información, por tanto, implica que necesitamos **recopilar, almacenar, organizar, relacionar y analizar datos.**

En este módulo nos ocuparemos, sobre todo, del almacenamiento de los datos y todos sus aspectos relacionados. Pero ¿por qué es tan importante este paso?

Los datos, aunque se procesan en la memoria del ordenador, deben almacenarse en otro lugar para asegurar su disponibilidad en cualquier momento. La memoria, aunque rápida para procesar datos, es volátil, lo que significa que cuando deja de recibir alimentación, o incluso al cerrar la aplicación correspondiente, se pierde su contenido. Por eso necesitamos dispositivos donde los datos puedan guardarse de forma permanente. El concepto de persistencia de la información hace referencia a la necesidad de disponer de dispositivos en los que se puedan guardar todo tipo de datos para una posterior utilización.



CASO PRÁCTICO 1

En otros módulos, como Fundamentos del Hardware o Implantación de Sistemas Operativos, aprenderás todo lo necesario sobre el funcionamiento de la memoria y los dispositivos de almacenamiento permanentes.

1.3. La gestión de los datos

Si solo tuviéramos que almacenar datos podría servir para ello una carpeta, un archivador o una libreta. Sin embargo, sabemos que los datos almacenados deben procesarse para obtener la información que buscamos. Según aumenta la cantidad de datos, algo que como ya hemos comentado cada vez es más frecuente en todo tipo de organizaciones, aumenta la complejidad de su gestión. Una gestión que incluye su recuperación, tratamiento (donde tendrían cabida el procesamiento, el análisis y la obtención de nuevos datos) y distribución de la información resultante. Estos procesos forman lo que se conoce como «ciclo de vida de los datos».



Ciclo de vida de los datos.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.



Para poder llevar a cabo todos los procesos de manera efectiva, rápida y correcta debemos tener un sistema que incluya los recursos necesarios para realizar cada paso. Ahí es donde recurrimos a la ayuda de la informática, que nos proporciona un hardware que permite el almacenamiento y las comunicaciones entre equipos o usuarios, pero también un software que desarrolla los procesos de recopilación, clasificación, recuperación, tratamiento y distribución de la información, considerando como información tanto los datos de entrada como los de salida. En este caso, cuando nos ayudamos de la informática para realizar todos los pasos del ciclo de vida de los datos, hablamos de un sistema informático. Trataremos más a fondo esta cuestión en el siguiente apartado.

> No olvidamos que el propio término "informática" tiene su raíz en la unión de dos palabras: **INFOR**mación y auto**MÁTICA**. De hecho, la informática podría definirse, en un sentido amplio, como la ciencia que se ocupa de estudiar e implementar las técnicas necesarias para llevar a cabo el procesamiento de la información de forma automática (no manual).

La gestión de los datos, por tanto, implica la capacidad de sistematizar todas las tareas necesarias para recopilarlos, almacenarlos, recuperarlos, hacer su tratamiento y obtener nuevos datos que, finalmente, podrán ser distribuidos, presentados y, posiblemente, almacenados de nuevo.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

2. Sistemas de información. Ficheros

2.1. Sistemas de información

Un **sistema de información** es un conjunto de recursos que sirven para recoger, guardar, tratar y repartir información dentro de una organización.

Los **sistemas de información** facilitan y optimizan las tareas relacionadas con la gestión de la información, desde su recogida y su almacenamiento hasta su procesamiento y distribución. La estructura básica de un sistema de información, por tanto, debería estar relacionada con el ciclo de vida de los datos que hemos visto en el apartado anterior. Una posible estructura sería:



Un sistema que maneje una cantidad importante de datos necesita automatizar ciertas tareas. La informática, como sabemos, nos ofrece multitud de recursos para gestionar datos de manera automática. Al aplicar la informática a un sistema de información podemos decir que disponemos de un sistema informático.

2.2. Sistema informático

Un sistema informático utiliza recursos tecnológicos para llevar a cabo las diferentes fases del ciclo de vida de los datos, desde la recopilación y el almacenamiento hasta el tratamiento y la distribución de resultados. Estos recursos se agrupan en dos categorías: hardware y software.

El hardware incluye cualquier recurso físico: ordenadores, impresoras, enrutadores, etcétera.

El software, por su parte, engloba cualquier tipo de programa o aplicación que ayude a llevar a cabo las distintas tareas de gestión de los datos, incluyendo los sistemas operativos, ya que intervienen en el funcionamiento del hardware.

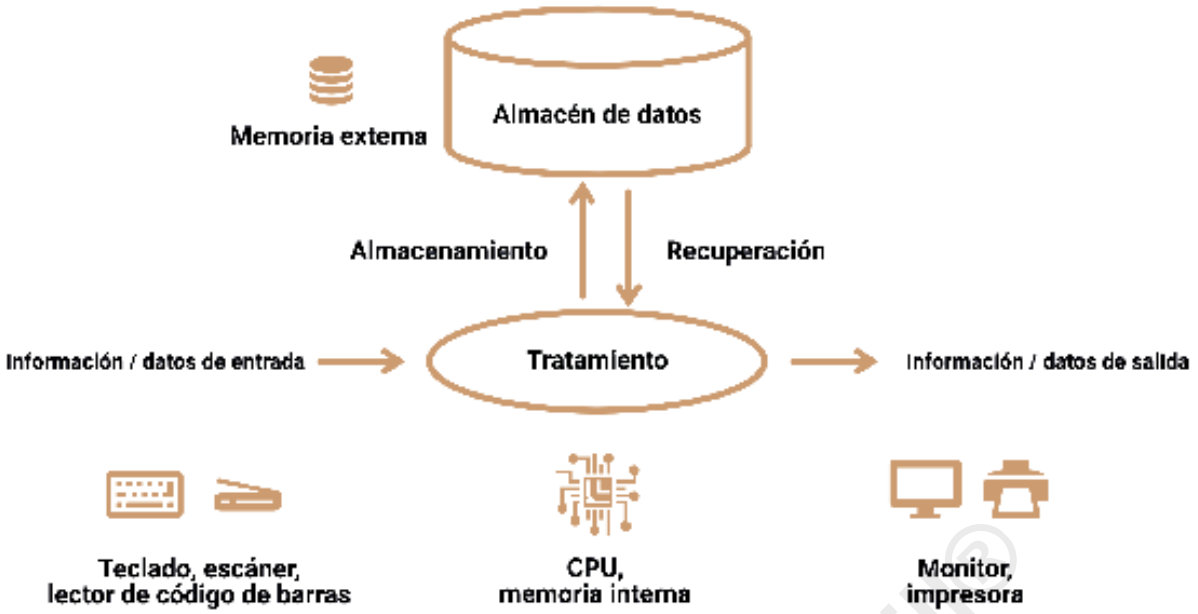
Algunos autores añaden otra categoría, el peopleware, un concepto que hace referencia a la relación del sistema con las personas implicadas en su diseño, desarrollo, mantenimiento y utilización. Aquí entrarían aspectos de diseño adaptable, estilos, software colaborativo, etcétera.

Trazando un paralelismo entre las partes de un sistema informático y las funciones de un sistema de información, la memoria interna (RAM) interviene en el tratamiento de los datos, los dispositivos permanentes sirven para el almacenamiento y los dispositivos de entrada ayudan a recopilar los datos, mientras que los dispositivos de salida permiten interactuar con los usuarios y mostrar el resultado de los tratamientos.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

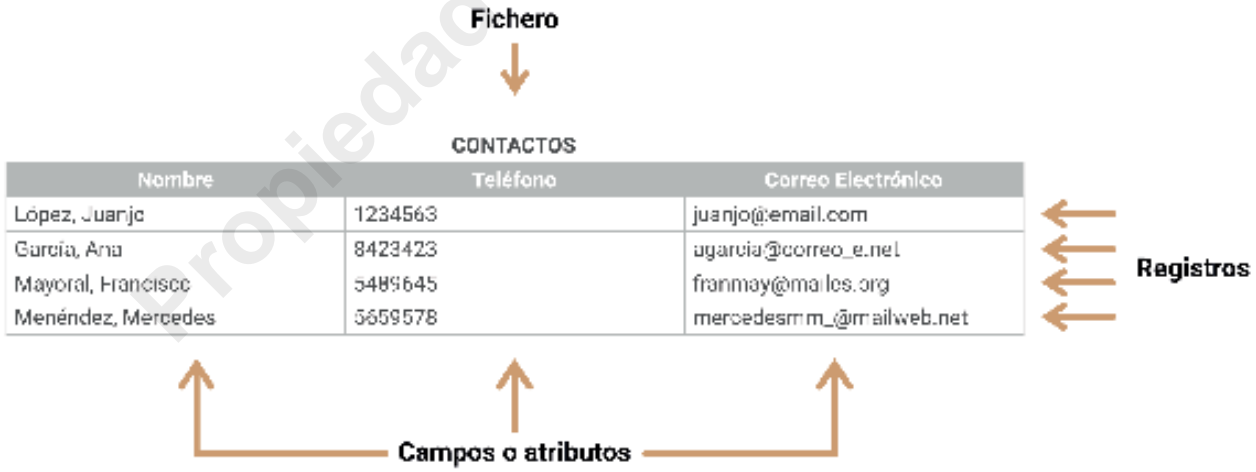
Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.



2.3. Ficheros

Para un mejor manejo y control de los datos gestionados, el sistema operativo los organiza en forma de ficheros. Un fichero sería una colección de datos de un mismo tipo. Así, podemos tener ficheros de productos, personas, datos estadísticos o ventas de una empresa. Dentro de cada fichero los datos vuelven a agruparse, esta vez en registros. Un registro es una ficha del fichero, con lo cual correspondería a uno de los elementos guardados en este: una venta, una persona o un producto.

Cada ficha puede incluir varios datos sobre el elemento tratado, y cada uno de esos datos sería un atributo o campo del registro. Para ilustrar mejor el concepto podemos pensar en una hoja de cálculo en la que guardáramos información sobre nuestros contactos telefónicos. La hoja de cálculo sería el fichero, cada fila sería un registro, y cada columna sería un campo o atributo.



Según su formato, los ficheros pueden ser de texto plano o estructurados. Los de texto plano tienen la ventaja de que pueden abrirse con cualquier editor de textos, mientras que los estructurados facilitan la búsqueda de datos concretos pero necesitan software adicional. El acceso a los ficheros puede ser secuencial (típico en archivos de texto plano), directo, indexado o una combinación de estos. En general, se incluye cierta información adicional que permite relacionar el contenido del fichero con su ubicación física, de forma que para acceder a un dato no hace falta recorrer todo el fichero desde el principio, salvo en el acceso secuencial.

Cuando por el tipo de acceso es necesario relacionar su contenido con su posición en los dispositivos de almacenamiento, necesitamos un sistema de gestión de ficheros.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Un **sistema de gestión de ficheros (SGF)** es un conjunto de programas que, conjuntamente con el sistema operativo o como parte del mismo, permiten organizar, gestionar y recuperar la información archivada en la memoria externa.

El sistema de gestión de ficheros recibe órdenes lógicas para trabajar con los datos y las convierte en operaciones físicas para encontrar el fichero en el dispositivo de almacenamiento, trasladarlo (todo o en parte) a la memoria interna para ser procesado, y actualizar su contenido en la memoria externa.

¿Puedes ampliar información sobre ficheros, sus tipos y gestión en el documento *What is File? Types of file organization*, que puedes encontrar la sección de enlaces web.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

3. Bases de datos: topologías y arquitecturas

La principal diferencia entre una **base de datos** y un conjunto cualquiera de ficheros es que en la base de datos los ficheros que la forman están relacionados entre sí. Además, esas relaciones se guardan en la propia base de datos, igual que se hace con el contenido de cada fichero. De esa forma es muy fácil llevar a cabo operaciones como incluir en un único listado datos provenientes de distintos ficheros, o localizar fácilmente información en un fichero a partir de un dato incluido en otro.

3.1. Características fundamentales de las bases de datos

Las principales características que diferencian una base de datos de otro sistema organizado de ficheros, además de la gestión de las relaciones entre sí, son:

ASEGURAN LA INTEGRIDAD DE LA INFORMACIÓN

Los datos y las relaciones que se guardan en la base de datos van a ser correctas en todo momento. Por ejemplo, en una base de datos con ficheros de clientes y facturas podemos impedir, si así lo deseamos, que se guarde una factura sin determinar a qué cliente pertenece.

MINIMIZAN LAS REDUNDANCIAS:

Cada dato solo se repetirá en diversos ficheros si es imprescindible para gestionar la relación entre sí. Por ejemplo, si en un fichero con datos del alumnado del centro tuviéramos el número de matrícula, el nombre, los apellidos, la dirección o el teléfono, en un fichero que guardase las notas solo haría falta incluir el número de matrícula para saber a quién pertenecería cada nota.

ALUMNADO

NIA	Nombre
123456	Jorge González
135736	Almudena Pérez
134893	Estrella López
143324	Rocío García

NOTAS

NIA	Módulo	Nota
123456	GBD	10
135736	GBD	9
134893	GBD	10
143324	GBD	9

Tabla 1.1. Ejemplo de redundancia de información

FACILITAN LA COMPARTICIÓN:

La compartición de la información entre los distintos usuarios, así como la gestión de estos (nombres, contraseñas, permisos). Esto significa que diferentes usuarios podrán estar haciendo gestiones sobre un mismo archivo sin que se pierdan cambios ni se comprometa la integridad de la información.

SON INDEPENDIENTES DE LAS APLICACIONES DE USUARIO

Una misma base de datos puede ser usada por diferentes aplicaciones, incluso escritas en diferentes lenguajes o para distintos sistemas operativos, algo que no ocurre en general con los sistemas de gestión de ficheros.

Cualquier organización que maneje cierta cantidad de ficheros, y sobre todo que necesite relacionarlos de forma rápida y eficaz, necesita una base de datos. Ejemplos: una biblioteca, un instituto, un taller de automóviles o una farmacia. La manera como se guardan los archivos y sus datos en una base de datos, así como la forma como se relacionan los distintos archivos entre sí, definen la topología de la base de datos o, como se suele llamar, el modelo. De los diferentes modelos de datos hablaremos en un apartado posterior.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

3.2. Arquitectura de una base de datos

En realidad, al hablar de la arquitectura de una base de datos estamos haciendo referencia a la manera como se gestionan sus diversos componentes. Esta función la llevan a cabo los sistemas gestores de bases de datos, que abordaremos más adelante. De momento podemos adelantar que la información correspondiente a una base de datos se guarda fundamentalmente a dos niveles:

- En un nivel físico estarían los propios datos.
- En un nivel lógico estaría la información sobre las relaciones entre los datos, cómo se agrupan y, en general, qué condiciones deben cumplir. Es decir, toda la información relevante sobre la base de datos que no sean los propios datos.

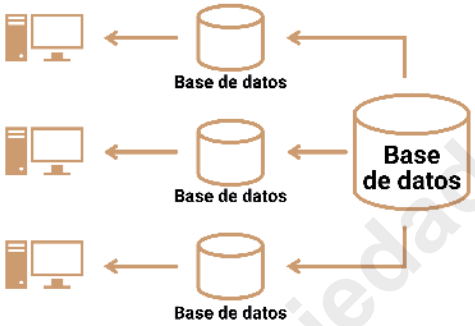
Volveremos más adelante sobre esta separación en niveles, que se estudiará con más detalle al hablar de los sistemas gestores de bases de datos.

Sí que podemos, de momento, establecer una clasificación de las bases de datos en cuanto a su ubicación física:



Centralizada

La base de datos está toda en el mismo ordenador, aunque este forme parte de un sistema más amplio, como un NAS (Network Attached Storage) o un RAID (Redundant Arrays of Independent Disks).



Distribuida

La base de datos almacena la información en distintos ordenadores (o máquinas virtuales) conectados entre sí. La decisión de distribuir los datos de esta forma puede hacerse por motivos de seguridad (cada departamento solo accede a sus datos), para optimizar los accesos remotos (por ejemplo, una sucursal de un banco accede localmente a ciertos datos, y remotamente a otros) o para distribuir la carga de trabajo entre varios ordenadores y no dirigir todas las peticiones a una misma máquina. La tecnología blockchain, en la que las operaciones sobre la base de datos primero se agrupan en bloques de transacciones, se verifican desde distintos nodos, y finalmente se registran en la base de datos principal, sería un ejemplo de esta modalidad.

En las bases de datos centralizadas resulta más sencillo mantener la seguridad e integridad de los datos, se consigue un mayor rendimiento y su implantación es más económica. Por otra parte, las bases de datos distribuidas ofrecen ventajas como una mayor disponibilidad de los datos en caso de fallo y mejores posibilidades de expansión.

La distribución de los datos puede hacerse de varias formas:

Replicación	Cada nodo de la red de distribución tiene una copia completa de la base de datos. El mantenimiento de los datos es más costoso, porque cada cambio debe replicarse en todas las copias.
Partición	La información está repartida entre los diferentes nodos sin duplicarse. El coste de almacenamiento y mantenimiento es menor, pero también es menos fiable en caso de fallos.
Combinando ambos modos	Cierta información se reparte entre los nodos sin duplicarse, mientras que los datos más importantes, o que se consultan con mayor frecuencia, pueden replicarse.

En la sección de enlaces puedes encontrar más información sobre bases de datos distribuidas.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

4. Sistemas gestores de bases de datos

Un **sistema gestor de bases de datos (SGBD)** incluye las herramientas necesarias para trabajar con bases de datos, asegurando que se cumplan todos los requerimientos que se han estudiado en anteriores apartados.

Por ello, para cada tipo de base de datos se necesitará un SGBD distinto. En adelante hablaremos, principalmente, de los **SGBD relacionales**, ya que la mayor parte del material posterior, salvo alguna unidad, tratará sobre este tipo de bases de datos. Ahora bien, hay algunas características comunes a todos ellos que vamos a ver a continuación.

4.1. Características fundamentales de los sistemas gestores de bases de datos

Las características principales que debe cumplir un sistema gestor de bases de datos son:

- Debe gestionar no solo los datos, sino **también las relaciones** entre ellos.
- Debe permitir la **creación, modificación y ampliación de la estructura de las bases de datos** y todos **sus componentes**.
- Debe **permitir la gestión de los usuarios de la base de datos**, así como de las operaciones que les será permitido llevar a cabo.
- En general deben **disponer de una interfaz gráfica**, o al menos accesible, que facilite las tareas de administración.
- Deben **permitir los accesos concurrentes** y **mantener la integridad de los datos.**
- Deben ser independientes de las aplicaciones que el usuario pueda utilizar para acceder a la base de datos y trabajar con ella. De esa forma se garantiza que una misma base de datos pueda usarse desde aplicaciones distintas, e incluso desde sistemas operativos diferentes. También se facilita, si es necesario, la migración de los datos a otra plataforma distinta.

Entre los mecanismos de los que dispone un sistema gestor de bases de datos para desarrollar dichas funciones encontramos las siguientes:

	Minimización de las redundancias
	Control y cumplimiento de las restricciones
	Opciones de regeneración de los datos después de una operación fallida o incompleta
	Gestión de los accesos simultáneos, estableciendo políticas de bloqueo
	Gestión de los usuarios y sus permisos

Propiedad de McGraw-Hill®

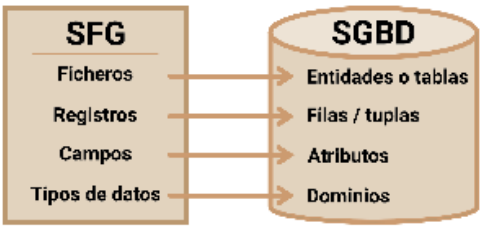
Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

4.2. Correspondencia entre sistemas gestores de ficheros y sistemas gestores de bases de datos

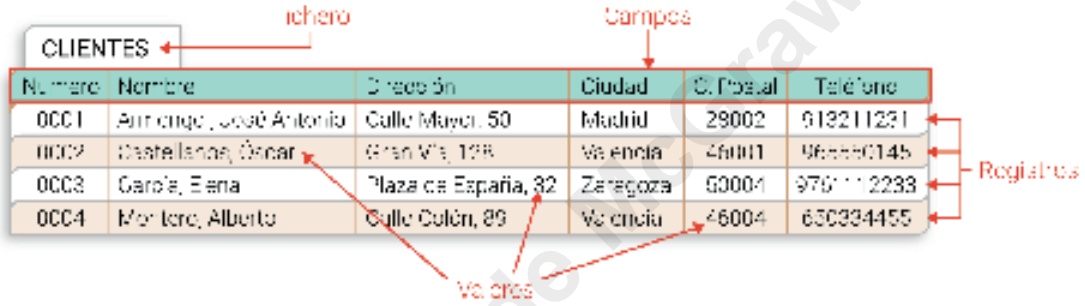
Anteriormente hemos visto la nomenclatura utilizada para especificar el trabajo con un sistema de gestión de ficheros. Se manejan ficheros que contienen registros y estos están divididos en campos. Veamos cuál sería la equivalencia en un sistema gestor de bases de datos relacionales, los más usados por el momento:

- Los ficheros en un SGBD relacional se llaman tablas.
- Los registros se llaman filas o tuplas.
- Los campos se llaman atributos.
- Los tipos de datos se llaman dominios.



Ten en cuenta que los conceptos de la figura anterior y los que veremos a continuación, aunque muchos son extrapolables a otros SGBD, se refieren sobre todo a los sistemas que trabajan con bases de datos relacionales. En una unidad posterior veremos cómo se organizan y gestionan otros tipos de bases de datos.

De acuerdo con las equivalencias vistas en la figura, una tabla de una base de datos relacional se vería de la siguiente forma:



Como puedes ver, una tabla en un sistema gestor de base de datos relacional aparece como una estructura con filas (registros o tuplas) y columnas (atributos). Cada fila corresponde a un elemento de la tabla, y en cada columna se guarda un dato correspondiente a dicho elemento. En cuanto al dominio, que no aparece en la figura, se refiere al tipo de datos de cada atributo, así como a ciertos requisitos que tal vez deba cumplir: ser un número entero, ser un número entero positivo, ser una cadena alfanumérica, ser una cadena alfanumérica de un máximo de 10 caracteres, una cadena alfanumérica de exactamente 5 caracteres, etcétera.

4.3. Componentes de los sistemas gestores de bases de datos

Los componentes de un SGBD son, a grandes rasgos, los siguientes:

	El **gestor o motor de la base de datos.**
	El **diccionario de datos** (información sobre cómo están organizados los datos en archivos, atributos, etcétera).
	Los **usuarios**, con sus tipos, roles y permisos.
	Los **lenguajes** que permiten crear y mantener la base de datos, con sus intérpretes y procesadores.
	**Interfaces** externas para facilitar su uso.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

La arquitectura de un sistema gestor de bases de datos hace referencia a la forma en que dichos componentes se distribuyen en el sistema. En general, se divide en tres niveles:

Nivel interno	En él se gestiona la forma como se guardan físicamente los datos. Es transparente al usuario, que no debe preocuparse de esta parte, puesto que el acceso se hace de forma interna y automática a través del software del sistema gestor.
Nivel conceptual	En este nivel se describe la organización lógica de los datos, cómo se agrupan en tablas, las relaciones entre estas, las restricciones, los usuarios con sus permisos, etcétera.
Nivel externo	También se conoce como esquema externo. Es la capa con la que interacciona el usuario final, bien directamente o, más frecuentemente, a través de aplicaciones. Se muestra una vista con la información requerida por el usuario o, en caso de desarrollar tareas de administración, las utilidades correspondientes para realizar dichas tareas.

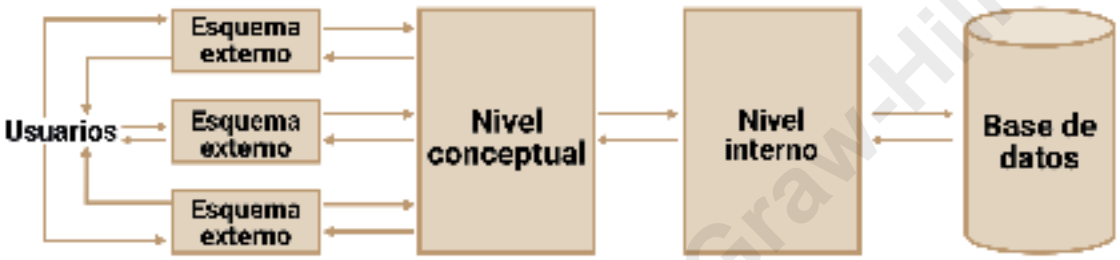


Fig. 1.1. Arquitectura de un SGBD.

En la figura puede verse cómo los usuarios finales interactúan siempre con el nivel externo y sus diferentes esquemas o vistas, proporcionados directamente por el sistema gestor de la base de datos o por las aplicaciones intermedias. En el caso de usuarios con rol de administrador, pueden interactuar también con otros niveles, principalmente con el conceptual, para poder hacer cambios en la estructura de la base de datos: añadir tablas, cambiar dominios, añadir o quitar columnas en una tabla, etcétera.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

5. Modelos de bases de datos

En realidad, un modelo de bases de datos es un concepto que, aunque siempre está relacionado con algún tipo de representación del contenido de la base de datos, puede tener dos significados similares pero distintos. Por un lado, puede referirse a los conceptos abstractos en los que se basa el diseño inicial de la base de datos: los elementos que se verán representados, la información que necesitaremos sobre dichos elementos, las relaciones entre ellos, sus propiedades, etcétera. Hablaríamos así de un modelo relacionado con el diseño lógico. Por otra parte, otros modelos de datos definen la estructura concreta de los datos y sus relaciones, tanto lógicas como físicas. Veamos ambos casos.

5.1. El modelo lógico

Los modelos lógicos no son, en realidad, exclusivos del mundo de las bases de datos. Un **modelo lógico** se usa para describir aspectos relacionados con la representación abstracta de objetos y elementos de la vida real de forma que sirva para empezar a diseñar un sistema que permita cubrir las necesidades de una organización. Sería como un esquema, una especie de boceto que usamos como punto de partida para diseñar la base de datos. En ocasiones también nos referimos a ellos como **modelos semánticos**.

El modelo lógico se centra en la información en abstracto, su descripción, semántica, restricciones y relaciones, más que en las operaciones que se harán con los datos concretos. El modelo lógico de la base de datos sería como un plano para su diseño, un plano que servirá para establecer cómo será su diseño conceptual y cómo se relacionará este con su implementación física.

Hay dos modelos lógicos principales, que son los más utilizados:

- El **modelo entidad/relación** (E/R)
- El **lenguaje unificado de modelado** (UML).

> En posteriores unidades nos ocuparemos a fondo del modelo entidad/relación, ampliamente usado como modelo lógico en bases de datos. El lenguaje unificado de modelado está, sin embargo, más relacionado con el uso de clases y objetos, conceptos pertenecientes al mundo de la programación y que por tanto se estudian en los ciclos de DAM y DAW.

5.2. El modelo conceptual/físico

El modelo conceptual se presenta en muchas ocasiones como modelo físico, puesto que los conceptos desarrollados en este esquema afectan también a la forma en la que los datos se almacenan, se recuperan y se relacionan entre sí físicamente.

El modelado conceptual y físico de las bases de datos va estrechamente unido a la evolución histórica de los sistemas gestores de bases de datos, así que en primer lugar vamos a hacer un pequeño repaso de dicha evolución.

5.3. Evolución de los sistemas gestores de bases de datos

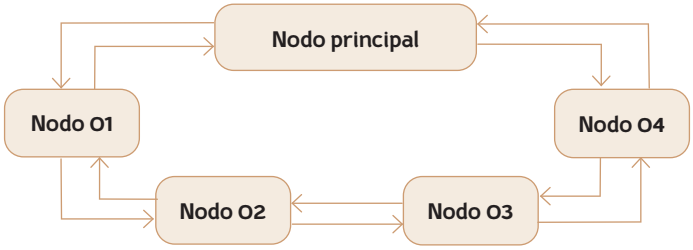
Las primeras bases de datos propiamente dichas surgen como consecuencia de la aparición de dispositivos de acceso directo, en los que ya se podían hacer búsquedas no secuenciales, saltos de unos datos a otros y maniobras interactivas con dichos datos.

Los primeros sistemas gestores de base de datos, allá por los años sesenta, eran **navegacionales**: los elementos que componen la base de datos se encuentran navegando a través de las referencias incluidas en los propios elementos. Entre ellos estaba IDS, que fue la base desde la que evolucionaron CODASYL o IDMS y modelos posteriores como los jerárquicos o los de red, que veremos posteriormente.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.



Esquema de un sistema navegacional.

En los años setenta, a partir de los trabajos de Edgar F. Codd en IBM, aparecen los **sistemas gestores de bases de datos relacionales**, cuyo documento fundacional podrás consultar en los enlaces adjuntos. En estas bases de datos, el sistema encadenado dejaba paso a otro basado en tablas relacionadas entre sí según las necesidades de navegación. Los trabajos de Codd dieron origen a sistemas como el System R de IBM, INGRES y todos los sistemas SQL que han llegado después, desde Sybase o Informix hasta SQL Server, PostgreSQL, MySQL, Oracle, MariaDB, etcétera. Veremos la estructura de una base de datos relacional en el siguiente punto.

Con el auge de la programación orientada a objetos, los sistemas gestores de bases de datos intentaron adaptarse a este paradigma, y surgieron distintas iniciativas que han dado lugar a una rama en la que se experimenta con bases de datos totalmente orientadas a objetos, y un campo más amplio que podemos llamar los **sistemas gestores de bases de datos NoSQL**. Estos últimos ya han sido ampliamente desarrollados y comercializados, con aplicaciones como MongoDB, Apache Cassandra o Redis. En general, se orientan a documentos más que a entidades con una estructura cerrada, y funcionan como almacenes de datos con una estructura ***clave → valor***, lo que las convierte en lo más cercano a una base de datos orientada a objetos.

```
{
    "_id": "5odf99234sdfsf92342",
    "nombre": "Ana",
    "apellidos": "Sánchez",
    "dirección": {
        "calle": "Gran Vía",
        "ciudad": "Madrid",
        "cpostal": "28013",
        "pais": "España"
    },
    "estudios": ["DAW", "DAM", "Universitarios"]
}
```

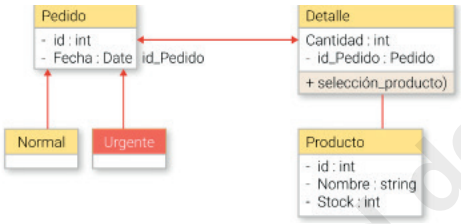
Mención aparte merecen los sistemas gestores de bases de datos que trabajan en la nube: se trata del llamado **cloud storage**. No se corresponden con modelos determinados: aunque tienden a parecerse más a los documentales, se pueden encontrar otros relacionales o híbridos. Los datos, como su nombre indica, se encuentran alojados en espacios virtualizados aportados por terceros, distribuidos entre diversos servidores físicos y accesibles solamente a través de determinados servicios en la web o, en algunos casos, mediante interfaces de usuario. Aunque servicios como Dropbox o Google Drive permiten almacenar datos en la nube, no se pueden considerar en realidad bases de datos como AWS (Amazon) o Azure (Microsoft), por citar un par de ellos. En una unidad posterior estudiaremos más a fondo estos tipos de bases de datos.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

5.4. Tipos de modelos conceptuales/físicos

Los modelos conceptuales de bases de datos han ido evolucionando de la mano de los sistemas gestores, por lo que su clasificación es bastante similar. Los principales modelos de bases de datos son:

<table>
<tr><td> 
Modelo jerárquico</td><td>Modelo jerárquico
Hay una jerarquía entre los ficheros, aquí llamados nodos, de forma que la relación entre ellos tiene forma de árbol con raíces y ramas. Fue el primero en aparecer y en la actualidad está en desuso.</td></tr>
<tr><td> 
Modelo en red</td><td>Modelo en red
Las relaciones no son exclusivamente jerárquicas, sino que cualquier nodo puede estar relacionado con cualquier otro nodo. Tendría forma de malla o red, de ahí su nombre. También se encuentra obsoleto, salvo para propósitos muy específicos.</td></tr>
<tr><td> 
Modelo relacional</td><td>Modelo relacional
La base de datos está formada por entidades (archivos) y relaciones entre ellas. Estas relaciones no vienen establecidas por una ruta prefijada, como en los modelos anteriores, sino por relaciones lógicas entre las distintas entidades. Cada entidad tiene forma de tabla donde las filas son los registros y las columnas, los atributos o campos.</td></tr>
<tr><td> 
</td><td>Modelo orientado a objetos
Modelo en expansión, sobre todo usado de manera conjunta con el relacional. La organización de la base de datos se asemeja a la de las clases en un programa orientado a objetos, lo que hace muy sencillo su mapeado (establecer equivalencia entre clases y tablas).</td></tr>
<tr><td> 
</td><td>Modelo documental
Se usa para entidades con una estructura variable que no se puede organizar fácilmente en tablas con filas y columnas, sino más bien en forma de documentos.</td></tr>
</table>

Existen otros modelos de datos muy específicos para llevar a cabo tareas muy concretas. Entre ellos podemos destacar el modelo **transaccional**, el **multidimensional** o el **deductivo**. Puedes ampliar información sobre estos modelos en los enlaces correspondientes.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Unidad 2.

Modelado de bases de datos. El modelo entidad/relación

Propiedad de McGraw-Hill®

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

1. Diseño lógico de una base de datos. Modelado

De la misma forma que para empezar a programar o diseñar una página web hay que hacer un análisis previo, lo mismo ocurre cuando queremos empezar a diseñar una base de datos. Este análisis previo es el punto de partida de la creación de nuestra base de datos y se conoce como **diseño lógico**. Este proceso empieza recopilando todos los datos que queremos incluir, continúa agrupándolos en conceptos relacionados con el mundo real y finalmente se completa estableciendo relaciones entre dichos conceptos. Toda esta información obtenida mediante el análisis debe representarse de alguna forma por dos razones fundamentales:

- Para tener un esquema que sirva de punto de referencia desde el que seguir el proceso de diseño de la base de datos.
- Para que podamos compartir nuestro análisis con otras personas en caso de estar trabajando en equipo.

Esa representación no puede llevarse a cabo de cualquier manera, sino que debe seguir una serie de estándares y especificaciones. De esa forma, tanto nosotros como cualquier otra persona que acceda a nuestro análisis será capaz de comprender lo que queremos representar en nuestra base de datos y cómo queremos hacerlo. Este proceso de representación del diseño lógico se conoce como **modelado semántico**, y el resultado es un modelo visual que muestra la estructura lógica de la base de datos que estamos empezando a construir.

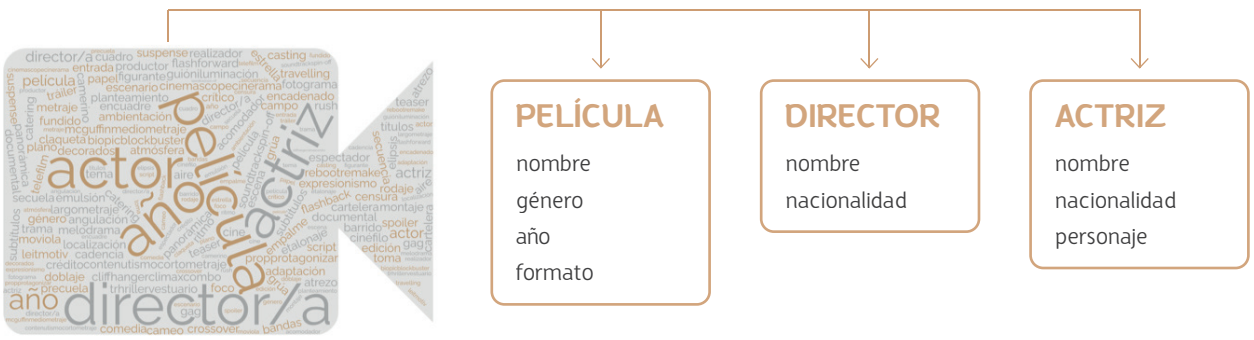
Otra ventaja que tiene el modelado semántico de una base de datos es su independencia respecto a su estructura física. De esa forma, el diseño lógico extraído de nuestros análisis será igualmente válido con independencia del tipo de modelo físico que vayamos a usar (Access, MySQL, MongoDB, etcétera), y tampoco se verá afectado por el hecho de que la base de datos sea centralizada o distribuida. Esa es la principal ventaja de empezar por el modelado semántico de la base de datos: las decisiones sobre su implementación física las podremos tomar más adelante y no se verán afectadas por el diseño lógico.

1.1. Recopilación de datos y definición de entidades

Así pues, el primer paso para empezar a diseñar una base de datos sería recopilar toda la información que queremos guardar y empezar a organizarla. ¿Cómo? Agrupándola en conceptos extrapolables a la vida real, concretamente al entorno o a la situación que queremos representar mediante nuestra base de datos: gestionar una colección de películas, libros, música o videojuegos, la información sobre nuestro centro educativo, las ventas de una empresa o los productos de un almacén. Esos conceptos asociados con elementos de la vida real se conocen con el nombre de **entidades**.

Si, por ejemplo, queremos empezar a diseñar una base de datos para gestionar nuestra colección de películas, el primer paso sería recopilar todos los datos que queremos tener guardados: nombres de actores y actrices, de películas, género al que pertenecen, año en que se estrenaron, etcétera.

Todos estos datos se recogen y se anotan de manera desorganizada, pero a continuación hay que intentar agruparlos en conceptos que sean equivalentes a los elementos de la vida real que vamos a representar. En nuestro caso, películas, directores, actores/actrices y cualquier otra agrupación que se nos ocurra.



Vemos en la figura cómo los datos recopilados empiezan a organizarse en entidades como película, director o actor. En cada entidad se incluyen los datos que guardan relación con la misma: en **PELÍCULA** tenemos el nombre, el género al que pertenece, el año de estreno y formato; en **DIRECTOR**, su nombre y nacionalidad, datos que también tenemos en la entidad **ACTOR**, además del nombre del personaje que representa. Este último dato, como veremos más adelante, deberá desarrollarse algo más, puesto que un actor puede representar múltiples personajes en diferentes películas, o incluso en la misma.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

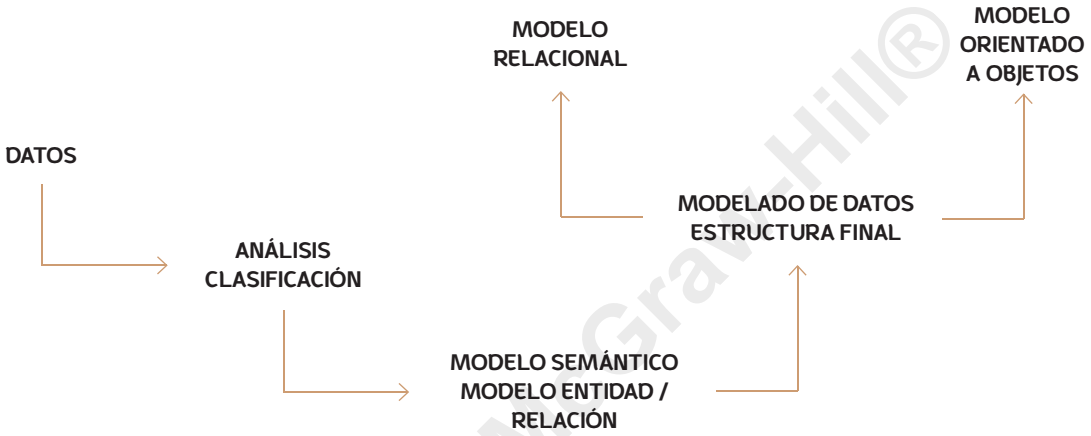
Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

1.2. Empezando con las relaciones

En la unidad anterior vimos que la principal diferencia entre un conjunto de ficheros y una base de datos es que en esta se guardan también las relaciones entre los distintos conceptos que forman parte de ella. En el caso de la figura anterior, de poco nos vale agrupar los datos en entidades como PELÍCULA, DIRECTOR y ACTOR si después no sabemos quién ha dirigido la película, así como qué actores y actrices han intervenido en ella. Es necesario, por tanto, identificar las **relaciones** entre los distintos conceptos que van a aparecer en nuestra base de datos:

- ¿Quién dirige la película?
- ¿Qué actores y actrices la interpretan?

El **modelado semántico**, a diferencia de otros modelos orientados a la estructura final de los datos, no busca describir el formato final de la base de datos, sino la información que representa y cómo se relaciona entre sí. Posteriormente ya se toma la decisión de qué modelo de datos se utilizará para implementar la base de datos: relacional, orientado a objetos, documental o cualquier otro.



Así pues, ya sabemos cómo empezar a diseñar una base de datos: recopilando la información, organizándola en entidades y preguntándonos por las relaciones que se pueden establecer entre dichas entidades. A lo largo de esta unidad y la siguiente estudiaremos una herramienta muy utilizada para representar el diseño lógico: el **modelo entidad/relación**. También descubriremos, en unidades posteriores, cómo el modelo entidad/relación nos sirve de base para empezar a diseñar la estructura final de la base de datos. Una estructura final que, a lo largo de este curso, veremos cómo se puede implementar.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

2. Modelo entidad/relación: entidades, atributos, claves

Un **modelo entidad/relación** básico es un esquema gráfico que se compone de entidades, en las que se clasifican los datos de interés, y de las relaciones que se producen entre dichas entidades. El primer paso, realizado el análisis y la obtención de los datos que queremos incluir en el modelo, es, por tanto, la agrupación de dichos datos en entidades.

2.1. Entidades

Una entidad es, como hemos visto en el apartado anterior, un concepto utilizado en el diseño de bases de datos que agrupa elementos del mundo real. Dichos elementos se caracterizan por compartir una serie de propiedades, lo que determina su agrupación como entidad. Estamos pues ante un concepto abstracto que se usa para representar las propiedades de un tipo específico de elemento, que puede ser un objeto físico (una persona, un ordenador, un automóvil, etcétera) o conceptual (un módulo, un ciclo formativo, una factura, una familia de productos, etcétera).

La clave está en que dichos objetos compartan una serie de características que sirvan para considerar que son del mismo tipo y diferenciarlos de otros objetos de otro tipo. Pongamos por ejemplo un automóvil y una bicicleta. A simple vista sabemos diferenciar uno de otra. ¿Por qué? Porque dos automóviles comparten una serie de características que nos hacen identificarlos como automóviles, y lo mismo ocurre con las bicicletas. A su vez, esas características compartidas hacen que resulte imposible confundir un automóvil con una bicicleta.

Si volvemos al ejemplo del apartado anterior, en el que empezábamos a diseñar una base de datos sobre películas, podemos recordar que habíamos definido tres entidades: película, director y actor.



PELÍCULA
nombre
género
año
formato

DIRECTOR
nombre
nacionalidad

ACTRIZ
nombre
nacionalidad
personaje

En el modelo E/R, una entidad se representa mediante un rectángulo con el nombre de la entidad en su interior.



2.2. Atributos

Los atributos son las propiedades que definen a los elementos que forman parte de cada entidad; es decir, los datos que queremos guardar de cada uno de ellos. Por ejemplo, en el caso de las entidades que acabamos de definir, los atributos de PELÍCULA por el momento son el nombre, el género, el año y el formato; en la entidad DIRECTOR tenemos el nombre y la nacionalidad, mientras que en la entidad ACTOR tenemos también el nombre y la nacionalidad, además de un nombre de personaje.

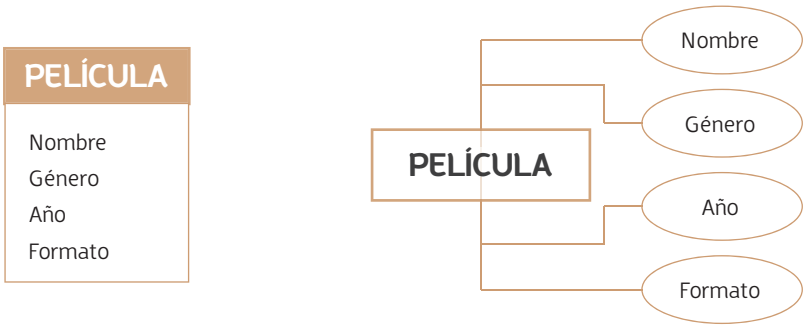
IMPORTANTE

Cuando definimos los atributos de una entidad no estamos haciendo referencia a valores concretos, sino a propiedades generales que comparten todos los elementos de dicha entidad. Así, de momento especificamos que de una película necesitamos guardar el título, pero sin referirnos a ninguna película concreta ni a un título específico.

En el modelo E/R, los atributos pueden incluirse dentro del rectángulo de su entidad correspondiente, o también representar cada atributo mediante una elipse que se enlaza con la entidad a la que pertenece. Así pues, las dos posibilidades que puedes ver en la figura siguiente serían correctas.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.



2.3. Claves

Como hemos visto, cada entidad es una unidad conceptual que incluye una colección de elementos con las mismas propiedades. Esos elementos, denominados "ocurrencias", necesitan ser diferenciados entre sí de alguna manera. Generalmente, para identificar a una ocurrencia de entre el resto no es necesario conocer el valor de todos sus atributos, sino que solo se requiere conocer el valor de uno (o de unos pocos) que nunca se van a poder repetir en la vida real. Ese atributo (o atributos) forman lo que se denomina clave o identificador de la entidad.

EJEMPLO

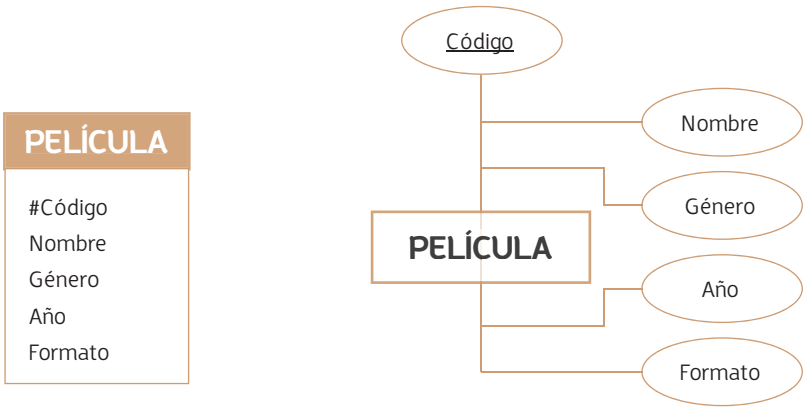
En una entidad, para representar automóviles, la clave podría ser el **atributo número de matrícula**, ya que no se repite de un automóvil a otro.

En una entidad, para representar personas, la clave podría ser el DNI, que tampoco se repite en personas distintas. Dicho de otro modo, para identificar a una persona concreta entre un millón no me hará falta ir comparando con cada una el valor de todos sus atributos (su nombre, sus apellidos, la fecha de nacimiento, la estatura, el color de ojos, etc.). Me bastará conocer el valor del atributo DNI, porque no va a haber otra ocurrencia de persona que tenga el mismo valor.

En ocasiones no hay un atributo o conjunto de ellos que claramente pueda usarse como clave. En esos casos podemos definir un atributo identificador, un código que tome obligatoriamente un valor distinto para cada una de las ocurrencias.

Ejemplos de estos identificadores serían un número de factura o un número de pedido. En nuestro caso, una película no puede identificarse y diferenciarse del resto por alguno de los atributos especificados, ya que puede haber dos películas diferentes con el mismo nombre. En situaciones así es conveniente definir un atributo extra que sirva de clave.

En el modelo E/R, la clave de cada entidad puede representarse mediante una almohadilla delante del atributo o atributos que forman la clave, o bien subrayándolos. Ambas opciones, que aparecen en la figura siguiente, serían de nuevo válidas. En ambos casos hemos añadido un atributo código que será la clave utilizada para la entidad PELÍCULA.



Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

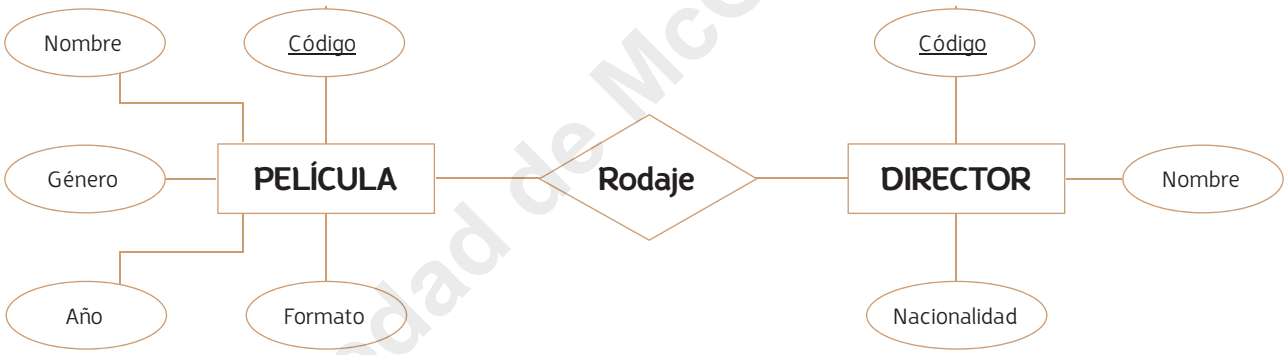
3. Modelo entidad/relación: relaciones, grados

Las relaciones entre entidades es lo que verdaderamente le da sentido a un modelo entidad/relación. La **relación** a la que se refiere el modelo es una asociación entre entidades en la que pueden intervenir un número indeterminado de ellas, siendo las más habituales las que involucran a dos o tres entidades. ¿Qué determina la existencia de una relación entre entidades? Básicamente, que las entidades compartan algún tipo de información o que exista algún tipo de proceso o relación en la vida real en el que dichas entidades se vean involucradas. Es decir, las relaciones del modelo entidad/relación responden siempre a alguna relación equivalente en el mundo real entre miembros de las diferentes entidades que componen la relación.

Recordemos alguno de los ejemplos que hemos visto en apartados anteriores. Por ejemplo, si tenemos la entidad PELÍCULA con su código, nombre, año de rodaje, etcétera, y por otro lado la entidad DIRECTOR con su código de identificación, su nombre y su nacionalidad, si nuestra base de datos debe dar respuesta a preguntas como "¿Quién ha dirigido tal película?", entonces en el modelo E/R que diseñemos tendremos que incluir una interrelación entre DIRECTOR y PELÍCULA. Esa información es la que representaremos en nuestro modelo entidad/relación mediante las relaciones.

3.1. Representación gráfica de las relaciones en el modelo E/R

Hay un par de formas de representar las relaciones entre entidades en un modelo entidad/relación. Las entidades relacionadas siempre se unen mediante una línea. Además, en ocasiones la relación se representa añadiendo una figura romboidal con el nombre de la relación en su interior. El nombre de la relación puede ser cualquiera, pero es importante que especifique claramente qué tipo de relación hay entre las entidades de la forma más breve posible. Esta es la manera tradicional de representar las relaciones en el modelo entidad/relación. En la figura podemos ver el ejemplo de PELÍCULA y DIRECTOR:



De esta forma representamos que las entidades PELÍCULAS y DIRECTOR no están aisladas en nuestra base de datos, sino que hay una relación entre ellas. Una relación que hemos denominado rodaje porque ese es el proceso en el que se ambas entidades acaban encontrándose: una película está rodada por un director (puede que por más, en algunos casos), y un director rueda una o varias películas. De esa forma especificamos que esa relación existe, y sentamos las bases para, más adelante, acceder a una de las entidades desde la otra para saber quién ha rodado una película o qué películas ha rodado un director.

Aunque en el texto usaremos esta notación, en el material complementario puedes encontrar información sobre otra forma de representar las relaciones. Hablaremos de ello más adelante cuando estudiemos las cardinalidades y cómo representarlas en el modelo E/R.

3.2. Grado de una relación

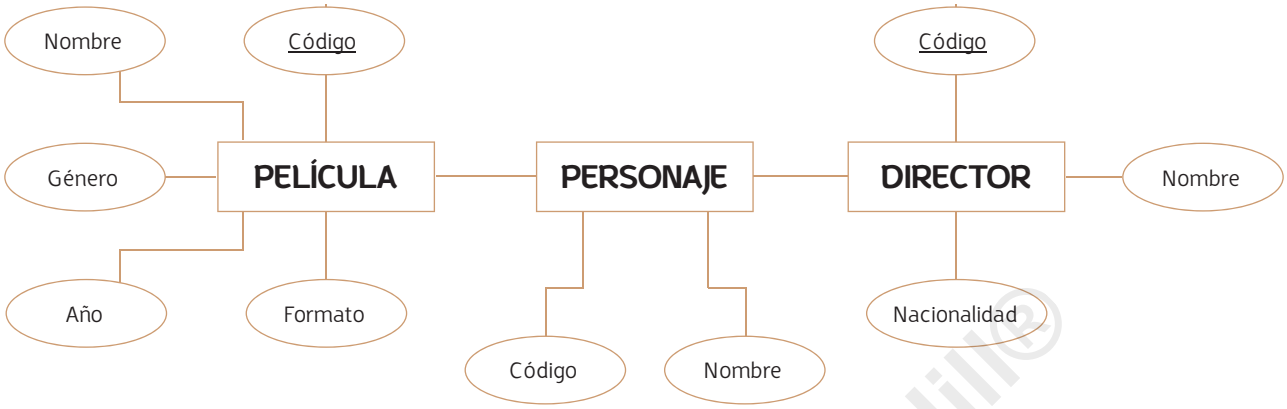
El número de entidades que se conectan mediante una relación determina su **grado**. Así, si una relación conecta tres entidades será de **grado 3 o ternaria**, mientras que una relación que conecte dos entidades será de **grado 2 o binaria**. En el caso que una relación conecte una entidad consigo misma, hablamos de una relación de **grado 1 o unaria**, aunque en realidad sería una relación binaria donde las dos entidades conectadas son la misma.

La relación entre PELÍCULA y DIRECTOR que hemos visto anteriormente sería una relación **binaria o de grado 2**, ya que conecta dos entidades.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Imaginemos ahora que queremos incluir en nuestro modelo E/R de películas una entidad PERSONAJE, en la que guardaremos información sobre los personajes que aparecen en cada película. Además, queremos saber qué actor o actriz ha interpretado a cada personaje en cada película. Por ejemplo, el personaje de Sherlock Holmes ha sido interpretado por diferentes actores a lo largo de la historia. Lo mismo ocurre con Batman o Superman. ¿Cómo saber quién ha interpretado al personaje en cada película? Con relaciones binarias entre PELÍCULA y ACTOR, o entre ACTOR y PERSONAJE, siempre nos faltaría un dato. En estos casos hay que recurrir a una **relación ternaria:**



Mediante esta relación estamos especificando que un actor o actriz interpreta un personaje en una película. Esa información es más completa que decir "un personaje aparece en una película" (sería el caso de una relación binaria entre PELÍCULA y PERSONAJE), "un actor/actriz interpreta un personaje" (en el caso de una relación binaria entre ACTOR y PERSONAJE) o "un actor/actriz interviene en una película" (en el caso de una relación binaria entre PELÍCULA y ACTOR). En los tres casos anteriores nos faltaría un dato, mientras que la relación ternaria nos da toda la información que necesitamos.

IMPORTANTE

Las relaciones se dan siempre entre entidades. Debemos tener cuidado para no relacionar una entidad con algo que en realidad sería una propiedad y no otra entidad. Por ejemplo, *el director* (entidad) *dirige una película* (entidad) es una relación. *La película fue rodada en un año* no sería una relación, simplemente el año en que fue rodada sería un atributo o propiedad de la entidad película.

Las relaciones unarias, es decir, que conectan una entidad con ella misma, se dan sobre todo cuando los miembros de una misma entidad se relacionan entre sí de una manera más o menos jerárquica. Por ejemplo, si tenemos una entidad EMPLEADO y queremos representar que hay algunos empleados que tienen a su cargo (son sus jefes a otros empleados, lo más adecuado es utilizar la propia entidad EMPLEADO, ya que los jefes no dejan de ser también empleados. Puedes ver la relación unaria para representar que unos empleados son jefes de otros en la siguiente figura.



No hay un procedimiento único para determinar las relaciones entre entidades, pero con la práctica iremos aprendiendo a encontrarlas y definirlas de una manera más rápida y eficiente. De momento podemos tener en cuenta el siguiente consejo: en general una relación viene determinada por un verbo que conecta dos conceptos que en el modelo hemos definido como entidades. Ejemplos: el alumnado ***se matricula*** de ciertos módulos de un ciclo, una profesora ***imparte*** un módulo o un grupo ***ocupa*** un aula.

IMPORTANTE

Cuidado con el verbo *tiene*: en general indica un atributo más que una relación. Un módulo *tiene* un nombre, un alumno *tiene* un número de teléfono, un actor *tiene* una nacionalidad, etcétera. En este caso "nombre", "número de teléfono" o "nacionalidad" no son entidades, no representan elementos complejos del mundo real sino propiedades de otros elementos, por lo que serían atributos de sus correspondientes entidades.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

4. Modelo entidad·/relación. Cardinalidades. Restricciones

La **cardinalidad** o cardinalidades de una relación representan la cantidad de ocurrencias de una entidad que se pueden (o deben) relacionar con una ocurrencia de otra entidad. En el modelo entidad/relación se usan para describir con más detalle la relación que se establece entre varias entidades. Para ello, en cada relación incluiremos la cardinalidad máxima y la mínima. A continuación veremos para qué sirve cada una.

4.1. Cardinalidad máxima

La **cardinalidad máxima** define el tipo de relación al especificar si una ocurrencia de una entidad puede relacionarse solo con una única ocurrencia de otra entidad o, por el contrario, puede relacionarse con más de una. En una relación binaria, la cardinalidad máxima va a encajar en uno de estos tres patrones:

RELACIÓN	DESCRIPCIÓN
1:1 (Uno a uno)	Solo se permite que cada una de las ocurrencias de cada entidad se interrelacione, como máximo, con una ocurrencia de la otra entidad y viceversa.
1:M (Uno a muchos)	De las dos entidades, solo una puede tener ocurrencias que se interrelacionen con más de una ocurrencia de la otra.
M:M (Muchos a muchos)	No hay restricción para la cantidad de ocurrencias con que se interrelaciona cada ocurrencia de cualquier entidad.

Por su simplicidad hemos escogido las relaciones binarias para explicar el significado de la cardinalidad, pero el mismo concepto se aplica a cualquier tipo de relación, sea del grado que sea. La forma de calcularla, eso sí, varía un tanto en el caso de las relaciones ternarias (en general, en cualquier relación de grado mayor que 2). En el material adicional encontrarás más información sobre este tema.

A continuación vamos a ver un ejemplo de cada cardinalidad máxima:

RELACIÓN	EJEMPLO
1:1	Un profesor/profesora solo puede tutorizar **un** grupo, y un grupo solo puede tener **un** tutor/tutora. La cardinalidad máxima en ambos casos es 1.
1:M	Un profesor/profesora puede impartir **varios** módulos, pero de cada módulo se encarga solo **un** profesor/profesora.
M:M	Un alumno/alumna puede matricularse de **varios** módulos, y en cada módulo hay matriculados **varios** alumnos o alumnas.

4.2. Cardinalidad mínima

En toda relación binaria existen dos cardinalidades mínimas, una por cada sentido, y para cada una de ellas hemos de establecer el valor 0 o 1. Por ejemplo, en una interrelación llamada MATRICULAR entre la entidad ESTUDIANTE y la entidad MÓDULO, existirá una cardinalidad mínima para el sentido de ESTUDIANTE --> MÓDULO (leído de estudiante a módulo) y otra para el sentido ESTUDIANTE <-- MÓDULO (leído de módulo a estudiante).

Para el sentido ESTUDIANTE --> MÓDULO, decidiríamos establecer la cardinalidad mínima en 1 si quisiéramos impedir que hubiera estudiantes que no estuviesen matriculados en ningún módulo. Sin embargo, estableceríamos la cardinalidad mínima en 0 si quisiéramos que la base de datos pudiera contener estudiantes que no estuvieran matriculados en ningún módulo (por ejemplo, porque quisiéramos guardarlos después de haber finalizado sus estudios).

Y para el sentido ESTUDIANTE <-- MÓDULO, decidiríamos establecer la cardinalidad mínima en 1 si quisiéramos impedir que hubiera módulos que no tuviesen matriculado a nadie. O estableceríamos la cardinalidad mínima en 0 si quisiéramos que la base de datos pudiera contener módulos que no tuvieran a nadie matriculado.

4.3. Representación gráfica de las cardinalidades

Hay dos formas de representar las cardinalidades en el modelo E/R. La primera consiste en ponerlas por escrito en cada participación de una entidad en una relación. Esta es tal vez la forma quizás más sencilla, y viene

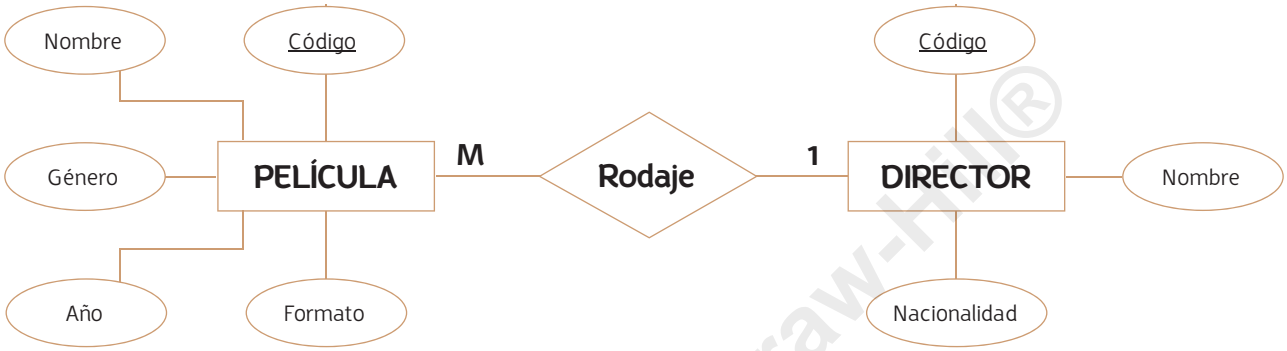
Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

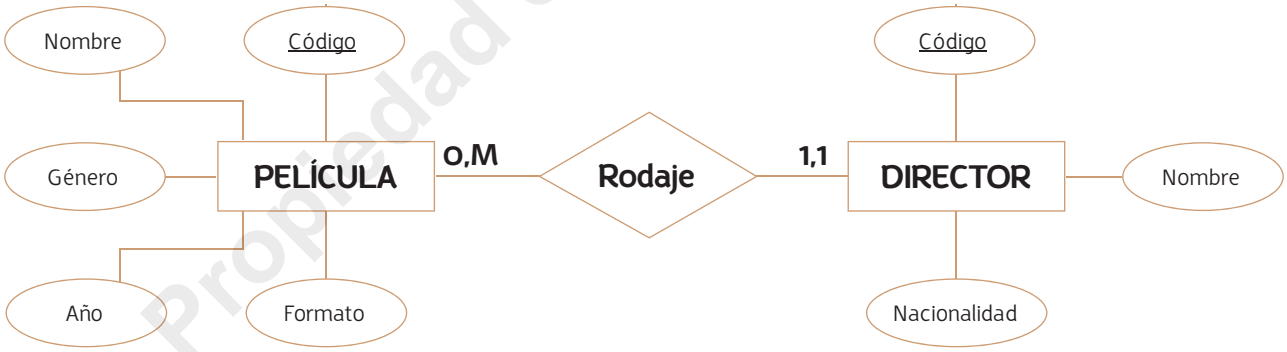
heredada de las primeras propuestas de Peter P. Chen. Veamos un ejemplo: volvamos a la relación que hemos visto en el apartado anterior entre las entidades PELÍCULA y DIRECTOR. Hasta ahora no nos hemos preocupado de la cardinalidades, pero ahora que ya conocemos su utilidad vamos a usarlas. En una relación binaria como esta, las preguntas que debemos hacernos son:

- ¿Es obligatorio que una película haya sido dirigida por alguien? ¿Es obligatorio que un director haya dirigido al menos una película para aparecer en nuestra base de datos? Las respuestas definirán la cardinalidad mínima de la relación para ambas entidades.
- ¿Una película puede haber sido dirigida por varios directores? ¿Un director puede haber dirigido más de una película? Las respuestas a estas cuestiones definirán la cardinalidad máxima.

Supongamos que una película solo puede haber sido dirigida por un director, y que un director puede haber dirigido más de una película. En este caso las cardinalidades máximas son 1 y M, por lo que la relación es 1:M o UNO a MUCHOS. La representación gráfica con el formato que estamos viendo sería la siguiente:



En la figura aparece solo la cardinalidad máxima, que indica que una película puede tener como mucho un director, y un director puede haber dirigido muchas películas. Nos falta añadir la cardinalidad mínima, que expresa la obligatoriedad (o no) de participar en la relación. En nuestro caso, una película necesariamente debe tener un director, con lo que la cardinalidad mínima en el lado de la entidad DIRECTOR será 1, y vamos a suponer que un director puede estar incluido en la base de datos aunque no haya dirigido todavía ninguna película, con lo que la cardinalidad mínima en el lado de la entidad PELÍCULA sería 0. Con este formato, la relación quedaría como puede verse en la figura siguiente:



La relación se leería como "*una película debe tener obligatoriamente un director, y como máximo habrá sido dirigida por 1 director, mientras que un director puede no haber dirigido ninguna película o puede haber dirigido muchas*".

La otra forma de representar las cardinalidades de las relaciones es más reciente y consiste en unir directamente las entidades mediante una línea (es decir, sin el rombo), de forma que los extremos de la línea cambian en función de las cardinalidades mínima y máxima. Es lo que se conoce como notación "pata de gallo" por el motivo que fácilmente adivinaremos cuando veamos cómo se representan gráficamente. En este caso, el 0 se representa mediante un círculo, el 1 mediante una línea vertical, y la M justamente mediante una especie de "pata de gallo". Veamos cómo quedarían las cuatro combinaciones de cardinalidades mínimas y máximas con esta notación:



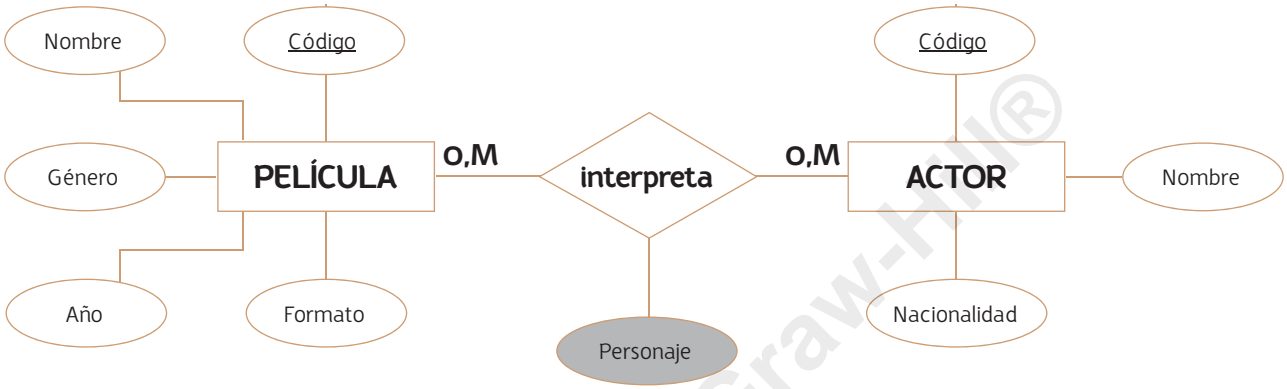
Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

4.4. Atributos en las relaciones M:M

Las relaciones M:M (muchos a muchos) pueden tener atributos. ¿En qué casos? En aquellos en los que una propiedad no puede incluirse en ninguna de las entidades relacionadas, puesto que dicha propiedad solo toma valor cuando se interrelacionan ocurrencias de ambas entidades.

Vamos a aclararlo con un ejemplo, otra vez de nuestra base de datos de películas. Supongamos que tenemos una relación M:M entre PELÍCULA y ACTOR, puesto que en una película pueden aparecer varios actores, y cada actor/actriz puede haber rodado muchas películas. Ahora imaginemos que queremos incluir como atributo el nombre del personaje interpretado (sin crear una entidad PERSONAJE, como hemos visto en un ejemplo anterior). Puesto que en una película aparecen muchos personajes, y un actor/actriz interpreta muchos personajes a lo largo de su carrera, el personaje no puede ser un atributo ni de la entidad PELÍCULA (nos faltaría saber quién lo ha interpretado) ni de la entidad ACTOR (nos faltaría saber en qué película lo ha interpretado). En este caso, el atributo se incluye en la relación de la siguiente manera:



La relación de la figura se lee de la siguiente manera: "*una película puede no tener ningún actor (en caso de ser una película de animación, por ejemplo) o puede tener muchos actores; por su parte, un actor puede aparecer en muchas películas pero podría no haber aparecido todavía en ninguna.* ***Para cada actor, en cada película****, guardamos el nombre del personaje*".

Hay que tener en cuenta que en el formato "pata de gallo" no hay un símbolo para la relación (el rombo de la notación tradicional). Por tanto, en el caso de que una relación M:M tenga atributos dicha relación se convierte en una entidad, denominada entidad de intersección, que se relaciona con las dos entidades originales con relaciones 1:M. En el material descargable encontrarás los ejemplos de este apartado representados en la notación "pata de gallo".

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Unidad 3.

El modelo entidad/relación extendido

Propiedad de McGraw-Hill®

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

1. Entidades fuertes y débiles

En una relación entre dos entidades puede darse el caso de que una de ellas **dependa** de la otra para existir, para que sus miembros puedan ser identificados y diferenciados unos de otros, o en ocasiones para ambas cosas a la vez. En ese caso hablamos de una **entidad débil** respecto a la otra entidad con la que se relaciona, que representaría el rol de **entidad fuerte**.

> La calidad de "fuerte" o "débil" de una entidad siempre viene dada dentro de una relación concreta. Es decir, una entidad no siempre es fuerte o débil: puede ser débil en una relación y no serlo en otra.

En una relación entre dos entidades, una de ellas puede ejercer el papel de entidad débil respecto a la otra por dos motivos. Cada uno de ellos responde a un tipo distinto de restricción, aunque en la práctica pueden darse ambos a la vez.

Restricción	Significado	Ejemplo
Existencia	La entidad débil depende de la entidad fuerte con la que se relaciona para poder existir. Si desaparece el elemento de la entidad fuerte con el que se relaciona el elemento de la entidad débil, este también desaparece. La restricción de existencia está relacionada con la cardinalidad mínima igual a 1: todas las ocurrencias de la entidad con restricción de existencia están obligadas a relacionarse con una ocurrencia (como mínimo) de la entidad fuerte.	Entidad **factura** respecto a la entidad **cliente**. La factura no puede existir con independencia del cliente: se crea cuando se da la relación, y si desaparece el cliente lo hace también la factura. Sin embargo, la factura puede ser identificada sin necesidad de especificar a qué cliente pertenece.
Identidad	Hasta ahora hemos asumido que una ocurrencia podía ser identificada de entre las demás gracias a que los valores de todos o algunos de sus atributos no se podían repetir. Sin embargo, la entidad débil depende la entidad fuerte para poder identificar a sus ocurrencias, ya que el atributo o atributos que formarían la clave primaria se repiten en función del elemento de la entidad fuerte con el que se relaciona el de la entidad débil.	Entidad **habitación** respecto a la entidad **hotel**. El número de habitación, que sería la clave primaria, se repite en los diferentes hoteles, de forma que para identificar una habitación y diferenciarla de todas las demás habitaciones de otros hoteles necesitamos especificar a qué hotel nos estamos refiriendo.

En casi todas las ocasiones, la restricción de identidad está acompañada también por la de existencia. Al revés, sin embargo, no siempre ocurre. En la tabla anterior, la factura tiene un número que la identifica sin necesidad de añadir a qué cliente pertenece, por tanto la factura es débil respecto a la entidad cliente solamente por restricción de **existencia**. La habitación, además de no poder existir con independencia del hotel y desaparecer si desaparece este, también necesita del hotel para poder ser identificada. En ese caso se dan ambas restricciones, la de existencia y la de **identidad**, de manera conjunta.

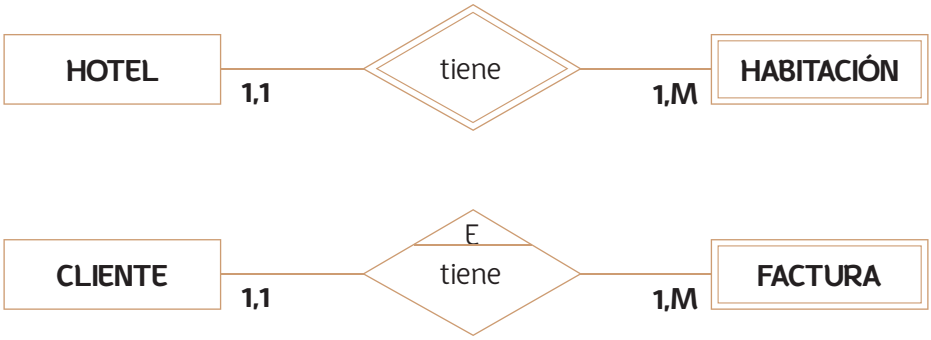
Representación gráfica de la relación entre una entidad fuerte y otra débil

Una entidad débil se representa habitualmente mediante un rectángulo cuyo perímetro se dibuja con doble línea. El tipo de restricción (identidad o existencia) suele indicarse mediante las letras I o E, bien en el rombo que representa la relación o en otro lugar de la misma. Puesto que una entidad puede ser débil en una relación y no serlo en otra, debemos encontrar alguna forma de especificar, al definir una entidad como débil, a cuál de todas sus relaciones nos estamos refiriendo. Una buena forma es incluir las letras I o E en la relación, aunque en otras ocasiones el propio rombo de la relación se dibuja también con una línea doble. Algunos autores, sin embargo, utilizan el rombo con doble línea solamente cuando la restricción es de identidad. En realidad, no es necesario indicar gráficamente una restricción de existencia, puesto que equivale a indicar que la cardinalidad mínima es mayor de cero.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Así, en la siguiente factura la relación entre HOTEL y HABITACIÓN tendría una restricción de identidad por la que la habitación necesita del hotel para ser identificada, mientras que la relación entre CLIENTE y FACTURA tendría una restricción solo de existencia pero no de identidad, puesto que la factura puede identificarse por su número sin necesidad de recurrir al cliente. En ambos casos, de cualquier manera, existe una entidad débil (HABITACIÓN en el primer caso, FACTURA en el segundo).



Para diferenciar entre ambas restricciones puedes hacerte una sencilla pregunta: ¿puede la clave primaria natural de la entidad débil identificar a cada miembro de dicha entidad de manera única e inequívoca? En el caso de la factura sí, puesto que cada factura tiene un número que la identifica sin tener en cuenta a qué cliente pertenece. En ese caso, la restricción es solamente de existencia. En el caso de la habitación, sin embargo, el número de una habitación de un hotel se repite en el resto de hoteles, con lo que resulta insuficiente para identificar a una única habitación dentro del modelo representado siempre que incluya varios hoteles. Si hablamos de la habitación 001 es imposible saber a qué habitación nos estamos refiriendo si no mencionamos en qué hotel se encuentra.

Otro ejemplo podría ser el de los episodios de una serie: todas ellas tienen un episodio 1; por tanto, para que la clave "1" identifique de manera única a un episodio concreto debemos mencionar también a qué serie pertenece. En este caso, como en el de los hoteles y las habitaciones, la restricción es de identidad.

La clave primaria de una entidad debe identificar siempre de manera única e inequívoca a cada miembro de dicha entidad y diferenciarlo del resto. Por ello, como veremos cuando hablemos del modelo relacional, las entidades débiles con restricción de identidad incorporan la clave primera de la entidad fuerte a la suya propia.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

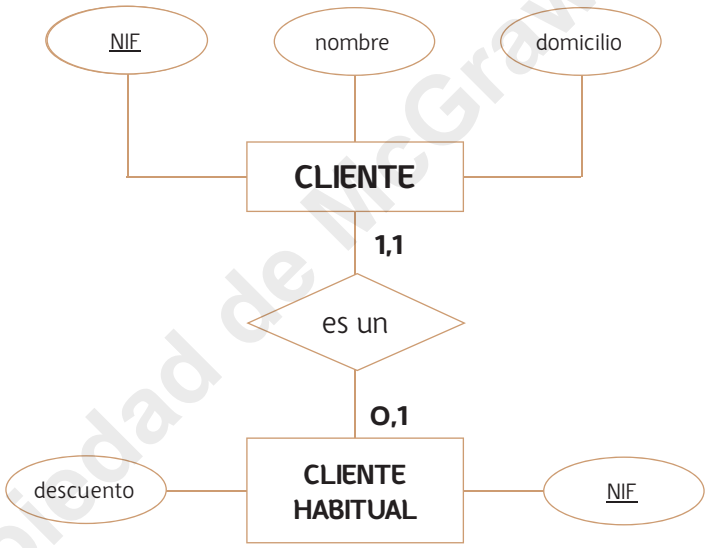
Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

2. Subentidades

El modelo entidad/relación original no contemplaba la posibilidad de representar de una forma diferenciada las relaciones en las que una entidad es un tipo especial de otra. Posteriormente, con las distintas ampliaciones que han dado lugar al modelo entidad/relación extendido, se han añadido diversas formas de representar este tipo de relaciones. En este apartado y en el siguiente iremos viendo cada una de ellas. Veamos primero el caso en la que una entidad es un tipo especial de otra. Sería, como hemos comentado en la introducción al apartado, el tipo de relación llamada **IS-A** en inglés, **ES-UN** en castellano.

Supongamos que tenemos una entidad **CLIENTE** con todos los datos personales, cuenta bancaria, forma de pago, etc., de nuestros clientes. Imaginemos que necesitamos identificar a un cierto tipo de cliente, digamos habitual, VIP, etcétera, que tiene algunos atributos especiales que no tienen el resto: un descuento, por ejemplo. En realidad no hablamos de dos entidades diferentes, sino de una entidad en la que ciertos miembros tienen algún atributo adicional. El hecho de que en una entidad haya atributos que no compartan todos los miembros de dicha entidad es algo que en el modelo entidad/relación, como en el modelo relacional que veremos más adelante, no se contempla.

Estos casos se resuelven, generalmente, creando una relación 1:1 entre la entidad principal (CLIENTE en nuestro caso) y una subentidad que en este ejemplo podríamos denominar CLIENTE HABITUAL. La clave primaria de ambas entidades sería la misma, y los atributos propios de la subentidad serían aquellos que no tienen todos los miembros de la entidad principal. Nuestro ejemplo de **CLIENTE** y **CLIENTE HABITUAL** se podría ver representando así:



El significado del modelo representado en la figura es el siguiente: tenemos una entidad **CLIENTE** en la que están todos los clientes y donde guardamos la información común a todos ellos. Por otro lado, tenemos una entidad **CLIENTE HABITUAL** en la que están solo ciertos clientes y donde se guardan los atributos específicos de los clientes habituales, en este caso el descuento. La clave primaria en ambas entidades es la misma, el NIF, para que desde la ficha de un cliente en una de las dos entidades podamos acceder fácilmente a su ficha equivalente en la otra. La cardinalidad mínima 0 en la relación, de parte de **CLIENTE HABITUAL**, indica que no todos los clientes tienen por qué aparecer en la subentidad. Sin embargo, todos los clientes habituales aparecen también en la entidad **CLIENTE** (cardinalidad mínima 1).

Como vemos, los atributos comunes a todos los clientes no se repiten en la subentidad **CLIENTE HABITUAL**. Si se repitieran, estaríamos contraviniendo el principio de mínima redundancia. La excepción es el NIF, repetición necesaria para poder relacionar la entidad principal con su subentidad. El resto de atributos comunes (nombre, domicilio, etcétera) aparecen solo en la entidad principal y la subentidad los **hereda**.

> La herencia es un concepto de la programación orientada a objetos que no implica duplicidad. Los atributos de la entidad principal no se replican en la subentidad, pero esta puede acceder a ellos a través de la clave primaria compartida, el NIF, que permite acceder desde la ficha de un cliente habitual a su ficha correspondiente en la entidad principal.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

En la práctica, el concepto de herencia de atributos se aplica así: si estamos en la ficha del CLIENTE HABITUAL con NIF 11222333X y queremos saber su nombre; usando el NIF accederemos a su ficha en la entidad CLIENTE y allí encontraremos no solo el nombre sino también el resto de datos personales. Por supuesto, la comunicación funciona en los dos sentidos: desde la ficha de un cliente podemos acceder a su descuento si dispone de ficha como cliente habitual.

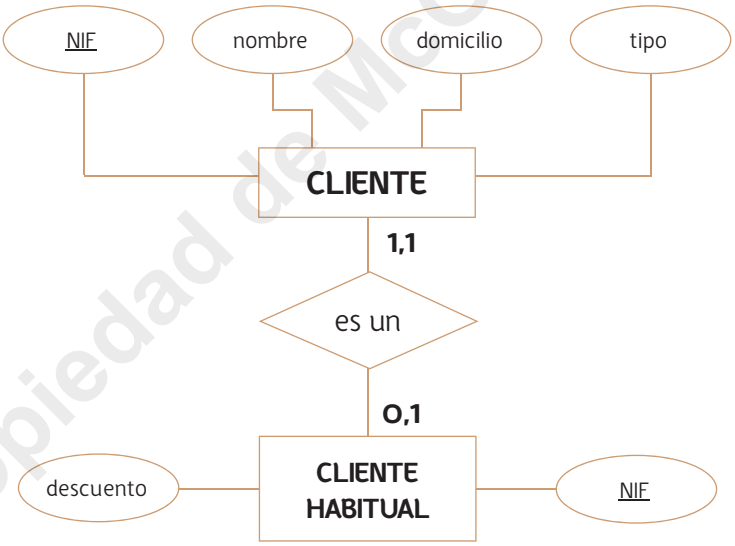
CLIENTE HABITUAL	
NIF	**DESCUENTO**
11222333X	5%
...	...

CLIENTE		
NIF	**NOMBRE**	**DOMICILIO**
11222333X	José Luis	C/ Mayor, 10
...	...	...

IMPORTANTE

Los conceptos de propagación de claves y claves ajenas, necesarios para plasmar en la base de datos física las relaciones que aparecen en el diseño lógico, los veremos más a fondo cuando estudiemos el modelo relacional.

Para evitar la búsqueda en la subentidad de fichas no existentes, en ocasiones añadimos un atributo discriminatorio en la entidad principal que diferencie a quienes tienen ficha en la subentidad de quienes no la tienen. En nuestro caso, añadiendo un atributo **`Tipo`** en la entidad principal que distinga entre los clientes normales y habituales, podemos ahorrarnos un acceso a la subentidad si ya sabemos de antemano que el cliente no es habitual.

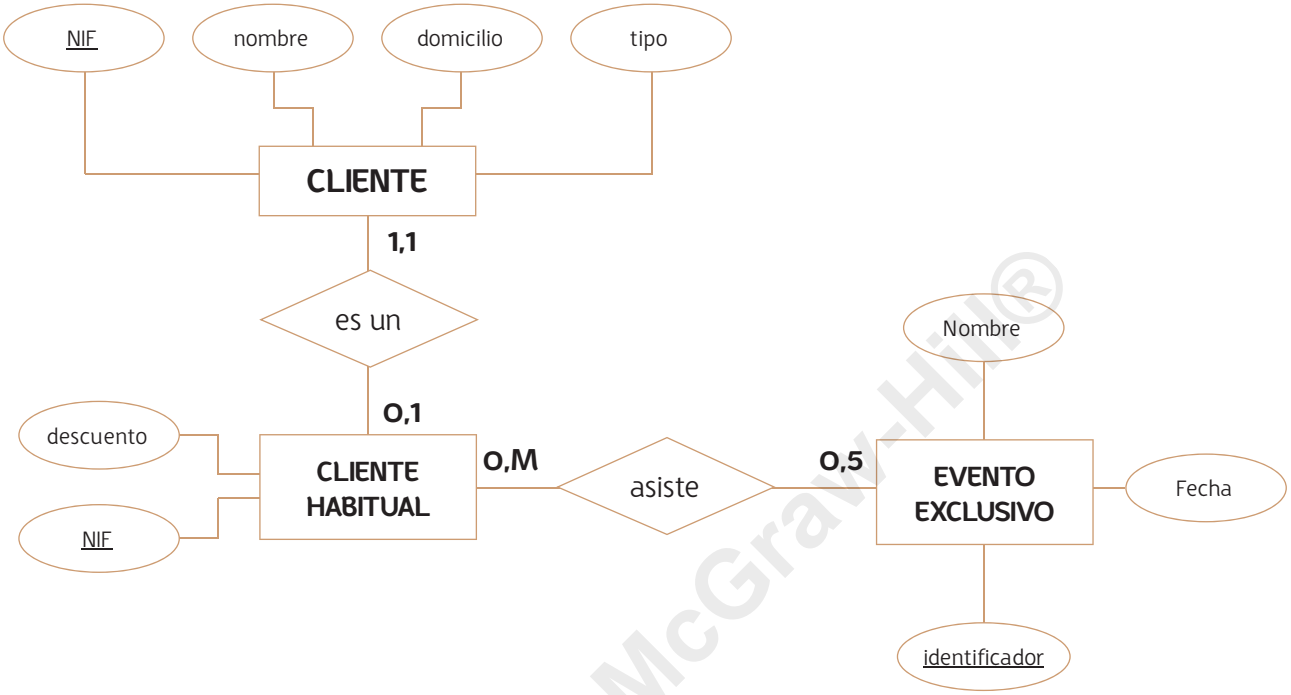


Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

En el ejemplo propuesto podríamos pensar que no hay necesidad de crear una subentidad solo para añadir un atributo nuevo que, en caso de pertenecer a la entidad CLIENTE y dejarse vacío (para los clientes no habituales) o rellenarse (para los clientes habituales) ya indicaría de qué tipo es cada cliente. Estaría más justificada la inclusión de la subentidad en los casos en que solo las ocurrencias de dicha subentidad participan en determinadas relaciones. Supongamos que los clientes habituales, además de tener un descuento, pueden disponer de hasta 5 entradas gratis a algún tipo de evento exclusivo. En este caso sí que sería necesario crear la subentidad para que quedara claro qué clientes acceden a dicha promoción y cuáles no. Nuestra relación, en este caso, quedaría como sigue:



Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

3. Especialización y generalización

Cuando una entidad principal da lugar a la aparición de diversas subentidades se produce lo que se denomina una **partición.** Sin embargo, lo que en el modelo aparece como una partición puede haberse originado como consecuencia de la fusión de dos entidades en una entidad superior. Así pues, la partición de una entidad en varias subentidades se puede producir por dos motivos:

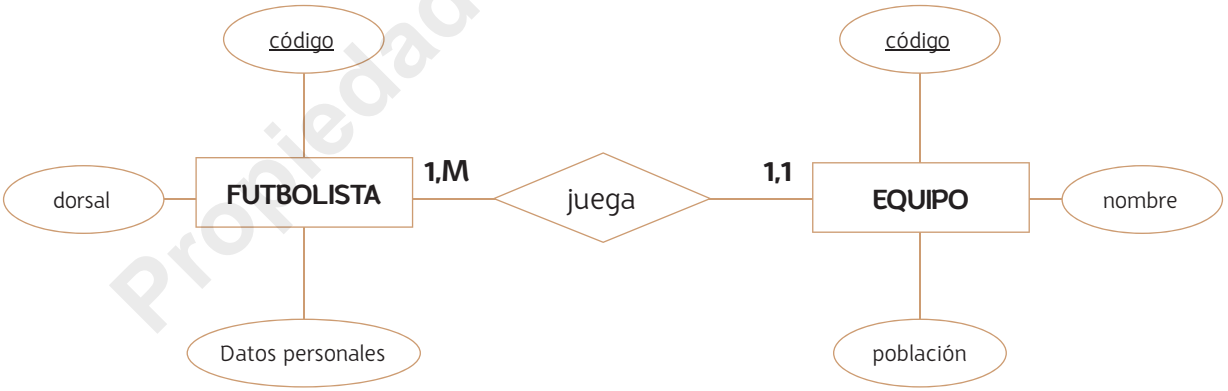
- Existen varios subconjuntos de miembros de la entidad principal con atributos específicos, relaciones específicas, o ambas cosas a la vez. En este caso es la entidad principal la que origina la división en subentidades
- Existen dos o más entidades en el modelo entidad/relación con ciertos atributos comunes, por lo que se decide crear una entidad principal que contenga dichos atributos comunes y dejar en las entidades originales aquellos atributos específicos de cada una de ellas. En este caso son las subentidades las que dan origen a la entidad principal.

En el primer caso hablamos de **especialización**, mientras que en el segundo caso se trataría de una **generalización.** La diferencia entre ambas la define, básicamente, el hecho de haber escogido la metodología de arriba hacia abajo (*top-down*, la entidad principal se divide, estamos ante una **especialización**), o de abajo hacia arriba (*down-up*, varias entidades se agrupan en una superentidad, hablamos entonces de **generalización**).

En ambos casos, por cada miembro de una de las subentidades existe siempre una ficha para ese mismo miembro en la entidad principal. Sin embargo, no siempre ocurre así en el sentido contrario: puede ocurrir que un miembro de la entidad principal no aparezca en ninguna de las subentidades. Hablamos entonces de una **partición parcial**, mientras que cuando cada miembro de la entidad principal tiene necesariamente que pertenecer al menos a una de las subentidades estamos ante una **partición completa.**

3.1. Especialización

Veamos un ejemplo de especialización. Supongamos que estamos modelando una base de datos para guardar información de personas que participan en un torneo de fútbol. Para cada futbolista tendríamos, por ejemplo, sus datos personales, su dorsal habitual y el año de nacimiento. También queremos saber en qué equipo juega, y para ello creamos otra entidad con la información de cada equipo.

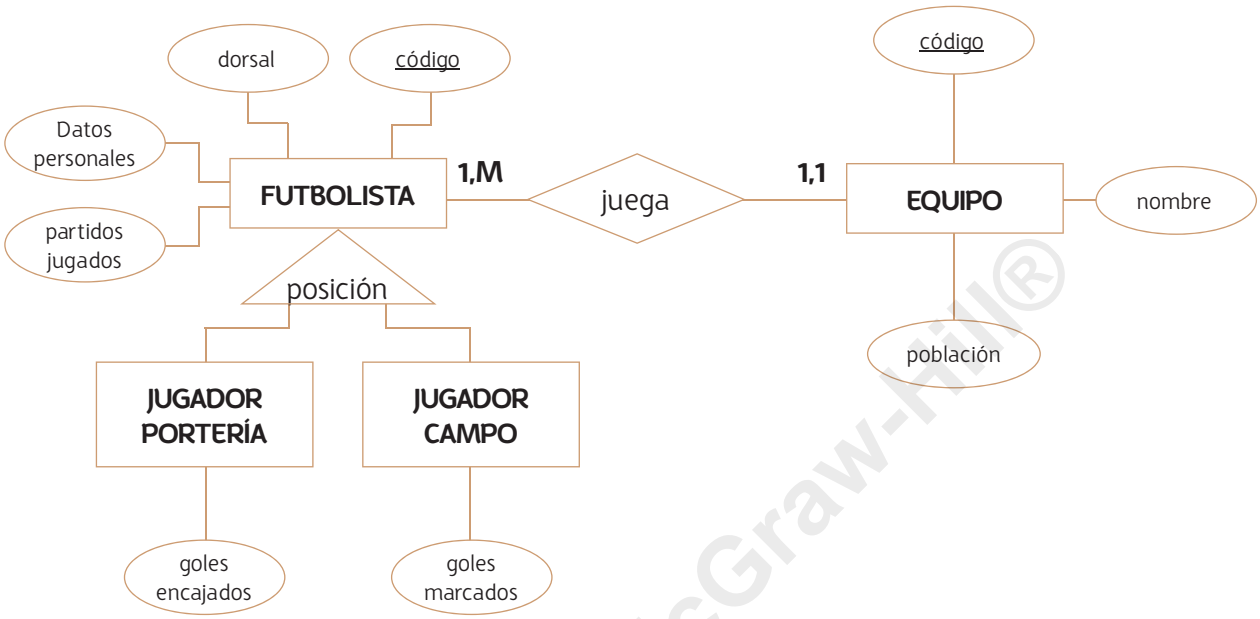


Propiedad de McGraw-Hill

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

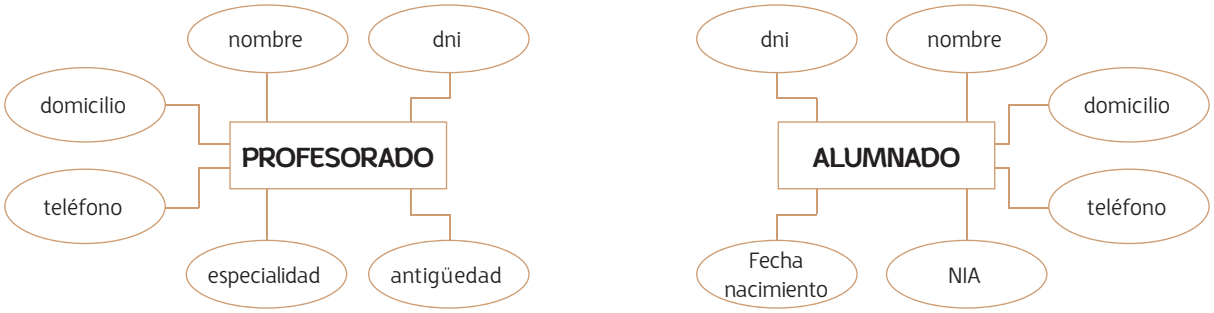
Ahora imaginemos que queremos añadir atributos como partidos jugados, goles encajados y goles marcados. Eso sí, con una salvedad: el atributo **`Goles encajados`** sería solo para futbolistas que ocupen la portería, y los goles marcados solo para quienes jueguen en otras posiciones (para facilitar el ejemplo no tenemos en cuenta de momento la posibilidad de que un portero marque goles). Habrá entonces dos subentidades que podríamos denominar **JUGADOR_PORTERÍA** y **JUGADOR_CAMPO**. Para diferenciar entre ellas, cada futbolista tendrá un atributo que indique la posición en la que juega habitualmente (P para la portería y C para otras posiciones). Con estos cambios, y la especialización de la entidad **FUTBOLISTA** en función de su posición habitual, la representación quedaría así:



En general, las particiones de tipo especialización suelen producirse de forma que existe un atributo que diferencia a unos miembros de la entidad (quienes ocupan la portería) de otros (quienes ocupan otra posición). Siendo así, no sería posible que un futbolista ocupase (habitualmente, habría que ver cómo representar lo que ocurre en un partido concreto) a la vez la portería y otra posición distinta. Cuando un miembro de una subentidad no puede aparecer en otra subentidad de la misma entidad principal hablamos de particiones **disjuntas**. En el caso de que el mismo miembro de una entidad pudiese aparecer en varias subentidades hablaríamos de particiones **superpuestas**.

3.2. Generalización

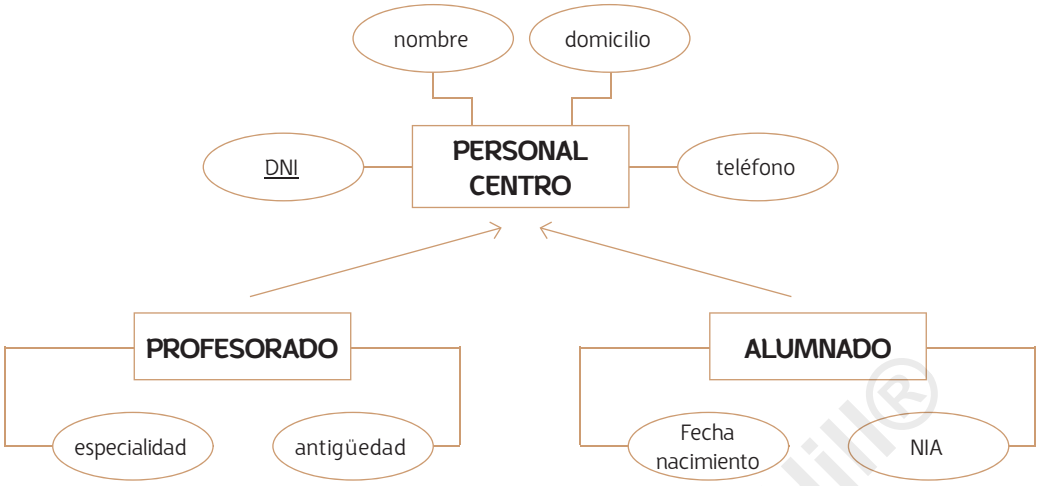
La generalización se da cuando disponemos de varias entidades con atributos comunes y decidimos crear una superentidad donde guardarlos, dejando en las entidades (ahora convertidas en subentidades) solo sus atributos específicos. Por ejemplo, supongamos que queremos guardar información del personal de nuestro centro, tanto del profesorado como del alumnado. Pongamos que obtenemos el siguiente modelo:



Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Como podemos ver, ambas entidades comparten atributos como el DNI, el nombre, el domicilio o el teléfono. En este caso es posible crear una superentidad, una **generalización**, que podríamos llamar PERSONAL CENTRO y que englobaría los atributos comunes a PROFESORADO y ALUMNADO. Cada una de estas entidades, que en el nuevo esquema pasarían a ser subentidades, conservarían los atributos específicos. El resultado sería el siguiente:



Como podemos apreciar, la forma de representar una generalización es diferente a la que usábamos para representar una especialización. De entrada, vemos que no existe un atributo en la entidad principal que diferencia entre miembros de PROFESORADO y miembros de ALUMNADO. Esto permite que la partición pueda ser **parcial** y **superpuesta**. Parcial porque puede haber personal en el centro (limpieza, conserjería, cantina, etcétera) que no pertenezcan a ninguna de las dos subentidades; superpuesta, porque se abre también la puerta a que un miembro de la subentidad PROFESORADO pueda pertenecer también a la subentidad ALUMNADO.

La dirección de las flechas, de abajo arriba, indica que la entidad principal ha surgido como consecuencia de una generalización: es decir, de abajo arriba y no de arriba abajo como en la especialización.

Propiedad de McGraw-Hill®

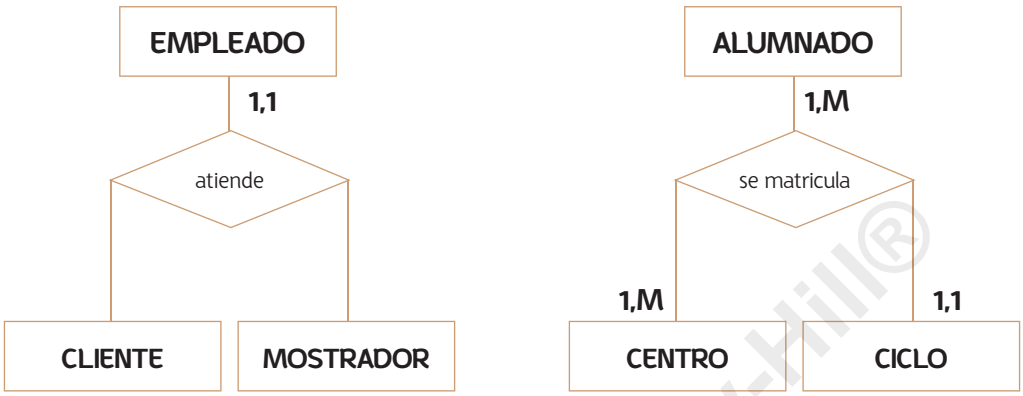
Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

4. Agregación

Una agregación es un proceso por el cual una relación, generalmente entre dos entidades, se convierte en una entidad virtual que, a su vez, podrá relacionarse con otras entidades y tener sus propios atributos. En general, las agregaciones surgen en situaciones muy concretas: básicamente aparecen cuando en una relación ternaria hay dos entidades que deben establecer una relación previa entre sí antes de poder relacionarse con la tercera.

Veamos dos ejemplos de relaciones ternarias:

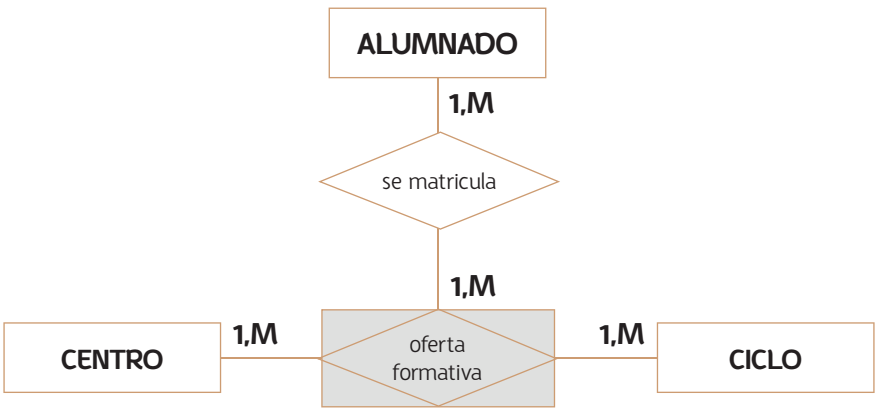


Aparentemente, ambos diagramas son prácticamente iguales. En el diagrama de la izquierda representamos que en una tienda hay empleados que se mueven por los distintos mostradores atendiendo a distintos clientes. En el de la derecha vemos el proceso de matriculación del alumnado en uno o varios ciclos de un centro.

Sin embargo, hay una sutil diferencia. A un cliente se le atiende en un mostrador por parte de un empleado, pero dicho empleado puede posteriormente estar atendiendo a otros clientes en otros mostradores. Sin embargo, un alumno no puede matricularse de cualquier ciclo en cualquier centro, porque no todos los centros ofrecen todos los ciclos.

Para saber si una relación ternaria es susceptible de convertirse en agregación debemos hacernos la siguiente pregunta: ¿es necesario que haya una relación previa entre dos de las entidades antes de que pueda incorporarse la tercera? Tal como hemos visto, en el caso de la izquierda no: un cliente puede acercarse a un mostrador cualquiera y ser atendido por el empleado que esté más cerca. Las tres entidades se incorporan a la relación al mismo tiempo. Estamos, pues, ante una relación ternaria pura. Sin embargo, en el caso de la derecha no es así: **antes de que el alumnado pueda matricularse, cada centro ya tiene unos ciclos asignados.** A la hora de la matrícula no puede escogerse cualquier ciclo en cualquier centro, sino que solo se puede escoger entre la oferta previamente diseñada. Es decir, **solo pueden escogerse parejas ciclo-centro que ya se hayan formado anteriormente al proceso de matriculación.**

La manera de representar esta relación previa entre ciclo y centro que debe existir para que el alumnado pueda matricularse, y al mismo tiempo que quede constancia de en qué ciclo y centro se ha hecho la matricula, es mediante una agregación. La relación quedaría de la siguiente manera:



Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Como podemos ver en la figura, la manera de representar la agregación entre entidades es crear una relación previa entre ellas y encerrarla en un rectángulo.

En este caso estamos especificando que previamente se produce una relación entre los centros y los ciclos formativos, como consecuencia de la cual se genera una oferta formativa. Cuando el alumnado decide en qué centro y ciclo desea matricularse, solamente puede escoger las parejas previamente definidas en la nueva entidad OFERTA FORMATIVA.

IMPORTANTE

Recuerda: para saber si una relación ternaria puede convertirse en una agregación tienes que preguntarte si es necesario establecer una relación previa entre dos de las tres entidades para que, posteriormente, se les pueda unir la tercera. No todas las relaciones ternarias pueden dar lugar a agregaciones.

En el diagrama que hemos obtenido como resultado final vemos que la relación entre ALUMNADO y OFERTA FORMATIVA es M:M. Esto significa que un miembro de la entidad ALUMNADO puede matricularse en más de una OFERTA FORMATIVA. Sin embargo, no es posible especificar en el diagrama que cada miembro de ALUMNADO solo puede estar matriculado de un ciclo en un único centro; es decir, aunque pueda matricularse en varios centros a la vez, debería hacerlo de ciclos distintos en cada centro. Este es un tipo de restricción que deberíamos incluir en la documentación de la base de datos para que se tenga en cuenta a la hora de desarrollar la aplicación que vaya a gestionarla.

Ciertos tipos de restricciones sí que pueden representarse gráficamente en el modelo entidad/relación, y precisamente de ellas nos ocuparemos en el siguiente apartado.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

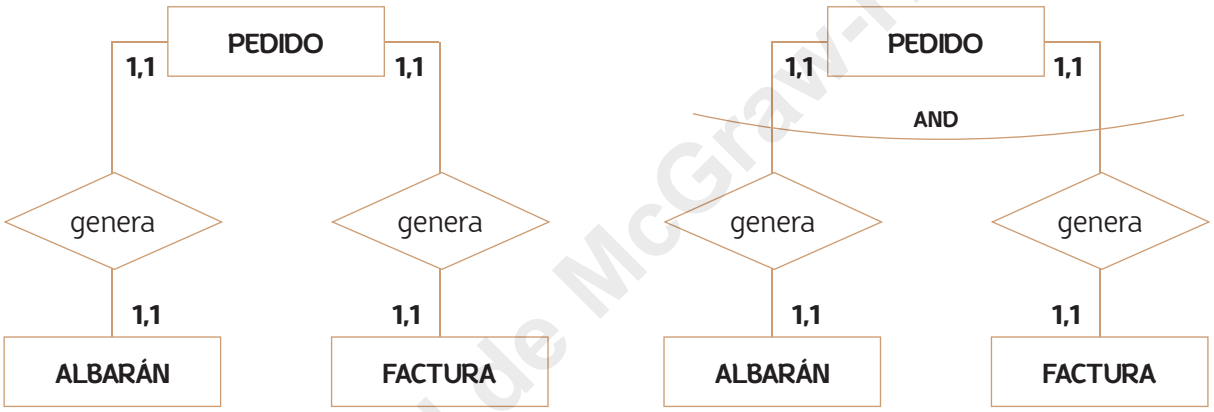
5. Otras restricciones

No todas las restricciones de una base de datos pueden mostrarse en el modelo entidad/relación. Algunas condiciones demandadas por el usuario, como que no se pueda facturar a un cliente moroso o que no se puedan vender más unidades de un producto de las que tenemos en almacén, habría que especificarlas en otro tipo de documentos. Sin embargo, sí que hay ciertas restricciones que no hemos visto hasta ahora y que, usando elementos del modelo entidad/relación extendido, se pueden especificar en el diagrama. A continuación vamos a ver algunas de ellas.

5.1. Inclusividad y exclusividad entre relaciones

Se produce una restricción de **inclusividad** entre relaciones cuando ambas deben aparecer conjuntamente, de manera que no pueden darse una sin la otra.

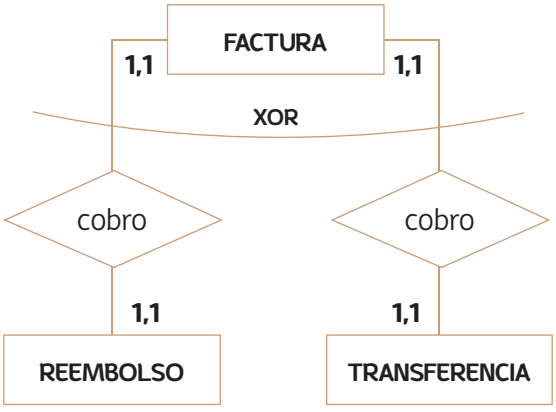
Imaginemos una base de datos en la que queremos representar que un pedido debe dar origen necesariamente a un albarán y a una factura. En la figura de la izquierda vemos ambas relaciones expresadas de la manera tradicional. En ese modelo no queda reflejado el hecho de que el pedido debe necesariamente originar un albarán y una factura. En la figura de la derecha puedes ver una forma de representar esa restricción de inclusividad con un arco y el símbolo **AND**.



Por otro lado, puede ocurrir justamente lo contrario: que entre varias relaciones posibles haya que escoger solo una, de manera que no puedan darse dos o más a la vez. En ese caso hablamos de **exclusividad** entre relaciones. En ocasiones este tipo de restricción entre relaciones se denomina con el término en inglés **Exclusive-OR**. Es un término ampliamente utilizado en informática que, abreviado, se representa como **XOR** y que indica que entre varios elementos existe una relación de exclusión mutua: solo uno de ellos puede darse a la vez.

Veamos un ejemplo: supongamos que queremos indicar en el diagrama que una factura, a la hora de cobrarla, da lugar a un reembolso o a una transferencia, pero nunca a ambas cosas a la vez.

Este tipo de restricción entre relaciones se expresa de la misma manera que la de inclusividad: dibujando ambas relaciones y uniéndolas mediante una línea curva. Sin embargo, en este caso, en lugar de utilizar el término **AND** para indicar que ambas deben darse a la vez, usaremos **XOR** para definir justo lo contrario: solo puede producirse una de ellas.



Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

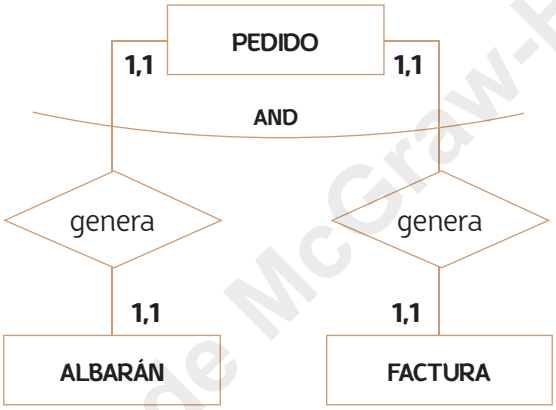
5.2. Precedencia entre relaciones

El modelo entidad/relación representa una visión prácticamente estática de una base de datos. Es decir, se especifican las entidades, las relaciones entre ellas y sus atributos, pero no el orden en el que ocurren las cosas. Ese factor tiempo se representa mejor con otras herramientas que se tratan en otros módulos.

Sin embargo, y aunque este tipo de restricciones no nos afectarán a la hora de diseñar físicamente nuestra base de datos sino más bien a la hora de desarrollar la aplicación que trabaje con ella, en ocasiones puede ser conveniente especificarlas. Por ejemplo, en el caso de la **precedencia** entre relaciones. Se trata de una mediante la cual especificamos que unas relaciones deben producirse en un orden concreto. Es decir, no puede darse una relación sin que previamente haya tenido lugar otra.

Este tipo de restricciones, como siempre, se entienden mejor con casos concretos. Volvamos al diagrama en el que veíamos cómo un pedido daba origen a un albarán y una factura. Si queremos especificar que primero debe generarse el albarán para que después puede hacerse lo propio con la factura, podemos usar la precedencia entre relaciones.

En el modelo entidad/relación extendido, la precedencia entre dos relaciones se indica mediante una flecha discontinua que sale desde la que debe darse en primer lugar y apunta hacia la que debe producirse con posterioridad. En nuestro caso la relación entre PEDIDO, ALBARÁN y FACTURA quedaría como se ve en la siguiente figura.



Con este conjunto de especificaciones añadidas al modelo queda más claro cuál es el tipo de relación entre las tres entidades:

- Cada pedido genera **obligatoriamente** un albarán y una factura (conector AND).
- Para poder generar la factura, **previamente** se debe haber generado el albarán (flecha de precedencia).

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Unidad 4.

Modelo relacional

Propiedad de McGraw-Hill®

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

1. Fundamentos del modelo relacional: tuplas y relaciones. Representación

Comencemos esta unidad resumiendo lo que hemos tratado hasta ahora.

Nuestro cometido es crear una base de datos que contenga toda la información relevante de un fragmento de la realidad (un instituto, un hospital, una ferretería, etc.), a fin de que un sistema informático pueda gestionarla representando su evolución (altas, bajas, modificaciones y listados).

En las unidades anteriores nos hemos valido de un instrumento, el modelo entidad/relación, para representar los datos que abstraemos de la realidad. Lo que obtuvimos fue una primera aproximación, una especie de maqueta. Se trataba de un diagrama que representaba la semántica de la base de datos final. Se establecieron **restricciones** que rigen el comportamiento de los datos, o sea, que preservan siempre su integridad, impidiendo que se llegue a cualquier estado inconsistente con la realidad.

Sin embargo, ese diagrama no nos informa acerca del contenido, es decir, de qué valores hay en las ocurrencias de cada entidad ni de cuáles de ellas están interrelacionadas con cuáles ocurrencias de qué otras entidades.

En la presente unidad vamos a tratar de solucionarlo. Recurriremos al **modelo relacional** para representar ambos aspectos: por un lado, las restricciones que se establecieron en el modelo E/R y, por otro, los datos que contienen las ocurrencias y sus interrelaciones.

El modelo relacional debe su nombre al concepto matemático de **relación**, un tipo especial de **conjunto**. Para comprenderlo, será necesario hacer una breve aproximación a los conjuntos.

IMPORTANTE

La relación (*relation* en inglés) del modelo relacional no es la *interrelación* (*relationship* en inglés) que se representaba con un rombo en el modelo E/R.

1.1. Conjuntos. Representaciones

Un conjunto es una colección de elementos. La forma más ilustrativa de representar gráficamente un conjunto es mediante un **diagrama de Venn**. Consiste en colocar sus elementos en el interior de un óvalo que se etiqueta con el nombre del conjunto. Por ejemplo, el conjunto «SEMANA» se podría representar en un diagrama de Venn así:



Otra manera, menos ilustrativa, de representar ese mismo conjunto es mediante llaves:

```
SEMANA = { sábado, lunes, viernes, domingo, martes, miércoles, jueves }
```

IMPORTANTE

Dos conjuntos son iguales si contienen los mismos elementos, sin importar el orden.

1.2. Tuplas

Otro tipo especial de conjunto es la **tupla**. Los elementos de una tupla no son valores atómicos o escalares, sino pares **atributo-valor**.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Para ejemplificarlo, imaginemos que estamos jugando al póker y queremos representar nuestra mano de cinco cartas, que está compuesta por el 3 de diamantes, el 8 de picas, el 10 de diamantes, la J de tréboles y la Q de corazones. Para representar una carta de la baraja no basta con indicar el número: necesitamos indicar también cuál es el palo. Por eso, cada carta es, en realidad, una tupla.

1.3. Esquema de tupla

El **esquema de tupla** de una carta de póker está formado por esos dos **atributos**: el número y el palo.

```
esq_carta = { numero, palo }.
```

Siguiendo ese esquema de tupla, nuestra mano de cartas se puede representar con cinco tuplas:

```
carta1 = { 3 , diamantes }
carta2 = { 8, picas }
carta3 = { 10 , diamantes }
carta4 = { J , tréboles }
carta5 = { Q , corazones }
```

1.4. Dominio

Además, el modelo relacional permite restringir los valores que puede tomar cada atributo indicando su **dominio**:

```
dom_numero = { A, 2, 3, 4, 5, 6, 7, 8, 9, 10, J, Q, K }
dom_palo = { picas, tréboles, diamantes, corazones }
```

1.5. Relación

Finalmente, si has comprendido el ejemplo anterior, no te va a costar nada entender qué es la relación del modelo relacional. Una **relación es un conjunto de tuplas con idéntico esquema**, o sea, con los mismos atributos y sus respectivos dominios.

Ahora estamos en condiciones de representar la relación CARTAS usando el modelo relacional:

```
CARTAS = {
        RESTRICCIONES = {
                dom_numero = { A, 2, 3, 4, 5, 6, 7, 8, 9, 10, J, Q, K }
                dom_palo = { picas, tréboles, diamantes, corazones }
                esq_tupla = { numero, palo }
        }
        TUPLAS = {
                { 3 , diamantes },
                { 8, picas },
                { 10 , diamantes },
                { J , tréboles },
                { Q , corazones }
        }
}
```

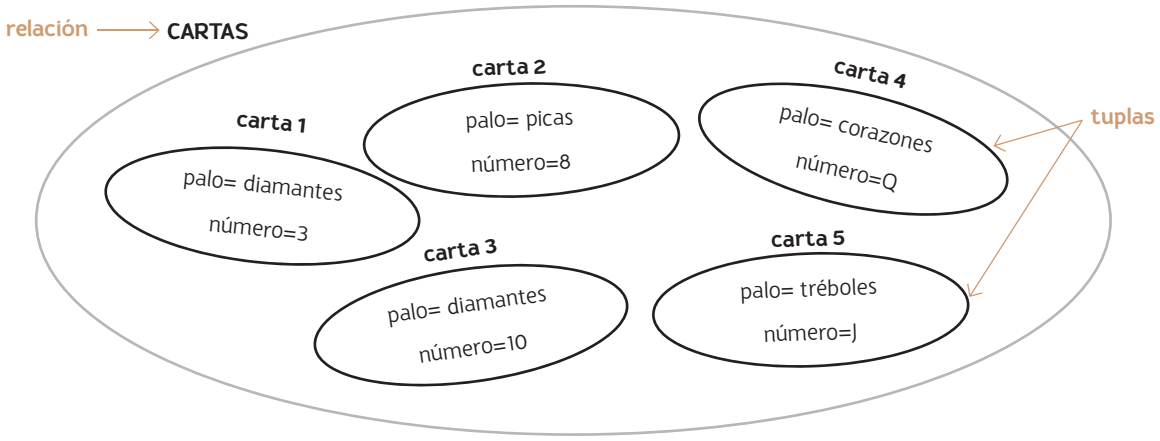
Observarás que, por un lado, se han conseguido representar las restricciones (dominios y esquema de tupla) y, por otro, el contenido de las ocurrencias (tuplas).

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Seguro que la visualización de este ejemplo como diagrama de Venn te va a reforzar la comprensión del concepto de relación del modelo relacional:



1.6. Representación de relaciones como tablas

Aunque la forma correcta de representar gráficamente las relaciones del modelo relacional sea mediante **diagramas de Venn** o mediante la notación de llaves, a menudo resulta cómodo representarlas como tablas.

Para representar una **relación** como **tabla**, en la fila de encabezado indicamos los nombres de los **atributos**, ahora también denominados **columnas** o **campos**. Debajo de ese encabezado, cada tupla se representa como una **fila** cuyas celdas contienen los valores según el orden de los atributos del encabezado (en una tabla, sí que es relevante el orden de los atributos).

La relación CARTAS del ejemplo inicial se representaría como tabla así:

CARTAS	
número	**palo**
3	diamantes
8	picas
10	diamantes
J	trébol
Q	corazones

La fila de encabezado son dos columnas con los nombres de cada atributo. Debajo aparecen las cinco tuplas, cada una en una fila.

IMPORTANTE

Al traducir *relación* como *tabla* se vuelve a cometer una «traición», puesto que una tabla no es un conjunto, pero, siempre que se tenga ello en mente, se disculpa por razones de comodidad.

Relación	Tabla
Tupla	Fila
Atributo	Columna
Grado	Número de columnas
Cardinalidad	Número de filas

Comparación de la terminología de relación y tabla.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Ejemplo de tabla:

Código	Nombre	Equipo	Dorsal
USI	Unai Simón	Athletic Club	23
JAL	Jordi Alba	FC Barcelona	18
PTO	Pau Torres	Villarreal CF	4
KOK	Koke	Atlético de Madrid	8
AMO	Álvaro Morata	Juventus FC	7

5 tuplas
Cardinalidad: 5

4 atributos
Grado: 4

Ejemplo de relación con cardinalidad 5 y grado 4.

1.7. Tipos de restricciones

A. RESTRICCIONES INHERENTES AL MODELO RELACIONAL

El modelo relacional impone varias restricciones inherentes a toda relación:

- No puede contener dos tuplas iguales. De ello se deduce la obligatoriedad de que exista una clave primaria, aunque está compuesta por todos los atributos del esquema de tupla.

  Esta restricción, como se verá más adelante, no lo es de SQL ni de muchos SGBD, que permiten la definición de clave primaria sin imponerla (es decir, permiten crear tablas sin clave primaria, aunque ello implique que el diseño no sea relacional).

- Ningún atributo que forme parte de la clave primaria de una relación puede tomar un valor nulo, esto es, un valor desconocido, vacío o inexistente (regla de integridad de entidad).
- El orden de las tuplas no es significativo.
- El orden de los atributos no es significativo.
- Cada atributo solo puede tomar un único valor del dominio sobre el que está definido.

B. RESTRICCIONES SEMÁNTICAS

Condiciones que deben cumplir los datos para su correcto almacenamiento.

- Restricción de clave primaria (**`PRIMARY KEY`**): permite declarar un atributo o un conjunto de atributos como clave primaria de una relación, con lo que todos sus valores no se podrán repetir en diferentes tuplas ni ser nulos.
- Restricción de valor único o de unicidad (**`UNIQUE`**): mediante ella se indica que los valores de un conjunto de atributos (uno o más) no pueden repetirse en una relación. Esta restricción permite la definición de claves alternativas.
- Restricción de obligatoriedad (**`NOT NULL`**): permite obligar a que ciertos atributos de las tuplas no admitan el valor nulo.
- Restricción de integridad referencial (**`FOREIGN KEY`**): un atributo de una tupla de una relación puede hacer referencia a un atributo de una tupla de otra relación o incluso de la propia. Esta restricción exige que el valor referenciado exista en la otra relación.
- Restricción de dominio: esta restricción exige que el valor que puede tomar un campo esté dentro del dominio definido.
- Restricción de verificación (**`CHECK`**): esta restricción permite comprobar si un valor de un atributo es válido de acuerdo con una expresión determinada por el usuario.
- Restricción genérica adicional o aserción (**`ASSERT`**): permite validar cualquiera de los atributos de una o varias relaciones.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

2. Entidades y relaciones 1:1, 1:M

El paso de un esquema en el modelo E/R al relacional está basado en los tres principios siguientes:

- Toda entidad se convierte en una relación.
- Toda interrelación 1:M se convierte en una nueva relación o bien da lugar a una *propagación de clave*.
- Todo tipo de interrelación M:M se transforma en una relación.

A primera vista, se podría deducir que en la transformación del modelo E/R al modelo relacional se pierde semántica, puesto que tanto las entidades como las interrelaciones se transforman en relaciones, de forma que ya no es posible distinguir entre unas y otras. Se constata asimismo que la pérdida de semántica es aún mayor en el caso de la propagación de clave, donde, como veremos, desaparece incluso la interrelación como nueva relación.

Ciertamente, cada vez que se hace una traducción se pierde algo por el camino. Como reza el dicho italiano, «traduttore, traditore». Pero es preciso destacar que la pérdida de semántica no implica necesariamente un peligro para la integridad de la base de datos, ya que, si la transformación se realiza correctamente, se respetarán las restricciones que aseguran la consistencia.

2.1. Transformación de entidades fuertes



Ejemplo de entidad que representa un futbolista en el modelo ER.

Siguiendo el primer principio del paso de un esquema en el modelo E/R a un esquema relacional, todo tipo de entidad fuerte se convierte en una relación, con el nombre en plural. Además, también tiene una trasformación directa cada uno de los atributos del modelo.

Una primera aproximación de cómo quedaría la relación sería:

```
FUTBOLISTAS (código, nombre, dorsal, equipo)
```

Como podemos observar, en el diagrama E/R el atributo *código* está **marcado** como atributo identificador. ¿Cómo indicamos que un campo es clave primaria en el modelo relacional? Añadiendo el símbolo **#** delante de los atributos que forman parte de esta. Por tanto, la especificación de la relación quedaría de la siguiente forma:

```
FUTBOLISTAS (#código, nombre, dorsal, equipo)
```

2.2. Transformación de relaciones

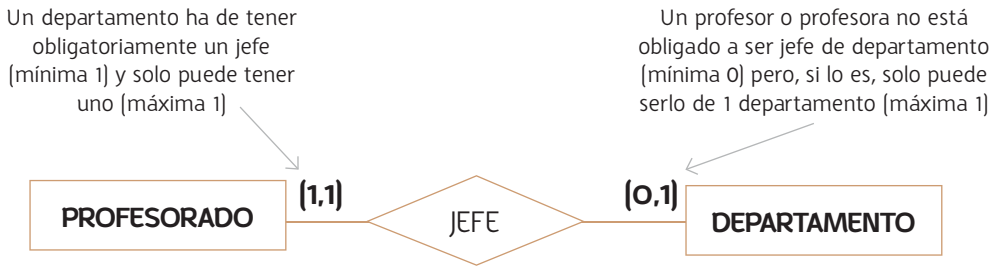
Siguiendo el segundo principio del paso de un esquema en el modelo E/R a un esquema relacional, todo tipo de interrelación 1:M conlleva crear una nueva relación o bien el fenómeno denominado *propagación de clave*.

En la transformación de la interrelación 1:M, vamos a distinguir el caso concreto de que M sea igual a 1, y el resto de casos.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

A.RELACIONES CON CARDINALIDAD 1:1



En este tipo de relaciones, se produce una propagación de clave, es decir, o bien la clave primaria de la relación **PROFESORADO** pasa a la relación **DEPARTAMENTOS**, o viceversa. Vamos a ver qué solución sería la correcta, teniendo en cuenta los datos de las siguientes relaciones:

PROFESORADO		
Id_prof	Nombre	Apellidos
RLB	Raúl Vicent	Lerma Blasco
JMA	José Alfredo	Murcia Andrés
FOL	Fidel	Oltra Landete
JCM	José	Chamorro Molina

DEPARTAMENTO	
Id_dpt	Nombre
INF	Informática
MAT	Matemáticas
ROB	Robótica

Si partimos de la premisa de que los cuatro profesores pertenecen al departamento de Informática, en el caso de que pasemos la clave primaria del departamento a la tabla profesorado, el atributo **id_dpt** solo tendría valor en una tupla de la tabla.

PROFESORADO			
Id_prof	Nombre	Apellidos	Id_dpt
RLB	Raúl Vicent	Lerma Blasco	
JMA	José Alfredo	Murcia Andrés	
FOL	Fidel	Oltra Landete	INF
JCM	José	Chamorro Molina	

DEPARTAMENTO	
Id_dpt	Nombre
INF	Informática
MAT	Matemáticas
ROB	Robótica

Luego esta no sería una buena opción.

Por el contrario, si pasamos la clave primaria **id_prof** a la tabla **DEPARTAMENTOS**, todos los campos tendrán la información correspondiente a sus jefes de departamento.

ALUMNADO		
NIA	Nombre	Apellidos
104578	Raül	Lerma Blasco
103568	José Alfredo	Murcia Andrés
107896	Fidel	Oltra Landete
106574	José	Chamorro Molina

DEPARTAMENTO		
Id_dpt	Nombre	Id_prof
INF	Informática	JCM
MAT	Matemáticas	MGB
ROB	Robótica	PIB

Hay que tener en cuenta que el valor del campo **Id_prof** de la tabla **DEPARTAMENTOS** ha de existir en el campo **id_prof** de la tabla **PROFESORADO**.

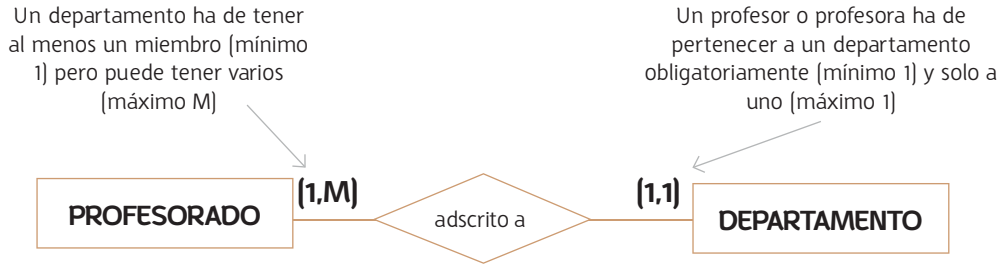
Por tanto, la solución correcta sería propagar el campo clave de la tabla **profesorado** a la tabla **departamento**, de forma que se convertiría en clave foránea (o ajena). El resultado sería:

```
PROFESORADO (#id_prof, nombre, apellidos)
DEPARTAMENTOS (#id_dpt, nombre, id_prof)
                              profesorado
```

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

B. RELACIONES CON CARDINALIDAD 1:M



En este tipo de interrelaciones no existe duda a la hora de propagar la clave. Siempre se propaga la clave a la relación donde la cardinalidad máxima es M.

En el ejemplo actual, el departamento puede tener varios miembros adscritos; por tanto, la propagación de clave se produce en la tabla PROFESORADO.

El resultado sería el siguiente:

```
PROFESORADO (#id_prof, nombre, apellidos, id_dpt)
                                          departamentos

DEPARTAMENTOS (#id_dpt, nombre)
```

Se puede observar que los cuatro profesores pertenecen al departamento de Informática.

PROFESORADO			
Id_prof	Nombre	Apellidos	Id_dpt
RLB	Raül Vicent	Lerma Blasco	INF
JMA	José Alfredo	Murcia Andrés	INF
FOL	Fidel	Oltra Landete	INF
JCM	José	Chamorro Molina	INF

DEPARTAMENTO	
Id_dpt	Nombre
INF	Informática
MAT	Matemáticas
ROB	Robótica

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

3. Relaciones M:M

Para el estudio de la transformación de las relaciones M:M (muchos a muchos) del modelo E/R al modelo relacional, vamos a ver un ejemplo ya visto en anteriores unidades didácticas.

Vamos a partir del siguiente modelo E/R que representa la relación del alumnado con los módulos matriculados de un ciclo formativo.



Ejemplo de interrelación muchos a muchos entre alumnado y módulo.

Como se puede observar, las cardinalidades ya están representadas. Como en ambos extremos de la relación la cardinalidad máxima es M, nos encontramos ante una relación M:M.

¿Cómo leemos estas cardinalidades?

- Un alumno o alumna puede matricularse en varios módulos; como mínimo, en uno.
- En un módulo, puede haber matriculados varios alumnos o alumnas; como mínimo, uno.

Si tenemos los siguientes datos en las relaciones:

ALUMNADO		
NIA	**nombre**	**apellidos**
104578	Raül Vicent	Lerma Blasco
103568	Joaquim	Matalí Ortiz
107896	Fidel	Oltra Landete
106574	Jose	Chamorro Molina

MÓDULOS	
código	**nombre**
PRG	Programación
BDD	Bases de Datos
EDD	Entornos de Desarrollo
LM	Lenguajes de Marcas

podríamos pensar que una posible solución sería añadir en la relación **ALUMNADO** tantas columnas como el número de módulos en los que está matriculado el alumno. Podríamos tener una primera solución de este tipo:

ALUMNADO					
NIA	**nombre**	**apellidos**	**módulo_1**	**módulo_2**	**módulo_3**
104578	Raül Vicent	Lerma Blasco	PRG	BDD	EDD
103568	Joaquim	Matalí Ortiz	BDD	PRG	EDD
107896	Fidel	Oltra Landete	PRG	EDD	BDD
106574	Jose	Chamorro Molina	EDD	BDD	PRG

Pronto nos daremos cuenta de que, si un alumno se quiere matricular en un cuarto módulo, haría falta añadir otro atributo. Y así para cada uno de los siguientes módulos que deseáramos matricular.

Además, se podría dar el caso que un alumno o alumna estuviera matriculado en más módulos que otro y quedar muchos atributos sin valores asignados (`null`).

Por tanto, no es una buena solución.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Si, por el contrario, añadimos los campos en la relación **MÓDULOS**, llegaremos a la misma conclusión:

MÓDULOS				
código	**nombre**	**alumno_1**	**alumno_2**	**alumno_3**
PRG	Programación	104578	103568	107896
BDD	Bases de Datos	104578	103568	107896
EDD	Entornos de Desarrollo	104578	103568	107896
LM	Lenguajes de Marcas	104578	103568	107896

Tampoco es una buena solución. Por tanto, se hace necesario crear una tercera relación para que almacene la información de las matrículas del alumnado en los diferentes módulos, siguiendo el principio de que **todo tipo de interrelación M:M se transforma en una nueva relación.**

El **esquema relacional** estará formado por el esquema de tupla de cada relación:

```
ALUMNADO (#NIA, nombre, apellidos)
MÓDULOS (#código, nombre)
MATRÍCULAS (#NIA, #código, nota)
```

Como se aprecia, la clave primaria de la relación **MATRÍCULAS** está formada por cada uno de los campos de las claves primarias de las relaciones **ALUMNADO** y **MÓDULOS**.

Un posible estado de la base de datos relacional sería:

ALUMNADO		
NIA	**nombre**	**apellidos**
104578	Raül Vicent	Lerma Blasco
103568	Joaquim	Matalí Ortiz
107896	Fidel	Oltra Landete
106574	Jose	Chamorro Molina

MÓDULOS	
código	**nombre**
PRG	Programación
BDD	Bases de Datos
EDD	Entornos de Desarrollo
LM	Lenguajes de Marcas

MATRÍCULAS		
NIA	**código**	**nota**
104578	PRG	10
103568	PRG	9
103568	BDD	10
107896	PRG	8
107896	BDD	9
107896	EDD	10
106574	PRG	9
106574	LM	9

Todo tipo de interrelación M:M se transforma en una nueva relación.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

4. Relaciones reflexivas y entidades débiles

4.1. Relaciones reflexivas

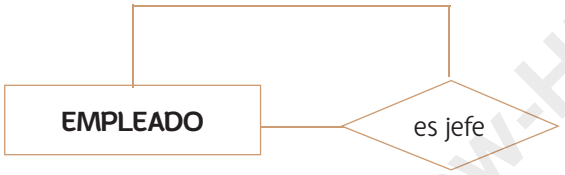
Aunque son menos frecuentes, también existen las relaciones de grado uno o unarias, también conocidas como relaciones reflexivas.

Vamos a distinguir dos casos en las relaciones reflexivas atendiendo a la cardinalidad.

A. RELACIONES REFLEXIVAS CON CARDINALIDAD 1:M

Imaginemos una relación del mundo real: en una empresa, nos encontramos que un empleado es jefe de otro empleado. **JEFE** no sería una entidad especial diferente de **EMPLEADO**, salvo que tuviera algunas propiedades distintas.

El jefe se puede considerar como un empleado más. La forma de representar que unos empleados son jefes de otros empleados sería la siguiente:



Relación de grado 1 o unaria.

Si partimos de la premisa de que un empleado solo tiene un jefe, pero un empleado puede ser jefe de varios empleados, tendremos una relación 1:M.

El proceso de trasformación del modelo E/R al modelo relacional sería:

- Se crea una relación para la entidad **EMPLEADO** añadiendo otra vez la clave primaria cambiada de nombre como clave ajena.

El resultado sería el siguiente:

EMPLEADOS (#DNI, Nombre, DNIJefe)

empleados

Un ejemplo con datos podría ser:

EMPLEADOS		
DNI	**nombre**	**DNIJefe**
11.111.111-A	Maria	
22.222.222-B	Javier	
33.333.333-C	Sara	11.111.111-A
44.444.444-D	Roberto	11.111.111-A

donde se puede observar que Maria es la jefa de Sara y de Roberto.

B. RELACIONES REFLEXIVAS CON CARDINALIDAD M:M

Por otro lado, podría ser que, en nuestra representación de la realidad, un empleado pudiera tener varios jefes y un empleado pudiera ser jefe de varios empleados.

En este caso, nos enfrentaríamos a una relación M:M.

En las relaciones reflexivas con cardinalidad M:M, se aplica la regla general para la transformación de relaciones; es decir, se debe crear una nueva relación cuya clave primaria sea el conjunto de atributos resultado de la unión de los atributos de la clave primaria de ambas relaciones. En este caso, al tener una única relación, se repetirán los campos de la clave primaria dos veces con nombres distintos.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

El resultado sería el siguiente:

```
EMPLEADOS (#DNI, Nombre)
JEFES (#DNI_empleado, #DNI_jefe)
```

Un ejemplo con datos podría ser:

EMPLEADOS	
DNI	**nombre**
11.111.111-A	Maria
22.222.222-B	Javier
33.333.333-C	Sara
44.444.444-D	Roberto

JEFES	
DNI	**DNIJefe**
22.222.222-B	11.111.111-A
33.333.333-C	11.111.111-A
44.444.444-D	11.111.111-A
33.333.333-C	22.222.222-B
44.444.444-D	22.222.222-B

Se puede observar que Maria es la jefa de Javier, Sara y Roberto, y que Javier, a su vez, es jefe de Sara y Roberto.

4.2. Entidades débiles

Una entidad débil respecto a otra con la que se relaciona lo puede ser por dos motivos. Cada uno de ellos responde a un tipo de restricción, y pueden darse por separado o ambos a la vez.

La transformación de entidades débiles en relaciones dependerá, por tanto, del tipo de restricción que exista.

- **Dependencia de existencia** → La clave primaria de la relación de la entidad débil será la unión de los atributos clave de la entidad débil; es decir, no se añaden atributos a los campos clave.
- **Dependencia de identificación** → La clave primaria de la relación de la entidad débil será la unión de los atributos clave de la entidad débil más los de la entidad fuerte.

Veamos un par de ejemplos:



Relación fuerte/débil con restricción de existencia.

Como vimos, la entidad débil es la entidad **factura** respecto a la entidad **cliente**. La factura no puede existir con independencia del cliente: se crea cuando se da la relación, y, si desaparece el cliente, lo hace también la factura.

En este caso no se añaden a la clave primaria los atributos de la clave primaria de la entidad fuerte, pero se hace necesario establecerles la condición de **NOT NULL**, pues no pueden estar vacíos.

El modelo relacional quedaría de la siguiente forma:

```
CLIENTES (#DNI, nombre, apellidos, empresa)
FACTURAS (#idFactura, fecha, DNICliente)
                              clientes
```

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.



Relación entidad fuerte – entidad débil.

Como vimos, la entidad débil es la entidad **HABITACIÓN** respecto a la entidad **HOTEL.** El número de habitación, que sería la clave primaria, se repite en los diferentes hoteles, de forma que, para identificar una habitación y diferenciarla de todas las demás, necesitamos indicar también en qué hotel está.

En este caso, la clave primaria de la relación **HABITACIONES** debe contener la clave primaria de la relación **HOTELES.**

El modelo relacional quedaría de la siguiente forma:

```
HOTELES (#idHotel, nombre, estrellas)
HABITACIONES (#idHotel, #numHabitacion, numCamas, esSuite)
```

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

5. Especializaciones y agregaciones

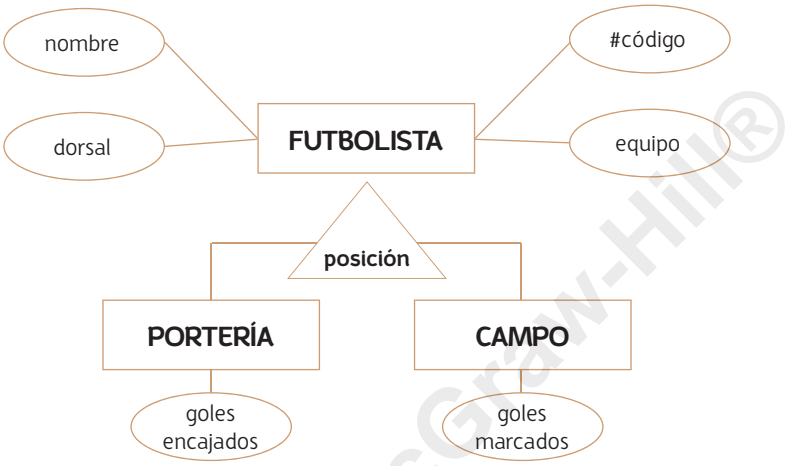
5.1. Generalizaciones y especializaciones

Como ya hemos visto, la aparición de diversas subentidades asociadas a una entidad principal conlleva lo que se denomina **partición** de entidades.

La partición de una entidad en varias subentidades se puede producir por varios motivos, y, dependiendo del proceso del diseño del modelo E/R, estaríamos hablando de una especialización o de una generalización.

En ambos casos, la trasformación del modelo E/R al modelo relacional se produce de la misma forma.

Vamos a ver este proceso a partir de un ejemplo que ya conocemos:



Especialización de futbolistas en función de su posición.

Para realizar esta trasformación tenemos cuatro opciones, y, en cada uno de los casos, elegiremos la que mejor se adapte a nuestra abstracción de la realidad.

1. Podemos generar una relación para la superclase y otras tantas para cada subclase incorporando el campo clave de la superclase a las relaciones de las subclases.

```
FUTBOLISTAS (#código, nombre, dorsal, equipo)
PORTEROS (#código, golesEncajados)
DECAMPO (#código, golesMarcados)
```

2. Podemos crear una relación para cada subclase incorporando todos los atributos de la clase padre, y no crear la relación de la superclase.

```
PORTEROS (#código, nombre, dorsal, equipo, golesEncajados)
DECAMPO (#código, nombre, dorsal, equipo, golesMarcados)
```

3. Podemos crear una sola relación para la superclase incorporando los atributos de todas las subclases, y añadir, para distinguir, un campo «tipo» que contendrá el tipo de subclase que representa cada tupla.

```
FUTBOLISTAS (#código, nombre, dorsal, equipo, , golesEncajados, ,
golesMarcados, tipo)
```

4. Por último, podríamos crear una sola relación para la superclase, como en la opción anterior, pero, en vez de un campo «tipo», se añadirían varios campos que indiquen si cumple un perfil.

```
FUTBOLISTAS (#código, nombre, dorsal, equipo, golesEncajados,
golesMarcados, esPortero, esDeCampo)
```

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

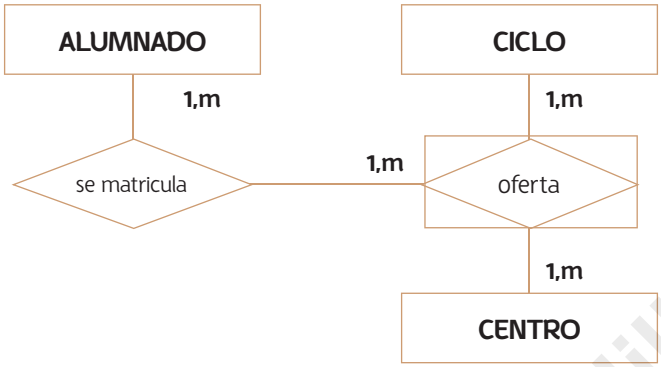
Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

5.2. Agregaciones

Todas las agregaciones se resuelven utilizando los principios en los que se basa la trasformación del modelo E/R en el modelo relacional.

Como en el modelo E/R ya se especifican las entidades que se crean, la trasformación al modelo relacional es directa.

Veamos un ejemplo que ya conocemos:



El esquema relacional sería el siguiente:

```
CENTROS (#códigoCentro, nombre, ciudad)
CICLOS (#idCiclo, nombre, familia)
OFERTAS (#códigoCentro, #idCiclo, plazas)
ALUMNADO (#NIA, nombre, apellidos)
MATRICULAS (#NIA, #códigoCentro, #idCiclo, fecha, convocatorias)
```

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Unidad 5.

Normalización

Propiedad de McGraw-Hill®

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

1. Teoría de la normalización

La **normalización** es el proceso de organizar datos en una base de datos. Esto incluye la creación de tablas y el establecimiento de relaciones entre esas tablas de acuerdo con unas reglas diseñadas tanto para proteger los datos como para hacer que la base de datos sea más flexible, al eliminar la redundancia y la dependencia inconsistente.

Los **datos redundantes** son aquellos que se repiten en distintos registros de la misma tabla o de tablas distintas y que desperdician espacio en disco y crean problemas de mantenimiento. Si los datos que existen en más de un lugar deben cambiarse, debe hacerse exactamente de la misma manera en todas las ubicaciones. Un cambio en la dirección de un/a alumno/a es mucho más fácil de implementar si esos datos solo se almacenan en la tabla Alumnado y en ningún otro lugar de la base de datos.

¿Qué es una **dependencia inconsistente**? Significa que tenemos atributos en una tabla que no dependen del valor de la clave primaria de la misma. Si bien es intuitivo para un usuario consultar en la tabla Alumnado la dirección de un/a alumno/a en particular, puede que no tenga sentido buscar allí el número de horas de un módulo que cursa ese alumno/a. El número de horas del módulo está relacionado con el módulo o depende de él y, por lo tanto, debe trasladarse a la tabla Módulos.

Las reglas formales de la teoría de la normalización son conocidas con el nombre de **formas normales**. Existen seis formas normales, de forma que, cuando la base de datos cumple las reglas de la primera forma normal, se considera que está en primera forma normal (1FN); cuando pasa la segunda, que está en segunda forma normal (2FN), etc.

También existe la forma normal de Boyce-Codd, que estudiaremos en el último apartado de esta unidad didáctica.

En adelante, utilizaremos la notación 1FN, 2FN, 3FN y FNBC con las correspondencias siguientes:

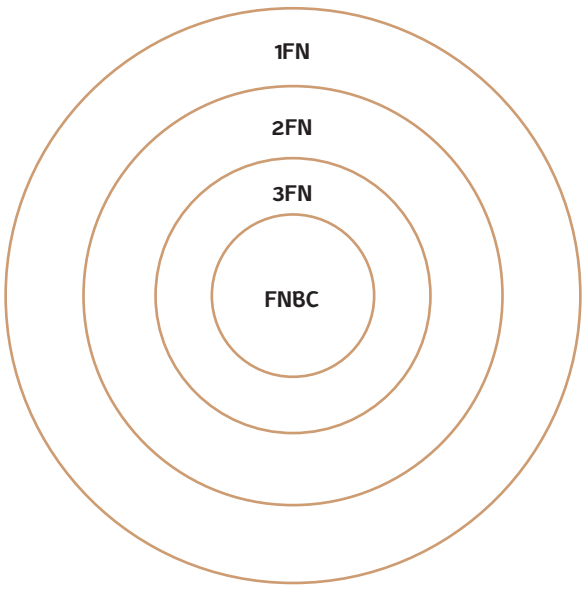
1.ª forma normal	1FN
2.ª forma normal	2FN
3.ª forma normal	3FN
Forma normal de Boyce-Codd	FNBC

Correspondencia de la notación a utilizar con su significado.

Como ocurre con muchas reglas y especificaciones formales, los escenarios del mundo real no siempre permiten un cumplimiento perfecto. Además, la normalización requiere tablas adicionales. Debemos tener en cuenta que, si no se cumplen las tres primeras reglas de normalización, tendremos que asegurarnos que la aplicación que utilice la base de datos anticipe cualquier problema que pueda ocurrir, como datos redundantes y dependencias inconsistentes.

De las seis formas normales, generalmente solo se aplican sobre las bases de datos las tres primeras, ya que se considera que una base de datos que está en 3FN es una base de datos correctamente diseñada.

Asimismo, una base de datos de la que se afirme que está en 2FN está también en 1FN, pues las formas normales se aplican de forma sucesiva.



Formas normales.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Las anomalías de inserción, borrado y modificación consisten en la aparición en la base de datos de estados no válidos en el mundo real.

Por ejemplo, si cada docente de una universidad tiene un departamento asociado a él o ella, así como los grupos a los que imparte clase:

PROFESORADO			
Id_profesor	**Nombre**	**Departamento**	**Grupo**
01054	José	Informática	1.º C
01078	Ana	Matemáticas	2.º A
01078	Ana	Matemáticas	2.º B
01069	Alejandro	Robótica	3.º A
01069	Alejandro	Robótica	1.º C

Una **anomalía de inserción** es la imposibilidad de agregar datos a la base de datos debido a la ausencia de otros datos. Por ejemplo, suponemos que el campo Grupo está definido para que no se permitan valores nulos. Si se contrata a un nuevo docente, pero no se le asigna inmediatamente un grupo, este docente no se podrá incluir en la base de datos. Esto da como resultado inconsistencias en la base de datos debido a omisiones.

Una **anomalía de borrado** es la pérdida involuntaria de datos debido a la eliminación de otros datos. Si el grupo de 1.º C se disolviera y fuera eliminado de la tabla anterior, el profesor José y el departamento de Informática dejarían de existir. Esto da como resultado inconsistencias en la base de datos, y al mismo tiempo es un ejemplo de cómo la combinación de información que realmente no pertenece a una sola tabla puede causar problemas.

Una **anomalía de modificación** es una inconsistencia de datos que resulta de la redundancia de datos y de una actualización parcial.

Si el departamento del profesor Alejandro no es correcto y debe cambiar, debe actualizarse al menos dos veces o habrá datos inconsistentes en la base de datos. Si el usuario que actualiza no se da cuenta de que los datos se almacenan de forma redundante, la actualización no se realizará correctamente.

Las anomalías de actualización, eliminación e inserción son muy indeseables en cualquier base de datos. Sin embargo, estas se evitan mediante el proceso de normalización.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

2. 1.ª forma normal

La 1FN espera que sigamos algunas reglas simples mientras diseñamos nuestra base de datos, y estas son:

- Atributos de un solo valor: Cada columna de nuestra tabla debe tener un solo valor, lo que significa que no debe contener campos multivaluados.
- El dominio de atributo no debe cambiar: En cada columna, los valores almacenados deben ser del mismo tipo. Por ejemplo: si hay una columna **fechaNacimiento** para guardar la fecha de nacimiento del alumnado, entonces no puede guardar en esa columna los *nombres* de algunos de ellos junto con la fecha de nacimiento de otros.
- Nombre único para atributos/columnas: Esto es para evitar confusiones en el momento de recuperar datos o realizar cualquier otra operación sobre los datos almacenados. Si una o más columnas tienen el mismo nombre, el sistema gestor de bases de datos quedará confuso.
- El orden no importa: El orden en el que se almacenan los datos en la tabla no es importante.

Siguiendo las anteriores reglas:

> Una base de datos se considera que está en 1FN si:
>
> - **Cada atributo (campo)** de una tabla contiene un solo valor atómico (simple), es decir, un atributo no puede tomar varios valores para un mismo registro (atributos multivaluados).
> - **Cada registro (fila)** es único.

Para desarrollar la teoría de normalización de una base de datos, tomaremos como ejemplo el diseño de la gestión de un instituto considerando solo la parte de las matrículas del alumnado en los diferentes módulos de los ciclos formativos.

Imaginemos que, después de analizar las necesidades del instituto para esta gestión, hemos obtenido una tabla de módulos. Establecemos que la clave primaria para identificar un módulo sea el código y, además, que se necesitan los campos siguientes:

MÓDULOS
#Código_módulo
Código_ciclo
Nombre_ciclo
Horas_ciclo
Familia_ciclo
Nombre_módulo
Curso_módulo
NIA_1
Nombre_1
Apellidos_1
Nota_1
Convocatoria_1
Horas_1
NIA_alumno_2
Nombre_2
Apellidos_2
Nota_2
Convocatoria_2
Horas_2
...
...
NIA_alumno_N
Nombre_N
Apellidos_N
Nota_N
Convocatoria_N
Horas_N

Diseño inicial de los módulos

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Analizando el diseño inicial de la tabla MÓDULOS, observamos la existencia de múltiples valores para los atributos (campos) siguientes: NIA, Nombre, Apellidos, Nota, Convocatoria y Horas.

Vemos que no cumple la condición de 1FN porque no cumple la condición de que no deben existir atributos con más de un valor.

La solución consiste en crear una nueva tabla, que podemos llamar MATRÍCULAS, a la cual se trasladan los datos repetitivos, en nuestro caso los datos referentes al alumnado.

MÓDULOS
#Código_módulo
Código_ciclo
Nombre_ciclo
Horas_ciclo
Familia_ciclo
Nombre_modulo
Curso_modulo

MATRÍCULAS
#Código_módulo
#NIA
Nombre
Apellidos
Nota
Convocatoria
Horas

Diseño de las tablas aplicando la 1FN.

Podemos comprobar que en la tabla MATRÍCULAS la clave primaria tiene como atributos el campo que identifica al alumnado y, también, que se ha trasladado el atributo de la clave primaria de la tabla original. Por lo general, cuando pasamos a 1FN las claves primarias de las nuevas tablas, estas están formadas por dos atributos.

Veamos un ejemplo de cómo podrían quedar los datos en la base de datos del instituto en 1FN:

MÓDULOS						
Código_ módulo	**Codigo_ ciclo**	**Nombre_ciclo**	**Horas_ciclo**	**Familia_ciclo**	**Nombre_ módulo**	**Curso_ módulo**
PRG	DAW	Desarrollo de Aplicaciones Web	2000	Informática y Comunicaciones	Programación	1.º
BDD	DAW	Desarrollo de Aplicaciones Web	2000	Informática y Comunicaciones	Bases de datos	1.º
EDD	DAW	Desarrollo de Aplicaciones Web	2000	Informática y Comunicaciones	Entornos de Desarrollo	1.º
DIW	DAW	Desarrollo de Aplicaciones Web	2000	Informática y Comunicaciones	Diseño de Interfaces Web	2.º
LM	DAM	Desarrollo de Aplicaciones Multiplataforma	2000	Informática y Comunicaciones	Lenguajes de Marcas	1.º
AAD	DAM	Desarrollo de Aplicaciones Multiplataforma	2000	Informática y Comunicaciones	Acceso a datos	2.º
PSP	DAM	Desarrollo de Aplicaciones Multiplataforma	2000	Informática y Comunicaciones	Programación de Servicios y Procesos	2.º

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

MATRÍCULAS						
Código_ módulo	**NIA**	**Nombre**	**Apellidos**	**Nota**	**Convocatoria**	**Horas**
PRG	104578	José	García	8	1.ª	256
PRG	103569	Ana	Martínez	9	2.ª	256
PRG	107896	María	Hernández	7	2.ª	256
BDD	104578	José	García	6	1.ª	160
BDD	103569	Ana	Martínez	8	1.ª	160
EDD	104578	José	García	5	2.ª	96
EDD	103569	Ana	Martínez	8	1.ª	96
EDD	107896	María	Hernández	7	1.ª	96
EDD	106574	Alejandro	Jiménez	6	1.ª	96

Como se puede observar, mucha información se repite.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

3. 2.ª forma normal

Dependencias funcionales

Entendemos como **dependencia funcional** la relación que tienen los atributos (campos) de una tabla con otros atributos de la propia tabla. Un campo tiene dependencia funcional si necesita información de otro/s campo/s para contener un valor concreto. Este concepto, introducido por E. F. Codd, ayuda a prevenir la redundancia de datos y está muy relacionado con los malos diseños de bases de datos.

La 2FN es un proceso de trasformación de tablas en el que se revisan las dependencias funcionales entre atributos que forman parte de la clave primaria con otros atributos de la misma tabla que no forman parte de la clave primaria. Por tanto, en este proceso solo tendremos que detenernos en aquellas tablas que tengan una clave primaria compuesta.

La 3FN es un proceso de trasformación de tablas en el que se revisan las dependencias funcionales entre atributos de una misma tabla que no forman parte de la clave primaria. Si esto sucede, en la tabla habrá información de varias entidades y la solución será dividir en dos tablas los campos de la tabla original.

Trasformación a 2FN

Se dice que una tabla está en segunda forma normal (2FN) si sucede que:

- Está en 1FN.
- Cada atributo (campo) no clave depende de la clave primaria completa, no de parte de ella.

Por tanto, una base de datos estará en 2FN si todas sus tablas lo están.

La idea intuitiva de la **2FN** es identificar todas las tablas con una clave compuesta, pues todas las que tienen clave simple están por defecto en **2FN** si están en **1FN**, y comprobar que cada uno de los campos de esta tabla depende de la clave completa.

En nuestro ejemplo, la tabla **MÓDULOS** se encuentra en **2FN**, pues está en **1FN** y su clave es simple. Sin embargo, la tabla **MATRÍCULAS** ha de ser analizada, dado que su clave es compuesta (está formada por dos atributos).

Analizando la tabla **MATRÍCULAS**, observamos que los atributos nombre y apellidos dependen únicamente del atributo **NIA**, y no de todos los atributos de la clave primaria. El nombre y los apellidos de un/a alumno/a dependen únicamente de qué alumno/a se trate y es completamente independiente de la matrícula, por lo cual estos dos atributos han de ser trasladados a una nueva tabla junto con el atributo clave **NIA**.

Además, si nos fijamos en el resto de campos de la tabla **MATRÍCULAS**, observamos que hay un atributo horas. Este atributo es una característica de los módulos, de modo que las horas no dependen de todos los campos de la clave primaria, sino de parte ella. Únicamente depende del campo **`Código_módulo`** y es independiente del NIA, por lo cual ha de ser devuelto a la tabla **MÓDULOS**.

MÓDULOS
#Código_módulo
Código_ciclo
Nombre_ciclo
Horas_ciclo
Familia_ciclo
Nombre_módulo
Curso_módulo
Horas

MATRÍCULAS
#Código_módulo
#NIA
Nota
Convocatoria

ALUMNADO
#NIA
Nombre
Apellidos

Diseño de las tablas aplicando la 2FN.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Veamos cómo podrían quedar los datos en la base de datos del instituto en 2FN:

MÓDULOS							
Código_ modulo	**Código_ ciclo**	**Nombre_ciclo**	**Horas_ ciclo**	**Familia_ciclo**	**Nombre_ módulo**	**Curso_ módulo**	**Horas**
PRG	DAW	Desarrollo de Aplicaciones Web	2000	Informática y Comunicaciones	Programación	1.º	256
BDD	DAW	Desarrollo de Aplicaciones Web	2000	Informática y Comunicaciones	Bases de datos	1.º	160
EDD	DAW	Desarrollo de Aplicaciones Web	2000	Informática y Comunicaciones	Entornos de Desarrollo	1.º	96
DIW	DAW	Desarrollo de Aplicaciones Web	2000	Informática y Comunicaciones	Diseño de Interfaces Web	2.º	120
LM	DAM	Desarrollo de Aplicaciones Multiplataforma	2000	Informática y Comunicaciones	Lenguajes de Marcas	1.º	96
AAD	DAM	Desarrollo de Aplicaciones Multiplataforma	2000	Informática y Comunicaciones	Acceso a datos	2.º	120
PSP	DAM	Desarrollo de Aplicaciones Multiplataforma	2000	Informática y Comunicaciones	Programación de Servicios y Procesos	2.º	60

MATRÍCULAS			
Código_módulo	**NIA**	**Nota**	**Convocatoria**
PRG	104578	8	1.ª
PRG	103569	9	2.ª
PRG	107896	7	2.ª
BDD	104578	6	1.ª
BDD	103569	8	1.ª
EDD	104578	5	2.ª
EDD	103569	8	1.ª
EDD	107896	7	1.ª
EDD	106574	6	1.ª

ALUMNADO		
NIA	**Nombre**	**Apellidos**
104578	José	García
103569	Ana	Martínez
107896	María	Hernández
106574	Alejandro	Jiménez

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

4. 3.ª forma normal

Se dice que una tabla está en tercera forma normal (**3FN**) si:

- Está en **2FN**.
- Todos los atributos que no son clave deben ser mutuamente independientes, es decir, un atributo no debe depender de otro atributo no clave de su tabla.

Por tanto, una base de datos estará en **3FN** si todas sus tablas lo están.

Si un atributo que no forma parte de la clave primaria depende de otro atributo que no forma parte de la clave primaria, posiblemente la tabla contiene datos sobre más de una entidad, de forma que se contradice el principio de que cada tabla almacene información de una entidad.

En nuestro ejemplo, podemos observar que las tablas **MATRÍCULAS** y **ALUMNADO** se encuentran en **3FN**. Sin embargo, la tabla **MÓDULOS** no está en 3FN, pues los atributos **`Nombre_ciclo`**, **`Horas_ciclo`** y **`Familia_ciclo`** dependen funcionalmente del atributo **`Código_ciclo`**, campo que no es clave. Por ello, debemos extraer estos atributos de la tabla **MÓDULOS** e incluirlos en una nueva tabla que haga referencia a los ciclos, tabla la que llamaremos **CICLOS** y que contendrá como clave primaria **`Código_ciclo`** y como atributos **`Nombre_ciclo`**, **`Horas_ciclo`** y **`Familia_ciclos`**.

Aplicando esto, nuestro diseño de facturas da lugar a las tablas que pueden verse a continuación:

MÓDULOS
#Código_módulo
Nombre_módulo
Curso_módulo
Horas
Código_ciclo

MATRÍCULAS
#Código_módulo
#NIA
Nota
Convocatoria

CICLOS
#Código_ciclo
Nombre_ciclo
Horas_ciclo
Familia_ciclo

ALUMNADO
#NIA
Nombre
Apellidos

Diseño de las tablas aplicando la 3FN.

Veamos cómo podrían quedar los datos en la base de datos del instituto en 3FN:

MÓDULOS				
Código_módulo	**Nombre_módulo**	**Curso_módulo**	**Código_ciclo**	**Horas**
PRG	Programación	1.º	DAW	256
BDD	Bases de Datos	1.º	DAW	160
EDD	Entornos de Desarrollo	1.º	DAW	96
DIW	Diseño de Interfaces Web	2.º	DAW	120
LM	Lenguajes de Marcas	1.º	DAM	96
AAD	Acceso a Datos	2.º	DAM	120
PSP	Programación de Servicios y Procesos	2.º	DAM	60

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

CICLOS			
Código_ciclo	**Nombre_ciclo**	**Horas_ciclo**	**Familia_ciclo**
DAW	Desarrollo de Aplicaciones Web	2000	Informática y Comunicaciones
DAM	Desarrollo de Aplicaciones Multiplataforma	2000	Informática y Comunicaciones

MATRÍCULAS			
Código_módulo	**NIA**	**Nota**	**Convocatoria**
PRG	104578	8	1.ª
PRG	103569	9	2.ª
PRG	107896	7	2.ª
BDD	104578	6	1.ª
BDD	103569	8	1.ª
EDD	104578	5	2.ª
EDD	103569	8	1.ª
EDD	107896	7	1.ª
EDD	106574	6	1.ª

ALUMNADO		
NIA	**Nombre**	**Apellidos**
104578	José	García
103569	Ana	Martínez
107896	María	Hernández
106574	Alejandro	Jiménez

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

5. Otras formas normales: FNBC, 4FN y otras

5.1. FNBC (forma normal de Boyce-Codd)

La **forma normal de Boyce-Codd** es una extensión de la tercera forma normal y también se conoce como **forma normal 3,5.**

Se dice que una tabla está en forma normal Boyce-Codd (**FNBC**) si:

- Está en **3FN.**
- Para cualquier dependencia **A → B**, A debería ser una **superclave.**

Por tanto, una base de datos estará en FNBC si todas sus tablas lo están.

El segundo punto suena un poco complicado, ¿verdad? En palabras simples, esto significa que para una dependencia **A → B**, A no puede ser un atributo no principal si B es un atributo principal.

Vamos a intentar entender este concepto en el ejemplo siguiente:

MATRÍCULAS		
NIA	**Módulo**	**Profesor**
0011	PRG	Jose
0011	BDD	Fidel
0022	PRG	Raül
0033	LMSGI	Alfredo
0044	PRG	José

En la tabla de arriba:

- Un estudiante puede inscribirse en varios módulos. Por ejemplo, el estudiante con **NIA 0011** ha optado por asignaturas de **PRG** y **BDD.**
- Para cada módulo, se le asigna un profesor.
- Puede haber varios profesores enseñando en un módulo, como en PRG.

¿Cuál crees que debería ser la clave principal?

En la tabla anterior, **NIA** y el módulo en conjunto forman la clave principal, porque al usar NIA y módulo podemos encontrar todas las columnas de la tabla. Otro punto importante a tener en cuenta aquí es que un profesor solo enseña un módulo, pero un módulo puede tener dos profesores diferentes. Por lo tanto, en este caso existe una dependencia entre el módulo y el profesor, en la que el módulo depende del nombre del profesor.

Esta tabla satisface la **1FN**, puesto que todos los valores son atómicos, los nombres de las columnas son únicos y todos los valores almacenados en una columna en particular son del mismo dominio. Esta tabla también satisface la **2FN**, ya que cada atributo depende de la clave completa. Y todos los atributos no clave son independientes, de modo que la tabla también satisface la **3FN.** Pero esta tabla no está en la **FNBC.**

¿Por qué esta tabla no está en FNBC?

En la tabla anterior, **NIA** y módulo forman la clave principal, lo que significa que la columna del módulo es un atributo principal. No obstante, hay una dependencia más: **profesor → módulo.** Y mientras que el módulo es un atributo principal, el profesor es un atributo no principal, que no está permitido por **FNBC.**

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

¿Cómo satisfacer FNBC?

Para que esta relación satisfaga la FNBC, descompondremos esta tabla en dos tablas, la de estudiantes y la de profesores. A continuación, tenemos la estructura de ambas:

ALUMNADO	
NIA	**ID_prof**
0011	01
0011	02
0022	03
0033	04
0044	01

PROFESORADO		
ID_prof	**Profesor**	**Módulo**
01	José	PRG
02	Fidel	BDD
03	Raül	PRG
04	Alfredo	LMSGI

5.2. 4NF

La **4FN** entra en escena cuando la dependencia de valor múltiple o dependencia multivalor ocurre en cualquier relación. Para que exista una dependencia de valor múltiple, la tabla debe tener al menos tres columnas.

Se dice que una tabla está en cuarta forma normal (**4FN**) si:

- Está en **FNBC**.
- No existe ninguna dependencia de valor múltiple.

Por tanto, una base de datos estará en **4FN** si todas sus tablas lo están.

Intentemos entender qué es la dependencia de valores múltiples mediante el ejemplo siguiente.

Imaginemos una tabla en la que vamos a almacenar datos sobre la matrícula de estudiantes de ciclos formativos en distintos módulos y en la que también se especifiquen los lenguajes de programación que les gustaría aprender durante sus estudios en el ciclo formativo.

MATRÍCULAS		
NIA	**Módulo**	**Lenguaje**
0011	PRG	Java
0011	BDD	C#
0022	EDD	Java
0022	LMSGI	C#

Como se puede ver en la tabla anterior, el estudiante con **NIA 0011** ha optado por matricularse de dos módulos, **PRG** y **BDD**, y le gustaría aprender dos lenguajes de programación, Java y C#.

Debes estar pensando en qué problema puede causar esto, ¿verdad?

Bueno, los dos registros para el o la estudiante con NIA 0011 darán lugar a dos registros más, como se muestra a continuación, porque para un estudiante existen dos lenguajes de programación; por lo tanto, estos lenguajes pueden darse a la vez junto con ambos módulos, lo que daría lugar a los registros siguientes:

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

MATRÍCULAS		
NIA	**Módulo**	**Lenguaje**
0011	PRG	Java
0011	PRG	C#
0011	BDD	Java
0011	BDD	C#

En la tabla anterior, no hay relación entre las columnas Módulo y Lenguaje; son independientes entre ellas.

Por lo tanto, existe una dependencia de valores múltiples, lo que conduce a la repetición innecesaria de datos y también a otras anomalías.

¿Qué debemos hacer para que la tabla esté en 4FN?

Para que la relación anterior satisfaga la cuarta forma normal, debemos descomponer la tabla en dos tablas.

MATRÍCULA_MÓDULOS	
NIA	**Módulo**
0011	PRG
0011	BDD
0022	EDD
0022	LMSGI

MATRÍCULA_LENGUAJES	
NIA	**Lenguaje**
0011	Java
0011	C#
0022	Java
0022	C#

Una tabla también puede tener una dependencia funcional junto con una dependencia de varios valores. En este caso, las columnas funcionalmente dependientes se mueven en una tabla separada y las columnas dependientes de valores múltiples se mueven a tablas separadas.

Estos problemas se pueden evitar fácilmente si diseñamos con cuidado nuestra base de datos.

5.3. 5FN y 6FN

Una tabla está en la **5FN** solo si está en **4NF** y no se puede descomponer en ninguna cantidad de tablas más pequeñas sin pérdida de datos.

La **6FN** de momento no está estandarizada y no será objeto de estudio en esta unidad didáctica.

En la normalización de bases de datos, generalmente la **5FN** y la **6FN** no se implementan en el diseño de bases de datos de la vida real.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Unidad 6.

SQL I: diseño físico y DDL

Propiedad de McGraw-Hill®

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

1. Diseño físico de la base de datos. Tablas

En esta unidad didáctica vamos a estudiar el **diseño físico** de la base de datos, es decir, la implantación final de la base de datos en un sistema informático, y, para ello, nos centraremos en el uso de SQL como lenguaje de bases de datos relacional.

Existen diferentes herramientas para el diseño físico de las bases de datos con SQL. Podemos distinguir entre los intérpretes de comandos y los entornos gráficos:

- **Línea de comandos mysql**

  El cliente del servidor MySQL (mysql-server) se llama *mysql*, y desde este intérprete de comandos podemos realizar todas las tareas necesarias sobre nuestras bases de datos, mediante sentencias sql.

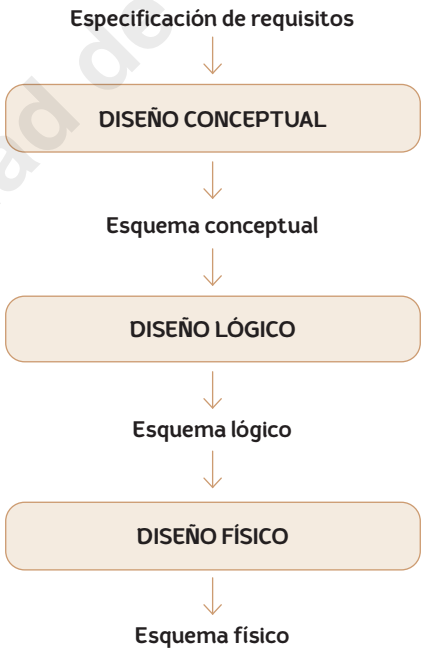
- **phpMyAdmin**

  PhpMyAdmin es una herramienta gratuita para manejar la administración de MySQL a través de un servidor web, utilizando un navegador web. La interfaz gráfica es muy intuitiva y permite acceder a todas las funcionalidades de MySQL.

- **MySQL Workbench**

  MySQL Workbench es una herramienta visual de escritorio para el diseño de bases de datos que ofrece dos ediciones diferentes, una *open-source* y otra comercial. Este software da al usuario la posibilidad de modelado de datos, desarrollo de SQL y herramientas de administración para la configuración de servidores.

Como se puede ver en la siguiente imagen, nos encontramos en la última fase de creación de una base de datos: el diseño físico. Consiste en elegir las estructuras más apropiadas para los ficheros de la base de datos de entre las opciones que ofrece el sistema gestor de bases de datos (SGBD), atendiendo al uso que se dará a los datos. Por esta razón, podemos afirmar que el diseño físico es muy dependiente del SGBD seleccionado, y es muy importante conocer las posibilidades que ofrece para implementar un esquema lógico sobre un hardware concreto.



Fases del desarrollo de una base de datos.

El objetivo general del diseño físico es determinar las estructuras de almacenamiento y estructuras de acceso para que las aplicaciones que accedan a la base de datos obtengan un buen rendimiento, y, para ello, cada SGBD ofrece varias opciones de organización de ficheros y caminos de acceso:

- **Índices:** se pueden utilizar diferentes tipos de índices para optimizar el acceso a datos, para garantizar la exclusividad y habilitar la agrupación en clúster.
- **Agrupamiento de registros** (de distinto tipo) relacionados, en los mismos bloques de disco (ficheros mixtos o clúster de ficheros).

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

- Distintos tipos de **técnicas de dispersión** (*hashing*).
- **Vistas de datos personalizadas:** una vista es una forma alternativa para presentar los datos que existen en una o más tablas.
- **Abreviar los nombres:** en el diseño físico se pueden abreviar los nombres de los campos que se han elegido durante el diseño lógico.

Tablas

Como sabemos, de las unidades anteriores, el esquema lógico de una base de datos relacional «tradujo» el esquema conceptual (el diagrama E/R), representándola en forma de relaciones.

El diseño físico va a «traducir» esas relaciones, implementándolas en forma de tablas. No nos resultará extraño, porque ya vimos que, por comodidad, a menudo ya solíamos representar las relaciones como tablas.

NOMBRE de TABLA				
Campo 1	**Campo 2**	...	**Campo n**	
XXX	XXX	...	XXX	→ Registro 1
XXX	XXX	...	XXX	→ Registro 2
...	...	...	...	...
XXX	XXX	...	XXX	→ Registro m

Cada tabla adopta el nombre de la relación de la que proviene. Dentro de una tabla, cada fila representa una tupla. A la fila también se la denomina **registro.**

Las columnas (o campos) de una tabla corresponden a los **atributos** de la relación. Los campos representan las diferentes propiedades o características de una tabla.

El conjunto finito de valores homogéneos permitidos para un campo se denomina **dominio.** Por ejemplo: números enteros, números decimales, fechas y horas, cadenas de caracteres, etcétera.

Un factor muy importante a la hora de realizar el diseño físico de la tabla es escoger correctamente el **tipo de dato** para establecer el dominio de cada columna. Cada celda podrá contener **valores** correspondientes exclusivamente al dominio establecido para su columna.

Al número de filas de una tabla se le denomina **cardinalidad.** El número de columnas se llama **grado.**

En el siguiente apartado de esta unidad didáctica estudiaremos los distintos tipos de datos disponibles, con la finalidad de poder escoger el adecuado para cada circunstancia.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

2. El Lenguaje SQL

El primer estándar internacional fue ANSI en 1986, y, un año después, ISO. SQL define un lenguaje interactivo de alto nivel para acceder y manipular los datos que están almacenados en un sistema gestor de bases de datos relacionales (SGBDR o RDBMS, *relational database management system*), como, por ejemplo, MariaDB, Oracle MySQL, Oracle Database, Microsoft SQL Server, Microsoft Access o IBM DB2, entre otros.

Igual que en la mayoría de los lenguajes de programación, en SQL, a la instrucción se la suele denominar con el anglicismo sentencia (por su traducción del inglés *sentence*). También es frecuente que se le llame *comando* (*command*) o *consulta* (*query*). Esta última, especialmente cuando involucra al sublenguaje DQL, como más adelante veremos.

El estándar actual de SQL (2016), compatible con ficheros JSON para la consola de la línea de comandos interactiva (CLI), no distingue entre mayúsculas y minúsculas en las sentencias (es *case-insensitive*). Establece una interfaz de programación de aplicaciones (API, *application programming interface*) que han de utilizar nuestros programas para lanzar sentencias SQL de una forma normalizada a través de esta serie de funciones bien definidas en el estándar, de manera que, para incrustar sentencias SQL, los fabricantes de SGBDR pueden crear sus controladores que admitan la API de SQL o del controlador del lenguaje de programación; por ejemplo, JDBC (Java Database Connectivity) u ODBC (Open Database Connectivity), entre otros.

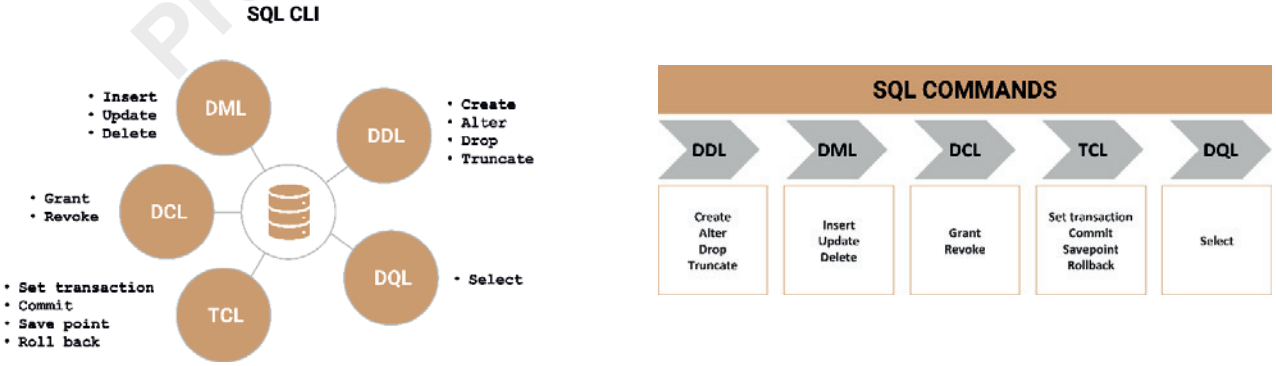
JDBC (Java Database Connectivity) Programa de conectividad de BB. DD. desde Java	ODBC (Open Database Connectivity) Programa de conectividad abierta de BB. DD.
Fue creado en 1996 por la empresa Sun Microsystems, que Oracle adquirió en 2009. Esta última se encarga de ir actualizándolo.	Lo creó en 1992 la empresa Microsoft, que se encarga de ir actualizándolo.
Define una API estándar solo para el lenguaje Java, que permite definir cómo puede un usuario acceder a sus datos desde la base de datos.	Define una API estándar abierta a cualquier SGBDR que lo admita.

Comparativa entre JDBC y ODBC.

Microsoft ha creado otra tecnología más potente, llamada OLE DB (*object linking and embedding for databases*), que permite trabajar con todo tipo de bases de datos, no solo con las relacionales, sino también con las de grafos, las orientadas a objetos, las nativas XML o las de NoSQL. OLE DB está incluida dentro de los Componentes de Microsoft para Acceso a Datos (MDAC, Microsoft Data Access Components).

2.1. Bloques de instrucciones de SQL

Las instrucciones SQL se pueden agrupar, dependiendo de su utilidad o funcionalidad, en cinco grandes bloques o sublenguajes: DDL, DQL, DML, DCL y TCL.



Sublenguajes de SQL.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

DDL *(data definition language)*	El lenguaje de definición de datos agrupa las instrucciones **`CREATE`**, **`ALTER`**, **`DROP`**, **`RENAME`**, **`TRUNCATE`**, **`COMMENT`**, **`SHOW`** y **`DESCRIBE`**.
DQL *(data query language)*	El lenguaje de consultas de datos solo utiliza la instrucción **`SELECT`**, que se puede usar de forma anidada.
DML *(data manipulation language)*	El lenguaje de manipulación de datos agrupa las instrucciones **`INSERT`**, **`UPDATE`**, **`DELETE`**, **`MERGE`**, **`CALL`**, **`EXPLAIN PLAN`** y **`LOCK TABLE`**.
DCL *(data control language)*	El lenguaje de control de datos agrupa las instrucciones **`GRANT`** y **`REVOKE`**.
TCL *(transaction control language)*	El lenguaje de control de transacciones agrupa las instrucciones **`SET TRANSACTION`**, **`COMMIT`**, **`ROLLBACK`** y **`SAVEPOINT`**.

En esta unidad nos centraremos en las instrucciones DDL y DQL. El resto se irán viendo en otras unidades.

2.2. Palabras reservadas de SQL

Como en cualquier lenguaje de programación, vamos a tener ciertos identificadores reservados. Dichas palabras solo se podrán utilizar en el lugar correcto para construir las sentencias SQL. Las que más vamos a utilizar son:

ALL	AND	ANY	AVG	BETWEEN	COUNT	EXISTS
FROM	GROUP BY	HAVING	IN	IS NULL	LIKE	MAX
MIN	NOT	OR	ORDER BY	SUM	UNIQUE	WHERE

Palabras reservadas más utilizadas en SQL.

No son las únicas, pues iremos descubriendo más a medida que las vayamos necesitando. También hay que comentar que es posible que cada SGBDR en concreto tenga algunas palabras reservadas propias, añadidas el estándar SQL, y que, por tanto, solo funcionan con ese fabricante. Nosotros vamos a tratar de seguir el estándar internacional de SQL de 2016, para que sea compatible con todos los fabricantes de SBGDR.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

3. Tipos de datos

Para que SQL pueda integrarse con los lenguajes de programación, necesitamos que sus tipos básicos de datos admitan los mismos dominios que estos, aunque puede que haya alguna variación de unos a otros. En SQL estándar están admitidos los siguientes:

3.1. Para gestionar números enteros o con decimales

- **`integer`** o **`int`** nos permite trabajar con números enteros. Será el HW concreto en donde se encuentre o del SGBDR en particular el que limitará su rango de valores. Existen tipos llamados **`tinyint`**, **`smallint`** y **`mediumint`** para rangos pequeños. O tipos llamados **`bigint`** para rangos más grandes.

Tipo (type)	Tamaño bytes (storage)	Valor mínimo con signo (minimum value signed)	Valor mínimo sin signo (minimum value unsigned)	Valor máximo con signo (maximum value signed)	Valor máximo sin signo (maximum value unsigned)
`TINYINT`	1	-128	0	127	255
`SMALLINT`	2	-32768	0	32767	65535
`MEDIUMINT`	3	-8388608	0	8388607	16777215
`INT`	4	-2147483648	0	2147483647	4294967295
`BIGINT`	8	-2^{63}	0	$2^{63}-1$	$2^{64}-1$

- **`numeric(p,d)`** nos permite trabajar con números decimales. El usuario definirá con el parámetro **`p`** el número de cifras enteras. y con el parámetro **`d`**, el número de cifras decimales.
- **`real`**, **`double`** nos permite trabajar con números en coma flotante; con **`real`**, de precisión simple, y con **`double`**, de doble precisión, utilizando el estándar IEEE 754.
- **`float(n)`** nos permite trabajar con números en coma flotante. En este caso, es el usuario quien especifica con el parámetro n el número de cifras que requiere para determinar la precisión.

3.2. Para gestionar fechas u horas

- **`datetime`** nos permite trabajar con valores que contengan partes de fecha u hora. En cada SGBDR puede variar. Los basados en el motor InnoDB, como Oracle MySQL, MariaDB o Amazon Aurora, recuperan y muestran los valores **`DATETIME`** con el formato «**`AAAA-MM-DD HH:MM:SS`**», y el intervalo admitido va del 1 de enero del año 1000 al 31 de diciembre del año 9999. O sea, el intervalo admitido es: «**`1000-01-01 00:00:00`**» a «**`9999-12-31 23:59:59`**».
- **`timestamp`** nos permite trabajar con valores que contengan partes de fecha u hora. Su rango va del 1 de enero de 1970 al 19 de enero de 2038. Aquí, el intervalo admitido es «**`1970-01-01 00:00:01`**» UTC a «**`2038-01-19 03:14:07`**» UTC. La diferencia con **`datetime`** es que hace referencia al huso horario UTC, por lo que, dependiendo del país en el que nos encontremos, podemos hacer el cálculo adecuado para mostrar el valor, según nuestro huso horario.
- **`date`** nos permite trabajar con valores que contengan solo la fecha con el formato «**`AAAA-MM-DD`**». El intervalo admitido va del 1 de enero del año 1000 al 31 de diciembre del año 9999 («**`1000-01-01`**» a «**`9999-12-31`**»).

Puede que alguna vez nos interese tener estos valores de las fechas y las horas por separado; por eso, disponemos de los siguientes tipos de campos.

- **`time`** nos permite trabajar con valores que contengan solo la hora, con el formato «**`HH:MM:SS`**». El intervalo admitido va del 1 de enero del año 1000 al 31 de diciembre del año 9999 («**`-838:59:59`**» a «**`838:59:59`**»). El hecho de que la parte de horas sea tan grande viene dado porque no solo se puede usar para representar una hora del día (que debe ser inferior a 24 horas), sino también para calcular el tiempo transcurrido, o un intervalo de tiempo entre dos eventos (que puede ser mucho mayor que 24 horas, o incluso negativo).
- **`Year`** nos permite trabajar con valores que contengan solo las cuatro cifras del año, dentro del rango de **`1901`** a **`2155`**. Si queremos guardar el año actual, lo podemos obtener con la función **`NOW`**().

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

3.3. Para gestionar valores alfanuméricos

- **`character`** o **`char(n)`** nos permite guardar una cadena de caracteres de longitud fija **`n`**.
- **`character varying`** o **`varchar(n)`** nos permite guardar una cadena de caracteres de longitud variable, aunque el usuario indicará con el valor n su tamaño máximo.

3.4. Para gestionar otros datos de diferentes tipos

- **`boolean`** o **`bit`** nos permite guardar valores ciertos (**`true`**) o falsos (**`false`**).
- Para campos más grandes, existen los tipos llamados **`binary`**, **`varbinary`**, **`blob`**, **`text`**, **`enum`** y **`set`**. Pueden servir para guardar imágenes u objetos multimedia, entre otros.
- Para terminar esta lista, hay que comentar que existen nuevos tipos de datos especiales, como **`point`**, que sirve para almacenar valores geodésicos, o **`json`**, para almacenar vectores (**`arrays`**) de texto que guarden el contenido de un fichero JSON con el que queramos trabajar.

Para obtener más información, podemos consultar directamente la web del fabricante. Por ejemplo, para MySQL podemos visitar **https://dev.mysql.com/doc/refman/8.0/en/data-types.html**.

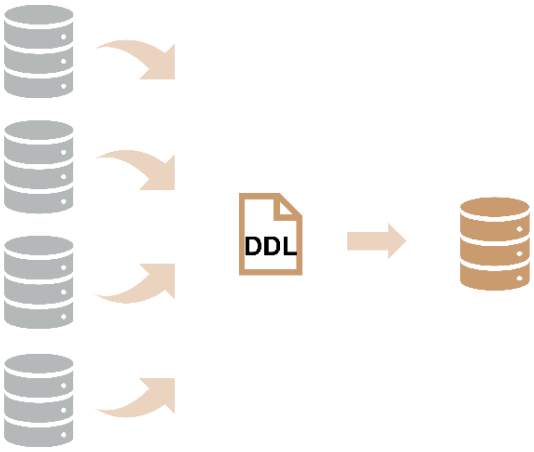
Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

4. SQL/DDL

Vamos a aprender cómo utilizar las sentencias **`CREATE`**, **`ALTER`**, **`DROP`**, **`RENAME`**, **`TRUNCATE`**, **`COMMENT`**, **`SHOW`** y **`DESCRIBE`** del lenguaje de definición de datos (DDL, *data definition language*).



Las tablas de nuestras bases de datos serán creadas con DDL sobre el SGBDR.

4.1. CREATE

Iremos comprendiendo su descripción, su sintaxis y algún ejemplo a partir de las bases de datos de **`mcgraw-cines`**, **`mcgrawpeliculas`**, **`mcgrawinstituto`** y **`mcgrawhoteles`**, que hemos ido diseñando y normalizando. Ahora es el momento de crearlas usando SQL.

> Dispones de estas bases de datos en el archivo **GBD_U06_A04_E.zip**, en la sección **Ejemplos para trabajar y analizar**.

`CREATE` es una potente instrucción que se utiliza para crear desde la propia base de datos hasta diferentes objetos de esta, como una tabla (**`table`**), un índice (**`index`**), una función (**`function`**), una vista (**`view`**), un procedimiento de almacenamiento (**`storage procedure`**) y un disparador o desencadenador (**`trigger`**). En esta unidad nos vamos a centrar en la creación de la base de datos y sus tablas, así como alguno de los índices.

A.CREATE (USE) DATABASE

Empecemos creando la base de datos:

Descripción
La instrucción **`CREATE DATABASE`** se utiliza para crear la base de datos.
Sintaxis básica
`CREATE DATABASE nombre_base_datos;` **`USE nombre_base_datos;`**
Ejemplo
`CREATE DATABASE mcgrawpeliculas;` **`CREATE DATABASE mcgrawinstituto;`** Con cada una de estas instrucciones hemos creado una base de datos en el SGBDR con el nombre que le hemos indicado. Cuando trabajamos en modo de línea de comandos, antes de proceder a manipular una base de datos hemos de indicársela al SGBDR, puesto que puede haber muchas. Ello se consigue con la instrucción **`USE`**. **`USE mcgrawpeliculas;`** Si utilizamos una herramienta gráfica o web, como **MySQL Workbench** o **phpMyAdmin**, simplemente tendremos que clicar sobre ella en la lista de bases de datos disponibles. Esto lo veremos en los casos prácticos ampliados.

Sintaxis de la sentencia CREATE (USE) DATABASE.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

B. CREATE TABLE

Cuando ya estamos *usando* la base de datos elegida, podemos empezar a crear sus tablas.

Descripción

La instrucción **CREATE TABLE** se utiliza para **crear tablas** en una base de datos. Al crear una tabla, debemos especificar los nombres de atributos, sus tipos y todas las demás características que definimos en el esquema relacional durante el diseño lógico.

Sintaxis básica

```
CREATE TABLE nombre_tabla (
<nombre_campo1> <tipo_de_dominio1>[ (tamaño) ]   [NOT NULL]  [índice1],
[<nombre_campo2> <tipo_de_dominio2>[ (tamaño) ] ]  [NOT NULL]  [índice2], ]
[ , ...]  [, CONSTRAINT <restricciones> [, ...] ] ) [ <opciones_de_la_base_
datos> ];
```

Ejemplo:

Vamos a crear la tabla *actor* de la base de datos **mcgrawpeliculas**.

Recordemos que el esquema de la relación *actor* era:

```
actor(#idActor, nombreActor, nacionalidadActor)
```

La implementación en SQL será:

```
CREATE TABLE actor (
  idActor int(11) PRIMARY KEY,
  nombreActor varchar(30) NOT NULL,
  nacionalidadActor smallint(4)unsigned NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_spanish2_ci;
```

Puedes observar que hemos forzado a que no se puedan dejar vacíos los atributos *nombreActor* y *nacionalidadActor* con sendas **restricciones de columna NOT NULL**.

El argumento **PRIMARY KEY** nos crea la clave primaria de la tabla como restricción de columna. Si estuviese compuesta por varios campos, se tendría que escribir como **restricción de tabla**, o sea, tras el paréntesis de antes del **ENGINE**. Así, si la *nacionalidad*, en vez de ser un campo, necesitase más datos, de forma que requiriese de su propia tabla, se escribiría **PRIMARY KEY** (**idActor**, **idNacionalidad**). En tal caso, además, haría falta crear una clave ajena a la nueva tabla *nacionalidad* así:

```
nacionalidadActor smallint(4)unsigned references nacionalidad
(idNacionalidad)
```

Si el campo referenciado, en nuestro caso **idNacionalidad**, no fuera clave primaria en la tabla nacionalidad, nos haría falta crear un índice sobre él de la siguiente manera:

```
index(nacionalidadActor)
```

Por último, para crear una clave ajena compuesta por más de un campo, lo haríamos como restricción de tabla:

```
foreign key (nacionalidadActor, idPais) references nacionalidad
(idNacionalidad, idPais);
```

ENGINE especifica el motor SQL usado para la gestión de la tabla. **CHARSET** y **COLLATE** nos indican el formato con que se van a almacenar los datos.

Ejemplo:

Vamos a crear la tabla *alumnado* de la base de datos **mcgrawinstituto**.

Recordemos su estructura: **alumnado (#NIA, nombreAlumnado, apellidosAlumnado)**

```
CREATE TABLE alumnado (
  NIA int(11) PRIMARY KEY,
  nombreAlumnado varchar(30) COLLATE utf8_spanish2_ci NOT NULL,
  apellidosAlumnado varchar(30) COLLATE utf8_spanish2_ci NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_spanish2_ci;
```

Sintaxis de la sentencia CREATE TABLE.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

C. CREATE INDEX

Cuando ya disponemos de nuestras tablas, podemos crearles índices:

Descripción

La instrucción **CREATE INDEX** se utiliza para **crear índices en nuestras tablas.**

Sintaxis básica

```
CREATE [UNIQUE] INDEX nombre_indice
ON nombre_de_la_tabla(
<nombre_campo1>[DESC][, <nombre_campo2> [DESC][, ...]])
[WITH {PRIMARY | DISALLOW NULL | IGNORE NULL}];
```

El argumento **UNIQUE** impide que el campo restringido acepte valores duplicados.

El argumento **DESC** crea en orden descendente el índice. Si lo omitimos, el índice se crea en orden ascendente.

El argumento **WITH PRIMARY** establece qué campos indexados formarán la clave principal de la tabla. Lo más habitual es que sea un único campo.

El argumento **WITH DISALLOW NULL** no permite valores nulos en el índice.

Ejemplo:

Vamos a crear un índice de clave primaria sobre el campo **idActor** de la tabla *actor* de la base de datos mcgrawpeliculas.

```
CREATE INDEX idActorPK
ON actor (
  idActor) WITH PRIMARY;
```

Ejemplo:

Vamos a crear un índice de unicidad sobre el campo **NIA** de la tabla *alumnado* de la base de datos mcgrawinstituto.

```
CREATE UNIQUE INDEX indice_valor_unico_NIA
ON alumnado (
  NIA
) WITH DISALLOW NULL;
```

Con el argumento **UNIQUE**, nos aseguramos de que no habrá valores duplicados, y con **WITH DISALLOW NULL** también nos aseguraremos de que no exista ningún valor nulo.

Sintaxis de la sentencia CREATE INDEX.

4.2. ALTER TABLE

Cuando ya disponemos de nuestras tablas o índices, puede que necesitemos hacer modificaciones sobre ellos:

Descripción

La instrucción **ALTER** se utiliza para **modificar la estructura de nuestras tablas o índices.**

Sintaxis básica

```
ALTER TABLE nombre_tabla predicado
```

donde el predicado puede ser cualquiera de los siguientes:

```
ADD COLUMN <nombre_campo_Nuevo> <tipo_dominio>[ (tamaño) ] [NOT NULL]
[CONSTRAINT <restriccion>]
ADD CONSTRAINT <restricciones_multicampo>
ALTER COLUMN <nombre_campo_existente> <tipo_dominio>[ (tamaño) ]
DROP COLUMN <nombre_campo>
DROP CONSTRAINT <restriccion>]
```

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

El argumento **ADD** añadirá nuevos campos (**COLUMN**) o restricciones (**CONSTRAINT**).

El argumento **ALTER** modificará un campo existente (**COLUMN**).

La sentencia **DROP** dentro de **ALTER TABLE** permite eliminar campos (**COLUMN**) o restricciones (**CONSTRAINT**).

Ejemplo:

Vamos a crear un nuevo campo **aliasActor** en la tabla *actor* de la base de datos mcgrawpeliculas.

```
ALTER TABLE actor (
ADD COLUMN aliasActor varchar(30);
```

Ejemplo:

Vamos a modificar a 40 caracteres el tamaño del campo **apellidosAlumnado** de la tabla *alumnado* de la base de datos mcgrawinstituto.

```
ALTER TABLE alumnado (
ALTER COLUMN apellidosAlumnado varchar(40) NOT NULL);
```

Sintaxis de la sentencia ALTER.

4.3. DROP TABLE o DROP INDEX

Si queremos eliminar una tabla o índice, con la sentencia **DROP**:

Descripción

La instrucción **DROP TABLE nombre_tabla** se utiliza para eliminar tablas.

La instrucción **DROP INDEX nombre_indice** se utiliza para eliminar índices de nuestras tablas.

Sintaxis básica

```
DROP TABLE nombre_tabla;
DROP INDEX nombre_indice ON nombre_tabla;
DROP INDEX nombre_tabla.nombre_indice;
```

Ejemplo:

Vamos a eliminar la estructura de la tabla *actor* de la base de datos mcgrawpeliculas.

```
DROP TABLE actor;
```

Ejemplo

Vamos a eliminar el índice de unicidad sobre el campo **NIA** de la tabla *alumnado* de la base de datos mcgrawinstituto.

```
DROP INDEX indice_valor_unico_NIA ON alumnado;
```

Otra manera:

```
DROP INDEX alumnado.indice_valor_unico_NIA;
```

Sintaxis de la sentencia DROP.

4.4. RENAME

Si queremos renombrar una tabla o índice, usamos la sentencia **RENAME**:

Descripción

La instrucción **RENAME TABLE nombre_tabla** se utiliza para cambiar el nombre de la tabla.

La instrucción **RENAME INDEX nombre_indice** se utiliza para cambiar el nombre del índice de nuestras tablas.

Sintaxis básica

```
RENAME TABLE nombre_tabla_actual TO nombre_tabla_nuevo;
RENAME INDEX nombre_indice_actual TO nombre_indice_nuevo;
```

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Ejemplo:

Supongamos que, para conseguir inclusión de género, queremos cambiar el nombre de la tabla *actor a artista* en la base de datos **mcgrawpeliculas**.

```
RENAME TABLE actor TO artista;
```

Ejemplo:

Vamos a cambiar el nombre del índice de unicidad sobre el campo **NIA**, que es muy largo, de la tabla *alumnado* de la base de datos **mcgrawinstituto**, que ahora se llamara **NIA_UK** (de **unique key**).

```
RENAME INDEX indice_valor_unico_NIA TO NIA_UK;
```

Sintaxis de la sentencia RENAME.

4.5. TRUNCATE

Esta sentencia solo se utiliza sobre tablas. Su función es eliminar todo su contenido completo. Si bien en un principio esta sentencia parecería ser DML, en realidad es una DDL, ya que borra internamente la estructura de nuestra tabla (**DROP TABLE**) y la vuelve a crear (**CREATE TABLE**).

```
TRUNCATE TABLE nombre_tabla;
```

4.6. COMMENT

El lenguaje SQL también nos permite introducir comentarios dentro nuestras instrucciones.

Los comentarios empiezan con dos guiones consecutivos **--**. Podemos ponerlos al principio de una línea o detrás de la sentencia SQL. Todo texto entre **--** y el final de la línea no se ejecutará y será tratado como un comentario.

4.7. SHOW

Otros comandos interesantes que tenemos que conocer son **SHOW** y **DESCRIBE**. Si no estamos trabajando en un entorno gráfico o web, con la sentencia **SHOW DATABASES** se nos mostrarán las bases de datos a las que tenemos acceso.

Tenemos otra opción, que es **SHOW TABLES;**. Nos muestra las tablas que existen dentro de la base de datos en uso.

4.8. DESCRIBE

Para conocer la estructura de una tabla, disponemos de la sentencia **DESCRIBE nombre_tabla**, que nos muestra la estructura de la tabla **nombre_tabla**.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

5. Importación de datos

Después de haber creado la base de datos **mcgrawinstituto** y sus tablas, podemos incorporar algunos datos en ella.

Recordemos la sentencia SQL que utilizamos para crear la tabla «**alumnado**»:

```
CREATE TABLE `alumnado` (
  `NIA` int(11) NOT NULL,
  `nombreAlumnado` varchar(30) COLLATE utf8_spanish2_ci NOT NULL,
  `apellidosAlumnado` varchar(30) COLLATE utf8_spanish2_ci NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_spanish2_ci;
```

Para importar los datos a las tablas, lo podremos hacer de dos formas distintas:

- Cargando los datos con la sentencia LOAD DATA desde un archivo con los datos de una tabla separados por tabulaciones.
- Cargando los datos con las sentencias INSERT correspondientes almacenadas en un fichero de texto.

5.1. Mediante carga de datos en una tabla

Supongamos que los registros de nuestro alumnado son los descritos en la siguiente tabla.

NIA	nombreAlumnado	apellidosAlumnado
103569	Ana	Martínez
104578	José	García
106574	Alejandro	Jiménez
107896	María	Hernández

Datos para insertar en la tabla «Alumnado» de la base de datos mcgrawinstituto.

Una forma sencilla de insertar estos registros en la tabla *alumnado* es crear un archivo de texto que contenga un registro por línea para cada uno de nuestros alumnos. Posteriormente, cargaremos el contenido del archivo en la tabla con una única sentencia.

Por tanto, debemos crear un archivo de texto «**alumnado.txt**» o «**alumnado.csv**» que contenga un registro por línea con valores separados por tabuladores, cuidando que el orden de las columnas sea el mismo que utilizamos en la sentencia **`CREATE TABLE`**. Para valores que no conozcamos, podemos usar valores nulos (**`NULL`**). Para representar estos valores en nuestro archivo, debemos usar **`\N`**.

Para cargar el contenido del archivo en la tabla *alumnado*, usaremos el siguiente comando:

```
mysql> LOAD DATA INFILE "alumnado.csv" INTO TABLE alumnado;
```

La sentencia **`LOAD DATA`** nos permite especificar cuál es el separador de columnas y el separador de registros. Por defecto, el tabulador es el separador de campos, y el salto de línea es el separador de registros. Si seguimos ese criterio, no necesitamos especificar nada para que la sentencia **`LOAD DATA`** lea correctamente el archivo e inserte los registros.

Desde phpMyAdmin, deberemos acceder a la base de datos y luego pulsar en la pestaña *Importar*. Escogeremos el fichero que queramos importar e indicaremos el carácter que separa los campos y si están encerrados entre comillas. Finalmente, pulsaremos el botón *Continuar* para hacer la importación.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

5.2. Mediante carga de tablas y datos desde un archivo .sql

Otra opción para importar datos en una tabla es mediante la ejecución de un *script* sql (fichero de texto con extensión .sql), que contiene todas las sentencias SQL que reproducen el estado actual de esta.

Puedes ver varios ejemplos descargando los archivos de las bases de datos **McGrawInstituto**, **McGrawPeliculas** y **McGrawBombones**, disponibles en el archivo **GBD_U06_A05_E.zip**, en la sección **Ejemplos para trabajar y analizar**.

Si nos centramos únicamente en insertar datos en una tabla ya creada, la sentencia SQL que debemos utilizar es **INSERT INTO**. A continuación, se muestra un ejemplo para introducir los datos de la tabla:

```
INSERT INTO `alumnado` (`NIA`, `nombreAlumnado`, `apellidosAlumnado`)
VALUES
(103569, 'Ana', 'Martínez'),
(104578, 'José', 'García'),
(106574, 'Alejandro', 'Jiménez'),
(107896, 'María', 'Hernández');
```

IMPORTANTE

La sentencia **INSERT INTO** se estudiará en las siguientes unidades didácticas. Se muestra en este apartado para ver el contenido que contiene el fichero **.sql** que se utilizará para la importación de datos en nuestras bases de datos.

Si esta sentencia la guardamos, junto a otras de creación de la base de datos, creación de tablas, restricciones, etcétera, en un mismo fichero, nos resultará muy sencillo ejecutarlas todas con una única sentencia desde la línea de comandos **Mysql**.

La sentencia es **SOURCE**, y, para importar la base de datos **McGrawInsituto**, quedaría de la siguiente manera:

```
mysql> source \ruta_al_fichero\mcgrawinstituto.sql;
```

Si utilizamos un entorno gráfico, como puede ser Mysql Workbench o phpMyAdmin, podemos importar el fichero de una forma muy sencilla.

Por ejemplo, en MySQL Workbench, escogemos la opción *Archivo > Abrir secuencia de comandos SQL* para abrir la secuencia de comandos SQL. Aparecerá una opción para elegir nuestro archivo .sql. Seleccionamos el archivo SQL «**mcgrawinstituto.sql**», y veremos que se carga en la aplicación. Después de esto, y para finalizar, debemos ejecutar el script.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Unidad 7.

SQL II. DQL

Propiedad de McGraw-Hill®

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

1. SQL / DQL. Consultas simples

El DQL *(Data Query Language)*, lenguaje de consultas de datos, trata las instrucciones **SELECT** en diferentes niveles. En este apartado vamos a comenzar por las más simples.

La siguiente instrucción podría ser un ejemplo de consulta simple:

```
SELECT nombre, apellido FROM alumnos WHERE telef IS NULL;
```

Al ejecutarla, nuestro SGBD mostraría el nombre y el apellido del alumnado del cual no tenemos registrado el número de teléfono.

Al resultado de una **SELECT** se le denomina **conjunto resultante** (*result set*), y podría ser semejante a esto:

nombre	apellido
Lucía	Ferrer
Javier	Llopis
Eva	Torres

Observemos que es como una tabla cuya primera fila es un encabezado con los nombres de las columnas. Debajo se muestran, fila a fila, los valores de dichas columnas para los alumnos que cumplen la condición de filtrado, o sea, la de no tener registrado su teléfono.

1.1. Instrucción SELECT



Con el **comando SELECT** especificamos las columnas cuyos valores queremos mostrar. Con la **cláusula FROM** indicamos las tablas de la base de datos en la que están dichas columnas. En el caso de que queramos mostrar los valores de todas las columnas, en lugar de especificarlas una a una podemos usar un asterisco '*'.

Y si además queremos filtrar el resultado, lo podremos hacer opcionalmente indicando la condición que queremos que cumpla mediante la **cláusula WHERE**. La **cláusula ORDER BY** es también opcional y permite mostrar los resultados en forma ascendente o descendente, según la columna o columnas que indiquemos como criterio de ordenación.

Sintaxis básica

```
SELECT [ {ALL | DISTINCT | DISTINCTROW} ]
    <nombre_campo> [, <nombre_campo> [, ...] ]
FROM <nombre_tabla> [, {<nombre_tabla>} [, ...] ]
[WHERE <condición> [ {AND | OR} <condición> [, ...] ]
[ORDER BY <condición> [ {AND | OR} <condición> [, ...] ] [DESC]
[LIMIT [<posición_inicial>], <número_registros>]
```

Ejemplos

```
SELECT * FROM Director;
```

Nos mostrará el valor de todos los campos de la tabla director sin filtros.

```
SELECT nombreDirector, edadDirector
FROM Director
WHERE salarioDirector > 10000 AND nacionalidadDirector = 'USA';
```

Lista el nombre y la edad de todos los directores de cine cuyo salario sea superior a 10.000 y, además, sean estadounidenses. El carácter ';' marca el final de la consulta, lo cual nos permite partir la instrucción en varias líneas.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

La cláusula opcional **`LIMIT [<posición_inicial>], <número_registros>`** es muy práctica para aplicaciones de escritorio o web porque el resultado de una consulta puede contener centenares de filas que no caben en la pantalla. **`LIMIT`** nos permite ir recuperando los registros de la consulta en bloques con el número de registros que indiquemos.

1.2. Tipos de operadores de SQL

En las instrucciones SQL vamos a poder crear expresiones complejas valiéndonos de operadores que nos den la potencia requerida para evaluarlas adecuadamente. Veamos los diferentes tipos y los símbolos o palabras reservadas que podemos utilizar.

A. OPERADORES ARITMÉTICOS

Los operadores aritméticos solo trabajan con números enteros. Supongamos que vamos a utilizar la variable **`operador1`**, que vale **`23`**, y la variable **`operador2`**, que vale **`11`**; entonces, para cada operador obtendremos los siguientes resultados:

Operador	Descripción	Ejemplo de uso y resultado
Sumar + **`(add)`**	Suma los valores a ambos lados del operador.	operador1 + operador2 = 34
Restar – **`(subtract)`**	Resta el valor de la derecha del valor de la izquierda.	operador1 - operador2 = 12
Multiplicar * **`(multiply)`**	Multiplica los valores a ambos lados del operador.	operador1 * operador2 = 253
Dividir / **`(divide)`**	Divide el operando de la izquierda entre el operando de la derecha.	operador1 / operador2 = 2
MOD o % **`(reminder)`**	Devuelve el resto de la división entera entre el operando de la izquierda y el operando de la derecha.	operador1 % operador2 = 1

Por ejemplo, ejecutar **`SELECT MOD(30,9)`**; o ejecutar **`SELECT 30%9`**; son equivalentes y en ambos casos el resultado será un 3; es decir, que el resultado es el resto de la división entera entre 30 y 9.

B. OPERADORES DE COMPARACIÓN

Todos los operadores de comparación dan como resultado CIERTO o FALSO. Veamos su significado y el resultado para los mismos valores de variables: op1=23 y op2=11.

Operador	Descripción	Ejemplo y resultado
Menor < **`(less than)`**	Comprueba si el valor del operando izquierdo es menor que el valor del operando derecho.	(**`op1 < op2`**) es falso
Menor o igual <= **`(less than or equal to)`**	Comprueba si el valor del operando izquierdo es menor o igual que el valor del operando derecho.	(**`op1 <= op2`**) es falso
Mayor > **`(greater than)`**	Comprueba si el valor del operando izquierdo es mayor que el valor del operando derecho.	(**`op1 > op2`**) es cierto
Mayor o igual >= **`(greater than or equal to)`**	Comprueba si el valor del operando izquierdo es mayor o igual que el valor del operando derecho.	(**`op1 >= op2`**) es cierto
igual = **`(equal to)`**	Comprueba si los valores de ambos operadores son iguales.	(**`op1 = op2`**) es falso
Distinto <> **`(distinct)`**	Comprueba si los valores de ambos operadores son distintos.	(**`op1 <> op2`**) es cierto

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Estos operadores pueden ir precedidos del **operador de negación**, que se representa con el signo de exclamación '**!**'. Así pues, **!=** significa **"no es igual"**; **!<** significa **"no es menor que"** y **!>** significa **"no es mayor que"**.

Los siguientes operadores propios de SQL, no tan conocidos, también son de comparación, en este caso sobre valores individuales:

Operador	Descripción
IS NULL	Compara un valor con el valor nulo **NULL**.
LIKE	Compara un valor con un literal. Permite el uso de operadores comodín.

Y estos operadores se utilizan para conjuntos de valores:

Operador	Descripción
ALL	Evalúa si cierta condición se cumple para **todos** los elementos de un conjunto dado.
ANY	Evalúa si cierta condición se cumple para **algún** elemento de un conjunto dado.
BETWEEN	Evalúa si un elemento está dentro de un rango de valores.
EXISTS	Evalúa si existe algún elemento en un conjunto dado.
IN	Evalúa si un elemento pertenece a un conjunto dado.

C. OPERADORES COMODINES

Cuando queremos buscar datos filtrándolos con la cláusula **WHERE** dentro de un rango de valores alfanuméricos podemos utilizar estos operadores que hacen la función de comodín, son el símbolo del tanto por ciento **%** o el carácter del guion bajo **_**.

Operador	Descripción
Símbolo del porcentaje **%** **(percent symbol)**	Puede coincidir con ninguno, uno solo o varios caracteres.
Símbolo del guion bajo **_** **(underscore)**	Solo puede coincidir con un único carácter.

Hay que comentar que el SGBDR Microsoft Access utiliza el carácter asterisco *****, en lugar del tanto por ciento **%**, y el carácter del símbolo para cerrar la interrogación **?**, en lugar del guion bajo **_** (tal como sucede también en consolas o terminales CLI, como el CMD de MS-DOS, el PowerShell de Windows o el Shell bash de GNU/Linux). Hay que recordar que en SQL el ***** sirve para mostrar todos los campos de la/s tabla/s sobre las que vayamos a hacer la consulta y se coloca entre el comando **SELECT** y la cláusula **FROM**.

Algunos ejemplos buscando el valor VALENCIA o derivados utilizando los comodines:

```
SELECT * FROM Hotel WHERE provinciaHotel LIKE 'VAL%';
SELECT * FROM Hotel WHERE provinciaHotel LIKE '%ALENC%';
SELECT * FROM Hotel WHERE provinciaHotel LIKE 'VALENCI_';
SELECT * FROM Hotel WHERE provinciaHotel LIKE '_ALENCIA';
SELECT * FROM Hotel WHERE provinciaHotel LIKE '_ALENCI_';
```

D. OPERADORES LÓGICOS

En la siguiente tabla os explicamos para qué sirven.

Operador	Descripción
AND - &&	Sirve para poder poner múltiples condiciones en la cláusula **WHERE**. Da como resultado **true (verdadero)** si TODAS las condiciones son verdaderas.
NOT - !	Sirve para invertir el valor del operador lógico al que se le aplique.
OR - ||	Sirve para poder poner múltiples condiciones en la cláusula **WHERE**. Da como resultado **true (verdadero)** si CUALQUIERA de las condiciones es **verdadera**.

Por ejemplo, ejecutar **SELECT true && true**; o ejecutar **SELECT true AND true**; son equivalentes y en ambos casos el resultado es cierto **(true)**.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

1.3. Campos calculados

Podemos utilizar los operadores de SQL para crear nuevos campos calculados a partir de las columnas de nuestras tablas. Por ejemplo, si sabemos que en la tabla EMPLEADO de la base de datos MCGRAWHOTELES tenemos una columna con el sueldo mensual de los empleados y queremos saber el sueldo total bruto del año en 12 o 14 pagas, podemos pedirle que nos lo calcule directamente en la consulta **SELECT** con campos calculados.

```
SELECT nombreEmp, ApellidoEmp, sueldoEmp, sueldoEmp*12, sueldoEmp*14
FROM empleado;
```

> Dispones de las bases de datos en la sección Ejemplos para trabajar y analizar de la unidad, descargando el archivo GBD_U07_01.zip.

Y si no queremos obtener todos los empleados, sino los que más cobran o los que están en una franja de sueldos, podemos aplicarle el filtro correspondiente con la cláusula **WHERE**. Por ejemplo, los que cobren más de 10.000 € en 12 pagas o menos de 16.000 € en 14 pagas.

```
SELECT nombreEmp, ApellidoEmp, sueldoEmp, sueldoEmp*12, sueldoEmp*14
FROM empleado WHERE sueldoEmp*12 > 10000 AND sueldoEmp * 14 < 16000;
```

1.4. Consultas ordenadas

Normalmente, el conjunto resultante mostrará los valores según el orden en que fueron almacenados. Para obtener los resultados ordenados, bien de forma ascendente o de forma descendente, utilizaremos la cláusula **ORDER BY**. Por ejemplo, vamos a utilizar la consulta anterior, pero ordenándola alfabéticamente según los apellidos de los empleados.

```
SELECT nombreEmp, ApellidoEmp, sueldoEmp, sueldoEmp*12, sueldoEmp*14
FROM empleado ORDER BY ApellidoEmp;
```

Pero en ciertas ocasiones, por ejemplo, cuando son cifras o números, puede que nos interese ordenar de mayor a menor. Para ello añadiremos al final la opción DESC.

```
SELECT nombreEmp, ApellidoEmp, sueldoEmp, sueldoEmp*12, sueldoEmp*14
FROM empleado ORDER BY sueldoEmp DESC;
```

A veces nos hará falta ordenar por más de una columna; en estos casos, pondremos detrás del **ORDER BY** las columnas que formen parte de la ordenación considerando su precedencia a la hora de aplicar el criterio.

1.5. Consultas con cabeceras

Si los resultados de nuestras consultas van a servir para mostrárselos a otras personas, igual nos interesa renombrar los títulos de las columnas para que se interprete mejor el conjunto resultante. Para conseguirlo vamos a crear un *alias* con la opción **AS** junto con el nuevo título de la columna entre comillas simples.

```
SELECT nombreEmp, ApellidoEmp, sueldoEmp AS 'Sueldo mensual', sueldoEmp *
12 AS 'Sueldo anual', sueldoEmp * 14 AS 'Sueldo anual con pagas extras'
FROM empleado;
```

1.6. Consultas sin valores repetidos

Si en el conjunto resultante hay valores duplicados, pero solo queremos que aparezcan una vez, podemos utilizar la opción **DISTINCT**. Por ejemplo, imaginemos que queremos hacer una lista con cuántos salarios diferentes reciben los empleados, pero en la que cada salario solo aparezca una vez. Es decir, si hay varios empleados que cobran 900 € o 1000 €, con esta opción solo aparece 900 € y 1000 € una vez, en lugar de varias veces.

```
SELECT DISTINCT sueldoEmp
FROM empleado ORDER BY sueldoEmp DESC;
```

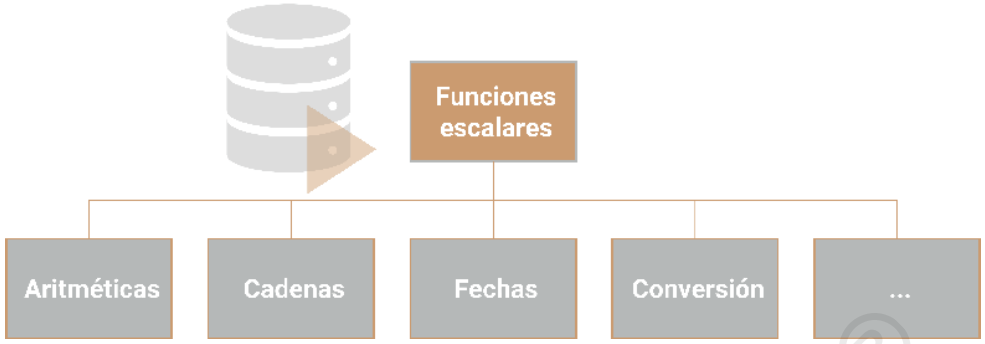
Por último, hay que comentar que podemos utilizar **DISTINCTROW** en lugar de **DISTINCT** si lo que queremos es que no aparezcan repetidas filas completas iguales.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

2. SQL/DQL. Uso de funciones propias de SQL

En este apartado nos vamos a centrar en presentaros las funciones más utilizadas para cada uno de estos dominios de valores básicos, incluso las relacionadas con JSON. Su aprendizaje nos puede ayudar a gestionar mejor nuestros datos.



2.1. Funciones sobre cadenas de texto

Las funciones que gestionan y manejan cadenas de texto nos van a permitir hacer casi cualquier cosa con estos campos de tipo **`char`** o **`varchar`**. Vamos a ver qué podemos hacer con estas:

Función	Descripción
`ASCII`	Nos devuelve el valor de la tabla ASCII del carácter que indiquemos.
`CHAR_LENGTH`	Devuelve la longitud de una cadena contando sus caracteres.
`CONCAT`	Nos permite unir o concatenar dos tiras de caracteres en una.
`FORMAT`	Nos permite darle formato a una cadena de caracteres que contenga un valor numérico al que le queramos dar formato, como por ejemplo "**`#.###.###,##`**".
`INSERT`	Nos permite insertar una cadena dentro de otra cadena en la posición que le indiquemos y para un cierto número de caracteres.
`LEFT`	Nos permite extraer varios caracteres de una cadena (por la izquierda).
`LENGTH`	Devuelve la longitud de una cadena.
`LOCATE`	Devuelve la posición de la primera aparición de una subcadena en una cadena.
`LOWER`	Nos permite convertir una cadena a minúsculas.
`LTRIM`	Quita espacios en blanco iniciales de una cadena (por la izquierda).
`MID`	Extrae una subcadena de una cadena (comenzando en cualquier posición).
`POSITION`	Devuelve la posición de la primera aparición de una subcadena en una cadena.
`REPLACE`	Reemplaza todas las apariciones de una subcadena dentro de una cadena, con una nueva subcadena.
`REVERSE`	Invierte una cadena.
`RIGHT`	Nos permite extraer varios caracteres de una cadena (por la derecha).
`RTRIM`	Quita espacios en blanco iniciales de una cadena (por la derecha).
`SPACE`	Devuelve el número especificado de espacio en blanco que contiene.
`STRCMP`	Compara dos cadenas de caracteres.
`SUBSTR`	Extrae una subcadena de una cadena (comenzando en cualquier posición).
`TRIM`	Quita espacios en blanco iniciales de una cadena (por ambos extremos).
`UPPER`	Nos permite convertir una cadena a mayúsculas.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

2.2. Funciones sobre valores numéricos

Las funciones que gestionan y manejan valores numéricos nos van a permitir hacer casi cualquier cosa con estos campos de tipo entero o con decimales. Vamos a ver qué podemos hacer con estas:

Función	Descripción
ABS	Nos devolverá el valor absoluto de un número.
SIGN	Nos devolverá el signo de un valor numérico.
RAND	Nos devolverá un valor numérico aleatorio.
ROUND	Redondea un número a un número especificado de decimales.
TRUNCATE	Trunca un número al número especificado de decimales.

También dispone de todo tipo de funciones relacionadas con el cálculo de la raíz cuadrada (**SQRT**) o la potencia de un número elevado a otro (**POWER** o **POW**); o cualquiera de las relacionadas con el cálculo de senos (**SIN**), cosenos (**COS**) o la tangente (**TAN**); o convertir grados a radianes (**RADIANS**) o el valor de constantes como **PI**. Existen muchas más, que podemos ir descubriendo.

2.3. Funciones agregadas

Aunque también son funciones numéricas, en SQL tienen su propia categoría, ya que como veremos se utilizan con una cláusula especial denominada GROUP BY que nos permitirá agrupar conjuntos o rangos de valores para obtener resultados con estas funciones. Veremos ejemplos de su uso en el apartado 3.1.

Función	Descripción
AVG	Nos devolverá la media aritmética del rango de valores indicado.
COUNT	Nos devolverá el número de elementos del rango de valores indicado.
MAX	Nos devolverá el valor máximo del rango de valores indicado.
MIN	Nos devolverá el valor mínimo del rango de valores indicado.
SUM	Nos devolverá el valor sumatorio del rango de valores indicado.

2.4. Funciones de fechas de SQL

En SQL es interesante utilizar funciones específicas de fechas para realizar operaciones con ellas dadas sus peculiaridades. Como sabemos, todos los meses no tienen el mismo número de días. Lo mismo pasa con los años.

Vamos a ver algunas de ellas.

Función	Descripción
ADDDATE	Agrega un intervalo de fechas a una fecha y, a continuación, devuelve la fecha.
ADDTIME	Agrega un intervalo de horas a una fecha y, a continuación, devuelve la hora y fecha.
CURRENT_DATE	Nos devuelve la fecha de hoy.
CURRENT_TIME	Nos devuelve la hora actual.
DATE	Nos devuelve la fecha de una expresión **DATETIME**.
DATEDIFF	Sirve para poder averiguar la diferencia entre dos fechas.
DATE_ADD	Agrega un intervalo de fechas a una fecha y, a continuación, devuelve la fecha. Es idéntica a ADDDATE.
DATE_FORMAT	Sirve para formatear la fecha según como nos pueda interesar.
DATE_SUB	Resta un intervalo de fechas a una fecha y, a continuación, devuelve la fecha.
DAY	Nos devuelve el valor numérico del día del mes.
DAYNAME	Nos devuelve el nombre del día de la semana de una fecha dada.
HOUR	Nos devuelve el valor de la hora.
MINUTE	Nos devuelve el valor de los minutos.
MONTH	Nos devuelve el valor del mes.
MONTHNAME	Nos devuelve el nombre del mes.

Propiedad de McGraw-Hill®

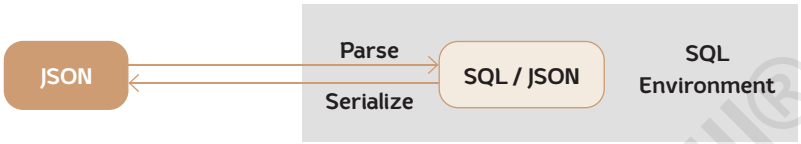
Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

NOW	Nos devuelve la fecha y hora actual en formato '**2022-02-22 22:22**'.
SECOND	Nos devuelve el valor de los segundos.
SYSDATE	Nos devuelve la fecha actual proporcionada por el sistema operativo.
TIME	Nos devuelve la hora de una fecha.
TIME_FORMAT	Da formato a una expresión **DATETIME** según el patrón especificado.
TIMEDIFF	Sirve para poder averiguar la diferencia entre dos horas.
YEAR	Nos devuelve el año de la fecha.

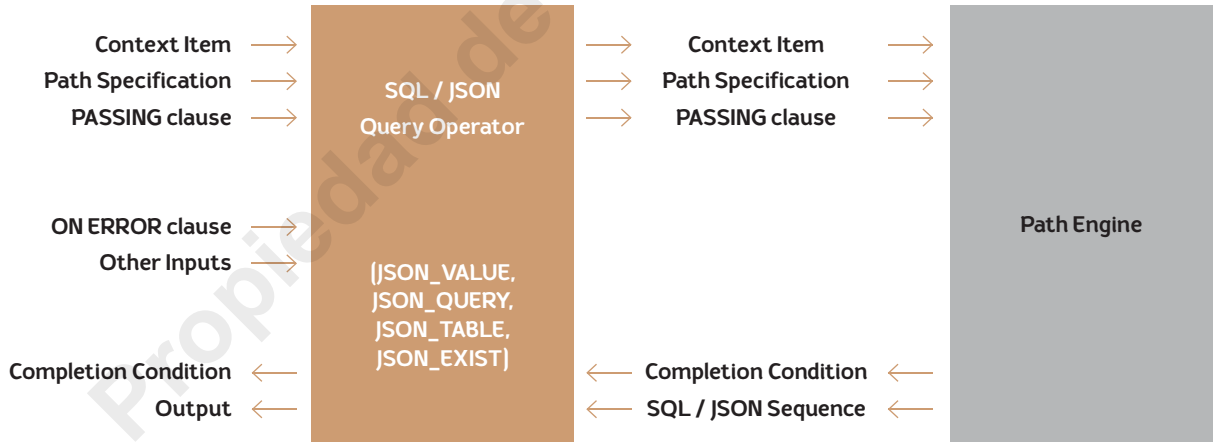
2.5. Funciones JSON de SQL

Desde 2016, el estándar SQL ha incorporado funciones especiales para convertir información en formato JSON a SQL o viceversa, a través de esta nueva pasarela SQL/JSON.



Esta nueva biblioteca SQL/JSON que añade nuevas funciones a SQL consiste básicamente en operadores de consulta (**Query Operator**) que importan o exportan valores, matrices u objetos desde una cadena JSON. Vamos a ver algunas de ellas.

Función	Descripción
JSON_VALUE	Nos permite extraer un valor escalar.
JSON_QUERY	Nos permite extraer un objeto o una matriz.
JSON_TABLE	Nos devuelve una tabla.
JSON_EXISTS	Nos permite saber si existe una expresión JSON.



Con estas funciones y algunas más que cada fabricante irá implementando vamos a ser capaces de importar o exportar nuestros datos en ambos sentidos.

2.6. Funciones avanzadas de SQL

En SQL es interesante utilizar funciones avanzadas para realizar conversiones o, por ejemplo, saber el nombre de la base de datos con la que estamos trabajando dentro del SGBDR.

Función	Descripción
BIN	Devuelve la representación binaria de un número.
CAST	Convierte un valor (de cualquier tipo) en otro tipo de datos especificado.
CONV	Convierte un número de un sistema base numérico a otro.
CONVERT	Convierte un valor en el tipo de datos o conjunto de caracteres especificado.
DATABASE	Devuelve el nombre de la base de datos actual.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

2.7. Funciones de programación en SQL

Otro bloque de funciones que pueden ser útiles son las relacionadas con la posibilidad de programar ciertas condiciones en nuestras sentencias.

Función	Descripción
CASE	Permite crear una selección múltiple.
IF	Devuelve un valor si una condición es **TRUE**, u otro valor si una condición es **FALSE**.
IFNULL	Devolver un valor si la expresión es **NULL**. De lo contrario devuelve la expresión.
ISNULL	Devuelve 1 o 0 dependiendo de si una expresión es **NULL**.
NULLIF	Compara dos expresiones y devuelve **NULL** si son iguales. De lo contrario, se devuelve la primera expresión.

2.8. Funciones propias de MySQL

A veces vamos a tener que recurrir a funciones propias del SGBDR que estemos utilizando, en nuestro caso MySQL o cualquiera de sus variantes, como MariaDB o AWS Aurora.

Función	Descripción
CURRENT_USER	Devuelve el nombre de usuario y el nombre de host de la cuenta MySQL que el servidor utilizó para autenticar al cliente actual.
LAST_INSERT_ID	Devuelve el valor que le ha correspondido a la última columna de tipo AU-TO_INCRE-MENT tras una inserción o modificación.
SESSION_USER	Devuelve el nombre de usuario y el nombre de host actuales de MySQL de la se-sión.
SYSTEM_USER	Devuelve el nombre de usuario y el nombre de host actuales de MySQL del sistema.
USER	Devuelve el nombre de usuario y el nombre de host actuales de MySQL.
VERSION	Devuelve la versión actual de MySQL.

Como siempre, cada fabricante de SGBDR, en su implementación ampliada de su SQL no estándar, dispone de sus propias funciones, por lo que al final tendremos que consultar el manual de ayuda.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

3. SQL/DQL. Consultas con valores de resumen

En este apartado nos vamos a centrar en utilizar las funciones de agrupación que nos permitan calcular valores concretos según lo que nos interese. Veremos las consultas agrupadas, también llamadas de totales, que son soportadas por el estándar SQL, y también os enseñaremos a crear consultas de referencias cruzadas.

3.1. Consultas agrupadas o de totales

Vamos a necesitar ampliar la sintaxis básica de nuestra orden **SELECT** para poder hacer consultas de este tipo. En concreto, añadimos la cláusula **GROUP BY** para poder agrupar por el campo o campos que pongamos aquí. Y también añade la cláusula **HAVING** para poder filtrar valores según la condición que le indiquemos.

Comando SELECT + Cláusula FROM + Cláusula WHERE + Cláusula GROUP BY + Cláusula HAVING + Cláusula ORDER BY

Veamos la sintaxis ampliada que vamos a incorporar para crear estas consultas de totales.

Sintaxis ampliada

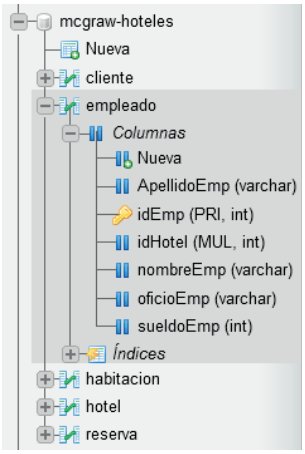
```
SELECT [ {ALL | DISTINCT | DISTINCTROW } ]
    <nombre_campo> [ , <nombre_campo> [ , ... ] ]
FROM <nombre_tabla>  [ , {<nombre_tabla>} [ , ...] ]
[WHERE <condición> [ {AND | OR} <condición> [ , ...] ]
[GROUP BY <nombre_campo> [ , <nombre_campo> [ , ...] ]
[HAVING <condición> [ {AND | OR} <condición> [ , ...] ]
[ORDER BY <condición> [ {AND | OR} <condición> [ , ...] ] [DESC]
```

Las funciones agregadas (que se presentaron en el apartado 2.3) operan sobre un grupo de filas y devuelven un valor numérico. Como hemos visto, son:

- El promedio (**avg**, **average**): muestra la media aritmética.
- contar (**count**, **cómputo**): muestra un recuento de los valores nulos.
- suma total (**sum**): muestra el sumatorio.
- máximo (**max**, **maximum**): muestra el valor máximo del conjunto.
- mínimo (**min**, **minimal**): muestra el valor mínimo del rango.



La siguiente imagen ilustra la estructura de una base de datos MySQL llamada MCGRAWHOTELES.



#	Nombre	Tipo	Cotejamiento	Atributos	Nulo	Predeterminado	Comentarios	Extra
1	idEmp	int(11)			No	Ninguna		AUTO_INCREMENT
2	nombreEmp	varchar(25)	utf8_spanish_ci		No	Ninguna		
3	ApellidoEmp	varchar(25)	utf8_spanish_ci		No	Ninguna		
4	oficioEmp	varchar(25)	utf8_spanish2_ci		No	Ninguna		
5	sueldoEmp	int(11)			No	Ninguna		
6	idHotel	int(11)			No	Ninguna		

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Por ejemplo, vamos a ver cuál es el sueldo más elevado de los empleados:

```
SELECT MAX(sueldoEmp) FROM empleado;
```

Si quisiéramos saber qué empleado cuyo apellido comienza en 'Len' es el que más cobra haríamos un filtrado de la siguiente manera:

```
SELECT MAX(sueldoEmp) FROM empleado WHERE apellidoEmp LIKE "Len%";
```

Este **SELECT** nos devolverá el máximo de la columna **sueldoEmp** de los empleados cuyo apellido comienza por "Len".

Veamos el esquema de la base de datos **MCGRAWCINES**:

> Dispones de las bases de datos en la sección Ejemplos para trabajar y analizar de la unidad, descargando el archivo **GBD_U07_01.zip**.



Para saber cuántas poblaciones tienen cine realizaríamos la siguiente consulta:

```
SELECT COUNT(DISTINCT poblacionCine) FROM cine;
```

Si queremos obtener valores estadísticos que correspondan no a filas individuales sino a un grupo de filas, deberemos utilizar la cláusula **GROUP BY**.

Para saber cuántas salas hay en total en todos los cines, lanzaríamos esta consulta:

```
SELECT COUNT(*) FROM sala;
```

Y si solamente queremos saber las que hay en el cine 1:

```
SELECT COUNT(*) FROM sala WHERE idCine=1;
```

Pero si queremos saber cuántas salas hay en cada cine, es cuando aplicaremos la agrupación:

```
SELECT idCine, COUNT(*) FROM sala GROUP BY idCine;
```

En cada cine hay una sala con un mayor número de butacas. Pues bien, la siguiente consulta muestra un ranking ascendente con el número de butacas de la sala con mayor capacidad de cada cine.

Con el modificador **DESC**, podríamos indicar que queremos que aparezcan de mayor a menor capacidad puesto que, por defecto, el orden es ascendente.

```
SELECT idCine, MAX(butacasSala) FROM sala GROUP BY idCine ORDER BY
MAX(butacasSala);
```

Y si queremos aplicar un filtro en la agrupación, existe otra cláusula llamada **HAVING** (recordemos que la cláusula **WHERE** filtraba filas, no agrupaciones de filas). Por ejemplo, para la consulta *"Muestra los códigos de los cines, junto a cuántas salas tiene cada cine, pero solo aquellas con más de 3 cines"* hemos de tener en cuenta que la condición a filtrar, en este caso **COUNT(*)**, también deberá formar parte de la consulta **SELECT** para poder utilizar ese campo en el **HAVING**.

```
SELECT idCine, COUNT(*) FROM sala GROUP BY idCine HAVING COUNT(*) >= 3;
```

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Para comprender mejor la diferencia entre **WHERE** y **HAVING** realizaremos otra consulta en la que aparezcan las dos. Por ejemplo, si queremos mostrar los códigos de cine que tienen más de 100 asientos, pero solo de los cines que tienen más de 2 salas, podríamos obtenerlo con esta consulta:

```
SELECT idCine, COUNT(*) FROM sala WHERE butacasSala > 50
                    GROUP BY idCine HAVING COUNT(*) > 2;
```

3.2. Consultas de referencias cruzadas

Otro tipo de consulta especial que se basa en la utilización de las consultas de totales que acabamos de ver son las consultas de referencias cruzadas. Aquí vamos a mostrar diferentes valores que dependerán del campo o campos que indiquemos en la parte del **PIVOT**.

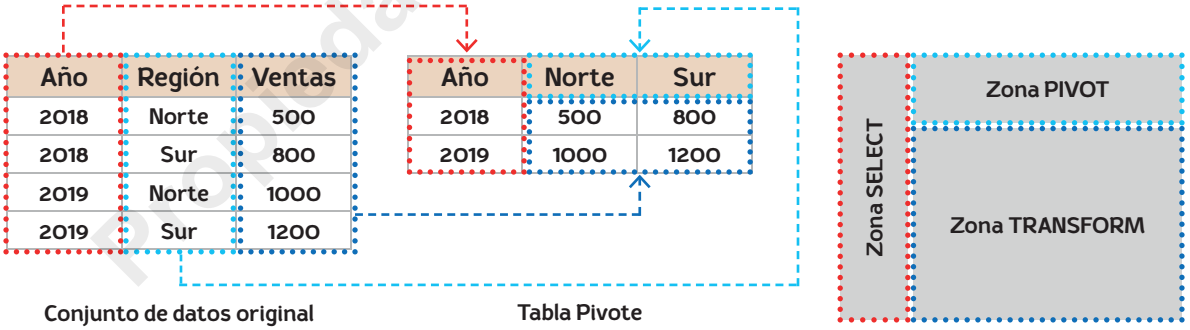
No están contempladas en el SQL estándar, por lo que de momento no están implementadas en SGBDR como MySQL y sus variantes, pero se pueden utilizar en otros fabricantes como Microsoft, en su producto Access o SQL Server, o en Oracle Database, entre otros. Por eso, pensamos que debemos conocerlas.



Sintaxis ampliada

```
TRANSFORM función(<nombre_campo>) [ , <nombre_campo> [ , ... ] ] AS Titulo
SELECT [ {ALL | DISTINCT | DISTINCTROW } ]
    <nombre_campo> [ , <nombre_campo> [ , ... ] ]
FROM <nombre_tabla>  [ , {<nombre_tabla>} [ , ...] ]
[WHERE <condición> [ {AND | OR} <condición> [ , ...] ]
[GROUP BY <nombre_campo> [ , <nombre_campo> [ , ...] ]
[HAVING <condición> [ {AND | OR} <condición> [ , ...] ]
[ORDER BY <condición> [ {AND | OR} <condición> [ , ...] ] [DESC]
PIVOT <nombre_campo> [ , <nombre_campo> [ , ... ] ]
```

El conjunto de datos original se transformará para mostrarlos de otra manera, según le indiquemos.



Este tipo de consultas especiales son como las tablas dinámicas que se pueden crear en una hoja de cálculo. Es otro recurso a utilizar si nos lo permite el SGBDR que estemos utilizando. Son muy utilizadas en la creación de paneles de control de datos (**dashboard**) para analizar grandes cantidades de datos (**Big data** o **Smart data**).

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

4. SQL/DQL. Consultas sobre varias tablas

Vamos a ver cómo crear consultas que combinen valores de varias tablas.

4.1. Consultas multitabla

La potencia del modelo relacional aparece cuando el conjunto resultante muestra los valores relacionando columnas de varias tablas. Lo comprobaremos con la base de datos **MCGRAWCINES** del material digital de esta unidad.

> Dispones de las bases de datos en la sección Ejemplos para trabajar y analizar de la unidad, descargando el archivo **GBD_U07_01.zip**.

Si en la tabla **CINE** tenemos los valores de cada cine y en la tabla **SALA** tenemos una columna **idCine** que nos indica a qué cine pertenece cada sala, sería interesante mostrar los valores de los cines y de sus salas combinando ambas tablas:

```
SELECT sala.idCine, cine.nombreCine, cine.poblaciónCine,
                sala.idSala, sala.butacasSala FROM cine, sala;
```

¿Qué pensáis? ¿Habrá funcionado correctamente esta consulta? Trata de ejecutarla y mira qué es lo que pasa.

La respuesta es **NO**. Hay un problema. Para solucionarlo necesitamos indicar cuáles son las columnas por las que se combinan las tablas. En este ejemplo, la columna **idCine** de ambas tablas:

```
SELECT cine.idCine AS "cine-cine", cine.nombreCine, cine.poblaciónCine,
       sala.idCine AS "cine-sala", sala.idSala, sala.butacasSala FROM cine,
sala;
```

Parecería que ya funciona, pues no da error, pero si observamos el conjunto resultante veremos que ha efectuado un **producto cartesiano**, combinando cada cine con todas las salas, sean suyas o no.

La solución pasa por añadir, como condición de filtrado, que el **idCine** de la tabla de **CINE** y el **idCine** de la tabla **SALA** coincidan. Lo podemos conseguir añadiendo un **WHERE**:

```
SELECT cine.idCine AS "Cine-cine", sala.idCine AS "cine-sala",
cine.nombreCine, cine.poblaciónCine, sala.idSala, sala.butacasSala
FROM cine, sala
WHERE cine.idCine = sala.idCine;
```

Si no hemos modificado las tablas originales, el conjunto resultante será:

Cine-cine	cine-sala	nombreCine	poblaciónCine	idSala	butacasSala
1	1	Colón	Alzira	1	100
1	1	Colón	Alzira	2	120
1	1	Colón	Alzira	3	111
2	2	Cervantes	Cullera	1	188

Resultado de la consulta multitabla relacionada por claves.

Sin embargo, la solución óptima es utilizar *combinaciones internas* (**LEFT JOIN**, **INNER JOIN** y **RIGHT JOIN**) en lugar del filtrado con **WHERE**. Para ello ampliamos la sintaxis básica del **SELECT** añadiendo la cláusula **JOIN..ON**. Con el **JOIN** indicamos qué tipo de combinación interna queremos realizar y con el **ON** indicamos las columnas a combinar, de forma similar a como lo hicimos antes en el **WHERE**.



Orden en que se han de escribir las cláusulas dentro de la orden SELECT.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Sintaxis completa

```
SELECT [ {ALL | DISTINCT} ]
    <nombre_campo> [ , <nombre_campo> [ , ... ] ]
FROM tabla_A  [{LEFT | INNER | RIGHT}
     [JOIN tabla_B ON tabla_A.idNombreCampo =  tabla_B.idNombreCampo]
[WHERE <condición> [ {AND | OR} <condición> [ , ...] ]
[GROUP BY <nombre_campo> [ , <nombre_campo> [ , ...] ]
[HAVING <condición> [ {AND | OR} <condición> [ , ...] ]
[ORDER BY <condición> [ {AND | OR} <condición> [ , ...] ] [DESC]
```

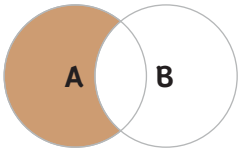
Sintaxis completa de la orden SELECT con uniones internas de SQL.

De los tres tipos de **JOIN** (**LEFT**, **INNER** y **RIGHT**), el más habitual es la composición interna **INNER JOIN**, o su subtipo llamado **NATURAL JOIN**, que nos ahorra indicar el **ON** cuando las columnas que queremos combinar tienen un nombre idéntico.

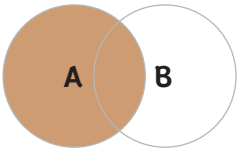
LEFT o **RIGHT** admiten la cláusula **OUTER** a continuación, ya que se trata de composiciones externas. La opción **FULL** [**OUTER**] **JOIN** permite obtener la combinación en ambos sentidos.

Tanto con la opción **LEFT** como con la **RIGHT** y con la **FULL** obtendremos, además de las filas combinadas, aquellas que no combinan, las cuales se rellenarán con **NULL**.

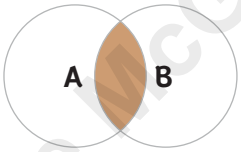
Y precisamente nos podemos servir del filtrado por valores **NULL** para saber los valores de cada tabla que no combinan con la otra tabla. Esto es muy práctico para conocer, por ejemplo, qué **CINES** tenemos sin **SALAS** (o qué **SALAS** existen sin **CINE**).



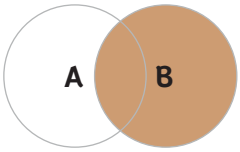

```
SELECT <select list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key=B.Key
WHERE B.Key IS NULL
```



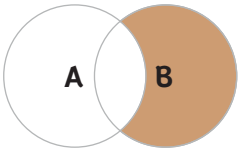

```
SELECT <select list>
FROM TableA
LEFT JOIN TableB
ON A.Key=B.Key
```




```
SELECT <select list>
FROM TableA A
INNER JOIN TableB B
ON A.Key=B.Key
```




```
SELECT <select list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key=B.Key
```




```
SELECT <select list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key=B.Key
WHERE A.Key IS NULL
```

Diferentes uniones internas que podemos utilizar con SQL JOIN.

La consulta anterior, optimizada usando **JOIN**, quedaría de la siguiente manera:

```
SELECT cine.idCine AS "cine-Cine", sala.idCine AS "cine-Sala",
cine.nombreCine, cine.poblaciónCine, sala.idSala, sala.butacasSala
FROM cine INNER JOIN sala ON cine.idCine = sala.idCine;
```

Como en este caso las columnas combinadas tienen idéntico nombre (**idCine**), podemos usar **NATURAL JOIN**:

```
SELECT cine.idCine AS "cine-Cine", sala.idCine AS "cine-Sala",
cine.nombreCine, cine.poblacionCine, sala.idSala, sala.butacasSala
FROM cine NATURAL JOIN sala;
```

Y como muestra todas las columnas de las dos tablas combinadas, equivale a:

```
SELECT * FROM cine NATURAL JOIN sala;
```

4.2. Consultas multitabla filtradas

Si a una consulta multitabla le añadimos la posibilidad de filtrar por el criterio que nos interese, podremos, por ejemplo, ver todos los cines SOLO de Cullera que tengan butacas dadas de alta en la tabla SALA. Cuando un nombre de una columna coincida en más de una tabla deberemos anteponerle el nombre de la tabla separado por un punto para especificar a cuál de ellas nos referimos. A menudo esto se produce cuando coincide el nombre de la columna de la clave primaria de una tabla con el nombre de la clave ajena de otra.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

He aquí un ejemplo para la columna **idCine**:

```
SELECT sala.idCine, nombreCine, poblacionCine, sala.idSala, butacasSala
FROM cine INNER JOIN sala ON cine.idCine = sala.idCine WHERE cine.
poblacionCine = "Cullera";
```

idCine	nombreCine	poblaciónCine	idCine	idSala	butacasSala
2	Cervantes	Cullera	2	1	188

Resultado de la consulta de Cines con Salas en Cullera desde phpMyAdmin.

Si ahora quisiéramos ver todos los cines de Cullera que tenemos dados de alta en nuestra tabla de cines, cambiaríamos la unión interna **INNER** por la de **LEFT**.

```
SELECT cine.idCine, nombreCine, poblacionCine, sala.idCine, sala.idSala,
butacasSala
FROM cine LEFT JOIN sala ON cine.idCine = sala.idCine WHERE cine.
poblacionCine = "Cullera"
```

idCine	nombreCine	poblaciónCine	idCine	idSala	butacasSala
2	Cervantes	Cullera	2	1	188
22	Cines Victoria	Cullera	NULL	NULL	NULL

Resultado de la consulta de todos los CINES de Cullera desde phpMyAdmin.

Si ahora quisiéramos saber cuántos cines de Cullera tenemos dados de alta en nuestra tabla de cines en las que aún no se han dado de alta sus salas con sus butacas, en vez de dos registros la consulta anterior podría haber devuelto un sinfín de registros, por lo que esta consulta solo muestra los que aún no tienen salas.

```
SELECT cine.idCine, nombreCine, poblaciónCine, sala.idCine, sala.idSala,
butacasSala FROM cine LEFT JOIN sala ON cine.idCine = sala.idCine WHERE
cine.poblaciónCine = "Cullera" AND sala.idCine IS NULL
```

idCine	nombreCine	poblaciónCine	idCine	idSala	butacasSala
22	Cines Victoria	Cullera	NULL	NULL	NULL

Resultado de la consulta de todos los cines sin salas de Cullera desde phpMyAdmin.

4.3. Consultas de unión

Estas consultas sirven para combinar tablas que no están relacionadas mediante claves ajenas.

Solo se pueden aplicar a conjuntos resultantes que tienen los mismos campos y los mismos tipos. La unión de ambos muestra otro conjunto resultante con todas las filas de ambas tablas.

Utilizaremos **UNION** si no quisiéramos valores repetidos y **UNION ALL** si los quisiéramos todos.

La siguiente consulta muestra un conjunto resultante fruto de unir los conjuntos resultantes de dos **SELECT**:

```
SELECT * FROM tabla1
UNION [ALL]
SELECT * FROM tabla2;
```

Como se ha indicado, solo funcionará si el conjunto resultante que se obtiene del primer **SELECT** y el del segundo son compatibles. O sea, en este caso, puesto que estamos indicando que se obtengan todas las columnas de cada tabla, si tabla1 y tabla2 tienen los mismos campos y los mismos tipos.

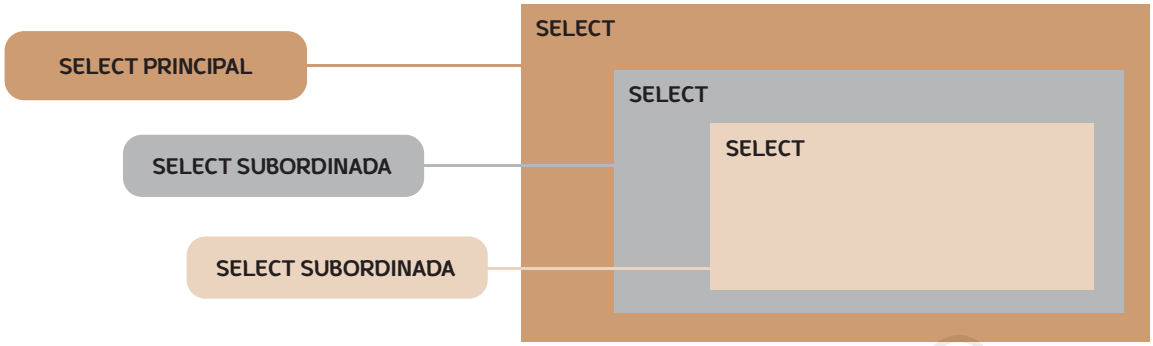
Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

5. SQL/DQL. Subconsultas

Se denominan **subconsultas** aquellas consultas en las que el resultado (**SELECT** principal) depende del resultado de otra **SELECT** (**SELECT** subordinada), la cual ha de ser resuelta con anterioridad. En consultas complejas, esta **SELECT** subordinada puede depender del resultado de una tercera **SELECT**, y así sucesivamente.



Subconsultas en SQL.

Una subconsulta puede ser escalonada o de lista:

- **Escalonada:** la **SELECT** principal filtra según el resultado de una **SELECT** subordinada que devuelve **un valor simple** fruto de una consulta agrupada (como por ejemplo una media aritmética o un valor máximo).
- **De lista:** similar a la anterior, pero cuya **SELECT** subordinada devuelve **una lista** de valores o de filas en lugar de un valor simple.

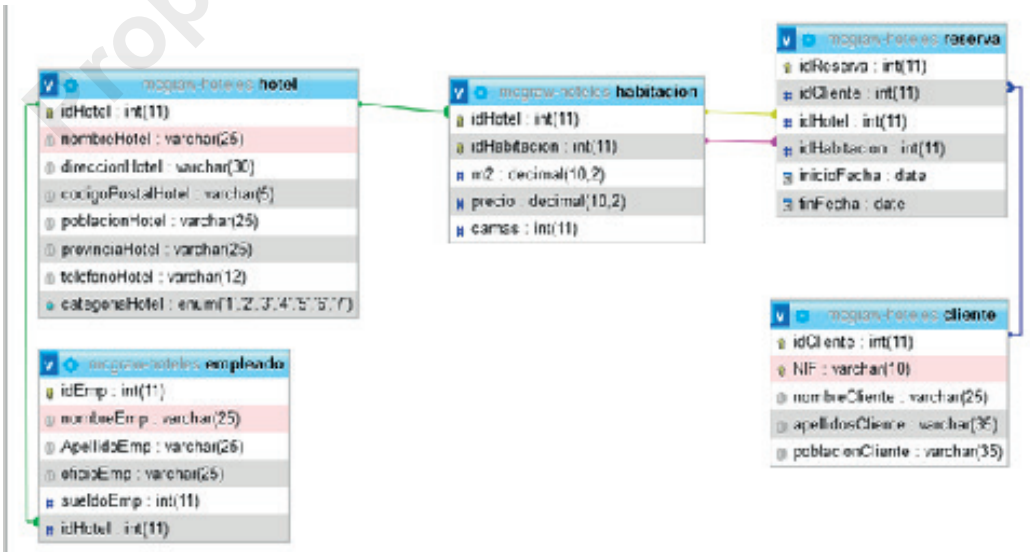
Además, diremos que una subconsulta, ya sea escalonada o de lista, es *correlacionada* cuando alguna de las tablas del **FROM** principal coincide con alguna de las del **FROM** subordinado.

5.1. Subconsultas escalonadas

Vamos a ver un ejemplo basado en la base de datos MCGRAWHOTELES.

> Dispones de las bases de datos en la sección Ejemplos para trabajar y analizar de la unidad, descargando el archivo **GBD_U07_01.zip**.

Primero vamos a ver el diseño de las cinco tablas que se utilizan en ella, que son: hotel, habitación, reserva, cliente y empleado.



Relaciones entre las cinco tablas de la base de datos de MCGRAWHOTELES.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Supongamos que la consulta que se nos pide es "**Mostrar los datos del empleado con mayor sueldo**".

Si lo tuviéramos que resolver a mano, el plan de actuación constaría de dos fases:

1. Resolver la **SELEC** subordinada: recorrer fila a fila la tabla empleado y obtener el valor mayor de **sueldoEmp**. En SQL expresaríamos esta **SELECT** subordinada así:

   ```
   SELECT MAX(sueldoEmp) FROM empleado);
   ```

2. Resolver la **SELECT** principal: recorrer de nuevo la tabla empleado, pero aplicando dicho valor como filtro y mostrando los datos de los empleados que tengan aquel valor (que ahora ya sabemos que es el máximo) como sueldo. En SQL sería:

   ```
   SELECT * FROM empleado WHERE sueldoEmp = SELECT subordinada;
   ```

Como el resultado de la **SELECT** subordinada es un valor simple, se trata de una subconsulta escalonada. Y como la tabla empleado se usa tanto en la **SELECT** subordinada como en la **SELECT** principal, se trataría también de una subconsulta correlacionada.

Aunque en las subconsultas siempre se resuelve la **SELECT** subordinada antes que la principal, en la instrucción se escribe primero la principal y, entre paréntesis, la subordinada. Así pues, la instrucción SQL que satisface la consulta sería:

```
SELECT * FROM empleado WHERE sueldoEmp = (SELECT MAX(sueldoEmp) FROM
empleado);
```

El resto de funciones agregadas en SQL estándar que podemos utilizar en este tipo de subconsultas son el sumatorio **SUM**, la media aritmética **AVG**, el valor mínimo **MIN** y el cómputo **COUNT**.

5.2. Subconsultas de lista

Los operadores de comparación que hemos venido utilizando hasta el momento (**>**, **<**, **=**, etc.) solo sirven para comparar dos valores simples. Sin embargo, en este tipo de subconsultas, el **SELECT** subordinado devuelve una lista de valores. Necesitamos nuevos operadores que puedan comparar un valor simple con una lista o conjunto de valores:

A. OPERADOR ALL

Este operador compara un valor simple con cada uno de los elementos de una lista y devuelve **CIERTO** si se cumple para **todos** ellos.

Supongamos que queremos expresar en SQL la consulta "**Muestra el nombre y apellido de los conserjes que cobran más que todos los directores**". Este sería un modo de resolverla:

```
SELECT nombreEmp, ApellidoEmp FROM empleado WHERE oficioEmp =
"Conserje" AND sueldoEmp > ALL (SELECT sueldoEmp FROM empleado WHERE
oficioEmp="Directivo");
```

B. OPERADOR ANY

Este operador compara un valor simple con cada uno de los elementos de una lista y devuelve **CIERTO** si se cumple para **alguno** de ellos.

Vamos a variar ligeramente la consulta anterior. "**Muestra el nombre y apellido de los conserjes que cobran más que alguno de los directores**".

```
SELECT nombreEmp, ApellidoEmp FROM empleado      WHERE oficioEmp =
"Conserje" AND sueldoEmp > ANY (SELECT sueldoEmp FROM empleado WHERE
oficioEmp="Directivo");
```

Evidentemente, el resultado incluirá a los conserjes de la consulta anterior.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

C. OPERADOR EXISTS

Este operador devuelve **CIERTO** si la lista de valores con la que opera no está vacía.

Supongamos que se nos pide: "Muestra los nombres de los hoteles que tienen más de una cama en todas sus habitaciones".

Podríamos valernos del operador **EXISTS** reformulando la consulta de este modo: "Muestra los nombres de los hoteles que no tienen ninguna habitación con menos de dos camas".

En la **SELECT** subordinada obtengo los nombres de los hoteles que tienen alguna habitación con menos de dos camas. En la **SELECT** principal descarto de todos los hoteles los que aparecen en aquella lista.

```
SELECT nombreHotel FROM hotel WHERE NOT EXISTS
SELECT* FROM habitacion WHERE habitacion.idHotel = hotel.idHotel and camas
< 2);
```

D. OPERADOR IN

Este operador compara un valor simple con cada uno de los elementos de una lista y devuelve **CIERTO** si lo es **alguno** de ellos.

Imaginemos que queremos saber el nombre completo de los empleados que trabajan en los hoteles de las poblaciones de Carlet o de Sueca. La subconsulta nos devolverá los códigos de los hoteles que tenemos dados de alta en la tabla que sean de las localidades de Carlet o de Sueca, y con ellos podremos saber quiénes son esos trabajadores que están empleados en alguno de estos hoteles.

```
SELECT DISTINCT nombreEmp, apellidoEmp FROM empleado WHERE idHotel
IN (SELECT idHotel FROM hotel WHERE poblacionHotel = "Carlet" or
poblacionHotel = "Sueca" );
```

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Unidad 8.

SQL III. DML. Seguridad

Propiedad de McGraw-Hill®

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

1. SQL/DML Altas: INSERT con o sin SELECT

El lenguaje de manipulación de datos (**DML, Data Manipulation Language**) nos permite realizar consultas de acción sobre nuestras tablas. Empezaremos tratando la sentencia **`INSERT`**, que añade información nueva. Si en las aplicaciones **CRUD**, la *R* de *READ* correspondía a **`SELECT`**, la *C* de *CREATE* corresponde a **`INSERT`**, o sea, añadir.



Veremos cómo insertar datos con la sentencia **`INSERT`**, pero también podemos introducir nuevos datos a partir de información que ya está contenida en nuestras bases de datos utilizando una sentencia **`SELECT`** dentro de un **`INSERT`**. Y para terminar, veremos cómo cargar información a partir de fuentes externas de datos con la sentencia **`LOAD DATA`**.



1.1. Insertar (INSERT INTO ... VALUES)

Existen tres opciones para trabajar con la sentencia **`INSERT`**.

- **Opción 1**: rellenando solo unos pocos atributos de cada fila. Especificaremos qué atributos queremos rellenar, sin importar el orden, y colocaremos sus valores en ese mismo orden. De los atributos omitidos, los de tipo **`AUTOINCREMENT`** se rellenarán con el valor que corresponda siguiendo su secuencia y el resto se rellenarán con el valor **`NULL`**. ¡Atención! La inserción de la fila puede fallar si sobre alguno de estos atributos pesan restricciones como **`NOT NULL`**.
- **Opción 2**: rellenando todos los atributos de cada fila. Si nos molestamos en colocar los valores de cada atributo en el orden en que aparecen en la descripción de la tabla –o sea, el orden en que los introdujimos cuando hicimos el **`CREATE TABLE`**– no hará falta indicar sus nombres. Recordad que podemos conocer la descripción de una tabla con **`DESC nombre_tabla`**.

Sintaxis

Opción 1. Especificando los nombres de los atributos que nos incumben:

```
INSERT INTO nombre_tabla (nombre_columna1, nombre_columna2, ...,)
VALUES (<valor_columna1>, <valor_columna2>, ...);
```

Como los SGBDR crean internamente una **secuencia** para cada tabla que contenga un atributo con la restricción **`AUTOINCREMENT`**, detrás de **`VALUES`** podemos escribir **`NULL`** en la posición que ocupen según la descripción de la tabla y así evitar molestarnos en ver antes qué valor toca.

Opción 2. Sin especificar ningún nombre de atributos:

```
INSERT INTO nombre_tabla
VALUES (<valor_columna1>, <valor_columna2>, … , <valor_columnaN>);
```

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Ejemplo

Efectuemos una inserción en la tabla ACTOR de la base de datos MCGRAWPELICULAS. Recordamos su estructura con **DESC**:

```
DESC actor;
```

Field	Type	Null	Key	Default	Extra
idActor	int(11)	NO	PRI	*NULL*	auto_increment
nombreActor	varchar(30)	NO		*NULL*	
nacionalidadActor	int(11)	NO	MUL	*NULL*	

Opción 1. Especificando los nombres de los atributos que nos incumben:

Comprobamos que el último valor de *idActor* insertado es el 14. Si desconocemos o no nos incumbe la nacionalidad de Emma Thompson, podríamos insertarla así.

```
INSERT INTO actor (`idActor`, `nombreActor`)
VALUES (15, "Emma Thompson");
```

Sin embargo, como el atributo **nacionalidadActor** tiene restricción NOT NULL, no funcionará.

Opción 2. Sin especificar los nombres de los atributos.

Como sabemos que Emma Thompson es inglesa, podemos especificar el valor 2 en la posición de **nacionalidadActor**. O sea, en la tercera posición:

```
INSERT INTO actor
VALUES (NULL, "Emma Thompson", 2);
```

El primer atributo en la descripción de la tabla ACTOR, **idAutor**, es autoincrementable, por lo que lo hemos obviado y hemos puesto **NULL** en la primera posición. Así, le dejamos al sistema gestor de bases de datos que le asigne el valor que corresponda siguiendo la secuencia que gestiona.

Resumen sentencia INSERT INTO ... VALUES.

La modalidad **INSERT** *múltiple* permite añadir varias filas con una sola instrucción.

Por ejemplo, podemos dar de alta todos actores de la serie *Stranger Things* con un solo **INSERT**. Para ello separamos con una coma cada fila insertada cerrando la última con el punto y coma:

```
INSERT INTO actor VALUES
    (NULL, 'Millie Bobby Brown', 1),
    (NULL, 'Natalia Dyer', 1),
    (NULL, 'Caleb McLaughlin', 1);
```

1.2. Insertar (INSERT INTO ... SELECT)

En este caso no vamos a ser nosotros quienes escribamos los valores a insertar con la cláusula **VALUES**, sino que será el *conjunto resultante* de una **SELECT**. Es una manera fácil y cómoda de generar múltiples registros a partir de información que ya existe en nuestra base de datos.

Descripción

La sentencia **INSERT INTO ... SELECT** se utiliza para añadir nuevos registros (filas) a una tabla desde otras tablas. La **SELECT** puede tener todas las opciones que ya conocemos. Resulta práctico cuando queremos hacer copias o pruebas.

Sintaxis

```
INSERT INTO <nombre_tabla_destino>(<nombre_campo1> [, ...])
SELECT <nombre_campo1>[, <nombre_campo2>] [, ...]
FROM <nombre_tabla_origen>;
```

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Ejemplo

```
INSERT INTO tabla_destino_copia (nombre_copia, apellidos_copia)
SELECT nombre, apellidos
FROM tabla_origen;
```

Si ambas tablas son iguales podríamos ejecutar simplemente:

```
INSERT INTO tabla_destino_copia
SELECT * FROM tabla_origen;
```

Resumen de la sentencia INSERT INTO ... SELECT.

1.3. INSERT IGNORE INTO

Disponemos de una opción que nos permite ignorar que, en el intento de dar de alta valores nuevos, nos hemos equivocado y resulta que estamos tratando de dar de alta una fila que ya existe en nuestra tabla. Si añadimos la opción **IGNORE** entre la sentencia **INSERT** y **INTO**, con ello nos aseguramos de que no nos dará error, pues se omitirá la acción de inserción y se mantendrá el valor actual.

Por ejemplo:

```
INSERT IGNORE INTO tabla_destino_copia
SELECT * FROM tabla_origen;
```

Funciona con cualquiera de las opciones vistas, por lo que se recomienda utilizarlo siempre a menos que queramos ser alertados de dicho error.

1.4. ON DUPLICATE KEY UPDATE

También existe una cláusula que nos permite no cometer errores al intentar dar de alta un valor repetido a algún atributo que no puede estar duplicado. Si añadimos esta cláusula, podemos tratar de arreglarlo de manera dinámica, sobre la marcha, mientras se están realizando las inserciones. Además, podemos hacer uso de la función **LAST_INSERT_ID()**, que devuelve el valor del último atributo clave insertado.

Como el valor 2 de *idPais* ya existe en la tabla PAIS, la siguiente instrucción nos dará error.

```
INSERT INTO pais (idPais, nombrePais)
    VALUES (2, 'England');
```

Sin embargo, con **ON DUPLICATE KEY UPDATE** lo podemos esquivar del siguiente modo;

```
INSERT INTO pais (idPais, nombrePais)
    VALUES (2,'England')
ON DUPLICATE KEY UPDATE
   idPais = LAST_INSERT_ID(idPais),
   nombrePais = VALUES(nombrePais);
SELECT LAST_INSERT_ID();
```

La última **SELECT** es opcional. Si la lanzamos a la vez que el **INSERT** nos devuelve que el valor 2 es el último valor de clave insertado.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

1.5. Cargar (LOAD DATA)

Por último, en este apartado de inserciones vamos a ver cómo cargar datos desde archivos externos, como por ejemplo un archivo de texto en formato **CSV**. Es un procedimiento que aún no está estandarizado en **SQL** por lo que la sintaxis puede variar según el SGBDR que utilicemos.

Descripción
La sentencia **LOAD DATA** permite obtener datos externos a nuestra base de datos.
Sintaxis

```
LOAD DATA INFILE <unidad><ruta><nombre_fichero_externo.csv>
INTO TABLE <tabla_destino>
FIELDS TERMINATED BY ','      -- Indica el símbolo que separa las filas.
ENCLOSED BY '"']              -- Símbolo que delimita los valores. Se omite si es el tabulador.
[LINES TERMINATED BY '\n']    -- Símbolo de fin de línea.
[IGNORE 1 ROWS;]              -- Indica si la primera fila es una cabecera con los nombres de atributos.
```

Ejemplo

```
LOAD DATA INFILE 'C:\McGraw_SQL\McGraw_CSV\Tabla_data.csv'
INTO TABLE provincia
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Resumen de la sentencia LOAD DATA.

El archivo *Tabla_data.csv* se ha generado a partir de la web del Instituto Nacional de Estadística (INE), **https://public.tableau.com/views/infografia_cumple/Dashboard12**. Dicha página muestra cuántas personas cumplen años cada día del año y lo ilustra en tablas e histogramas.

Hemos seleccionado todo el periodo del 1 de enero al 31 de diciembre de la tabla *"Número medio diario de cumpleaños por mes (periodo completo)"* y lo hemos descargado en un archivo de texto plano con formato *csv*.

> Puedes descargar el archivo de la sección *Ejemplos para trabajar y analizar* de la unidad, con el nombre **GBD_U08_A01_E.zip**

Recuerda que antes de hacer la carga de este archivo de intercambio deberemos crear la nueva tabla en la base de datos con los campos **Día;Mes..letra.completo.;Ranking;Número.de.personas**. Fíjate que estamos usando como separador de campos el símbolo del punto y coma **;** y que la primera fila contiene la cabecera de los datos.

IMPORTANTE

Esta instrucción puede ser muy útil para añadir información de fuentes de datos abiertas (**OPEN DATA**) de las que cada vez tenemos más a nuestra disposición desde las administraciones públicas u organizaciones internacionales. Bien sea desde el propio Instituto Nacional de Estadística (**www.ine.es**) o desde portales específicos de información pública abierta o transparencia, en nuestro país tenemos el portal **https://datos.gob.es**. A nivel de Europa, en el portal **https://data.europa.eu/data/datasets?locale=es**. Y desde la ONU, **https://data.un.org**, que persigue el Objetivo 2030 con sus 17 Objetivos de Desarrollo Sostenibles (ODS), que debemos apoyar para conseguir un mundo mejor.



Iniciativas y conjuntos de datos en el portal de datos.gob.es en 2022.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

2. SQL/DML. Modificaciones: UPDATE

La sentencia **UPDATE** sirve para modificar la información existente en las bases de datos. Es la de **U** de **UPDATE** de las aplicaciones **CRUD** . Al igual que **INSERT**, forma parte del sublenguaje del SQL llamado DML (Data Manipulation Language), el cual se encarga de la manipulación de datos.

Es una lástima que **UPDATE**, a diferencia de **INSERT**, no permita embeber datos a partir de sentencias **SELECT**, puesto que ello permitiría automatizar más fácilmente algunas actualizaciones. Su formato es completamente diferente al de la sentencia **INSERT**.

En esta sección veremos también cómo volcar información de una tabla temporal que hayamos cargado con datos externos sobre nuestra tabla de trabajo con una única sentencia **UPDATE**.

Y para terminar abordaremos la sentencia **MERGE**, que nos permite decidir previamente si tenemos que hacer una actualización en caso de que los datos ya existieran o si, por el contrario, se trata de una **ALTA NUEVA**, para la cual habrá que proceder con una sentencia **INSERT**.



2.1. Modificar (UPDATE) sobre una tabla

Descripción
La sentencia **UPDATE** se usa para **modificar** valores existentes en filas de una tabla.
Sintaxis

`UPDATE nombre_tabla`
`SET columna1 = valor1 [ , columna2 = valor2 [ , ...] ]`
`[WHERE condición];`

Ejemplo
`UPDATE empleado` `SET SueldoEmp = SueldoEmp * 1.1`

2.2. Modificar (UPDATE) entre tablas

Puede que hayamos hecho una carga de datos con la sentencia **LOAD DATA**. Lo habitual será hacerla sobre una tabla temporal que nos sirva para tenerlos en nuestra base de datos. Pero luego, si queremos añadir dichos datos a una tabla real de explotación, una vez revisada y nos hayamos asegurado de que no contiene valores erróneos, podemos hacer una actualización haciendo uso de esta nueva cláusula de la sentencia **UPDATE**.

Descripción
La sentencia **UPDATE** con **JOIN** se usa para **copiar** registros existentes en una tabla temporal u otra de explotación donde nos interese incorporar dichos valores.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Sintaxis

```
UPDATE nombre_tabla_destino  ntd   - - podemos poner un alias a nombre_
tabla_destino
JOIN nombre_tabla_origen nto          - - podemos poner un alias a nombre_
tabla_origen
ON nombre_tabla_destino.campoclave = nombre_tabla_origen.campoclave
SET nombre_tabla_destino.columna1 = nombre_tabla_origen.columna1
 [ , columna2 = valor2 [ , ...] ];
```

Ejemplo

```
UPDATE director d
JOIN director_temporal dt
ON d.idDirector = dt.idDirector,
SET d.idDirector = dt.idDirector,
d.nombreDirector = dt.nombreDirector,
d.edadDirector = dt.edadDirector,
d.nacionalidadDirector = dt.nacionalidadDirector
d.salarioDirector = dt.salarioDirector;
```

2.3. Insertar o Modificar (MERGE)

Hemos dado por supuesto que teníamos claro cuándo había que insertar filas nuevas y cuándo modificar las existentes. ¿Pero qué pasa si no sabemos si las filas ya existen o no? ¿Toca hacer un **INSERT** o un **UPDATE**? Para resolver esta disyuntiva, el estándar SQL:2008 creó una nueva instrucción llamada **MERGE**.

Descripción

La sentencia **MERGE** puede lanzar tres consultas a la vez. Se usa tanto para poder modificar los registros existentes en una tabla como para insertar nuevos o incluso borrar existentes, los de la tabla destino que no estén en la tabla origen.

Sintaxis

```
MERGE INTO  <tabla_destino>      <alias _tabla_destino>
  USING {<tabla_origen> | (sentencia SELECT)} -- Buscamos el origen datos
  [ON(<criterio_composicion>)] -- Aplicamos la unión interna entre tablas
WHEN MATCHED <condición> [{AND | OR} <condición> [,...]] THEN
-- aquí indicamos la acción cuando coinciden, normalmente UPDATE
[WHEN NOT MATCHED [BY <tabla_destino>] [{AND | OR}] <condición> [, ...] ]
THEN
-- aquí indicamos la acción cuando no coinciden por tabla destino,
normalmente INSERT
[WHEN NOT MATCHED [BY <tabla_origen>] [ [{AND | OR}] <condición> [, ...]]
THEN
-- aquí indicamos la acción cuando no coinciden por tabla origen,
normalmente DELETE
;
```

Hay que comentar que esta instrucción del estándar SQL de 2008 puede que aún no esté soportada por todos los SGBDR. En el momento de elaborar esta unidad esta sentencia aún no estaba implementada en Oracle MySQL, MariaDB ni Amazon Aurora.

Analicemos la sentencia **MERGE** para entender mejor su funcionamiento:

- **MERGE INTO** selecciona la tabla_destino (**TARGET**) sobre la que aplicar cambios.
- **USING** selecciona la tabla_origen (**SOURCE**) de la cual vamos a tomar los datos; puede ser una tabla real o una virtual construida con la sentencia **SELECT**.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

- **ON** realiza un **JOIN** por campo/s clave/s entre la tabla_origen y la tabla_destino.
- **WHEN MATCHED <condición> THEN UPDATE** indica que la acción que se debe llevar a cabo si existe el registro origen en el destino será efectuar un UPDATE.
- **WHEN NOT MATCHED BY tabla_destino THEN INSERT** indica que la acción que se debe llevar a cabo si no existe será efectuar un **INSERT**.
- **WHEN NOT MATCHED BY tabla_origen THEN DELETE** indica que la acción que se debe llevar a cabo si existe será efectuar un **DELETE**.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

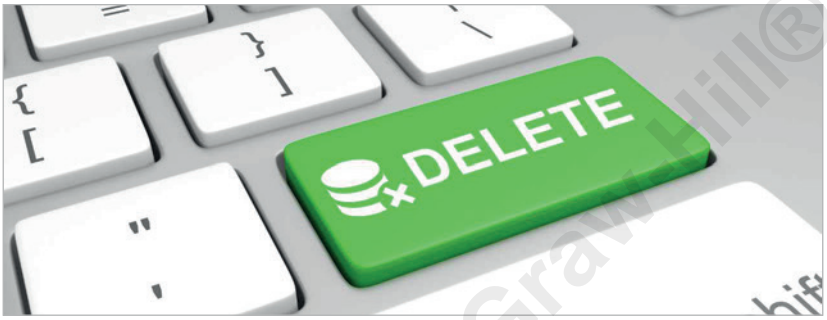
3. SQL/DML Bajas: DELETE

La sentencia de SQL/DML **DELETE** borra filas de tablas. Corresponde a la **D** de las siglas de **CRUD**. Con ella completamos las cuatro operaciones clásicas a realizar por toda aplicación:

- Altas: **INSERT**
- Bajas: **DELETE**
- Modificaciones: **UPDATE**
- Listados: **SELECT**

Es recomendable, antes de proceder a un borrado de filas, conocer cuál va a ser su efecto. Esto lo podemos conocer lanzando previamente la correspondiente sentencia **SELECT**.

Este apartado concluye abordando el coste de la ejecución de las consultas SQL con la sentencia **EXPLAIN**.



3.1. Borrar (DELETE)

Para quitar definitivamente registros/filas de una tabla usaremos la instrucción **DELETE**. Tenemos que especificar la tabla de donde queremos borrar filas, así como la condición que tienen que cumplir las filas a eliminar. Si omitiéramos esa condición, efectuada con la cláusula **WHERE**, se borrarían todos los registros de la tabla. Así pues, hay que tener mucho cuidado con esto, ya que una vez ejecutada, si no tenemos copia de nuestros datos, estos habrán sido dados de baja definitivamente.

Sintaxis

`DELETE FROM <nombre_tabla>`
`WHERE <condición> [ {AND | OR} <condición> [, ...] ]`
`[ORDER BY <nombre_campo1> [,<nombre_campo2>[, ...]] [DESC]]`
`[LIMIT [<posición_inicial>], <número_registros>]`

Ejemplo
`DELETE FROM empleados;` -- borra todos los empleados `DELETE FROM empleados WHERE idHotel = 9;` -- Borra las filas de la tabla EMPLEADOS, cuyo valor en el atributo *idHotel* es 9 (si las hubiere).

Hay que comentar que esta sentencia **DELETE** admite las cláusulas **IGNORE** para ignorar los errores que se puedan producir al ejecutarse. O también podemos priorizar el borrado con la cláusula **LOW_PRIORITY**, esperándonos a atender primero todas las posibles peticiones de consulta que estén llegando a la base de datos antes de proceder al borrado. Si se omite, se llevará a cabo el borrado inmediatamente. Mencionar, por último, que en el **WHERE** podemos utilizar subconsultas con **SELECT** anidados para filtrar los datos a eliminar.

IMPORTANTE

Tanto con la opción de **DELETE** sin **WHERE** como con la sentencia **TRUNCATE**, incluida en el estándar SQL:2008, obtenemos el mismo resultado. Cuando la tabla contiene miles o millones de filas, **TRUNCATE** es más eficiente, pues elimina la tabla (**DROP TABLE**) de la base de datos para luego crearla de nuevo (**CREATE TABLE**).

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

A. EJEMPLOS CON MCGRAWHOTELES

Dispones de las bases de datos para descargar en la sección "Ejemplos para trabajar y analizar", con el nombre de fichero **GBD_U08_A03_E.zip.**

Veamos algunos ejemplos más.

Ya sabemos que un **DELETE** sin condición elimina todos los registros/filas de una tabla:

```
DELETE FROM hotel;
```

Si queremos eliminar solo los hoteles de Sueca o de Carlet podemos escribir:

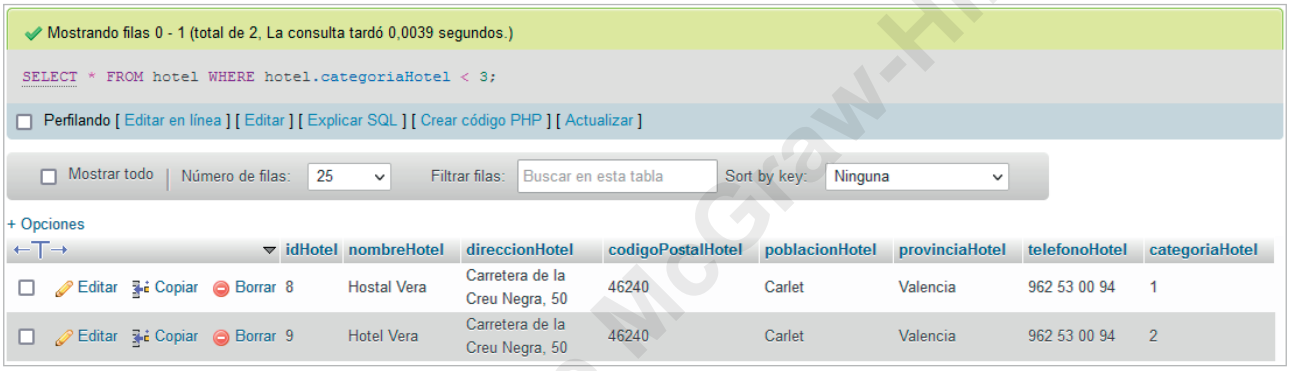
```
DELETE FROM hotel WHERE poblacionHotel = 'Sueca' OR poblacionHotel =
'Carlet';
```

Si queremos eliminar solo los hoteles de menos de 3 estrellas podemos lanzar esta consulta:

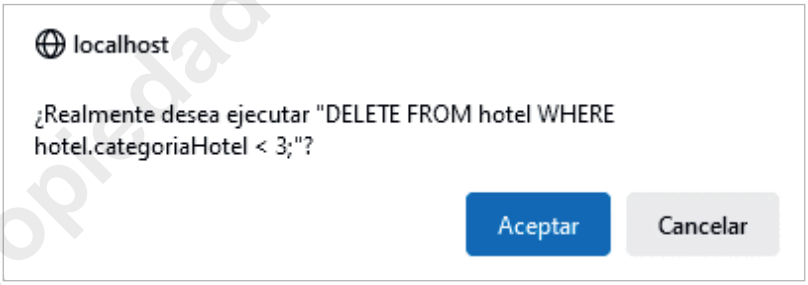
```
DELETE FROM hotel WHERE categoriaHotel < 3;
```

Recuerda que puede ser interesante conocer las filas que se eliminarán. Para ello ejecutamos antes la misma instrucción, pero sustituyendo el **DELETE** por **SELECT ***.

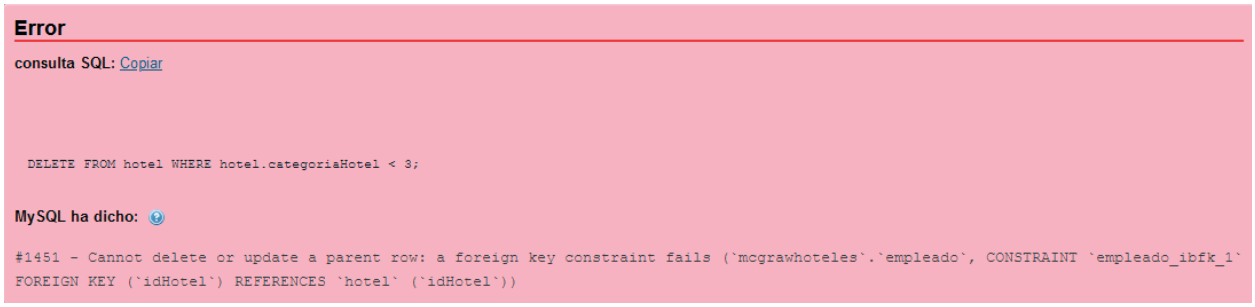
```
SELECT * FROM hotel WHERE categoriaHotel < 3;
```



Una vez verificado y convencidos de que se van a eliminar los valores deseados, lanzamos la consulta. Al ser una instrucción que no tiene vuelta atrás, el SGBDR nos puede mostrar una advertencia:



La ejecución de esta instrucción nos dará error.

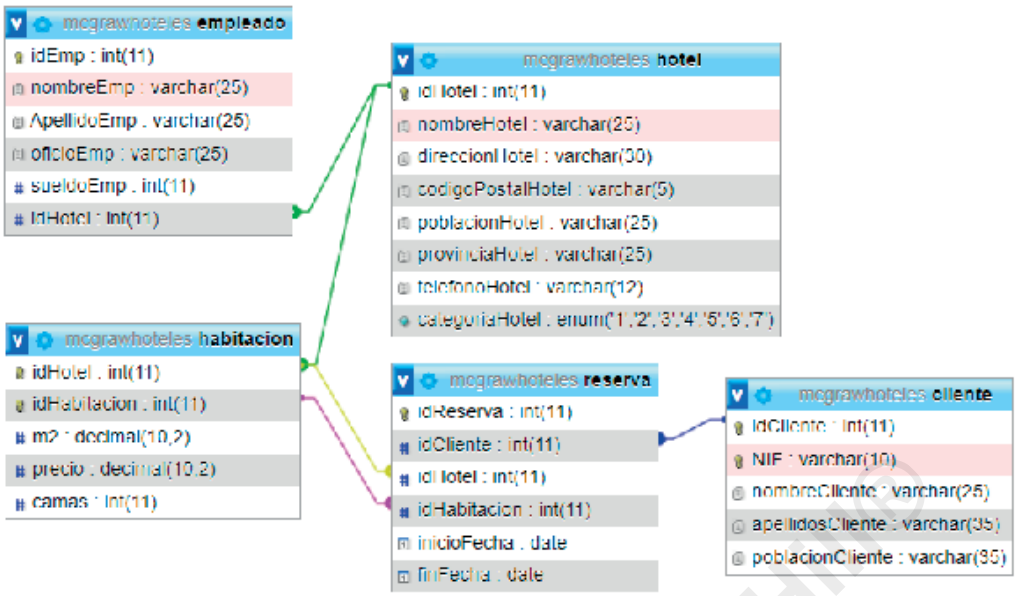


Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

El SGBDR ha detectado una restricción para las eliminaciones en la tabla HOTEL por estar relacionada con la de EMPLEADOS.



O sea, debe haber algún empleado cuyo hotel tiene menos de 3 estrellas. Eliminar los hoteles con menos de 3 estrellas afectaría al contenido del atributo **`idHotel`** de esos empleados. Si pese a ello queremos proceder al borrado de los hoteles de menos de 3 estrellas, o bien activamos el *borrado en cascada* para la columna **`idHotel`** de EMPLEADO o bien eliminamos antes las filas de empleados cuyo **`idHotel`** es el código de algún hotel de menos de 3 estrellas. Como en nuestro caso son el 8 y el 9, podríamos borrar antes las filas de empleado cuyo **`idHotel`** es 8 o 9.

IMPORTANTE

Nuestra base de datos está preparada para que un empleado solo pueda trabajar en un hotel, pero si resulta que varios hoteles comparten trabajadores (cardinalidad M:N), entonces deberemos hacer cambios en nuestras base de datos y añadir una nueva tabla intermedia entre hoteles y empleados para que se pueda admitir esta posibilidad.

Observemos cómo los valores numéricos (**`INT`**, **`FLOAT`**, etcétera) se colocan sin comillas en la condición, mientras que los valores alfanuméricos (**`VARCHAR`**, **`CHAR`**, **`ENUM`**, etcétera) se colocan entre comillas simples o acentos graves, dependiendo del SGBDR que utilicemos.

La opción **`LIMIT`** permite restringir el número de registros a borrar desde una posición inicial que indiquemos; si omitimos este primer argumento empezará por el registro cero.

3.2. Planificar (EXPLAIN)

La sentencia **`EXPLAIN`** en SQL nos muestra información sobre el plan de ejecución de las consultas SQL que vamos a realizar contra nuestra base de datos y analizarlo para optimizarlas. En nuestras pequeñas bases de datos puede que no tenga demasiado sentido analizarlo, pero cuando tratemos bases de datos gigantescas, con enorme cantidad de tablas y miles o millones de registros/filas, entonces sí que puede ser interesante.

IMPORTANTE

`EXPLAIN` sirve para analizar las consultas SQL o DML. Es equivalente al **`DESCRIBE`** que ya conocemos y que sirve para mostrar la estructura de las tablas. Por tanto, salvando las distancias, ambas sentencias son homólogas. La primera para analizar el resultado de la consulta a fin de optimizarla, y la segunda para analizar la estructura de la tabla también para tratar de mejorarla.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Descripción
La sentencia **EXPLAIN** se utiliza para que el SGBDR nos explique el coste de la instrucción que queremos analizar. Algo así como a cuántas tablas, claves, filas y filtros involucra. Se puede aplicar a instrucciones de tipo **SELECT**, **UPDATE** o **DELETE**.
Sintaxis
EXPLAIN <nombre_tabla> **EXPLAIN [EXTENDED | PARTITIONS] {SELECT select_options | UPDATE update_options | DELETE delete_options};** **SHOW WARNINGS; -- Si queremos que nos muestre advertencias**
Ejemplo elaborado con dos subconsultas de lista donde queremos saber qué hoteles de Carlet o Alzira tienen un conserje que gana más dinero que el directivo. Seguro que no es la forma más adecuada de averiguarlo, pero nos vale para empezar a optimizarla.
EXPLAIN SELECT nombreEmp, apellidoEmp, hotel.idHotel, hotel.poblacionHotel **FROM empleado INNER JOIN hotel ON empleado.idHotel = hotel.idHotel WHERE oficioEmp="Conserje" AND sueldoEmp > ALL** **(SELECT sueldoEmp** **FROM empleado WHERE oficioEmp="Directivo") AND hotel.idHotel IN** **(SELECT hotel.idHotel** **FROM hotel WHERE poblacionHotel="Carlet" OR poblacionHotel="Alzira");**

Esa instrucción nos devuelve el resultado del análisis en una tabla:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	PRIMARY	hotel	ALL	PRIMARY	*NULL*	*NULL*	*NULL*	9	
1	PRIMARY	hotel	eq_ref	PRIMARY	PRIMARY	4	mcgrawhoteles.hotel.idHotel	1	Using where
1	PRIMARY	empleado	ref	idHotel	idHotel	4	mcgrawhoteles.hotel.idHotel	1	Using where
2	SUBQUERY	empleado	ALL	*NULL*	*NULL*	*NULL*	*NULL*	14	Using where

Hay que destacar que el campo de tipo de selección (**select_type**) tiene categorías predefinidas como **SIMPLE, PRIMARY, UNION, DERIVED** y alguna más. Si queréis saber qué significa cada una, tenéis toda la información en la web oficial de MariaDB.

Relacionado con la ejecución ampliada (**EXPLAIN EXTENDED**), podemos ejecutar a continuación la orden **SHOW WARNINGS**, con las posibles advertencias que deberemos revisar para mejorar la consulta que estamos analizando.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

4. SQL/DCL. Usuarios y permisos

En este apartado vamos a aprender algunas de las instrucciones del sublenguaje de control de datos DCL (Data Control Language) que englobará la concesión de permisos (**GRANT**) o su retirada (**REVOKE)** a cada usuario (**USER**). También trataremos algún aspecto del lenguaje de control de sesiones, SCL (Session Control Language), como la creación de roles de usuarios y el control de las sesiones que puedan realizar. Con algunas de sus opciones gestionaremos el resto de los objetos de usuario, como el cambio de contraseña.

4.1. Gestión de Usuarios

Hasta ahora hemos estado administrando las bases de datos con privilegio absoluto accediendo al sistema gestor como usuario maestro (**root**), pero en un futuro puede que existan diferentes categorías de usuarios (empleados, miembros del consejo de dirección, personal de secretaría, de recursos humanos, etc.) que tengan que acceder o interactuar con ciertas bases de datos y tablas específicas. Y, por supuesto, sin requerir conocimientos de SQL, sino operando mediante formularios desde una aplicación web, por ejemplo.

Por seguridad, para evitar que dichos usuarios accedan y operen con impunidad sobre toda base de datos, conviene ajustarles los privilegios restringiéndoselos estrictamente a aquellas acciones y objetos que estén autorizados, sin otorgarles permisos para nada más.

A. USUARIO (USER)

Veamos las sentencias SQL de alta (**CREATE**), baja (**DROP**) y modificación (**ALTER**) de usuarios.

CREATE USER y ALTER USER

Descripción
Las sentencias **CREATE USER** y **ALTER USER** nos permiten respectivamente dar de alta y modificar un usuario en nuestro SGBDR.
Sintaxis
{CREATE | ALTER} [OR REPLACE] USER [IF NOT EXISTS] **nombre_usuario IDENTIFIED {BY 'password' | BY PASSWORD 'password_hash'}** **[,especificaciones_usuario ...]** **[REQUIRE {NONE | opcion_TLS [[AND] opcion_TLS ...]}]** **[WITH opcion_recursos [opcion_recursos...]]** **[opciones_bloque] [opcion_password]**
Ejemplo
CREATE USER 'raul'@'%'

DROP USER

Descripción
Destruir o eliminar es siempre más rápido que construir o modificar. La sentencia **DROP USER** solo se necesita saber el nombre del (o de los) usuario/s actuales para dejarlos sin acceso.
Sintaxis
DROP USER [IF EXISTS] nombre_usuario [,nombre_usuario] ...
Ejemplo
DROP USER IF EXISTS 'raul'@'%';

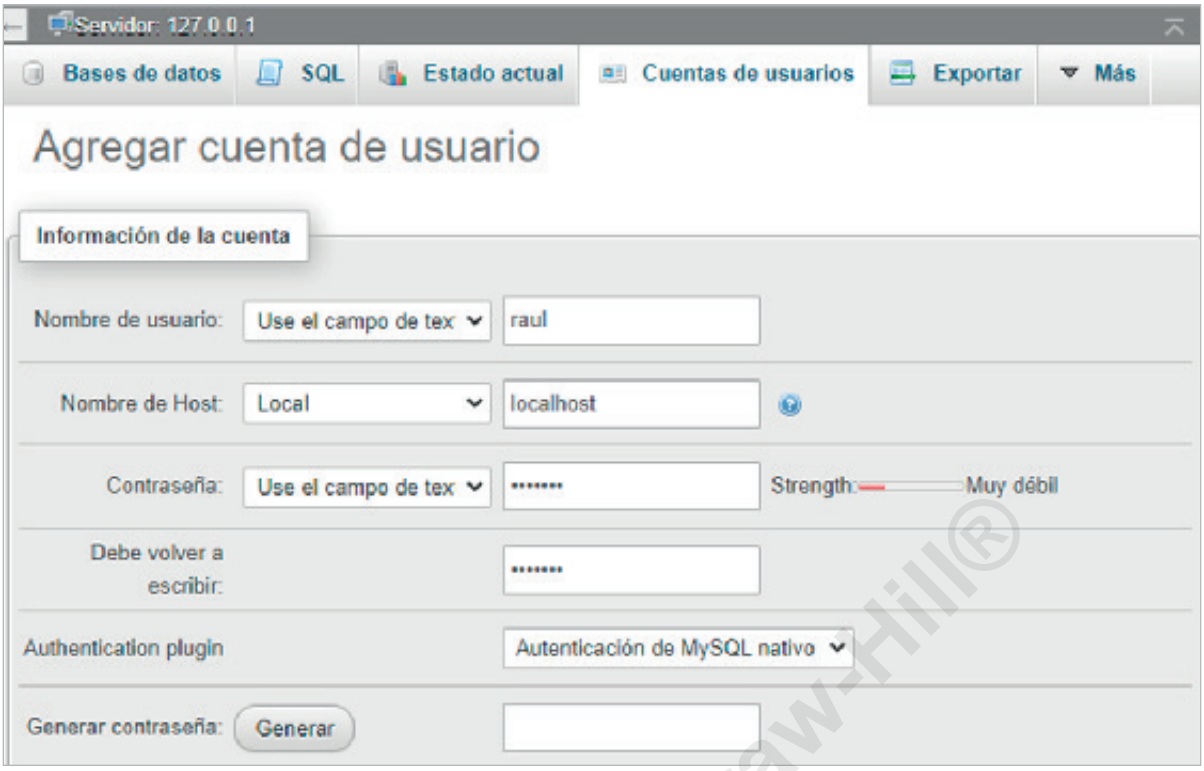
Las sentencias SQL relacionadas con los usuarios se pueden ejecutar con comandos, pero también las podemos indicar mediante el asistente del SGBDR.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Vamos a ver todo lo que podemos gestionar con el asistente de MySQL phpMyAdmin;



Una vez conectados como root, desde la pestaña *Cuentas de usuarios* vamos a poder indicar:

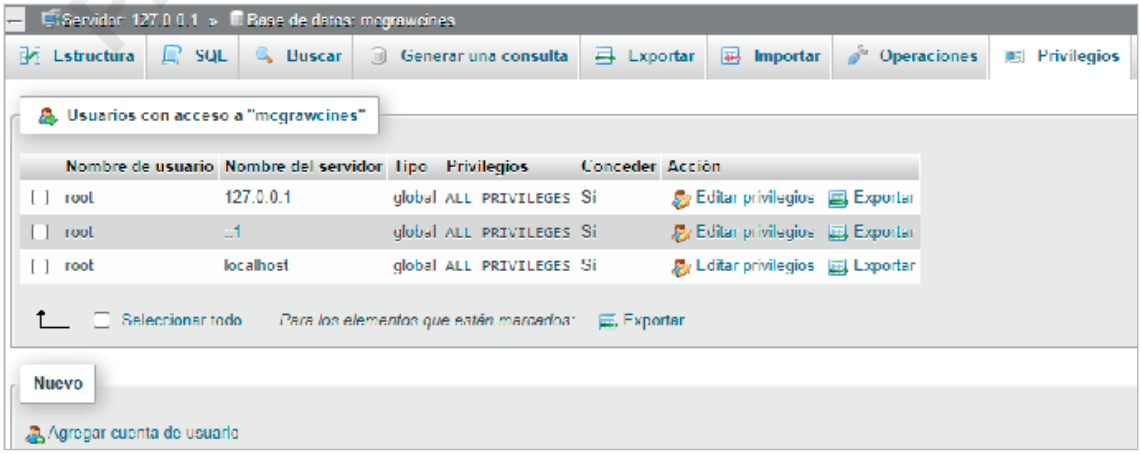
Nombre de usuario, sin espacios en blanco ni acentos ni diéresis.

Nombre de host, donde podemos elegir entre *Cualquier servidor* (**%**), *Local* (**localhost**), *Este host* (**localhost**), *Usar tabla de host* o *Usar el campo de texto* (caso de tener nuestro propio dominio).

Contraseña, que podemos elegir *Sin contraseña* o introducirla. También nos permite que phpMyAdmin nos genere una contraseña fuerte automáticamente con el botón de generar.

Autenticación, por defecto es *MySQL nativo*.

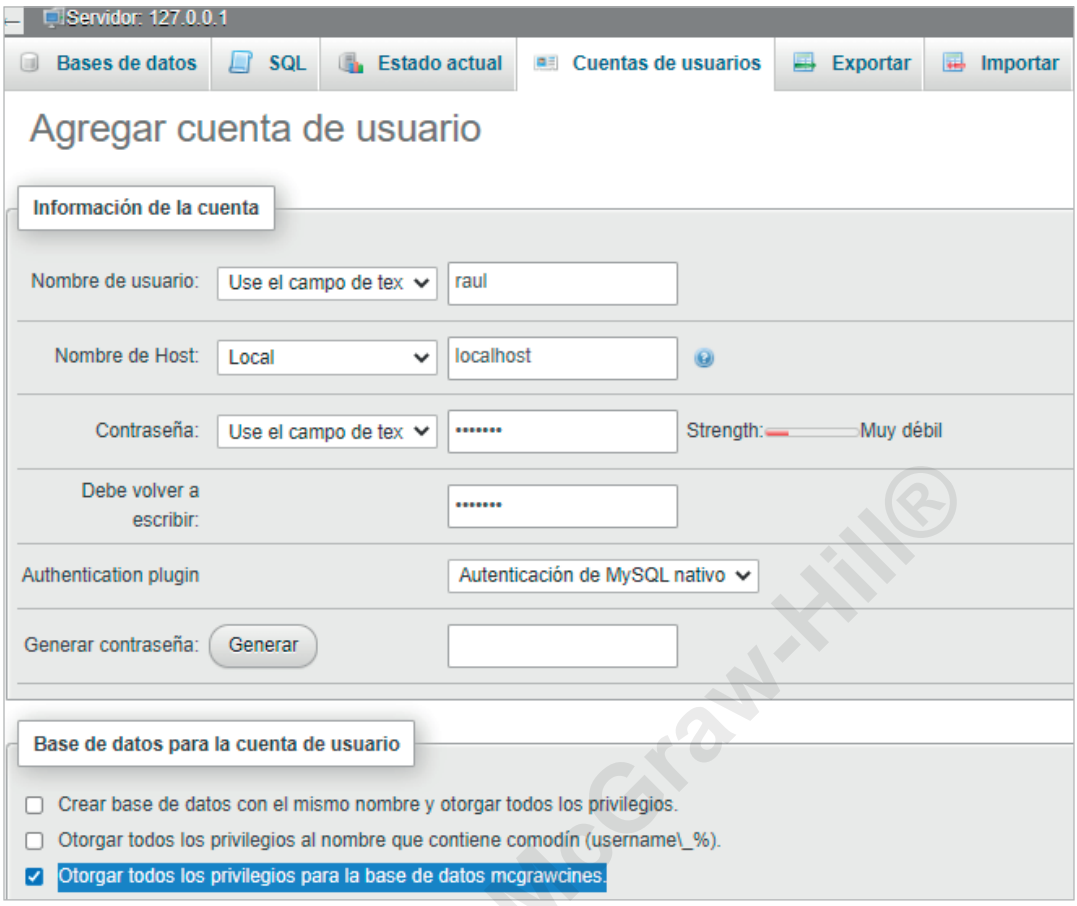
Como ejemplo, creamos un usuario con nombre "raul" autenticado con la contraseña "**rauluar**". Podríamos concederle todos los permisos marcando "**Seleccionar todo**" en "**Privilegios Globales**", lo cual marcará todos permisos del grupo de "**Datos**", "**Estructura**" y "**Administración**". Esto equivaldría a la concesión de todos los permisos a *raul* como si fuera root. Lo habitual es que solo la cuenta root tenga privilegios totales. Por eso, es recomendable activar antes la base de datos en la que queremos otorgarle permisos al usuario en cuestión. Por ejemplo, clicamos a la izquierda en la de MCGRAWCINES y luego, en la pestaña *Privilegios*, en el vínculo *Agregar cuenta de usuario*.



Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Aunque los campos a rellenar son los mismos que antes, ha aparecido una nueva casilla de verificación activada por defecto: **Otorgar todos los privilegios para la base de datos mcgrawcines.**



Además, podemos limitar el máximo número de consultas, actualizaciones o conexiones por hora, así como el número máximo de usuarios concurrentes. E incluso decidir si los usuarios han de conectarse con SSL para que el acceso se realice con una conexión segura HTTPS.



Los últimos bloques limitarán los recursos y configurarán el SSL de la base de datos para el usuario con phpMyAdmin.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

4.2. Concesión de permisos (GRANT)

Empezamos viendo cómo conceder permisos a todos estos objetos y gestionar a los usuarios dentro de esta. Dentro de ella iremos poniendo el resto de las opciones.

Descripción

La sentencia **GRANT** nos permite **conceder permisos a los usuarios** en nuestro SGBDR.

Sintaxis

```
GRANT priv_type [(column_list)] [, priv_type [(column_list)]] ...
    ON {tbl_name | * | *.* | db_name.*}
    TO user [IDENTIFIED BY [PASSWORD] 'password']
        [, user [IDENTIFIED BY [PASSWORD] 'password']] ...
    [REQUIRE
        NONE |
        [{SSL| X509}]
        [CIPHER 'cipher' [AND]]
        [ISSUER 'issuer' [AND]]
        [SUBJECT 'subject']]
    [WITH [GRANT OPTION | MAX_QUERIES_PER_HOUR count |
                          MAX_UPDATES_PER_HOUR count |
                          MAX_CONNECTIONS_PER_HOUR count |
                          MAX_USER_CONNECTIONS count]] [REQUIRE {NONE | tls_
option [[AND] tls_option ...] }]
  [WITH resource_option [resource_option ...]]
  [lock_option] [password_option]
```

Ejemplo de GRANT

```
# Privilegios por defecto para el usuario raul sobre la base de datos
mcgrawcines.
GRANT ALL PRIVILEGES ON `mcgrawcines`.* TO `raul`@`localhost` IDENTIFIED BY
PASSWORD '*B7CF8EE76575F27F3F4A57ECF3B426B177548330' REQUIRE SSL WITH GRANT
OPTION MAX_QUERIES_PER_HOUR 1 MAX_UPDATES_PER_HOUR 1 MAX_CONNECTIONS_PER_HOUR
1;
```

4.3. Revocar permisos (REVOKE)

Descripción

La sentencia **REVOKE** nos permite quitar los permisos a usuarios.

Sintaxis

Opción 1: versión rápida, lo quitamos todo.

```
REVOKE ALL PRIVILEGES, GRANT OPTION
    FROM usuario [,usuario] ...
```

Opción 2: versión específica, solo quitamos detallando lo que queremos.

```
REVOKE
    tipos_privilegios [(lista_columna)]
      [,tipos_privilegios [(lista_columna)]] ...
    ON [tipo_objeto] nivel_privilegio
    FROM usuario [,usuario] ...
```

Ejemplo

```
REVOKE ALL PRIVILEGES ON *.* FROM 'raul'@'%';
```

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

IMPORTANTE

La sentencia **`REVOKE ALL PRIVILEGES *.* FROM <usuario>`** retira todos los permisos del usuario que indiquemos. Así, si luego queremos concederle privilegios nos aseguramos de que no surgirán conflictos entre los que no se revocaron y los nuevos.

4.4. Otras sentencias relacionadas

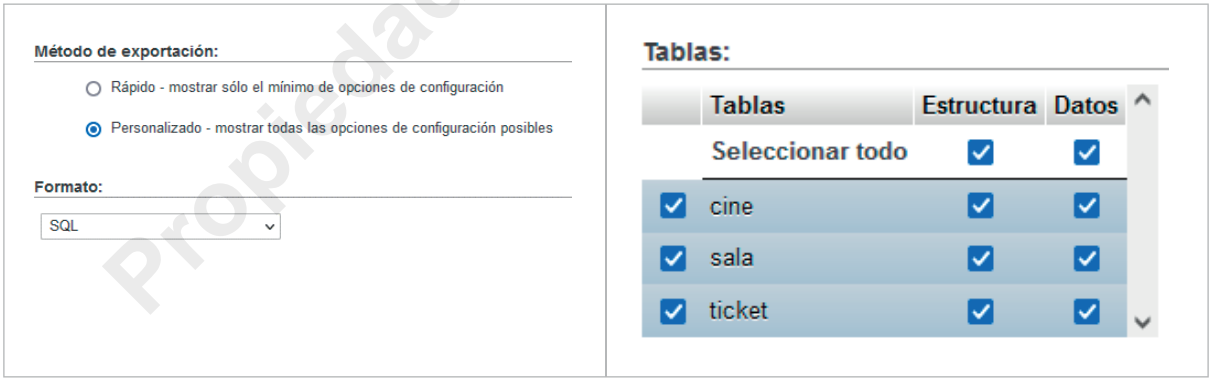
No vamos a poder entrar en detalle con todas las sentencias relacionadas con los usuarios y los permisos. Hemos visto las más importantes, pero os dejamos una relación de otras sentencias SQL para que las conozcáis y podáis buscar más información sobre ellas si os interesa profundizar.

- Para cambiar la contraseña que tenemos actualmente por una nueva: **`SET PASSWORD.`**
- Para gestionar roles de usuarios tenemos las sentencias: **`CREATE ROLE, DROP ROLE, SET ROLE / SET DEFAULT ROLE y MANDATORY ROLES`**.
- **`SESSION`**, para trabajar con sesiones.
- **`SHOW GRANTS FOR`**, para poder ver los permisos del usuario que le indiquemos.
- Para poder ver el usuario o perfil (rol) actual: **`CURRENT_USER / CURRENT_USER() / CURRENT_ROLE.`**
- Para quitar o vaciar de privilegios a algún usuario tenemos la sentencia **`FLUSH PRIVILEGES`**.

De estas, en el lenguaje de control de sesiones gestiona las sesiones de trabajo (SCL) con los diferentes roles de los usuarios. Agrupa las instrucciones **`ALTER SESSION`** y **`SET ROLE`**

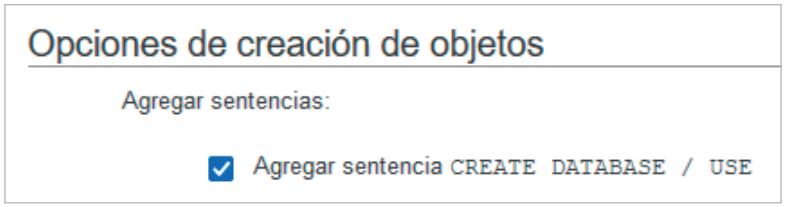
4.5. Crear copias de seguridad

En phpMyAdmin, para crear una copia de seguridad seleccionaremos una de las bases de datos, clicaremos sobre la pestaña *Exportar* y podremos hacer una exportación que contendrá las sentencias SQL que generan el estado actual de la base de datos. En el apartado "Método de exportación" podemos escoger, en lugar del rápido, el personalizado, indicando si solo queremos exportar una o varias tablas y si queremos solo sus estructuras o si también precisamos los datos que contienen actualmente.



IMPORTANTE

En la exportación personalizada podemos activar la casilla para crear automáticamente de la base de datos tras importarla. En caso contrario, deberemos crearla manualmente antes de llevar a cabo la importación.



Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Unidad 9.

SQL IV. Programación y TCL

Propiedad de McGraw-Hill®

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

1. Programación en SQL

El estándar actual, **SQL:2016**, compatible con ficheros JSON para la consola de la línea de comandos interactiva **SQL CLI**, **no distingue entre mayúsculas y minúsculas** (**case insensitive**) a la hora de escribir sus sentencias. Establece una interfaz de programación de aplicaciones (**API, application programming interface**) que han de utilizar nuestros programas para lanzar sentencias **SQL** de una forma normalizada a través de esta serie de funciones bien definidas en el estándar. De manera que los fabricantes de **SGBDR** pueden crear controladores que soporten el **API** de **SQL** y/o del controlador del lenguaje de programación; por ejemplo, **JDBC** (**Java Database Connectivity**) u **ODBC** (**Open Database Connectivity**), entre otros que vayamos a utilizar para embeber nuestras sentencias **SQL**.

	Comparativa entre JDBC y ODBC	
	JDBC (Java Database Connectivity)	**ODBC (Open Database Connectivity)**
Denominación	Programa de conectividad de BBDD desde Java.	Programa de conectividad abierta de BBDD.
¿Cuándo se creó?	En 1996, por la empresa Sun Microsystems, que fue adquirida por Oracle en 2009 y que se encarga de ir actualizándolo.	En 1992, por la empresa Microsoft, que se encarga de ir actualizándolo.
¿Qué es?	Define una API estándar solo para el lenguaje Java que permite definir cómo un usuario puede acceder a sus datos desde la base de datos.	Define una API estándar abierta a cualquier SGBDR que lo soporte.

Microsoft ha creado otra tecnología más potente llamada OLE DB (Object Linking and Embedding for Databases) que permite trabajar con todo tipo de BBDD, no solo las relacionales sino también con las de grafos, orientadas a objetos, nativas XML o NoSQL (que se tratan en el módulo de lenguaje de marcas). OLE DB está incluida dentro de los componentes de Microsoft para acceso a datos (MDAC, Microsoft Data Access Components).

1.1. SQL embebido en otros lenguajes de programación

Las aplicaciones de software suelen estar desarrolladas con un lenguaje de programación específico (como Java, C#, Python, etcétera), mientras que los Sistemas Gestores de Bases de Datos solo aceptan consultas en lenguaje SQL.

A través de estas bibliotecas: JDBC —que es una API de datos específica para el lenguaje de programación Java— u OBDC —más abierta para el resto de los lenguajes— podemos insertar de manera embebida nuestras instrucciones del lenguaje SQL dentro del código del **`programa.SQL`**: sintaxis y operaciones básicas

A continuación, se presentan las posibilidades que SQL nos facilita para elaborar algo parecido a un programa usando exclusivamente instrucciones del propio lenguaje SQL.

A. COMENTARIOS EN SQL

En primer lugar, veamos cómo escribir diferentes tipos de comentarios en SQL:

```
/*
* Como en Java o C podemos crear comentarios para varias líneas
consecutivas.
* Disponible desde el estándar SQL99.
*/
-- Con dos guiones creamos un comentario de una línea o lo que quede de
ella.
{Comentario con llaves de una línea o lo que quede de ella }
# Esto es un comentario de una línea o lo que quede de ella
REM Esto es un comentario de una línea o lo que quede de ella
```

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Puede que alguno de estos comentarios no esté soportado en todos los SGBD, como las llaves, la almohadilla o el comando **REM**, más típicos de programación de **Shell** scripts en sistemas operativos.

Como peculiaridad, hay que comentar que MySQL/MariaDB dispone también de una variante denominada *comentario ejecutable*.

```
/*!##### Código específico de MySQL o MariaDB a partir de la versión ##### */
```

Supongamos que se desea incrustar una porción de código que solo debe ser ejecutado si la versión de MySQL/MariaDB es la 5.1.0 o posterior. En ese caso introduciríamos el comentario ejecutable de la siguiente forma:

```
/*!50100 El código de MySQL y MariaDB 5.1.0 (y superior) va aquí. */
```

Por ejemplo, una de las características más demandadas por los usuarios que se introdujo en la versión de MySQL 8.0.16 fue la restricción **CHECK**. Si tenemos instalada una versión igual o superior a la 8.0.16, podremos hacer uso de ella, pero si no, no funcionará.

Si estamos desarrollando una mejora en una aplicación y no sabemos cuál es la versión de MySQL en la que va a aplicarla cada usuario, conviene que encapsulemos esa instrucción dentro de un comentario ejecutable para evitar que se muestren desagradables mensajes de error.

B. CARACTERÍSTICAS DE SQL

Algunas características importantes de SQL para tener en cuenta son:

- No exige que se especifique el tipo de las variables, se deduce por su contenido y uso.
- El nombre de las variables de usuario empieza por el signo arroba `'@'`. Puede incluir números y guiones bajos, pero no empezar con un número ni contener espacios o caracteres especiales distintos del guion bajo. Las palabras reservadas no pueden usarse en los nombres de las variables.
- Las variables del SGBD empiezan con dos signos de arroba `'@@'`.
- Las cadenas pueden delimitarse con comillas dobles ("**cadena**") o simples ('**cadena**').
- Las instrucciones acaban con un punto y coma.
- SQL no distingue entre mayúsculas y minúsculas **(case insensitive).**

1.2. Operadores de SQL

Los principales operadores de SQL son:

Aritméticos		De comparación		Lógicos	
+	Suma	=	Igual a	&&	y (**and**). Devuelve true si todas las expresiones evaluadas son verdaderas
-	Resta	!= o <>	Distinto de		
*	Multiplicación	>, >=	Mayor que, mayor o igual que	||	o (**or**). Devuelve false si todas las expresiones evaluadas son falsa.
/	División entera	<, <=	Menor que, menor o igual que		
%	Módulo (resto)	= o :=	Asignación de variables (depende del contexto en que se utilice)	!	no (**not**). Devuelve true si la expresión evaluada es falsa, y false si es verdadera.

1.3. Comando HELP

Disponemos de un comando de ayuda muy útil que nos dará información de cualquier otro del que no conozcamos o no recordemos la sintaxis.

```
HELP palabra_reservada_buscada
-- Por ejemplo CREATE, si aparece en varias opciones nos mostrara una lista
HELP CREATE
```

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

1.4. Variables de usuario en SQL

Podemos declarar variables en SQL desde la línea de comandos o en la pestaña *SQL* del asistente (phpMyAdmin, MySQL WorkBench o similar):

```
SET @variable = valor;
```

Una vez definida, podremos utilizarla para realizar otras instrucciones SQL.

```
SET @INICIAL="A%";
SELECT * FROM alumnado WHERE alumnado.nombreAlumnado LIKE @INICIAL;
```

Este comando mostrará la información del alumnado cuyo nombre empiece por "**A**".

NIA	nombreAlumnado	apellidosAlumnado
103569	Ana	Martínez
106574	Alejandro	Jiménez

Resultado de una consulta con variables.

También podemos asignarle un valor a una variable con el resultado de una consulta usando la cláusula **INTO** en la sentencia **SELECT**.

```
SELECT COUNT(*) INTO @cuantoAlumnado FROM alumnado;
```

Así guardamos la cantidad total de registros de la tabla ALUMNADO en la variable **@cuantoAlumnado**. Ahora podemos presentar un mensaje que muestra el resultado.

```
SELECT CONCAT("Total alumnado: ",@cuantoAlumnado) AS "Total alumnado";
```

Total alumnado
Total alumnado: 4

También podemos reasignar valores a las variables de usuario en sentencias SQL. Por ejemplo, si en la base de datos MCGRAWCINES queremos contar las filas que nos devuelve una consulta **SELECT**. Hay que tener en cuenta que dentro de la consulta **SELECT** la asignación se representa con **':='** a diferencia del símbolo **'='** que, como sabemos, es el operador de comparación por igualdad.

Así pues, para que solo se nos muestre los cines de VALENCIA:

```
SET @numerar = 0;
SELECT @numerar := @numerar + 1 AS Numerar,
       idCine,
       nombreCine,
       poblacionCine
FROM cine
WHERE poblacionCine = 'VALENCIA';

SELECT @numerar := @numerar + 1 AS Numerar, c.idCine, c.nombreCine,
c.poblacionCine FROM cine AS c WHERE c.poblacionCine = 'VALENCIA';
```

Podemos ver que nos ha creado una primera columna que va contando las filas que nos devuelve la consulta que hemos lanzado.

Numerar	idCine	nombreCine	poblacionCine
1	6	ABC Saler	Valencia
2	7	ABC Park	Valencia
3	8	Autocine Star	Valencia
4	9	Cines Babel	Valencia
5	10	Cines Lys	Valencia
6	11	Cinestudio d´Or	Valencia
7	12	Filmoteca Valencia	Valencia
8	13	Hemisfèric	Valencia
9	14	Ocine Aqua	Valencia
10	15	Yelmo Cines Mercado de Campanar	Valencia

Esta posibilidad de guardar el resultado de una consulta en una o más variables nos será muy útil más adelante.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

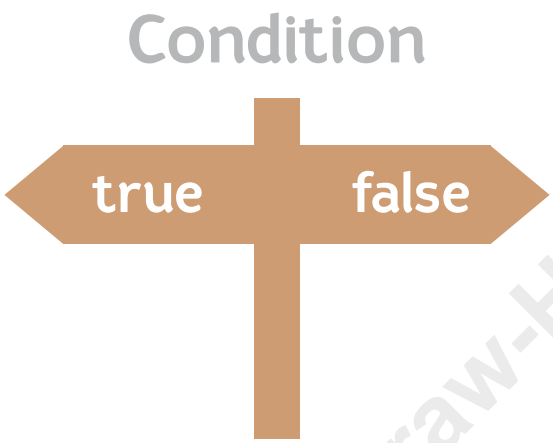
Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

2. Estructuras de control en SQL

Todo lenguaje de programación, y SQL no es una excepción, requiere de sentencias que permitan tomar decisiones a partir de condiciones que vamos planteando a lo largo de nuestro código. Las estructuras de control pueden ser tanto alternativas como repetitivas, del mismo modo que en otros lenguajes de programación.

2.1. Sentencias alternativas

Nos permiten determinar el flujo de ejecución que tomará nuestro algoritmo según el resultado de cierta condición. A menudo depende de la interacción con el usuario o del estado de la base de datos.



Sentencias alternativas ¿Qué camino coger?

A. SIMPLES: IF...ELSE

La sintaxis para realizar una bifurcación simple con la sentencia **IF...ELSE** es la siguiente:

Estructura IF...ELSE simple	ELSEIF enlazados

`IF condición THEN`
`...`
`ELSE`
`...`
`END IF;`
	`IF condición1 THEN`
	`...`
	`ELSEIF condición2 THEN`
	`...`
	`ELSE`
	`...`
	`END IF;`

Como vemos, la estructura **IF...ELSE** permite la elección simple (se cumple la condición o no) o la elección múltiple (evaluar varias condiciones enlazando varias sentencias con **ELSEIF**).

La estructura **IF...ELSE** solo se utiliza dentro de rutinas almacenadas. Para usar algo parecido desde la línea de comandos de SQL podemos hacer uso de la función **IF**, cuya sintaxis es:

```
IF(expr1, expr2, expr3)
```

La primera expresión es la condición a evaluar, la segunda expresión se utilizará cuando la condición sea cierta, y la tercera expresión se utilizará cuando sea falsa. Si devolvemos un valor en ambos casos, este puede ser asignado a una variable.

```
SET @variable = IF(condición, valorSiEsVerdadera, valorSiEsFalsa);
```

El ejemplo anterior que mostraba la cantidad de alumnado se podría mejorar de la siguiente forma:

```
SELECT COUNT(*) INTO @cuantoAlumnado FROM alumnado;
SET @respuesta = IF(@cuantoAlumnado > 0, CONCAT("Total alumnado:
",@cuantoAlumnado),"Todavía no se ha matriculado nadie");
SELECT @respuesta AS "Resultado consulta";
```

Si no hubiese nadie matriculado, la respuesta sería "Todavía no se ha matriculado nadie" en vez de "Total alumnado: 0".

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

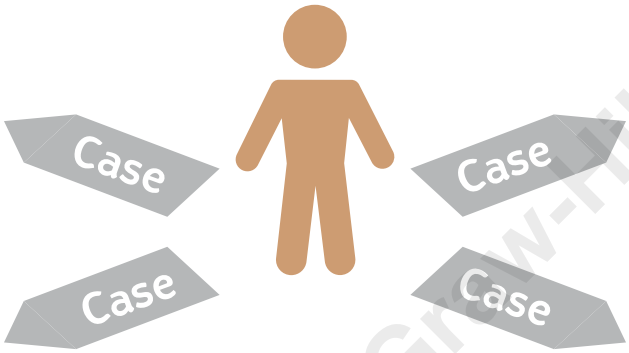
Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

B. MÚLTIPLES: CASE

La sintaxis de la estructura de selección múltiple **CASE** es:

```
CASE expresión
WHEN valor1 THEN
...
WHEN valor2 THEN
...
WHEN valorN THEN
...
ELSE
...
END CASE;
```

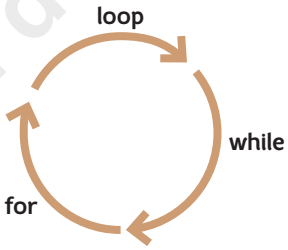
Usaremos la estructura **CASE** dentro de nuestras rutinas almacenadas.



Sentencia de selección múltiple ¿Qué camino coger entre todos?

2.2. Sentencias repetitivas

Tenemos tres tipos de bucles en SQL: **REPEAT**, **WHILE** y **LOOP**.



Diferentes tipos de bucles que podemos utilizar en SQL.

A. BUCLE MIENTRAS - WHILE

El bucle **WHILE**, como en el resto de los lenguajes de programación, se repite mientras se cumpla la condición especificada. El bucle **WHILE** puede no ejecutarse, puesto que la condición se evalúa antes de entrar. Por lo tanto, si no se cumple la primera vez, no se ejecutará.

Sintaxis WHILE	

`[begin_label:] WHILE search_condition DO`
`  statement_list`
`END WHILE [end_label]`
	`etiqueta: WHILE condición DO`
	`...`
	`END WHILE etiqueta;`

Ejemplo WHILE	

`SET @variable = 0;`
`WHILE @variable <= 10 DO`
`  SET @variable = @variable + 1;`
`END WHILE;`	

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

B. BUCLE REPEAT

El bucle **REPEAT** se diferencia del **WHILE** en que se ejecuta hasta que se cumpla la condición especificada. Además, siempre se repite al menos una vez, porque la condición se evalúa al final.

Sintaxis REPEAT

```
[begin_label:] REPEAT
    statement_list
UNTIL search_condition
END REPEAT [end_label]
```

```
etiqueta: REPEAT
…
UNTIL condición
END REPEAT etiqueta;
```

Ejemplo REPEAT

```
SET @variable = 0;
REPEAT
      SET @variable = @variable + 1;
UNTIL @variable > 9
END REPEAT;
```

C. BUCLE CIRCULAR - LOOP

En el bucle **LOOP** debemos decidir en qué momento queremos acabar (**LEAVE**). Según como creemos la condición, podemos convertirlo en un **WHILE** o en un **REPEAT**.

En esta sintaxis y ejemplo funciona como un **WHILE**: mientras cumpla la condición del **IF** se continuará ejecutando el bucle.

Bucle LOOP

```
[begin_label:] LOOP
    statement_list
    IF search_condition THEN
      ITERATE label;
    END IF;
    LEAVE label;
END LOOP [end_label]
```

```
etiqueta: LOOP
…
IF condición THEN
   ITERATE etiqueta; --reinicia
END IF;
…
LEAVE etiqueta; --finaliza
END LOOP etiqueta;
```

Ejemplo LOOP

```
SET @variable = 0;
etiqueta: LOOP
   IF @variable <= 10 THEN
       SET @variable = @variable + 1;
       ITERATE etiqueta; --reinicia
   END IF;
  LEAVE etiqueta; --finaliza
END LOOP etiqueta;
```

Los bucles se utilizarán en las funciones o los procedimientos de las rutinas almacenadas.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

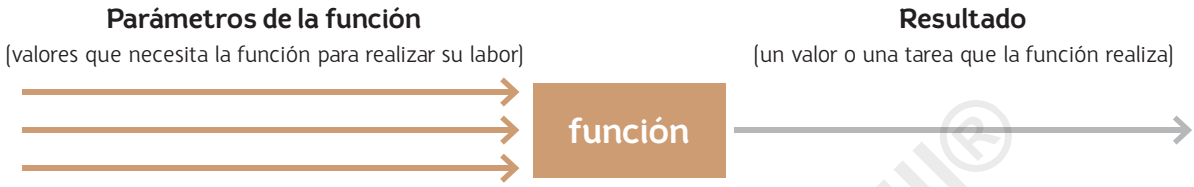
3. Funciones y procedimientos

3.1. Introducción

Igual que el resto de los lenguajes de programación, SQL proporciona funciones preinstaladas y nos permite crear funciones o procedimientos "a medida". Son las funciones y procedimientos de usuario.

A. FUNCIONES DE USUARIO

Una función de usuario es un bloque de código que, a partir de unos datos de entrada opcionales, devuelve un valor como resultado.



Obviamente, para poder utilizar una función de usuario, antes la tendremos que definir. Dicha definición consta de los bloques **cabecera**, **inicio**, **cuerpo** y **final**.

```
CREATE FUNCTION nombre_funcion ([parámetro tipo, parámetro tipo…]) RETURNS tipo
BEGIN
…
RETURN resultado;
END;
```

La **cabecera**, que suele escribirse en una línea, comienza con las palabras reservadas **`CREATE FUNCTION`**. A continuación, se indica el nombre de dicha función seguido de, entre paréntesis, los parámetros de entrada (si los hay) especificando el tipo de cada uno. Tras cerrar el paréntesis escribiremos la palabra reservada **`RETURNS`** y el tipo de dato del resultado que devuelve.

El bloque de **inicio** consiste únicamente en la palabra reservada **`BEGIN`**.

En las líneas sucesivas escribimos el **cuerpo** con el código de la función. La última línea del cuerpo está formada por la palabra reservada **`RETURN`** y el resultado que devuelve la función seguido del símbolo "`;`".

El bloque **final** consta de la palabra reservada **`END`** seguida del símbolo "`;`".

Si creamos la función desde la línea de comandos o la pestaña de SQL del asistente tendremos que cambiar el delimitador de instrucciones (opción **`DELIMITER`** del cliente MySQL/MariaDB) por otro que no sea el punto y coma "`;`", que es el valor por defecto. De no hacerlo, al llegar al primer punto y coma se ejecutaría la instrucción sin esperar a que la función estuviese definida del todo, por lo que es obligatorio hacer este cambio para poder definir correctamente la función.

Veamos un ejemplo de función que suma dos números y devuelve el resultado:

```
DELIMITER // -- Cambiamos el delimitador a las dos barras.
CREATE FUNCTION suma(num1 INT, num2 INT) RETURNS INT
BEGIN
DECLARE resultado INT;
SET resultado=num1+num2;
RETURN resultado;
END //
DELIMITER ; -- Volvemos a cambiar el delimitador a punto y coma.
```

Como vemos, dentro de las funciones, las variables se crean con la instrucción **`DECLARE`**, especificando de qué tipo son de manera obligatoria. Posteriormente, con **`SET`** les podemos asignar valores.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Una vez definida la función podremos usarla, por ejemplo, dentro de una **SELECT**. Para ello la invocamos indicando el nombre de la función y, entre paréntesis, los argumentos de llamada en el mismo orden en que definimos los parámetros cuando la creamos.

```
SELECT suma(10, 20) AS "Resultado de la suma";
```

Resultado de la suma
30

Resultado de la llamada a la función suma.

Por supuesto, no vale la pena crear una función solo para que sume dos números. Veamos un ejemplo más interesante: vamos a crear una función a la que le pasaremos el código de un módulo y el código de su ciclo para que nos devuelva la cantidad de aprobados.

En el cuerpo usaremos una consulta **SELECT** que filtrará el resultado a partir de los parámetros introducidos.

```
DELIMITER //
CREATE FUNCTION aprobados(ciclo INT, modulo VARCHAR(4)) RETURNS INT
BEGIN
DECLARE totalAprobados INT;
SELECT COUNT(*) INTO totalAprobados FROM matricula WHERE idModulo = modulo
AND idCiclo = ciclo AND nota >= 5;
RETURN totalAprobados;
END //
DELIMITER ;
```

Ahora podemos ver cuánta gente ha aprobado FOL (**idModulo=0493**) en el ciclo de DAM (**idCiclo=3**):

```
SELECT aprobados(3,"0493") AS "Aprobados de FOL en primero de DAM";
```

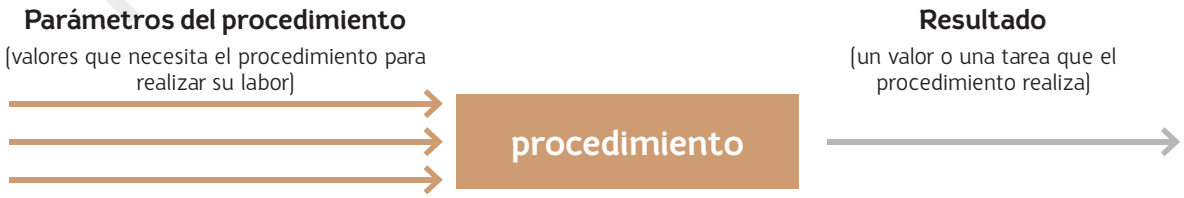
O en el ciclo de DAW (**idCiclo=4**):

```
SELECT aprobados(4,"0493") AS " Aprobados de FOL en primero de DAW";
```

A.PROCEDIMIENTOS DE USUARIO

Las funciones tienen una limitación: solo pueden devolver un valor. ¿Y qué ocurre si necesitamos que una función devuelva varios valores o ninguno? Pues la solución pasa por usar procedimientos.

Un procedimiento, a diferencia de una función, no devuelve valores directamente. Los datos que queremos que devuelva se los pasaremos como hicimos en las funciones con los parámetros de entrada, pero precediéndolos de la palabra reservada **OUT**. Los parámetros "normales" de entrada se definen como **IN**. Finalmente, si algún parámetro de entrada fuese también parámetro de salida, se definiría como **INOUT**.



Procedimientos en SQL.

La sintaxis para crear un procedimiento es muy similar a la de las funciones. No hay límite para el numero de parámetros, y podemos poner tantos como requiramos.

```
CREATE PROCEDURE nombre_procedimiento ([{IN|OUT|INOUT} nombre_parámetro
tipo_dato, {IN|OUT|INOUT} nombre_parámetro tipo_dato {,…}])
BEGIN
…
END;
```

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Como vemos, a diferencia de la sintaxis de las funciones de usuario, no hay cláusulas **RETURNS** ni **RETURN**.

Veamos cómo sería nuestra función suma si la definiéramos como procedimiento.

```
DELIMITER // -- Cambiamos el delimitador a las dos barras.
CREATE PROCEDURE suma_proc(IN num1 INT, IN num2 INT, OUT resultado INT)
BEGIN
SET resultado=num1+num2;
END //
DELIMITER ; -- Volvemos a cambiar el delimitador a punto y coma.
```

Observemos que pasamos el resultado como parámetro **OUT** .

Si lo pasamos como parámetro **INOUT** nos podría servir para calcular un sumatorio de varios valores:

```
DELIMITER // -- Cambiamos el delimitador a las dos barras.
CREATE PROCEDURE sumatorio_proc(IN num1 INT, INOUT sumatorio INT)
BEGIN
SET sumatorio = sumatorio +num1;
END //
DELIMITER ; -- Volvemos a cambiar el delimitador a punto y coma.
```

Ahora, desde la línea de comandos SQL podemos escribir:

```
SET @numero1=10;
SET @numero2=15;
CALL suma_proc(@numero1, @numero2, @numero3);
SELECT @numero3 AS "El resultado de la suma es";
SET @sumatorio=0;
CALL sumatorio_proc(@numero1, @sumatorio);
SELECT @sumatorio AS "El resultado del sumatorio es";
CALL sumatorio_proc(@numero2, @sumatorio);
SELECT @sumatorio AS "El resultado del sumatorio es";
CALL sumatorio_proc(@numero3, @sumatorio);
SELECT @sumatorio AS "El resultado del sumatorio es";
```

Dado que un procedimiento no devuelve nada, no se puede llamar con una **SELECT**. Lo hacemos con un **CALL**, pasándole tanto las variables **IN (@numero1** y **@numero2)** como las **OUT (@numero3)** o las **INOUT (@sumatorio)**. Después del procedimiento sí que podemos consultar el valor de **@numero3** con una **SELECT**.

IMPORTANTE

Las funciones y los procedimientos pueden crearse de una manera más sencilla desde un asistente tipo phpMyAdmin, MySQL Workbench o HeideSQL. También puedes probarlos desde el mismo asistente. Dispones de más funciones y procedimientos en el material complementario "Más ejemplos de funciones y procedimientos", en la sección "Documentos de descarga de la unidad".

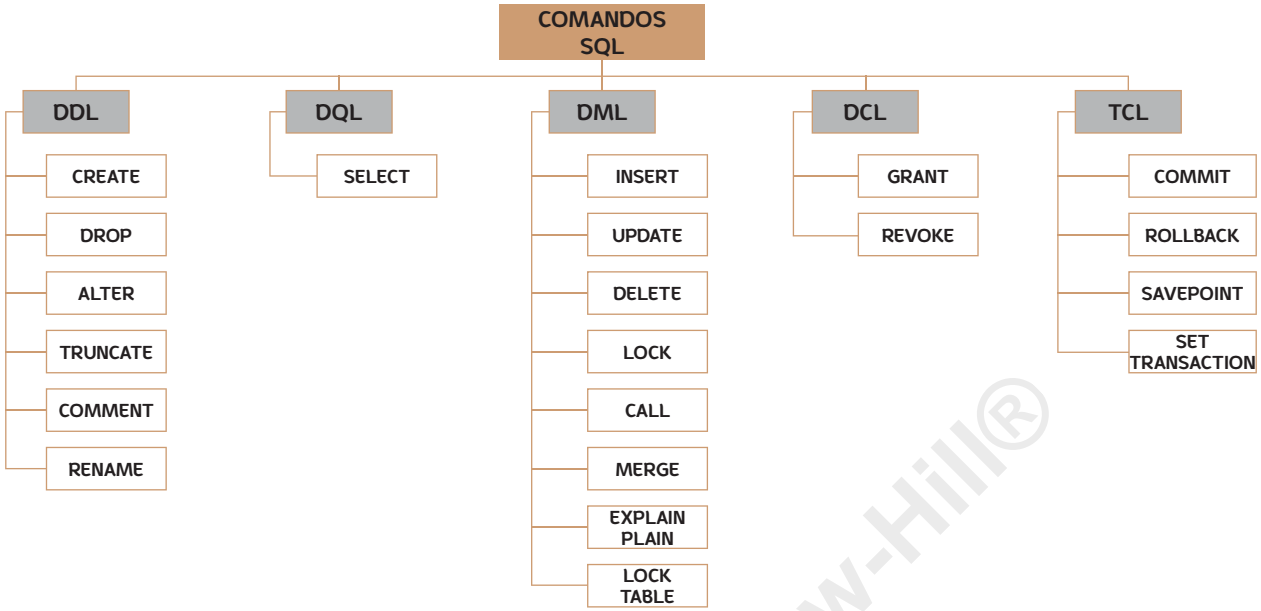
Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

4. TCL: Transacciones y políticas de bloqueos

Las instrucciones, las sentencias o los comandos SQL y sus funciones o procedimientos se pueden agrupar dependiendo de su utilidad o funcionalidad en seis grandes bloques:



Los 6 bloques de sublenguajes de SQL.

Ahora toca aprender el lenguaje de control de transacciones (TCL). Veremos qué es una transacción y lo que representa, teniendo en cuenta los tipos de bloqueos que se pueden aplicar al utilizarlas.

TCL (Transaction Control Language)	El lenguaje de control de transacciones agrupa las instrucciones **`SET TRANSACTION`**, **`COMMIT`**, **`ROLLBACK`** y **`SAVEPOINT`**.

Pero antes de empezar con los contenidos de este apartado vamos a recordar que algunas de las características fundamentales de las bases de datos son asegurar su integridad, minimizar las redundancias, facilitar la compartición y que sean independientes de las aplicaciones de usuario. Pues bien, las transacciones y las políticas de bloqueos nos van a permitir seguir cumpliéndolas con las medidas de seguridad que se van a aplicar con ellas.

4.1. Transacciones

Una **transacción** es un conjunto de instrucciones SQL que funcionan como una única operación. Es decir, es un bloque indivisible: o se llevan a cabo todas las operaciones hasta el final o no se lleva a cabo ninguna. Por tanto, necesitaremos mecanismos para detectar los errores y revertir los cambios realizados hasta el momento en caso de no poder completar toda la transacción.

Una **transacción** sobre una base de datos debe cumplir lo que se conoce como las propiedades **ACID**.

Propiedades ACID		
A	Atomicidad (Atomicity)	Todas las operaciones de la transacción deben haberse hecho correctamente. En caso contrario, se restaurará el estado del inicio de la transacción.
C	Consistencia (Consistency)	Al finalizar la transacción, la base de datos debe mantener su integridad y la consistencia de sus datos.
I	Aislamiento (Isolation)	Los cambios realizados en el transcurso de una transacción no son visibles por otras transacciones ni por cualquier operación externa a la transacción hasta que esta termina completamente.
D	Durabilidad (Durability)	Los cambios realizados por la transacción, en caso de haber llegado hasta el final, deben ser persistentes.

La idea de transacción de la instrucción **`TRANSACTION`** consiste en encapsular todas las operaciones que queremos hacer sin interrupciones en un mismo bloque de código SQL para, posteriormente, usar o bien la instrucción **`COMMIT`**, y con ello hacer los cambios en la base de datos, o bien usar **`ROLLBACK`** para anular dichos cambios y volver al estado anterior antes de haber iniciado la transacción.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Por ejemplo, supongamos que tenemos una tabla con cuentas bancarias en el campo **numeroCuenta** (que sabemos que tienen 20 cifras, pero aquí utilizaremos solo tres para simplificarlo) y sus saldos en el campo **saldo**. Queremos hacer una transferencia de **100 €** desde la cuenta **111** a la **222**. Imaginemos que escribimos las instrucciones de modificación (**UPDATE**) que se han de ejecutar en este orden:

```
UPDATE cuenta SET saldo = saldo + 100 WHERE numeroCuenta="222";
UPDATE cuenta SET saldo = saldo - 100 WHERE numeroCuenta="111";
```

Lógicamente, ambas operaciones deberían forman un conjunto indivisible. Si, por algún motivo, fallara el segundo **UPDATE**, la base de datos perdería su consistencia (la C de nuestro ACID) porque una cuenta tendrá 100 € más, pero la otra no tendría 100 € menos.

Por ejemplo, esta inconsistencia podría ocurrir si escribiésemos mal la segunda instrucción:

```
UPDATE cuenta SET saldo = saldo + 100 WHERE numeroCuenta="222";
UPDATE cuenta SET saldo = saldo - 100 WHERE numerosCuenta="111";
```

Al poner "**numerosCuenta**" en el segundo **UPDATE**, un nombre de columna que no existe, se producirá un error. Pero el primer **UPDATE** ya se habrá ejecutado. Por tanto, han aparecido 100 € de la nada en una cuenta sin haberlos quitado de la otra.

Podríamos pensar que, si cambiamos el orden de las instrucciones, se hubiera solucionado este problema. Tendríais razón, en parte. Para este caso de dos instrucciones podría funcionar: al cometer el error en la primera instrucción no pasaría nada, puesto que la segunda no se llegaría a ejecutar y, por tanto, la base de datos no perdería su consistencia.

```
UPDATE cuenta SET saldo = saldo - 100 WHERE numerosCuenta="111";
UPDATE cuenta SET saldo = saldo + 100 WHERE numeroCuenta="222";
```

Pero una transacción puede que tenga decenas o incluso cientos de instrucciones, por lo que no podemos depender de que el programador sepa en qué orden ha de colocarlas.

En estos casos, la solución es crear una transacción:

```
START TRANSACTION;
UPDATE cuenta SET saldo = saldo + 100 WHERE numeroCuenta="222";
UPDATE cuenta SET saldo = saldo - 100 WHERE numerosCuenta="111";
COMMIT;
```

Al ejecutar este código solo se llevarán a cabo los cambios al llegar al **COMMIT**. Si el segundo **UPDATE** provoca un error no se ejecutará el **COMMIT** y los cambios efectuados por el primero tampoco quedarán registrados. Por así decirlo, estamos llevando a cabo los cambios en una memoria temporal, pero no se fijarán como válidos hasta que terminemos y los confirmemos (**COMMIT**), validando así la transacción como correcta.

Si en lugar de ejecutar la transacción como un bloque desde una aplicación o desde la pestaña SQL de un asistente estamos trabajando interactivamente desde la línea de comandos y lanzando una a una las instrucciones, actuaríamos de manera similar. Empezaríamos con **START TRANSACTION** y acabaríamos con **COMMIT**. O, en caso de detectar un error en alguna operación intermedia, podríamos lanzar un **ROLLBACK** y devolver la base de datos al estado previo al inicio de la transacción.

IMPORTANTE

En la mayoría de bases de datos relacionales existe una variable de entorno llamada **autocommit** o similar. Si su valor es **1** o **ON**, cualquier operación que se haga dentro de una transacción se guardará inmediatamente en la base de datos (como si tras cada una hiciésemos un **COMMIT**).

Si preferimos que el **COMMIT** sea siempre manual, deberíamos modificar el valor de dicha variable de entorno y ponerla a **0** u **OFF** con la instrucción:

```
SET autocommit = 0;
```

Podemos consultar el valor actual de la variable con:

```
SELECT @@autocommit;
```

Más información del desarrollador sobre el funcionamiento de **autocommit**, **COMMIT** y **ROLLBACK** en los enlaces de la unidad.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

4.2. Políticas de bloqueos

El acceso simultáneo de varios usuarios a una misma base de datos conlleva posibles problemas de concurrencia. Esto puede proceder de dos tipos de situaciones:

- Dos usuarios intentando modificar los mismos datos.
- Un usuario viendo información que otro usuario está modificando y que pronto será obsoleta.

El Sistema Gestor de Bases de Datos debe asegurar la consistencia de la información que se ofrece al usuario y también la integridad de los datos. Para ello es necesario:

- Bloquear los datos que se están modificando para que no se lancen otros cambios sobre ellos.
- Mostrar únicamente la información guardada realmente en la base de datos, reflejando los cambios solo cuando estén completos. Todos los usuarios deben ver, en un momento dado y salvo restricciones por permisos, los mismos datos.

En general, la mayor parte de motores de bases de datos realiza los bloqueos de manera automática. El motor InnoDB, ampliamente utilizado, lleva a cabo de manera automática bloqueos a nivel de fila, de manera que otro usuario no pueda leer o escribir dicha fila mientras esté bloqueada.

A nivel de tabla, los bloqueos pueden hacerse con **`LOCK TABLE`**:

```
LOCK TABLE nombreTabla tipoBloqueo;
LOCK TABLES nombreTabla tipoBloqueo, nombreTabla tipoBloqueo…;
```

El tipo de bloqueo puede ser:

Opción	Tipo de bloqueo
READ	No permite escribir sobre los registros bloqueados, pero sí leerlos. Otros accesos concurrentes también podrán leerlos.
WRITE	Bloqueo exclusivo: los registros bloqueados pueden ser modificados y leídos por el usuario que los bloquea, pero no por el resto.
LOW_PRIORITY WRITE	Da prioridad a las operaciones de escritura sobre las de lectura, que deben esperar a que no haya ninguna escritura pendiente.

En general no usaremos **`LOCK TABLE`** si el motor de base de datos es InnoDB. En caso de bloquear una o varias tablas, debemos recordar ejecutar un **`UNLOCK TABLES`** para desbloquearlas cuando terminemos las operaciones. Como se acaba de mencionar, el bloqueo para las filas es automático con InnoDB, aunque el tipo de bloqueo puede variar en función de ciertas variables del sistema o del tipo de aislamiento que indiquemos al hacer una transacción.

InnoDB permite realizar ciertos tipos de bloqueo para los campos autoincrementables, de manera que la sucesión de valores asignados sea correcta. Puedes ver más información sobre este tipo de bloqueos en el siguiente enlace: **https://mariadb.com/kb/en/auto_increment-handling-in-innodb/.**

IMPORTANTE

Algunos Sistemas Gestores de Bases de Datos, en función del motor utilizado, permiten hacer bloqueos selectivos de varias filas para determinados tipos de operaciones. Por ejemplo:

```
SELECT * FROM matricula WHERE idCiclo = 3 FOR SHARE;
```

Otras transacciones podrán leer los registros bloqueados, pero no modificarlos ni borrarlos.

```
SELECT * FROM matricula WHERE idCiclo = 3 FOR UPDATE;
```

Otras transacciones no podrán modificar ni borrar los registros bloqueados, ni tampoco aplicar otro tipo de bloqueo sobre ellos. Solo podrán leerse en determinados niveles de aislamiento.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

5. SQL/JSON

Las siete partes del estándar SQL:2016 son estas. Nosotros vamos a centrarnos en la sexta parte, que trata la creación de una pasarela SQL/JSON para poder unir ambos mundos o entornos.

Partes del estándar SQL:2016 en la web	
ISO/IEC TR 19075-1:2011	Part 1: XQuery Regular Expression Support in SQL
ISO/IEC TR 19075-2:2015	Part 2: SQL Support for Time-Related Information
ISO/IEC TR 19075-3:2015	Part 3: SQL Embedded in Programs using the JavaTM programming language
ISO/IEC TR 19075-4:2015	Part 4: SQL with Routines and types using the JavaTM programming language
ISO/IEC TR 19075-5:2016	Part 5: Row Pattern Recognition in SQL
ISO/IEC TR 19075-6:2017	Part 6: SQL support for JavaScript Object Notation (JSON)
ISO/IEC TR 19075-7:2017	Part 7: Polymorphic table functions in SQL

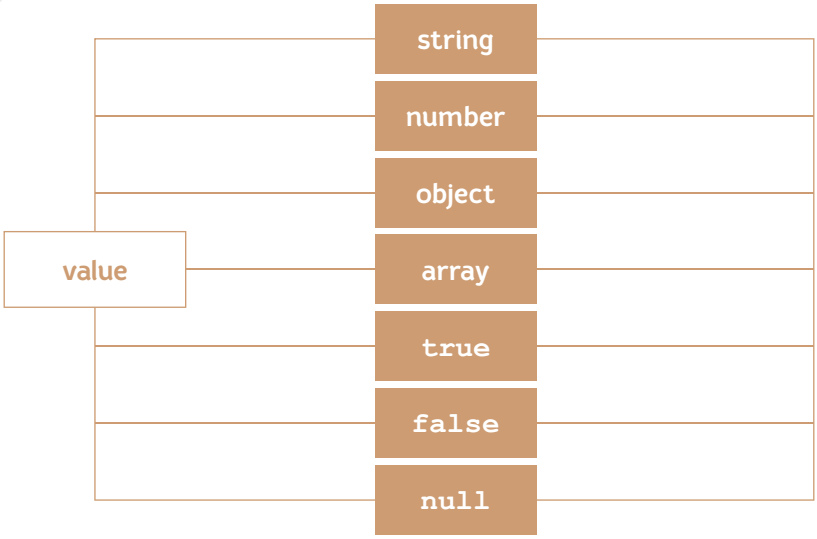
5.1. JSON

JSON es un formato independiente del lenguaje de programación. Los servicios que comparten información utilizando este procedimiento disponen de bibliotecas para codificar y decodificar cadenas JSON, de manera que, por ejemplo, el emisor puede ser una aplicación hecha con PHP u otro lenguaje y el receptor otra aplicación; esta vez, hecha con Java o JavaScript u otro lenguaje. Es muy utilizada en el mundo de las bases de datos NoSQL, a veces llamado "no solo SQL" (Not Only SQL).

A. TIPOS DE DATOS EN JSON

En JSON disponemos de los siguientes tipos de elementos:

- **Cadena (`string`)**: una cadena alfanumérica es una secuencia establecida de cero o más caracteres Unicode. Las cadenas se escriben entre comillas dobles (" ").
- **Número (`number`)**: pueden ser enteros, decimales o en notación exponencial.
- **Objeto (`object`)**: son colecciones desordenadas de parejas de valores.
- **Matriz (`array`)**: son listas de valores ordenados de `0` a más valores. Su rango es de `0` a `N-1`, siendo `N` el número de elementos incluidos en este.
- **Valor booleano**: admite valores de verdadero con `true` o falso con `false`. Los valores se escriben sin comillas.
- **Valor vacío**: admite el valor nulo con `null`. Se escriben sin comillas.



Tipos de datos y valores soportados en JSON.

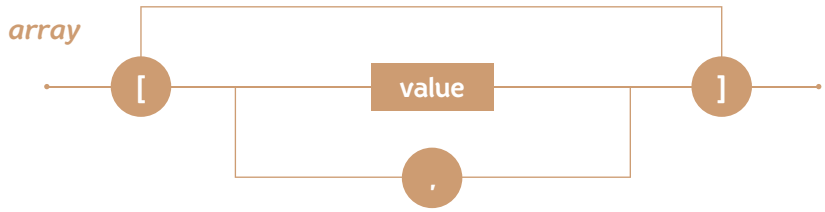
Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

B. SINTAXIS DE JSON

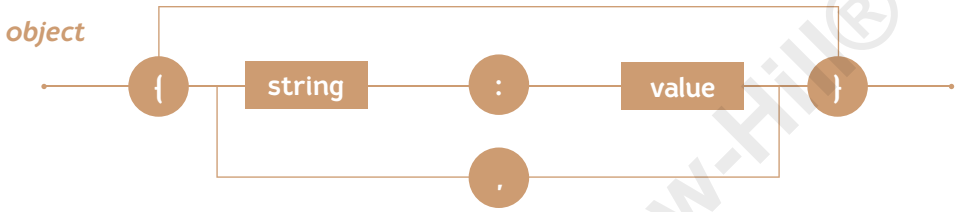
En JSON disponemos de dos tipos de elementos estructurados:

- **matrices** (**`arrays`**): son listas de valores separados por comas y se escriben entre corchetes **`[ ]`**, pudiendo contener dentro de ellos objetos JSON.



Sintaxis de los arrays en JSON.

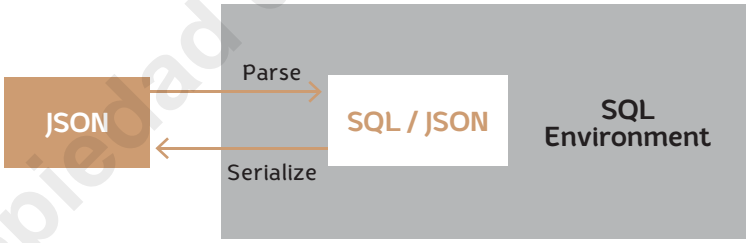
- **objetos** (**`objects`**): son listas de parejas **`nombre:valor o clave:valor`**. Las parejas están separadas por comas. Los objetos se escriben entre llaves **`{ }`**, y los nombres o claves de las parejas se escriben siempre entre comillas dobles. Y el valor, solo si son cadenas de texto.



Sintaxis de los objetos en JSON.

5.2. SQL/JSON

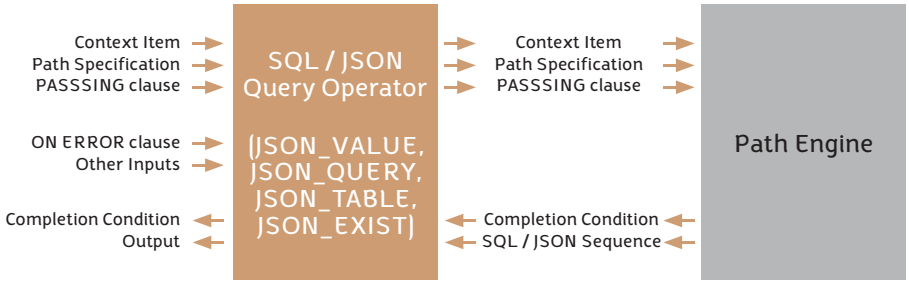
A partir de este nuevo estándar SQL:2016, tanto el emisor como el receptor van a poder codificar y decodificar estas cadenas en formato JSON con sentencias SQL. Este estándar diseña un elemento llamado SQL/JSON que entienda ambos lenguajes, y que con la función de análisis (**`parse`**) sepa convertir esos valores JSON a formato SQL y viceversa, y además, con la función de serializar (**`serialize`**), volver a fabricar en serie los valores según sus tipos de datos y su sintaxis en JSON.



Relaciones entre "JSON" y "SQL/JSON" del estándar SQL:2016.

Esta nueva biblioteca SQL/JSON que añade nuevas funciones a SQL son básicamente estos operadores de consulta (**`Query Operator`**) que importan o exportan valores, matrices u objetos desde una cadena JSON:

- **`JSON_VALUE`**: nos permite extraer un valor escalar.
- **`JSON_QUERY`**: nos permite extraer un objeto o una matriz.
- **`JSON_TABLE`**: nos devuelve una tabla.
- **`JSON_EXISTS`**: nos permite saber si existe una expresión JSON, devolviendo verdadero.



La arquitectura del lenguaje de ruta de acceso "SQL/JSON" del estándar SQL:2016.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Con estas funciones, y algunas más que cada fabricante irá implementando, vamos a ser capaces de importar o exportar nuestros datos en ambos sentidos. Para más información sobre todas las funciones JSON disponibles, ver la página oficial de MariaDB.

5.3. SQL DQL/DML vs JSON

Como ya sabemos, disponemos de un nuevo tipo de datos en el estándar SQL:2016 de tipo JSON en MySQL desde la versión 5.7.8 para almacenar vectores (**arrays**) de texto que contenga contenido JSON con el que queramos trabajar. El SGBDR la creará con el tipo de dato **longtext** de SQL, que es el más grande, donde permite incluir hasta 4 GB caracteres.

Disponer de este tipo de datos nativo en la base de datos conlleva varias ventajas:

- Podremos validar los formatos automáticamente.
- Podremos normalizar y unir valores JSON.
- Podremos disponer de las funciones espaciales llamadas SQL/JSON.

A. AÑADIR UN CAMPO DE TIPO JSON

Por ejemplo, vamos a crear un nuevo campo llamado **actorJSON** al final de la tabla actor de la base de datos MCGRAWPELICULAS, donde guardaremos en formato JSON los valores de la fila:

```
ALTER TABLE actor ADD actorJSON JSON NOT NULL AFTER nacionalidadActor;
```

B. ACTUALIZAR EL VALOR DEL CAMPO JSON

Ahora vamos a rellenar el campo llamado **actorJSON** para el registro de Harrison Ford:

```
UPDATE actor SET actorJSON = ' [{"idActor":2,"nombreActor":"Harrison
Ford","nacionalidadActor":1}] ' WHERE actor.idActor = 2;
```

C. SELECT CON JSON

Podremos empezar a fusionar ambos mundos uniendo funciones y expresiones de JSON dentro de las consultas SQL que ya conocemos. Por ejemplo, otras funciones interesantes para manipularlos son:

- **JSON_EXTRACT**: Nos permite extraer el contenido del tipo JSON.

Si queremos extraer el contenido del campo **actorJSON** del actor Harrison Ford:

```
SELECT JSON_EXTRACT(actorJSON, '$')
FROM ACTOR
WHERE actor.idActor = 2;
```

Nos debería devolver el contenido del array JSON con los diferentes objetos de parejas *clave: valor* que hemos introducido antes con la consulta **UPDATE**. El símbolo **$** de la consulta representa el array JSON con los tres objetos que tenemos en dicho campo.

```
[{"idActor":2,"nombreActor":"Harrison Ford","nacionalidadActor":1}]
```

Así, con cualquier otra consulta con la ayuda del resto de funciones JSON vamos a poder ir ampliando la potencia de las acciones que queramos realizar con ellas.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Unidad 10.

Bases de datos en la nube: SQL y NoSQL

Propiedad de McGraw-Hill®

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

1. Paradigmas NoSQL vs SQL: escenarios de uso

Como se ha tratado en las primeras unidades y todos sabemos:

> Una base de datos es una herramienta para recopilar, organizar y relacionar datos para buscar de una forma rápida y con consultas más o menos complejas, según objetivos.

Aunque esta definición es perfectamente válida, a lo largo de las últimas dos décadas, y al calor de la expansión de las apps, las redes sociales y el ubicuo uso de smartphones para cualquier cosa, han ido surgiendo necesidades especiales de rendimiento y escalabilidad a las que las bases de datos relacionales clásicas no han podido dar cobertura, o no de una forma lo suficientemente óptima.

En definitiva, la limitada concepción original de las bases de datos no ha podido digerir millones de usuarios concurrentes a nivel mundial de las últimas dos décadas, usando Facebook, Twitter, Angry Birds, Candy Crush o Netflix con tiempos de respuesta necesariamente muy cortos.

Por otro lado, también debemos asumir que el mundo del dato está sufriendo grandes transformaciones en la era del Big Data y el Data Science, o ciencia de los datos. Ya no se trata solo de almacenar información organizada y relacionada que tiene que ver con el día a día de una empresa (tales como clientes, facturas o artículos), sino de ampliar el rango de lo almacenado (dispositivos, uso del carrito de compra, tiempos de reacción, frecuencia de visitas, circulación de stocks e inventario) para poder analizar estadísticamente y mediante inteligencia artificial esos datos, generar modelos de previsión y ayudar a la toma de decisiones que anticipen los acontecimientos.

Estos dos hechos, la explosión del uso de webs y apps, junto con el auge del Big Data y la ciencia de los datos, ha determinado la generación de un extensísimo ecosistema de paradigmas y herramientas que tienen en el *cloud* su hábitat natural:



Fuente: fragmento de "2021 Machine Learning, AI and Data (MAD) Landscape" **mattturck.com/data2021**

¿Parece muy complejo? En realidad, es un ínfimo fragmento de la visión general del estado de herramientas y servicios de Data Science actualmente (puedes verlo en el enlace). Por un momento vamos a olvidarnos de la ciencia del dato y simplifiquemos dejando a un lado todas las herramientas y los servicios de análisis, los modelos de previsión y la inteligencia artificial.

> Aun centrándonos solo en la visión limitada del estado y el uso de bases de datos allá por 2016, se puede concluir que es un amplio y dinámico universo de difícil descripción. Te recomendamos que vayas a la infografía del enlace *"State of the database landscape"* de la consultora 451 Research. Ahí podrás ver una representación clasificada a modo de mapa estaciones de metro, de gran interés para comprenderlo.

1.1. SQL vs. NoSQL: comparativa

En primer lugar, vayamos a lo conocido: las bases de datos relacionales tradicionales, a las que llamaremos "soluciones SQL". Son la estructura fundamental de las bases de datos, y su presencia es ubicua y estandarizada porque son la elección ideal para gran parte de los casos.

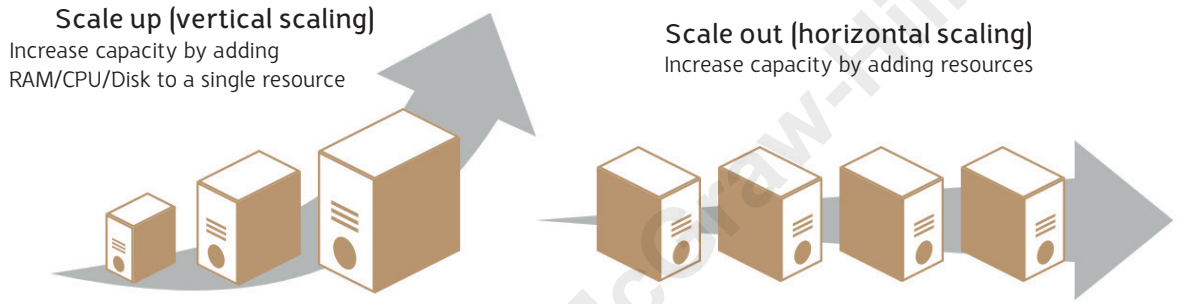
Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

En las soluciones SQL tenemos un modelo de datos definido y uniforme a todos los elementos, con relaciones bien trabajadas y un enfoque centrado en la calidad de los datos y en la consistencia. Si el sistema tiene que escalar lo podemos hacer verticalmente sin paralelizar nada. No necesitamos una gran capacidad de lecturas/escrituras por segundo ni un rendimiento extremo en la ejecución de consultas, las cuales tienen la enorme ventaja de programarse en SQL estándar.

IMPORTANTE

Un concepto fundamental de los sistemas en la nube es el "escalado horizontal" frente al "escalado vertical". Cuando queremos "escalar" un sistema nos referimos a aumentar su dimensión porque se nos ha quedado pequeño para nuestras necesidades. En el escalado vertical simplemente aumentamos el hardware existente (mejores discos, memoria o procesadores), lo cual en el fondo es sencillo de implementar, pero puede ser caro e insuficiente a ciertos niveles. En el escalado horizontal se trata de añadir más recursos en paralelo troceando y paralelizando las cargas de trabajo, lo cual puede requerir cambios en ellas y no es tan sencillo, pero así la capacidad puede crecer ilimitadamente y añade redundancia y tolerancia a fallos al sistema.



Las soluciones NoSQL son un cajón de sastre que trata de dar soluciones parciales y especializadas a problemas y limitaciones de las bases de datos relacionales tradicionales. Se usan solo cuando no hay más remedio, con volúmenes ingentes de datos o requerimientos extremos de lecturas/escrituras que necesitan escalado horizontal, o bien se tienen esquemas de datos no fijos, que evolucionan de forma flexible y la consistencia no es requerida en todo o en parte.

Simplificando mucho, y con amplias salvedades en uno y otro lado, estas podrían ser las grandes diferencias comunes a la mayor parte de paradigmas SQL y NoSQL:

Soluciones SQL	Soluciones NoSQL
- Diseño elaborado y de calidad - Esquema fijo - Escalado vertical - Rendimiento y disponibilidad complejos de adaptar a grandes volúmenes de trabajo - Ideal para consultas complejas, transacciones y fuerte consistencia. - Uso de SQL, integración clásica con entornos de desarrollo y lenguajes de programación	- Simplicidad de diseño - Flexibilidad de esquema - Escalado horizontal - Rendimiento y disponibilidad predecibles ante grandes volúmenes de trabajo - Transacciones, consistencia y JOIN no son importantes - Acceso mediante API, orientado a la programación y entornos de desarrollo. SQL inexistente o limitado

Características de las soluciones SQL vs. soluciones NoSQL

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

1.2. SQL vs. NoSQL: escenarios de uso

Los escenarios de uso de soluciones SQL son los más extendidos, donde se requiere un sistema consistente y estándar para mantener y migrar y altamente compatible con cualquier herramienta para programación y análisis. Como se ha comentado antes, se utilizarán siempre por defecto salvo que no quede más remedio. Básicamente, si nuestra estructura de datos es compleja, se ajusta bien a un esquema fijo relacional, y por otro lado se estima que razonablemente las necesidades de rendimiento van a poder cubrirse con escalado vertical, las soluciones SQL son ideales y eficaces:

- Sistemas de gestión empresarial (ERP).
- Sistemas de gestión documental.
- Sistemas de gestión y control de procesos.
- Cualquier otro sistema que requiera transacciones, consistencia y calidad en los datos.

Los escenarios de uso de soluciones NoSQL, por tanto, son las que no son cubiertas por lo anterior. De nuevo, consistencia relativa, datos semiestructurados y que requieren flexibilidad, así como fuertes necesidades de rendimiento y fiabilidad solo cubiertas por escalado horizontal son la clave:

- Redes sociales, blogs, recomendaciones, chats…
- Aplicaciones IoT.
- Juegos sociales.
- Aplicaciones de móviles de uso masivo.
- Cualquier portal o gestor de contenidos con acceso masivo.
- Cualquier aplicación con tipos de datos poco adecuados en modelo relacional.

IMPORTANTE

La consistencia de los datos es un aspecto básico en soluciones SQL clásicas, pero puede relajarse mucho en el caso de los NoSQL. Las bases de datos relacionales tienen un modelo de consistencia sólido llamado ACID que garantiza un funcionamiento estricto y confiable:

- Atomicidad: todo o nada en transacciones.
- Consistencia: coherencia e integridad de datos y reglas.
- Aislamiento: las transacciones no se afectan entre sí (isolate en inglés).
- Durabilidad: las transacciones son definitivas una vez confirmadas.

Mientras tanto, las soluciones NoSQL, como norma general, siguen un modelo de consistencia BASE en su diseño que implica una alta disponibilidad y orientación al rendimiento, lo que permite datos obsoletos o respuestas aproximadas o incompletas:

- Basically Available, está operativo la mayoría del tiempo.
- Soft state, los datos en las diferentes réplicas no tienen que ser mutuamente consistentes en todo momento.
- Eventually Consistent, asegura la consistencia solo después de que pase cierto tiempo.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

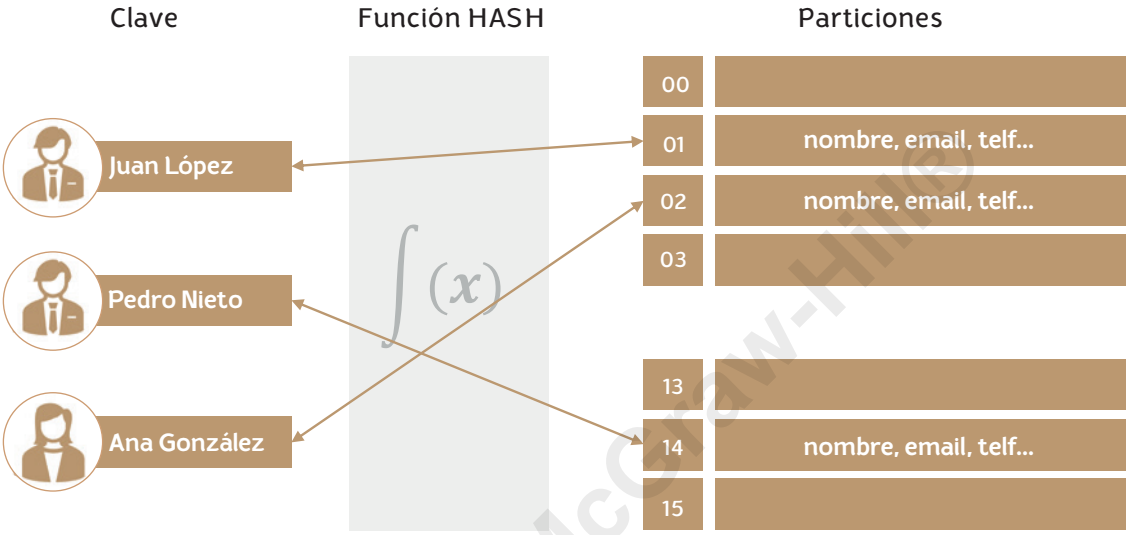
Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

2. Soluciones NoSQL: tipología e implementaciones en cloud

Como se ha visto, las soluciones NoSQL se reservan a requerimientos específicos cuando no queda más remedio, así que vamos a simplificar al máximo y tratar sólo algunos paradigmas habituales y con cierta extensión de uso.

2.1. Soluciones NoSQL clave-valor

Una base de datos *Key-Value* o clave-valor almacena los datos como un conjunto de pares en los que la clave es un identificador único que da acceso a los datos. Ejemplos importantes son *DynamoDB, Redis, Memcached* o *Azure CosmosDB*. Está orientada al rendimiento, a las peticiones masivas, a esquemas de datos sencillos y que pueden variar en cada individuo, y al escalado horizontal.

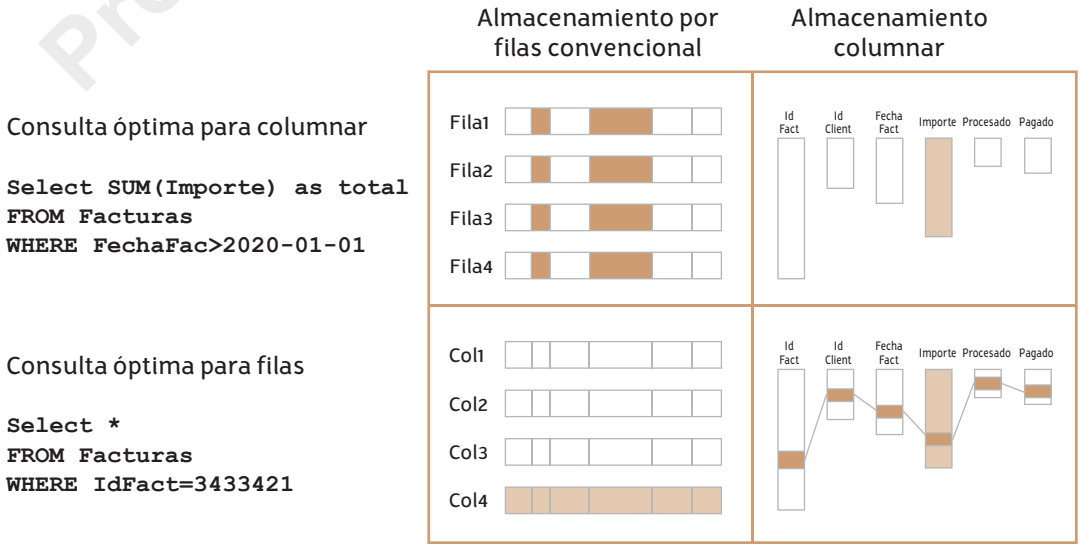


Almacenamiento clave

Como verás en un posterior caso práctico sobre *DynamoDB*, no se puede especificar como el modelo relacional solamente en base a la información, sino que el modelado dependerá fuertemente de un diseño de datos, claves, particiones, índices, etcétera, orientado al caso concreto y sus necesidades, garantizando así el rendimiento y escalado horizontal.

2.2. Soluciones NoSQL columnares

Una base de datos columnar almacena los datos en columnas, no en filas, y se utiliza normalmente en aplicaciones analíticas. De una manera intuitiva se puede entender que algunos tipos de operaciones sobre columnas (como los agrupamientos, muy usados en análisis), tendrán una mejora del rendimiento:



Almacenamiento columnar

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Es un grupo también heterogéneo, ya que está técnica la utilizan entornos de Datawarehousing como *AWS Redshift, Azure Synapse Analytics, MariaDB/PostgreSQL* (en algunas versiones) y opciones muy orientadas a la paralelización -y por tanto al escalado horizontal- *como HBase* o *Cassandra* (híbrida de columnar y clave-valor).

> Los conceptos y uso de terminología relacionada con el *Big Data* como *Datawarehouse, Data Lake* y *ETL/ELT* serán tratados en el segundo curso. De momento basta con saber que un Datawarehouse o Almacén de datos es un repositorio central de datos, normalmente históricos, altamente transformados y agrupados para realizar análisis y modelos de previsión.

2.3. Soluciones NoSQL documentales

Una base de datos documental permite almacenar y consultar datos como documentos, generalmente de tipo JSON o similar, de una manera muy orientada a la programación, a la flexibilidad y a la escalabilidad horizontal.

Junto a otras importantes como *CouchDB,* el ejemplo más paradigmático es *MongoDB*, solución NoSQL documental (híbrida clave-valor) en JSON que permite una estructura flexible donde cada documento es único y evoluciona con el tiempo, con una orientación directa a las aplicaciones y almacenamiento intuitivo de objetos como documentos.

Relacional

IdEmpl	Nombre	Email	Ciudad	fechaNac	Salario
1	Juan López	juan@empresa.es	Alicante	20/10/1984	31.000

IdDep	IdEmpl	Descripción
1	1	Contabilidad
2	1	Finanzas

IdCuenta	IdEmpl	Descripción
21	1	Juguetes La peña
22	1	Modas Galante
23	1	ElectroAlberto

IdDisp	IdEmpl	Descripción	Alta
101	1	iPhone	01/03/2020
107	1	Macbook Air	01/03/2020

MongoDB

```
[
{
  "nombre": "Juan Lopez",
  "email": "juan@empresa.es",
  "ciudad": "Alicante",
  "fechaNac": "20/10/1984",
  "salario": "31000",
  "departamento": ["Contabilidad", "Finanzas"],
  "cuentas": ["Juguetes la peña",
              "Modas Galante",
        "ElectroAlberto"
      ],
  "dispositivos": [
    {"descripcion":"iPhone",
     "alta": "01/03/2020"
  },
    {"descripcion":"iPhone",
   "alta": "01/03/2020"
  }
  ]
}
]
```

Almacenamiento relacional frente a documental en MongoDB rehacer por ilustrador

Tanto en las bases de datos documentales como en las clave-valor la estructura de los datos y sus relaciones deberán ser bien diseñadas para el uso que se pretenda dar y necesidades de rendimiento o escalabilidad, sobre todo en el caso de las relaciones, que habitualmente estarán como documentos embebidos dentro del principal, como en la figura anterior.

Así, según diseñemos tendremos más rendimiento, ya que recuperamos el documento completo sin JOINs, pero por otro lado hay que entender que tendremos datos duplicados (el mismo documento embebido estará en varios sitios) y podemos perder coherencia: será responsabilidad de la aplicación actualizar todas las duplicidades que haya y eso nos la puede complicar.

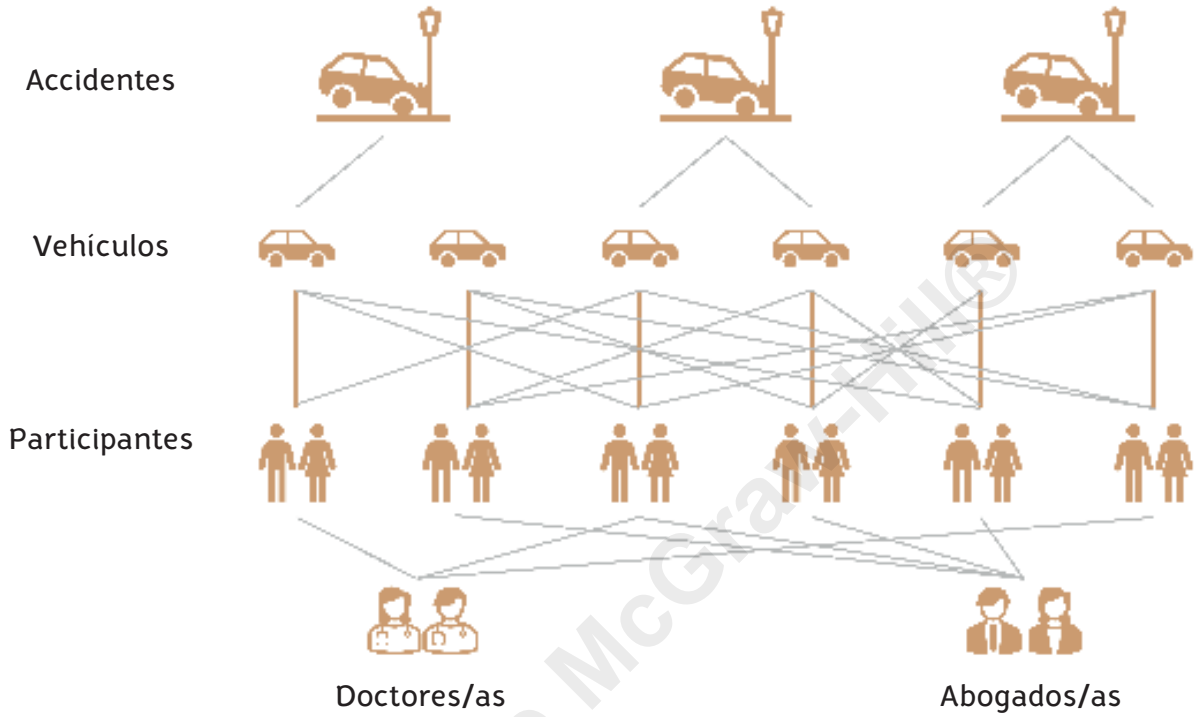
Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

2.4. Soluciones NoSQL de grafos

Las bases de datos de grafos representan la información como nodos de un grafo (con datos variables) y sus relaciones como aristas entre ellos, dirigidas (con puntas de flecha) o no. Ejemplos importantes de bases de datos de grafos son *Neo4j, AWS Neptune* o *Azure Cosmos DB.*

En definitiva, podemos pensar que es un tipo de bases de datos documentales pero centradas en las relaciones entre sus elementos, que pueden ser complejas: redes sociales, rutas, conglomerados empresariales, detección de fraude, sistemas de recomendación, sistemas de permisos y recursos, etcétera.



En la figura se puede ver intuitivamente la utilidad de un grafo para la detección de fraude, si existiera una concentración anormal de incidentes, pleitos e indemnizaciones que implicaran siempre a los mismos abogados, doctores y talleres en múltiples accidentes sin relación aparente.

2.5. Teorema CAP en la actualidad

En todo momento estamos indicando que las soluciones NoSQL tienen en la escalabilidad horizontal un punto fuerte, es decir, son soluciones basadas eminentemente en sistemas distribuidos, en nodos y particiones de procesamiento paralelo, lo cual tiene ciertos problemas y limitaciones.

El Teorema de Brewer o teorema CAP indica que es imposible para un sistema computacional distribuido ofrecer simultáneamente las siguientes tres garantías:

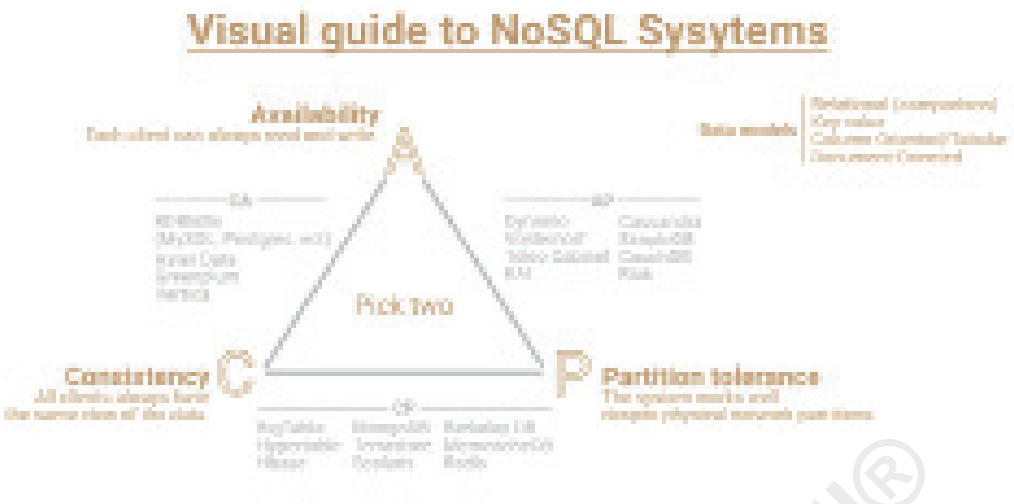
Consistencia (Consistency)	que todos los nodos vean los mismos datos al mismo tiempo
Disponibilidad (Availability)	garantizar que las peticiones encuentran una copia de los datos solicitados, aunque haya nodos caídos.
Tolerancia a la partición (Partition)	el sistema continúa funcionando a pesar de la pérdida de nodos o fallos parciales del sistema.

Es el equivalente, en paradigmas de arquitectura de datos, a los típicos carteles que podemos encontrar en un taller de reparaciones "Realizamos tres tipos de trabajo: el barato, el bueno y el rápido. Elija sólo dos cosas a la vez".

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

En base a este teorema, se han hecho clasificaciones de bases de datos como esta, que particionan de forma estricta cada base de datos según sus capacidades:



Clasificación de soluciones NoSQL por Nathan Hurst
Fuente: **blog.nahurst.com/visual-guide-to-nosql-systems**

Hay que tener en cuenta que el teorema CAP data del año 2000 y los sistemas evolucionan a gran velocidad, por lo que en muchas ocasiones y según versión y usos un sistema podrá estar presente en las tres dimensiones en mayor o menor medida. Seguramente tiene más sentido un diagrama radial para cada una de ellas de forma individual, según el uso que le vayamos a dar.

2.6. Conclusiones: ¿qué base de datos usar?

Las **soluciones NoSQL** están **orientadas a la paralelización y al rendimiento** con grandes tasas de lectura y escritura, así como **dinamismo en los esquemas**, pero las **consultas** que se pueden hacer son **más limitadas** y muchas veces requieren diseños complejos o **cargar la responsabilidad a la aplicación**, aunque cada vez hay más integración con herramientas de análisis.

En caso de necesitar **soporte transaccional, informes con consultas complejas, gran compatibilidad y calidad en los datos es mejor usar una solución SQL, un sistema relacional**. Por otra parte, los sistemas relacionales medianamente híbridos y de *datawarehousing* han evolucionado mucho y pueden cubrir muchas de las necesidades.

¿Por qué no ambas? Lo ideal es usar para cada tarea la herramienta adecuada. O sea, usar una **combinación de soluciones SQL y NoSQL** respectivamente en aquellas partes del sistema que sean más susceptibles de cubrir sus necesidades con cada tipo concreto.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

3. Servicios SQL (relacionales) en cloud

Los servicios relacionales implementados en cloud normalmente son más sencillos de entender y gestionar, y sus ventajas y automatizaciones resultan atractivas desde el primer momento, dado que conceptualmente son similares a las instalaciones propias, pero en el cloud.

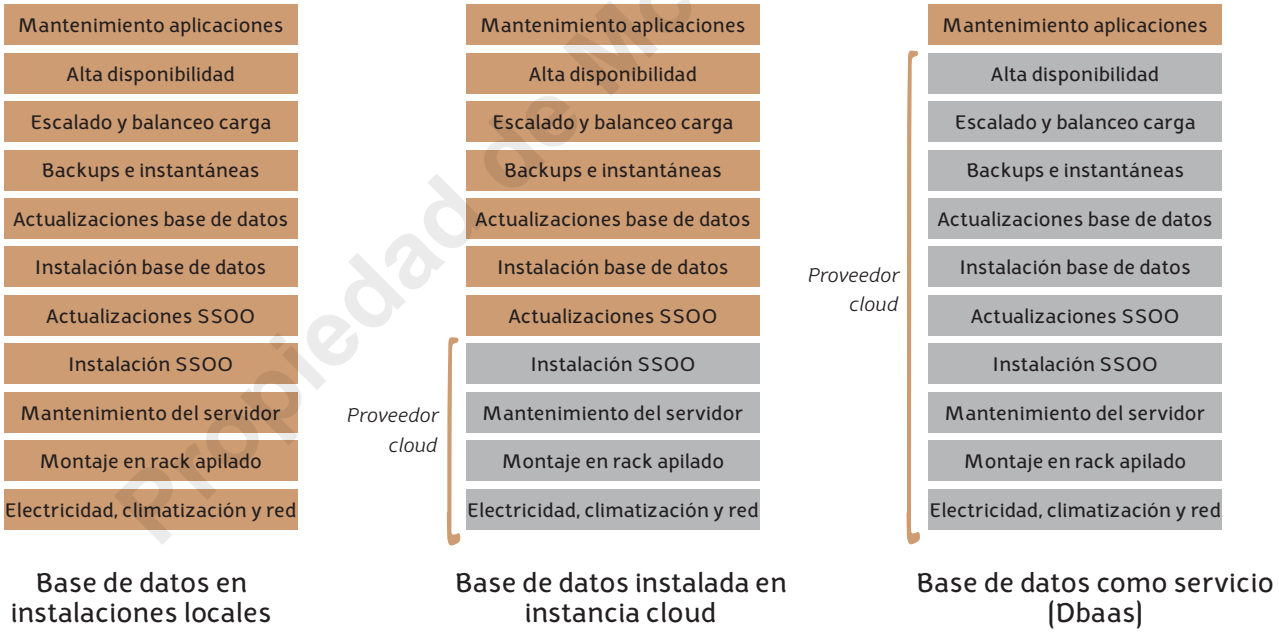
3.1. Servicios gestionados (DBaaS) vs. instalación en instancia cloud

La primera decisión que se debe tomar en el uso de base de datos relacionales en cloud tiene que ver con el grado de control que se desea en la implementación.

Partamos del supuesto de un sistema gestor de base de datos instalado localmente en una empresa. Hay que ocuparse de todo: instalaciones y seguridad física, energía y refrigeración, hardware, sistema operativo y su mantenimiento; más luego todo lo que tenga que ver con la base de datos en sí, su mantenimiento, copias de seguridad, alta disponibilidad, escalado, etcétera.

A partir de ahí tenemos dos generaciones de migración al cloud:

- **Una primera generación** fue utilizar una máquina virtual en la nube (una instancia cloud) en la que instalamos la base de datos exactamente igual que en instalaciones locales, pero aprovechando las instalaciones físicas y seguridad del cloud, así como otras ventajas, como el escalado vertical. En los comienzos del cloud (2007-2010) esta era la única opción.
- **Una segunda generación**, derivada de la explosión de desarrollo de los servicios cloud, es el uso de bases de datos como servicio (DBaaS), donde solo hay que ocuparse de definir y configurar una arquitectura que cubra nuestras necesidades, incluyendo copias, escalado y alta disponibilidad, siendo su buen funcionamiento responsabilidad total del proveedor.



Generaciones de implementación de servicios relacionales en cloud.

Esta evolución no ha venido de la noche a la mañana. Partiendo de la primera generación, todas las buenas prácticas de implementación de bases de datos en la nube se automatizaron poco a poco, haciendo atractivo para los usuarios descargarse de la responsabilidad de actualizaciones de seguridad de sistema operativo y motor de base de datos, copias de seguridad, alta disponibilidad y autoescalado, etcétera, que están incluidas en un DBaaS en cloud.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

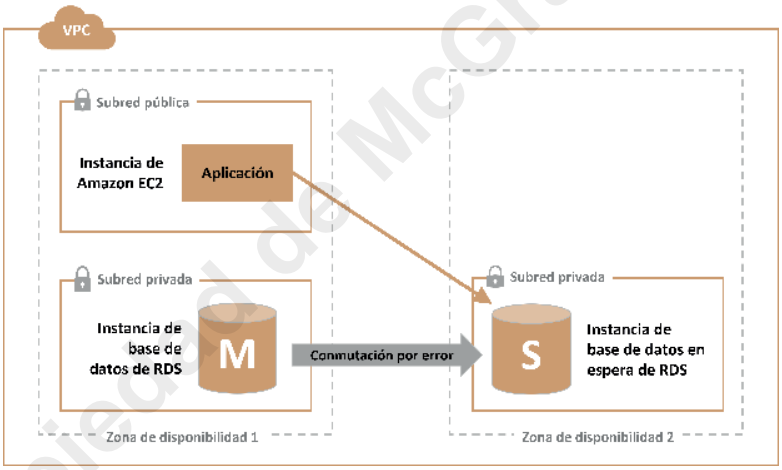
Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

3.2. RDS: servicio de base de datos relacional en AWS

Es un servicio de base de datos de Amazon Web Services que facilita la instalación, operación y escalabilidad de una base de datos relacional en la nube.

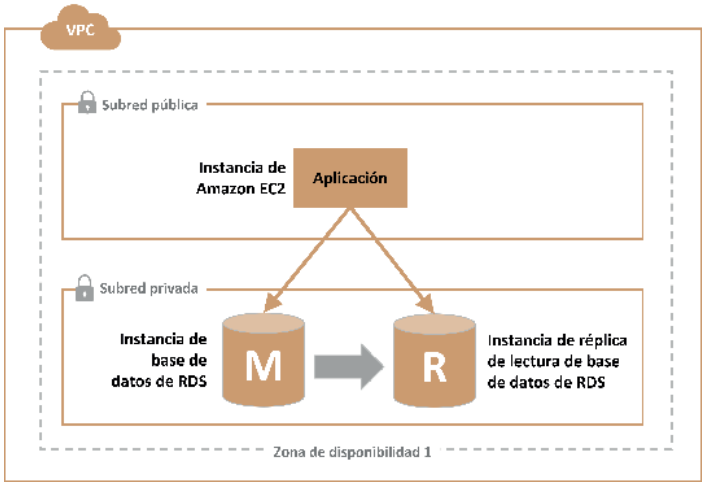
- Implementado para los motores más populares de base de datos relacionales: MySQL, MariaDB, PostgreSQL, SQL Server y Oracle. También tienen una variante de MySQL y PostgreSQL optimizada para la nube llamada Aurora, que luego trataremos.
- Es un servicio de automatización transparente, es decir, se paga conforme al hardware de las instancias cloud usadas (servidores virtuales con una CPU y memoria determinadas). Es decir, no tiene ningún coste adicional al uso de las máquinas en sí, pero goza de todas las ventajas de mantenimiento y automatización.
- Da acceso como administrador al sistema gestor de base de datos, pero no al sistema operativo ni a la instancia cloud (máquina virtual en la nube) subyacente.
- Proporciona de forma transparente copias de seguridad automáticas, balanceo de carga, alta disponibilidad, etcétera. Esto es compatible con cualquiera de los motores comentados, habiendo opciones mejoradas específicas para el cloud si se utiliza Aurora.
- Mantenimiento y actualizaciones automatizadas, tanto del sistema operativo como del SGBD.

RDS ofrece una alta disponibilidad mediante el despliegue multiAZ, donde genera un esclavo en una zona de disponibilidad diferente (en un centro de datos separado) que promociona automáticamente en caso de fallo sin cambiar nada en la aplicación. También sirve para hacer las copias de seguridad sobre el esclavo sin detener al maestro en ningún momento; así se pueden hacer copias con gran frecuencia, sin esperar a horas de baja actividad y sin ventanas de indisponibilidad.



Ejemplo de despliegue multiAZ para copia y failover.

RDS también proporciona alta disponibilidad mediante la creación automática de réplicas de lectura síncronas con el maestro (hasta 5) para repartir la carga de consultas sobre este. Esto es compatible con todos los motores de base de datos vistos, pero el balanceo de carga, es decir, el reparto de las consultas entre las réplicas, debemos programarlo en la aplicación: no es automático, al contrario que en el motor *Aurora,* donde sí lo es.



Ejemplo de uso de réplicas de lectura.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

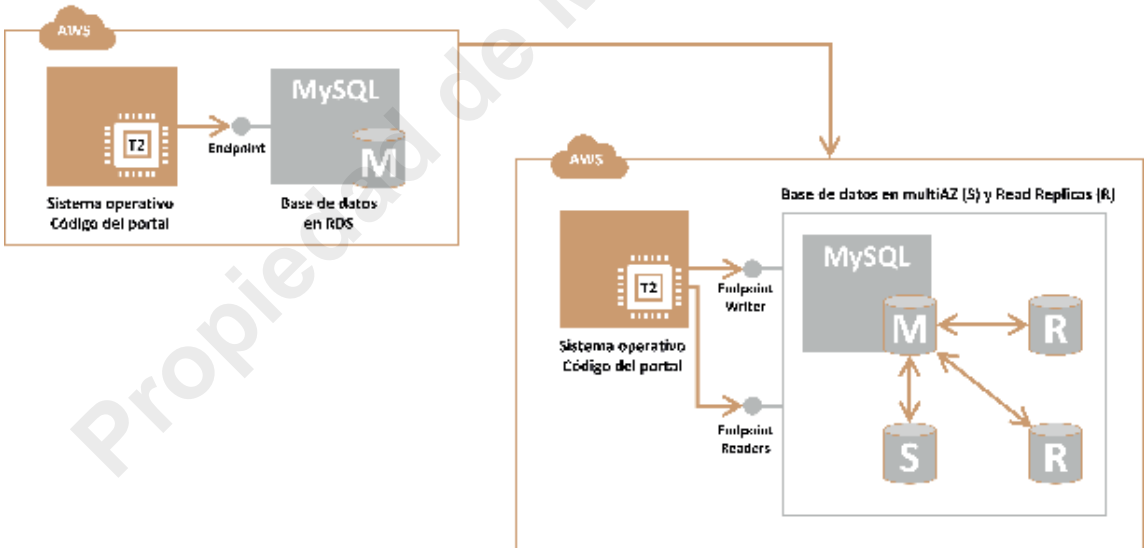
Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Mención aparte dentro de RDS merece el motor Aurora compatible con MySQL y PostgreSQL implementado por AWS. Al estar desarrollado de forma nativa por Amazon se adapta mejor a su infraestructura teniendo importantes ventajas:

	Si se desea escalado vertical, de CPU y RAM, se puede hacer en cuestión de minutos.
	Los discos aumentan de tamaño automáticamente hasta 128 TB, y es posible aumentar las IOPS de los mismos en cuestión de minutos.
	Las réplicas de lectura pueden ser hasta 15 y estar en regiones distantes del maestro.
x3	Los discos están replicados en todo momento en tres zonas de disponibilidad separadas (en centro de datos separados), siendo extremadamente tolerante a fallos y errores.
	Permite volver, en menos de un minuto y para cualquier tamaño, al estado de cualquier transacción exacta de la base de datos hasta 72 horas antes, almacenando el binlog en caché.

Aparte de lo descrito, la característica más potente de Aurora es la posibilidad de crear clústeres de forma transparente accesibles mediante endpoints, al estilo de un servicio de Datawarehouse.

Así, se puede escalar de forma transparente la base de datos desde una simple instancia o un multiAZ con un esclavo (como en la figura anterior) a un clúster complejo simplemente desglosando el endpoint, donde las peticiones SELECT se balancean automáticamente entre las distintas réplicas de lectura, mientras que los INSERT, UPDATE y DELETE van a uno o varios maestros, de forma transparente, sin cambiar nada en la aplicación que lanza las peticiones:



Ejemplo de crecimiento de una máquina simple a un clúster complejo en AWS Aurora.

3.3. Azure SQL: servicio SQL server relacional

En el caso de Azure, el servicio nativo más potente que ofrecen es Azure SQL, una implementación de SQL Server en diferentes modalidades. En caso de necesitar otros motores de base de datos como MySQL o PostgreSQL existen los servicios denominados Azure Database for MySQL o PostgreSQL, como forma de dar compatibilidad dentro del proveedor, pero no son servicios centrales que sean su punto fuerte ni tienen las mismas posibilidades.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

El servicio de Azure SQL (y en menor medida los servicios de compatibilidad descritos) proporciona características similares a RDS especializadas para motor SQL Server:

- Se da acceso como administrador al SGBD, pero no al sistema operativo ni máquina o estructura subyacente en modo alguno.
- Proporciona de forma transparente copias de seguridad automáticas, balanceo de carga, alta disponibilidad, etcétera, eso sí, según nivel de servicio, no de forma general.
- Mantenimiento y actualizaciones automatizadas.

Este servicio tiene dos modalidades: Azure SQL Database y Azure SQL Managed Instance.

Azure SQL Database

Permite la creación de bases de datos SQL Server compatibles a la última versión. Al contrario que RDS —en el que se elige un servidor virtual con un hardware concreto, una CPU y memoria determinadas—, en este caso los recursos asociados a la base de datos se pueden definir/facturar de dos maneras:

Basado en DTU (unidades de rendimiento)	Constituye un "paquete" de ciclos de CPU, memoria y entrada/salida a disco que corresponde, simplificando mucho, a una "transacción media" en una base de datos. En este modelo debemos estimar para cuántas de esas transacciones vamos a necesitar recursos de forma concurrente, y nos serán facturadas.
Basado en núcleos	Orientado a hardware, se definen núcleos de procesador y memoria a utilizar, y por el uso de esos recursos nos facturarán.

Existen calculadoras y técnicas para estimar los DTU o núcleos que podamos necesitar. La cantidad de recursos no se puede escalar indefinidamente, sino que hay unos límites mínimos y máximos según el nivel de servicio que elijamos: básico, empresa, premium e hiperescala.

Independientemente de cómo definamos los recursos, cada base de datos se considerará una **base de datos única**, donde está aislada y tiene una cantidad de recursos dedicados para la misma. Si en un diseño tenemos varias de estas bases de datos podemos agruparlas en un **grupo elástico**, de manera que, en momentos de baja carga en algunas de ellas, pueden prestarles recursos a otras. De nuevo tenemos niveles de servicio (básico, empresa y premium) que limitan el escalado de recursos esta vez para todo el grupo.

La estructura de costes es compleja de calcular, requiere estudio detallado y está encajonada entre niveles de servicio, por lo que muchas veces redirige a representantes de ventas para definir cálculos y opciones y límites de servicio. Es aconsejable, por tanto, una formación muy específica o la participación de una empresa externa partner de Microsoft.

Azure SQL Managed Instance

Es un servicio mucho más transparente y parecido a RDS: simplemente permite la creación de bases de datos SQL Server en servidores virtuales con unas características hardware definidas (CPU y memoria), pagando por hora de uso y por almacenamiento.
Por ello es el recomendado para migraciones e hibridación, donde no desee entrar en los complejos cálculos del servicio anterior, sino tener un hardware y software definidos y administrados en la nube con todas las funcionalidades de DBaaS (actualizaciones y copias de seguridad automáticas, alta disponibilidad, etcétera), que reducen enormemente la sobrecarga de administración.
Sí que mantiene de nuevo la estructura de niveles de servicio, aunque más simple, con menor o mayor coste y funcionalidades:

- **El nivel de "uso general"**, más económico, pero con límites en escalado de disco en tamaño y velocidad, potencia de CPU y en opciones de alta disponibilidad.
- **El nivel "crítico para la empresa"**, que es más parecido a RDS, donde tenemos límites más amplios en tamaño y velocidad de disco, zonas de disponibilidad y hay réplicas de lectura.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

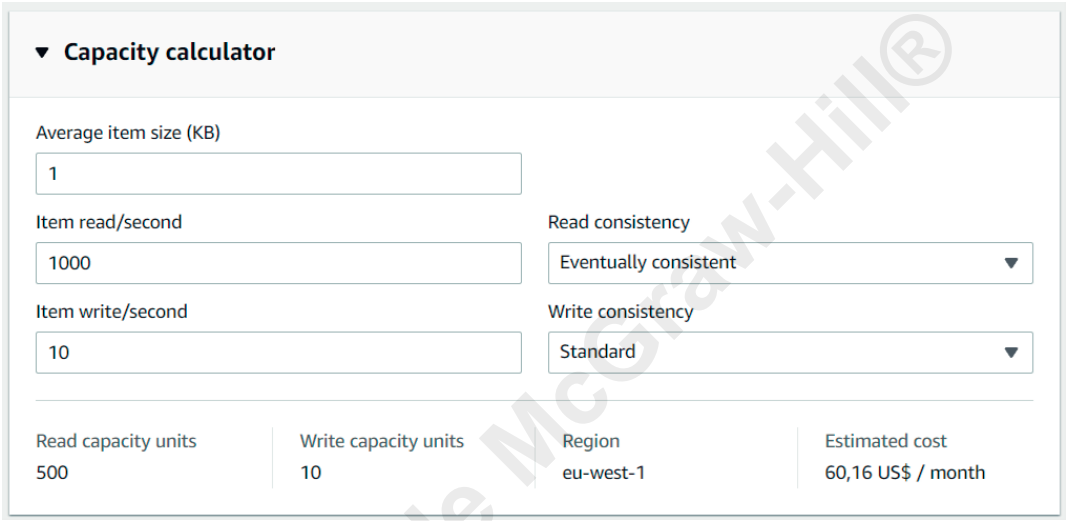
Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

4. Servicios NoSQL en cloud

Vamos a explorar algunas implementaciones cloud de servicios NoSQL populares de clave-valor *(AWS DynamoDB)*, documental híbrida (MongoDB) y el enfoque multimodo de Azure CosmosDB. Nos dejamos en el tintero implementaciones columnares orientadas al Big Data y Datawarehousing (como Redshift) y las de grafos, no por ser de menor interés, sino porque en ambos casos tienen implicaciones complejas y se requeriría mucho más tiempo para su estudio.

4.1. Servicio AWS DynamoDB (NoSQL clave/valor)

Es un almacén de clave/valor, flexible y sin estructura fija, diseñado para garantizar un determinado rendimiento, así como una determinada disponibilidad para cada tabla (en NoSQL suele haber pocas tablas). Es decir, se definen parámetros por tabla y se paga según lo exigido en cada una. Es, por tanto, una base de datos serverless.

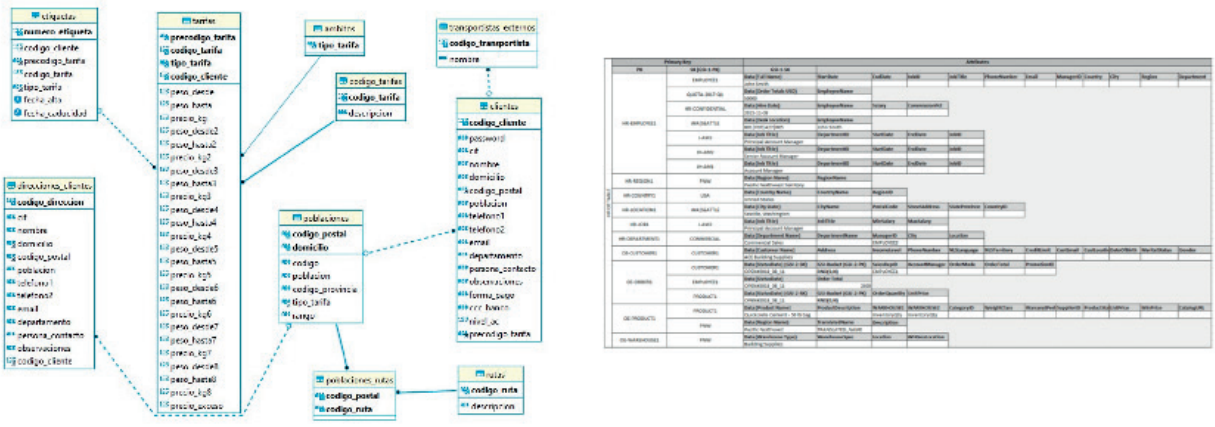


Especificación de requisitos de rendimiento en una tabla de DynamoDB (consola AWS)

Por ejemplo, en esta tabla concreta especificamos un rendimiento garantizado de 1.000 lecturas/s y 10 escritura/s, con una consistencia eventual (es decir, que permite desorden en lecturas y escrituras, como por ejemplo para un chat), y eso nos costará $60,16 al mes en Europa occidental.

DynamoDB garantiza baja latencia y soporte a millones de peticiones, pero para conseguir este rendimiento no podemos utilizar un modelo relacional clásico donde podemos usar SQL y hacer JOIN de forma indiscriminada.

En su lugar, se juntan muchas tablas y relaciones en una o pocas tablas con un sistema de claves de partición y ordenación, y orientado a unas consultas concretas: si con el tiempo estas necesidades de consulta cambian, esto podría alterar el modelo, al contrario que uno relacional:

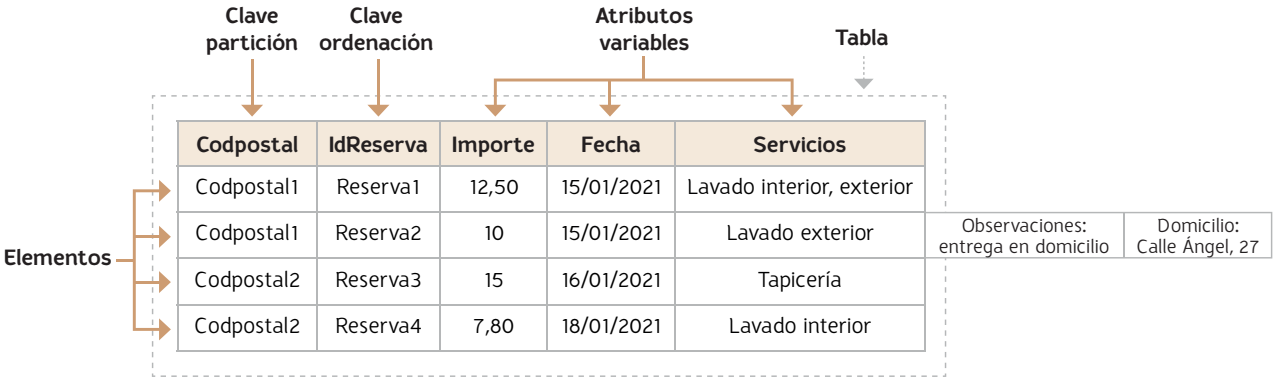


Aspecto típico de un diseño relacional frente a uno en DynamoDB

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

A la izquierda hay un diseño relacional clásico con tablas y relaciones, mientras que en DynamoDB —a la derecha— lo tendremos todo en una o pocas tablas y claves de partición y ordenación. Este último es el aspecto clave para garantizar el rendimiento: deberemos elegir una clave que particione los datos en conjuntos que permitan su escalado horizontal, así como su búsqueda y ordenación rápidas en cada partición.



Estructura de una tabla en DynamoDB.

Además, como se puede ver, cada elemento puede tener el número de atributos que se desee, lo que permite flexibilidad en el esquema. En un caso práctico extendido profundizaremos en estos aspectos de diseño y operación frente a un modelo relacional.

Un desafío habitual en este tipo de bases de datos con fuertes requerimientos de rendimiento y baja latencia es la escalabilidad a nivel global, con usuarios dispersos por todo el mundo. Es aquí cuando la estructura y la dimensión del cloud a nivel mundial entran en juego y permiten resolver el problema.

Por ejemplo, en el caso de DynamoDB existe un servicio de "tablas globales" que sincroniza las tablas individuales que tengamos en cada región donde preveamos tener peticiones masivas, de forma que funcionan como una sola a nivel mundial, con la mínima latencia posible.

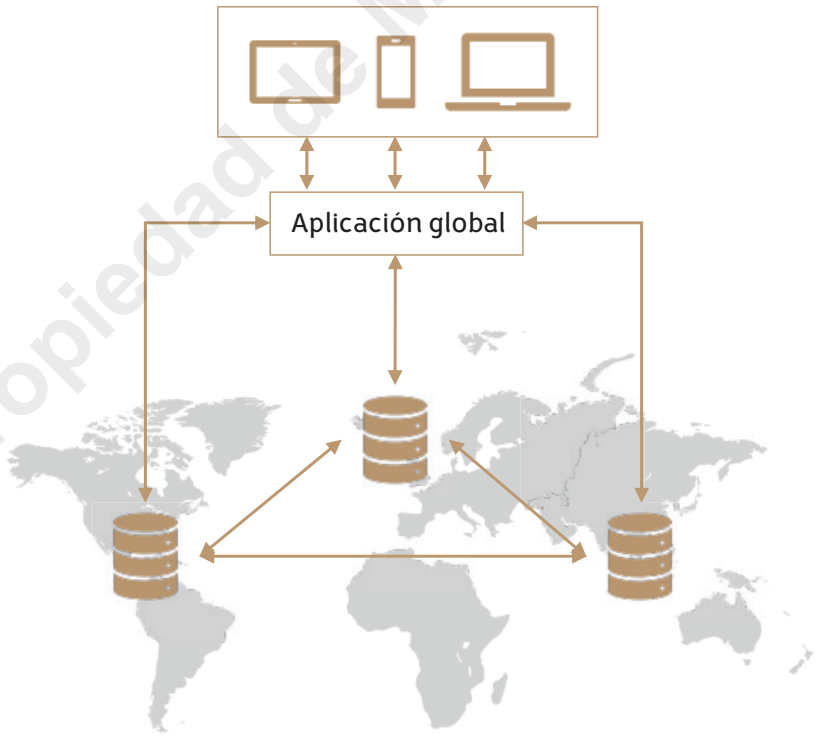


Tabla de acceso único en DynamoDB

Propiedad de McGraw-Hill®

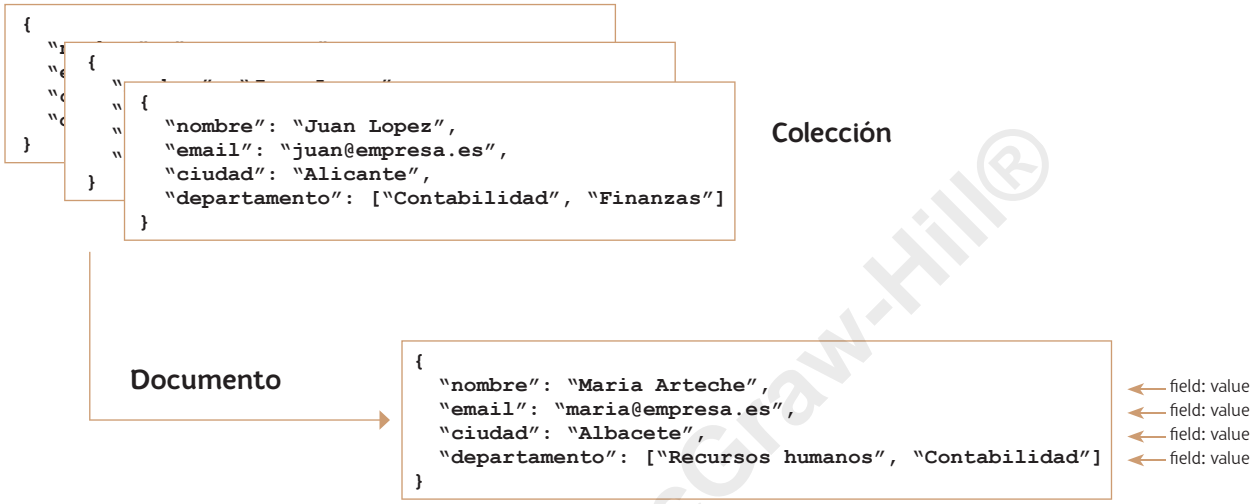
Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

4.2. MongoDB: base de datos documental híbrida

MongoDB es una base de datos que nativamente se concibió como documental, de forma que los datos se almacenan en documentos JSON, en un intento de ser una base de datos que resuelva ineficiencias del modelo relacional clásico en aspectos de excesiva complejidad, flexibilidad, orientación a la programación y escalabilidad horizontal.

En MongoDB, la información se almacena en colecciones de documentos presentados en JSON, aunque internamente se integran de forma binaria (BSON). El formato JSON orientado a objetos que pueden contener elementos simples, arrays y otros documentos anidados (evitando relaciones), unido al JavaScript como lenguaje de manejo, denota su fuerte orientación a la programación con decenas de drivers nativos para muchos lenguajes.



Estructura de elementos JSON en una base de datos MongoDB.

Una tabla en MongoDB sería una "colección" y un registro sería un "documento" que puede tener campos simples, compuestos o incluso un documento entero como campo. Así, MongoDB puede aglutinar gran cantidad de entidades en un documento, frente a un modelo relacional completamente normalizado. Esto resulta atractivo y natural para la programación.

También es atractivo por su flexibilidad de esquema. Si en el ejemplo de arriba, en el caso de necesitar almacenar datos de contacto (email, varios teléfonos, Twitter...) en lugar de un teléfono, bastaría con cambiar el campo a un documento embebido de contactos. Pero claro, toda esta flexibilidad no es automática. Aunque no es obligatorio, es recomendable crear un proceso de versiones que unifique:

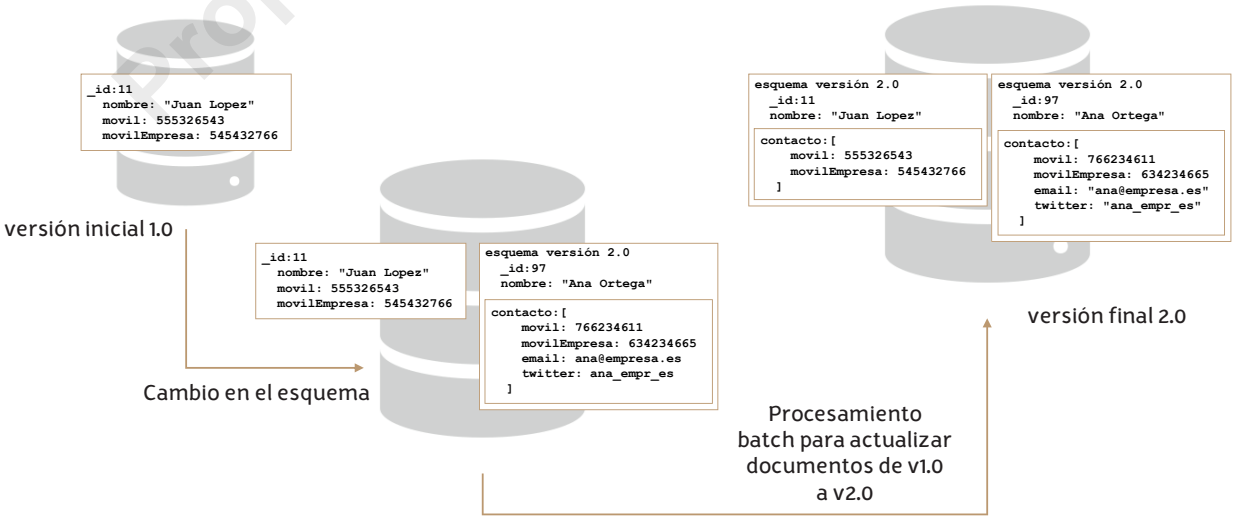


Figura 10.18 Gestión de versiones de esquema.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Igualmente, en función de la complejidad del esquema y los requerimientos de consultas, es necesario hacer un buen diseño de relaciones, debiendo elegir fundamentalmente entre enlazar (evita duplicidades, pero es más complejo en recogida de datos) o embebido (recogida más simple, pero habrá duplicidades). En la figura hay un ejemplo para una relación 1-1, extensible a otros tipos:



Relación 1-1 en MongoDB: implementación enlazada frente a embebida.

En un esquema más complejo para una aplicación es cuando empezarían las preguntas de diseño: ¿con qué frecuencia se accede a datos embebidos? ¿Las consultas suelen incluir datos embebidos? ¿Con qué frecuencia se cambian los datos embebidos (y por tanto hay que vigilar duplicidades)?

Finalmente, respecto al uso y la disponibilidad de MongoDB, y más al ser una base de datos OpenSource, existen diferentes opciones:

- **Instalación local**, muy usada para desarrollo y pruebas. Será mucho más complicado de operar si queremos acceder a sus características de rendimiento y escalado horizontal, para lo cual habría que implementar un clúster.
- Usar **plantillas de infraestructura como código de los proveedores cloud**, que proveen para desplegar automáticamente una infraestructura clúster compatible.
- Usar una **implementación compatible del proveedor cloud**, ya que al ser OpenSource habitualmente programan MongoDB total o parcialmente para una versión de su API. Este es el caso de DocumentDB en AWS o la base de datos multimodo CosmosDB en Azure.

Si se desea utilizar en cloud, lo mejor es la evolución de las dos últimas, la **implementación cloud propia de MongoDB denominada Atlas**, impulsada desde 2017, que está presente en AWS, Azure y Google Cloud con plena integración, pudiendo elegir en qué nube se despliega de forma transparente. Lo trataremos en un caso práctico extendido.

4.3. Azure CosmosDB: base de datos multimodo global

Azure CosmosDB es una gran apuesta de Microsoft en el NoSQL que tiene unas características muy particulares y ambiciosas:

- **Es agnóstica al esquema.** Una base de datos en CosmosDB almacena contenedores (como tablas o colecciones) que tienen ítems (como registros o documentos) en su interior. Los índices son automáticos y pueden ser convencionales o espaciales (v.g. datos geográficos).
- **Es multimodo.** Puede funcionar de forma relacional, documental, columnar, clave-valor o de grafos. Para ello tiene varias API que tratan los datos internamente de formas diferentes:

API o "motor"	Tipo	Nombre de contenedores e ítems
SQL	Relacional	Tablas y registros
MongoDB	Documental/clave-valor	Colecciones y documentos
Gremlin	Grafos	Grafos y nodos
Cassandra	Columnar/clave-valor	Tabla y columna
Azure Table	Clave-valor	Tabla e ítem
Etcd	Clave-valor	Clave y valor

APIs multimodo que implementa CosmosDB.

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.

- **Carácter global.** Las bases de datos en CosmoDB pueden configurarse para ser accedidas desde cualquier región de Azure con copias locales, cuyo nivel de consistencia y sincronización se puede elegir entre varios grados desde una baja consistencia en la que puede haber retrasos en las escrituras entre regiones a una consistencia máxima.
- **Herramientas de análisis.** Está conectado con diferentes herramientas de Big Data y análisis mediante la creación de un almacén separado a tal fin a partir de datos transformados del esquema principal, aunque con fuertes limitaciones.

Realmente tiene características muy ambiciosas que integran todas las necesidades que se pueden tener en un sistema gestor de base de datos, pero no está teniendo un gran éxito de adopción en casos reales complejos.

Una posible razón puede ser la fuerte dependencia de las limitaciones de cada API, que además suelen ser compatibles solo con determinadas versiones respecto a los originales. Así, puede haber temor de estar atado a unos desarrollos de evolución futura incierta, decidiendo en muchas ocasiones usar los originales de los diferentes sistemas en lugar de uno multimodo.

Por ejemplo, si se desea usar un sistema relacional se coge un SQL Server, PostgreSQL, Oracle, etcétera, o si se desea un MongoDB o Cassandra es mejor usar el original. La operativa en CosmosDB será introducida en un caso práctico tutorial.

Propiedad de McGraw-Hill®

Propiedad de McGraw-Hill®. Prohibida la reproducción total o parcial sin el permiso adecuado.
Uso exclusivo para el contenido de los cursos FPActiva® de McGraw-Hill.